Learn Being Forward

COURSES · CAMPUS PARTNERSHIPS · PLACEMENTS

COURSE NOTES · BASIC TO ADVANCED

Vue.js
Building Modern Web Interfaces

Complete notes for the Learn Being Forward Vue.js syllabus: every module, topic and subtopic explained step by step, with diagrams, verified examples and a practice checkpoint for each module.

21	3	42	281
modules	levels	minimum days	components and programs run

How to use these notes

These notes follow the **Learn Being Forward Vue.js syllabus** exactly: the same 21 modules, in the same order, with every topic and subtopic explained step by step. Each subtopic number (for example **3.7.2**) matches the syllabus, so you can always find where you are.

Level	Modules	Minimum	Outcome
1. Beginner	1 to 8	11 days	Build interactive Vue apps with templates, data binding, reactivity, computed properties, lists, events and forms
2. Intermediate	9 to 15	16 days	Create reusable components with props, emits and slots, use watchers, lifecycle hooks, composables, Vue Router, Pinia and APIs
3. Advanced	16 to 21	15 days	Use TypeScript, master reactivity and performance, extend Vue with directives and plugins, test thoroughly and build full-stack apps with Nuxt
Total	1 to 21	42 days	About 7 weeks at 6 study days per week

Read
Read the concept for each subtopic and note the key points in your own words.

Type & run
Type out and run every example. Do not copy and paste code. Compare with the OUTPUT box.

Checkpoint
Finish the practice checkpoint at the end of each module before moving on.

Revise
At the end of each level, revise its modules and build one small project.

Every Vue example in these notes was run, not just written: the `.vue` Single-File Components were compiled and bundled with esbuild and executed in **Chromium**, and the picture under the code is a screenshot of what the browser displayed, taken after really clicking and typing where the caption says so. Console output, warnings and rendered HTML were captured from the same running page, plain JavaScript was run with **Node.js 22** and TypeScript was checked with the real **tsc** compiler. Because npm packages could not be installed in the offline build environment, the examples ran on a compact runtime written for these notes that implements the public API of **Vue 3.5** (reactivity, the template compiler, the SFC compiler with `<script setup>`, built-in components and directives) with the same behaviour and warning messages, so the code is written exactly as for a real `npm create vue@latest` project. Vue Router, Pinia, Axios, VeeValidate, Vitest and Vue Test Utils were run against small stand-ins with the same API, and code for tools that cannot run here (the Vite dev server itself, Vue DevTools, UI component libraries, deployment platforms, CI pipelines) is labelled **representative**.

HOW TO READ THE BOXES

Dark boxes with a gold top line are **Vue, JavaScript, HTML and CSS code**. Dashed cream boxes labelled **OUTPUT** show what the program prints. Gold-bordered boxes are notes and tips, red boxes warn about common mistakes, and navy boxes hold the key idea to remember.

Course plan at a glance

Module	Topic	Level	Min days
01	Introduction to Vue.js	Beginner	1
02	JavaScript essentials for Vue	Beginner	1

Module	Topic	Level	Min days
03	Setup and Single-File Components	Beginner	1
04	Template syntax and data binding	Beginner	2
05	Reactivity fundamentals	Beginner	2
06	Conditional and list rendering	Beginner	1
07	Event handling and methods	Beginner	1
08	Forms and v-model	Beginner	2
09	Components: props, events and slots	Intermediate	3
10	Watchers and lifecycle hooks	Intermediate	2
11	Advanced component features	Intermediate	2
12	Composables	Intermediate	2
13	Routing with Vue Router	Intermediate	3
14	State management with Pinia	Intermediate	2
15	Working with APIs and form validation	Intermediate	2
16	TypeScript with Vue	Advanced	2
17	Reactivity in depth and performance	Advanced	2
18	Advanced patterns, plugins and UI libraries	Advanced	2
19	Testing Vue applications	Advanced	2
20	Full-stack Vue with Nuxt	Advanced	4
21	Deployment, projects and interview preparation	Advanced	3

Table of contents

LEVEL 1 · BEGINNER LEVEL · MODULES 1 TO 8

Module 01 · Introduction to Vue.js 7

1.1 What is Vue.js? 7
1.2 History of Vue.js 11
1.3 Why do we need Vue.js? 12
1.4 Where is Vue.js used? 13
1.5 Vue vs React vs Angular 14
1.6 Options API vs Composition API 14

Module 02 · JavaScript essentials for Vue 17

2.1 Modern syntax 17
2.2 Arrays and objects 19
2.3 Modules 21
2.4 Asynchronous JavaScript 21
2.5 JavaScript concepts behind reactivity 23

Module 03 · Setup and Single-File Components 27

3.1 Ways to use Vue 27
3.2 Creating a project 29
3.3 Project structure 30
3.4 Single-File Components (.vue) 32
3.5 Tooling 35

Module 04 · Template syntax and data binding 37

4.1 Text interpolation 37
4.2 Directives 40
4.3 Attribute binding 42
4.4 Class and style bindings 44
4.5 Other built-in directives 45

Module 05 · Reactivity fundamentals 49

5.1 ref() 49
5.2 reactive() 51
5.3 Computed properties 54
5.4 How reactivity works (overview) 57

Module 06 · Conditional and list rendering 60

6.1 Conditional rendering 60
6.2 List rendering 63

Module 07 · Event handling and methods 71

7.1 Listening to events 71
7.2 Event modifiers 73
7.3 Key and mouse modifiers 74
7.4 Functions in script setup 76

Module 08 · Forms and v-model 79

8.1 v-model basics 79
8.2 v-model modifiers 82
8.3 Value bindings 83

8.4 Form submission 84
8.5 Validation 85

LEVEL 2 · INTERMEDIATE LEVEL · MODULES 9 TO 15

Module 09 · Components: props, events and slots 90

9.1 Component basics 90
9.2 Props 92
9.3 Emitting events 97
9.4 Slots 99
9.5 Fallthrough attributes 103

Module 10 · Watchers and lifecycle hooks 110

10.1 Watchers 110
10.2 Lifecycle hooks 115
10.3 Template refs 119

Module 11 · Advanced component features 124

11.1 Component v-model 124
11.2 Provide and inject 127
11.3 Dynamic and async components 130
11.4 Built-in components 132
11.5 Transitions and animations 135

Module 12 · Composables 141

12.1 What composables are 141
12.2 Writing composables 143
12.3 Common composables 145
12.4 VueUse library 150

Module 13 · Routing with Vue Router 154

13.1 Setup 154
13.2 Navigation 156
13.3 Dynamic routes 158
13.4 Nested routes and layouts 161
13.5 Navigation guards 163
13.6 Lazy loading routes 166
13.7 Other features 167

Module 14 · State management with Pinia 171

14.1 Why a store 171
14.2 Pinia basics 172
14.3 Using stores 174
14.4 Organising stores 179
14.5 Plugins 181
14.6 Pinia in Vue DevTools 181

Module 15 · Working with APIs and form validation 184

15.1 API communication 184
15.2 Server-state libraries 190
15.3 Form validation libraries 192
15.4 File uploads 195

LEVEL 3 · ADVANCED LEVEL · MODULES 16 TO 21

Module 16 · TypeScript with Vue 202

16.1 Setup 202
16.2 Typing components 205
16.3 Typing reactivity 211
16.4 Typing the ecosystem 214

Module 17 · Reactivity in depth and performance 221

17.1 Reactivity internals 221
17.2 Rendering mechanism 226
17.3 Performance techniques 227
17.4 Measuring performance 233

Module 18 · Advanced patterns, plugins and UI libraries 236

18.1 Custom directives 236
18.2 Plugins 240
18.3 Render functions and JSX 242
18.4 Component patterns 244
18.5 Styling in Vue 248
18.6 UI libraries 252
18.7 Accessibility and internationalisation 253

Module 19 · Testing Vue applications 261

19.1 Testing fundamentals 261
19.2 Vitest 262
19.3 Component testing 265
19.4 Testing the ecosystem 269
19.5 End-to-end testing 273
19.6 Coverage and CI 275

Module 20 · Full-stack Vue with Nuxt 278

20.1 Why Nuxt 278
20.2 Nuxt basics 280
20.3 Routing and layouts 282
20.4 Data fetching 286
20.5 Server routes 289
20.6 State, SEO and modules 291
20.7 Deploying Nuxt 296

Module 21 · Deployment, projects and interview preparation 300

21.1 Deployment 300
21.2 Best practices 302
21.3 Capstone projects 305
21.4 Interview preparation 307
21.5 Next steps 312

LEVEL 1

Beginner level

Outcome: build interactive Vue apps with templates, data binding, reactivity, computed properties, lists, events and forms.

8 MODULES · 11 DAYS MINIMUM

Module 01 · Introduction to Vue.js	1 day
Module 02 · JavaScript essentials for Vue	1 day
Module 03 · Setup and Single-File Components	1 day
Module 04 · Template syntax and data binding	2 days
Module 05 · Reactivity fundamentals	2 days
Module 06 · Conditional and list rendering	1 day
Module 07 · Event handling and methods	1 day
Module 08 · Forms and v-model	2 days

MODULE 01 · BEGINNER

Introduction to Vue.js

Minimum duration **1 day** | Level **Beginner**

This module explains what Vue.js is, why developers choose it, and where it is used, before any project is created. It defines Vue as a progressive JavaScript framework, explains what "progressive" means, introduces declarative rendering, reactivity and components, traces Vue's history from Evan You's first release in 2014 through Vue 2, Vue 3 and the Composition API to Vapor Mode, describes where Vue is used, compares it with React and Angular, and introduces the two ways of writing components (Options API and Composition API) and why this course uses the Composition API with `<script setup>`.

IN THIS MODULE

1.1 What is Vue.js?
1.2 History of Vue.js
1.3 Why do we need Vue.js?
1.4 Where is Vue.js used?
1.5 Vue vs React vs Angular
1.6 Options API vs Composition API

1.1 What is Vue.js?

1.1.1 Definition: a progressive JavaScript framework for building user interfaces

Vue.js (pronounced "view") is an open-source **JavaScript framework for building user interfaces**. You describe what the page should look like for the current data, and Vue keeps the page in sync as the data changes. Vue focuses on the view layer but comes with official tools for everything a real application needs: routing (Vue Router), state management (Pinia), a build tool (Vite) and a full-stack framework (Nuxt).

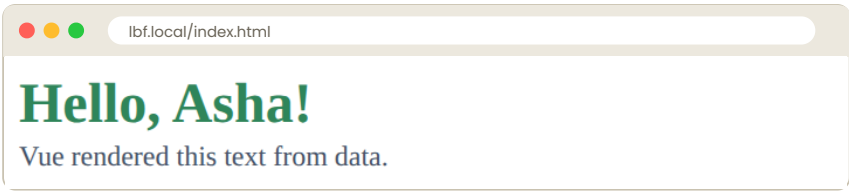
The smallest Vue program needs only one HTML file. Vue is loaded with a `<script>` tag, an app is created from an object, and it is mounted on an element. This page was opened in a real browser:

INDEX.HTML HTML

```html
<!DOCTYPE html>
<html>
<head>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
  <div id="app">
    <h1>Hello, {{ name }}!</h1>
    <p>Vue rendered this text from data.</p>
  </div>

  <script>
    const { createApp, ref } = Vue

    createApp({
      setup() {
        const name = ref('Asha')
        return { name }
      }
    }).mount('#app')
  </script>
</body>
</html>
```



VUE REPLACED `{{ NAME }}` WITH THE VALUE OF `NAME`

createApp()

Creates an **application instance** from a root component (here a plain object with a `setup()` function).

mount("#app")

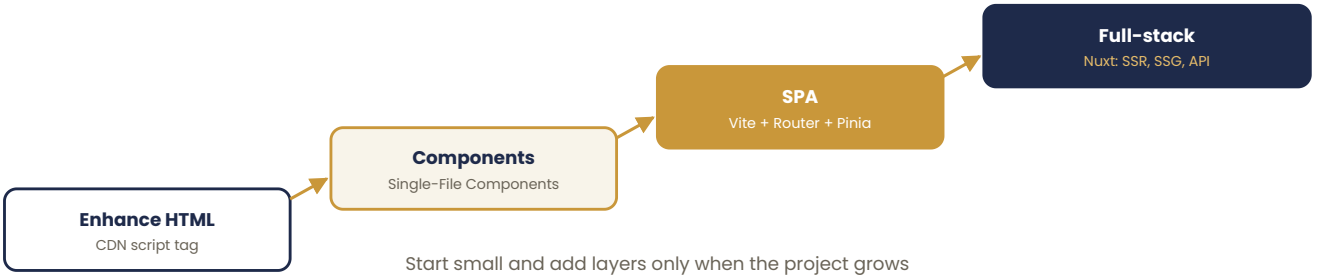
Takes over the element with `id="app"`; its inner HTML becomes the **template**.

{{ name }}

Text interpolation: shows the current value of `name` and updates it when it changes.

1.1.2 What "progressive" means: use as little or as much as you need

Vue calls itself a **progressive framework** because you can adopt it step by step. The same core library works for a small widget on an existing page and for a large single-page application. You add the router, the store, the build tool and server rendering only when the project needs them.



THE PROGRESSIVE ADOPTION PATH: EVERY LAYER IS OPTIONAL

Level of use	What you write	Typical use
Script tag	HTML + a `<script>` with `createApp`	add interactivity to a server-rendered page
Single-File Components	`.vue` files built by Vite	a new interactive app or dashboard
Single-page application	components + Vue Router + Pinia	large apps with many screens
Full-stack	Nuxt (server rendering, API routes)	SEO-friendly sites, blogs, stores

1.1.3 Declarative rendering and reactivity

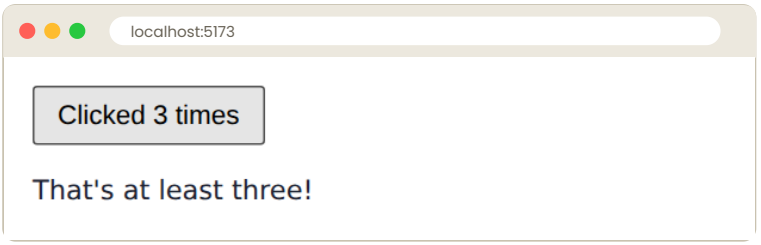
Declarative rendering means you describe the *result* ("show the count"), not the steps ("find the element, change its text"). **Reactivity** means Vue tracks which data each part of the page uses and updates exactly those parts when the data changes. The example below has no DOM code at all; the button was really clicked three times before the screenshot:

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'

const count = ref(0)
</script>

<template>
  <button @click="count++">Clicked {{ count }} times</button>
  <p v-if="count >= 3">That's at least three!</p>
</template>
```



AFTER THREE REAL CLICKS: `COUNT` CHANGED AND VUE UPDATED THE BUTTON TEXT AND SHOWED THE PARAGRAPH

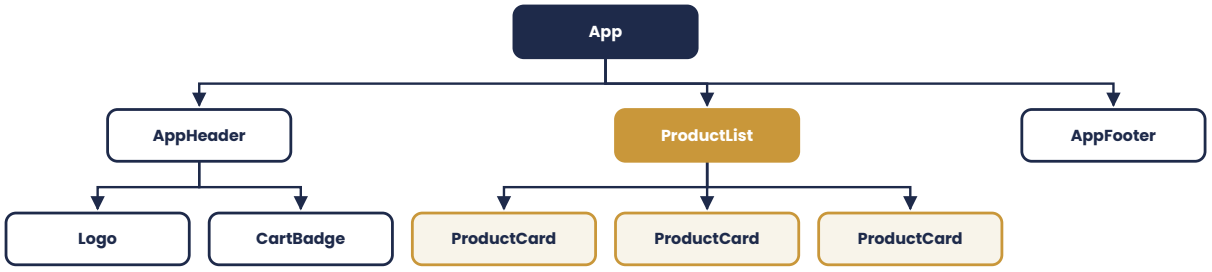
Imperative (plain DOM code)	Declarative (Vue)

`let count = 0`
`btn.addEventListener("click", () => {`
`  count++`
`  btn.textContent = `Clicked ${count} times``
`})`
	`const count = ref(0)`
	`<button @click="count++">`
	`  Clicked {{ count }} times`
	`</button>`

You must remember to update every place that shows `count`.	Vue updates every place that uses `count` automatically.

1.1.4 Component-based architecture

A Vue application is a **tree of components**. Each component is a reusable, self-contained piece of the interface with its own template, logic and styles, usually written in a single `.vue` file. A parent passes data down with **props** and children report events up with **emits**.



A SHOP PAGE AS A COMPONENT TREE: PRODUCTCARD IS WRITTEN ONCE AND USED THREE TIMES

SRC/COMPONENTS/PRODUCTCARD.VUE HTML

```
<script setup>
defineProps({ name: String, price: Number })
</script>

<template>
  <div class="card">
    <h3>{{ name }}</h3>
    <p>&#8377;{{ price }}</p>
  </div>
</template>

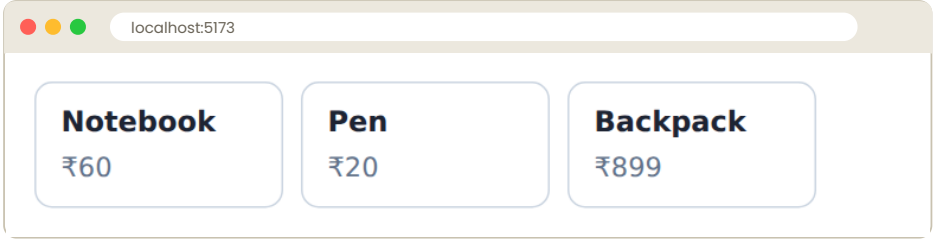
<style scoped>
.card { border: 1px solid #cbd5e1; border-radius: 10px; padding: 10px 14px; min-width: 110px; }
h3 { margin: 0 0 4px; font-size: 16px; }
p { margin: 0; color: #64748b; }
</style>
```

SRC/APP.VUE HTML

```
<script setup>
import ProductCard from './components/ProductCard.vue'
</script>

<template>
  <div class="grid">
    <ProductCard name="Notebook" :price="60" />
    <ProductCard name="Pen" :price="20" />
    <ProductCard name="Backpack" :price="899" />
  </div>
</template>

<style scoped>
.grid { display: flex; gap: 10px; }
</style>
```



ONE COMPONENT DEFINITION, THREE INSTANCES WITH DIFFERENT PROPS

1.2 History of Vue.js

1.2.1 Created by Evan You (former Google engineer), released in 2014

Vue was created by **Evan You** while he worked at Google Creative Lab, where he used AngularJS for prototypes. He wanted to keep the parts he liked (templates and data binding) in a much lighter package. He released the first version in **February 2014**. Vue is not owned by a big company; it is developed by an independent core team and funded by sponsors.



VUE'S HISTORY AT A GLANCE

1.2.2 Vue 2 (2016) and its wide adoption

Vue 2 (2016) introduced a **virtual DOM**, server-side rendering and the Options API style that made Vue famous for being easy to learn. It was widely adopted, especially with the Laravel PHP community and in China (Alibaba, Xiaomi).

1.2.3 Vue 3 (2020): Composition API, better performance, TypeScript support

Vue 3 ("One Piece", September 2020) was rewritten in TypeScript. It added the **Composition API** (organising logic by feature in `setup()`), a Proxy-based reactivity system, multiple root nodes in a template (fragments), `<Teleport>`, `<Suspense>`, a faster and smaller runtime, and first-class TypeScript support. `<script setup>` (Vue 3.2, 2021) made the Composition API the shortest way to write components, and Vue 3.5 (2024) made reactivity faster and lighter.

1.2.4 Vue 2 end of life (2023)

Vue 2 reached **end of life on 31 December 2023**: it no longer gets updates or security fixes. All new projects use Vue 3, and this course teaches Vue 3 only.

1.2.5 The Vue ecosystem: Vite, Pinia, Vue Router, Nuxt

Tool	Job	Module
Vite	dev server with instant reload and production build tool (also created by Evan You)	3, 21
Vue Router	maps URLs to components for multi-page single-page apps	13
Pinia	official state management (shared stores)	14
Vue DevTools	browser extension to inspect components, state and performance	3, 17
Vitest + **Vue Test Utils**	unit and component testing	19
Nuxt	full-stack framework: server rendering, file-based routing, API routes	20

1.2.6 Vapor Mode: compiling to code without the virtual DOM (overview)

Vapor Mode is a new compilation strategy. Normally Vue compiles a template into a render function that builds a virtual DOM tree, and Vue compares the old and new trees to find what changed. In Vapor Mode the compiler instead generates code that updates exactly the DOM nodes that depend on each piece of state, with no virtual DOM at all. Components opt in one at a time, and the code you write stays the same. It is covered again in Module 17.

1.3 Why do we need Vue.js?

1.3.1 Gentle learning curve with HTML-based templates

Vue templates are **valid HTML** with a few special attributes. Anyone who knows HTML, CSS and JavaScript can read a Vue component on day one, which is why Vue is often the first framework students learn.

1.3.2 Excellent performance and small size

The Vue runtime is small (around 34 KB minified and gzipped for the full runtime with the compiler, much less for the runtime-only build) and its **compiler-informed** virtual DOM skips the parts of a template that can never change. Fine-grained reactivity means only components that use changed data re-render.

1.3.3 Official router, state management and tooling

Unlike a view-only library, Vue's core team maintains the router, the store, the build tool, the DevTools and the language tools, so they are designed to work together and are versioned together.

1.3.4 Single-File Components keep template, logic and style together

SRC/COMPONENTS/LIKEBUTTON.VUE

HTML

```
<script setup>
import { ref } from 'vue'
const liked = ref(false)
</script>

<template>
  <button :class="{ on: liked }" @click="liked = !liked">♥ Like</button>
</template>

<style scoped>
.on { color: crimson; }
</style>
```

One `.vue` file holds everything about one component: the **logic** in `<script setup>`, the **markup** in `<template>` and the **styles** in `<style scoped>`, which apply only to this component. Module 3 covers Single-File Components in detail.

1.3.5 Clear, friendly documentation

The official documentation at **vuejs.org** is known for its clarity. Every page can switch between the Options API and Composition API, and the online **SFC Playground** runs examples in the browser without installing anything.

1.4 Where is Vue.js used?

1.4.1 Web apps and dashboards (GitLab, Alibaba, Xiaomi, BMW, Nintendo sites)

Vue is used in production by companies of every size, for example in parts of **GitLab**'s web interface, **Alibaba** and **Xiaomi** web properties, and marketing sites of **BMW** and **Nintendo**. Admin dashboards, internal tools and e-commerce front ends are especially common.

1.4.2 Laravel full-stack projects

The **Laravel** PHP framework adopted Vue early, and its starter kits (with Inertia.js) let a Laravel back end render Vue pages without building a separate API.

1.4.3 Full-stack apps with Nuxt

Nuxt builds on Vue to add server-side rendering, static site generation, file-based routing and server API routes in one project (Module 20).

1.4.4 Mobile apps with Ionic Vue or Quasar

Ionic Vue and **Quasar** wrap a Vue app into native iOS and Android apps (through Capacitor), reusing web skills for mobile.

1.4.5 Adding interactivity to existing server-rendered pages

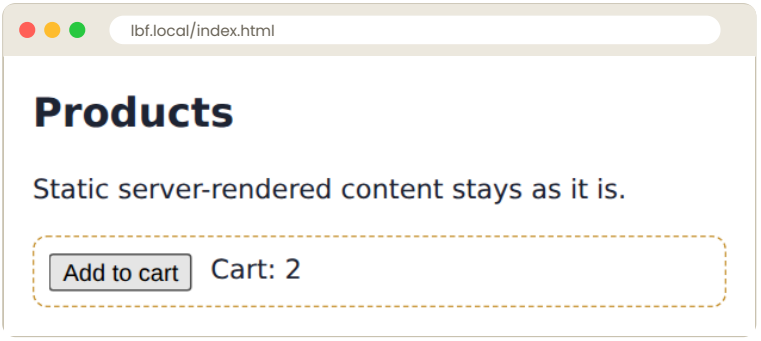
Because Vue can mount on any element, it can add interactive "islands" (a search box, a cart widget) to pages rendered by PHP, Django, Rails or plain HTML, without rewriting the site.

INDEX.HTML HTML

```html
<!-- a page rendered by any server; Vue only controls #cart-widget -->
<h2>Products</h2>
<p>Static server-rendered content stays as it is.</p>

<div id="cart-widget">
  <button @click="items++">Add to cart</button>
  <span>Cart: {{ items }}</span>
</div>

<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
<script>
  Vue.createApp({ data: () => ({ items: 0 }) }).mount('#cart-widget')
</script>
```



A VUE "ISLAND" INSIDE A STATIC PAGE, AFTER TWO REAL CLICKS

1.5 Vue vs React vs Angular

1.5.1 Key differences at a glance

	Vue	React	Angular
Type	progressive framework	UI library	full framework
Created by	Evan You (independent, 2014)	Meta (2013)	Google (2016, AngularJS 2010)
Templates	HTML-based templates (JSX optional)	JSX (JavaScript + HTML)	HTML templates with Angular syntax
Reactivity	automatic, fine-grained (refs, Proxies)	re-render on `setState`, hooks	signals + change detection
Language	JavaScript or TypeScript	JavaScript or TypeScript	TypeScript
Official router / store	yes (Vue Router, Pinia)	no (community choices)	yes (built in)
Learning curve	gentle	moderate	steeper
Full-stack framework	Nuxt	Next.js	Angular SSR / Analog

All three are excellent and used widely. Vue's strengths are its approachable templates, automatic reactivity and the fact that one team maintains the core tools. The same counter in each, to compare the style:

Vue	React	Angular

`const n = ref(0)`
`<button @click="n++">`
`{{ n }}</button>`
	`const [n, setN] = useState(0)`
	`<button onClick={() => setN(n + 1)}>{n}</button>`
		`n = signal(0)`
		`<button (click)="n.update(v => v + 1)">`
		`{{ n() }}</button>`

1.6 Options API vs Composition API

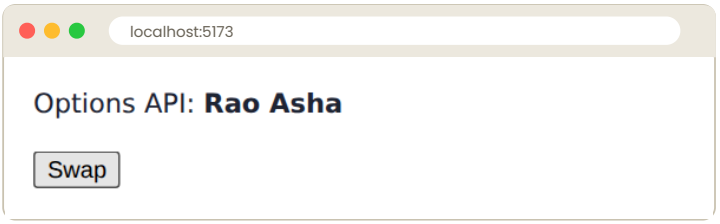
1.6.1 The two styles and why this course uses Composition API with <script setup>

Vue components can be written in two styles. The **Options API** (from Vue 2) groups code by *type of option*: `data`, `computed`, `methods`, `watch`, lifecycle hooks, and uses `this`. The **Composition API** (Vue 3) groups code by *feature*: reactive state, derived values and functions are declared together with plain functions such as `ref()` and `computed()`. Both produce the same component; here they are side by side, both run:

SRC/COMPONENTS/OPTIONSNAME.VUE HTML

```
<script>
export default {
  data() {
    return { first: 'Asha', last: 'Rao' }
  },
  computed: {
    full() { return this.first + ' ' + this.last }
  },
  methods: {
    swap() { [this.first, this.last] = [this.last, this.first] }
  }
}
</script>

<template>
  <p>Options API: <b>{{ full }}</b></p>
  <button @click="swap">Swap</button>
</template>
```



OPTIONS API COMPONENT AFTER ONE REAL CLICK ON SWAP

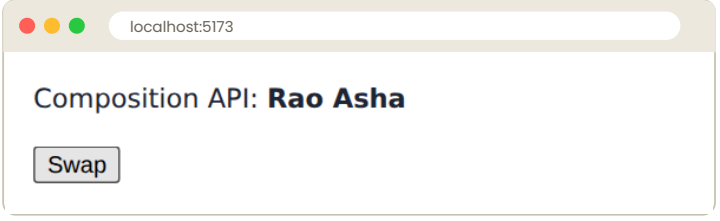
SRC/COMPONENTS/COMPOSITIONNAME.VUE HTML

```
<script setup>
import { ref, computed } from 'vue'

const first = ref('Asha')
const last = ref('Rao')
const full = computed(() => first.value + ' ' + last.value)

function swap() {
  [first.value, last.value] = [last.value, first.value]
}
</script>

<template>
  <p>Composition API: <b>{{ full }}</b></p>
  <button @click="swap">Swap</button>
</template>
```



THE SAME COMPONENT WITH THE COMPOSITION API AND `<SCRIPT SETUP>`, AFTER ONE REAL CLICK

	Options API	Composition API (script setup)
Organised by	option type (data, methods, computed…)	feature: related code stays together

	Options API	Composition API (script setup)
Accessing state	`this.first`	`first.value` (no `this`)
Reusing logic	mixins (name clashes, unclear sources)	composables: plain functions (Module 12)
TypeScript	works, but `this` typing is complex	natural type inference
Boilerplate	more (export default, return, this)	less: top-level bindings are usable in the template

NOTE

This course uses the **Composition API with `<script setup>`**, the style recommended by the Vue team for new projects and used by Vue Router, Pinia, Nuxt and VueUse. The Options API still works in Vue 3 and you will meet it in older code, so Module 5 and Module 10 point out the Options API equivalents.

PRACTICE CHECKPOINT

Explain in your own words what "progressive framework" means and compare Vue with React and Angular in three points.

Model answer. "Progressive" means I can adopt Vue a little at a time: start with a script tag that adds interactivity to one part of an existing page, then move to Single-File Components and Vite, and add Vue Router, Pinia or Nuxt only when the project needs them. Compared with the others: (1) Vue uses HTML-based templates, React uses JSX and Angular uses HTML templates with its own syntax; (2) Vue's reactivity is automatic (change a ref and the page updates), React re-renders when state setters are called, Angular uses signals and change detection; (3) Vue and Angular ship official routers and stores, while React leaves those choices to the community.

Vue is a progressive framework: a small, approachable core (declarative templates, automatic reactivity, components) plus official tools you add as needed. This book uses Vue 3 with the Composition API and `<script setup>`.

MODULE 02 · BEGINNER

JavaScript essentials for Vue

Minimum duration **1 day** | Level **Beginner**

Vue builds on modern JavaScript, so this module revises the features used in every Vue project: let, const, arrow functions and template literals; destructuring, spread and rest; optional chaining and nullish coalescing; the array methods map, filter, reduce, find and some; Object.keys, values and entries; ES modules with import and export; promises, async/await, fetch() and JSON; and finally the two JavaScript features that make Vue's reactivity possible, getters and setters and Proxy objects. Every program in this module was run with Node.js 22.

IN THIS MODULE

2.1 Modern syntax
2.2 Arrays and objects
2.3 Modules
2.4 Asynchronous JavaScript
2.5 JavaScript concepts behind reactivity

2.1 Modern syntax

2.1.1 let, const, arrow functions, template literals

Use `const` for a binding that is never reassigned (the default choice) and `let` when it will be; both are **block-scoped**. Avoid `var`. In Vue, refs are almost always declared with `const`: the variable keeps pointing at the same ref object while its `.value` changes.

SYNTAX.JS — main.js

```
const course = 'Vue.js'          // never reassigned
let lessons = 21                  // will change
lessons = lessons + 1

// arrow functions: short syntax, no own "this"
const double = (n) => n * 2
const greet = (name) => `Hello, ${name}! Welcome to ${course}.`   // template literal
const toCard = (title, price) => ({ title, price })                 // object needs ( )

console.log(double(21), lessons)
console.log(greet('Asha'))
console.log(toCard('Pen', 20))
console.log(`Total: ${2 * 60 + 20} rupees`)  // any expression inside ${ }
```

OUTPUT

```
42 22
Hello, Asha! Welcome to Vue.js.
{ title: 'Pen', price: 20 }
Total: 140 rupees
```

Feature	Use in Vue code
`const`	`const count = ref(0)`, `const total = computed(...)`
arrow function	`computed(() => a.value + b.value)`, `watch(src, (n, o) => ...)`
template literal	`` `/api/students/${id}` ``, building class names and messages

2.1.2 Destructuring, spread and rest

Destructuring unpacks values from arrays and objects into variables. **Spread** (`...`) copies elements or properties into a new array or object; **rest** (`...` in a parameter or pattern) collects the remaining ones.

DESTRUCTURING.JS — main.js

```
const student = { name: 'Ravi', marks: 82, city: 'Pune' }
const { name, marks, grade = 'B' } = student        // default value
console.log(name, marks, grade)

const [first, second, ...others] = [10, 20, 30, 40]
console.log(first, second, others)

const updated = { ...student, marks: 90 }            // copy + override
console.log(updated, student.marks)                   // original unchanged

const merged = [...[1, 2], ...[3, 4]]
const sum = (...nums) => nums.reduce((a, b) => a + b, 0)   // rest parameter
console.log(merged, sum(1, 2, 3, 4))
```

OUTPUT

```
Ravi 82 B
10 20 [ 30, 40 ]
{ name: 'Ravi', marks: 90, city: 'Pune' } 82
[ 1, 2, 3, 4 ] 10
```

WATCH OUT

In Vue, destructuring a `reactive()` object copies plain values and **loses reactivity** (Module 5.2.2). Spread is how you replace arrays immutably, for example `items.value = [...items.value, newItem]`.

2.1.3 Optional chaining and nullish coalescing

`a?.b` returns `undefined` instead of throwing when `a` is `null` or `undefined`. `x ?? y` gives `y` only when `x` is `null` or `undefined` (unlike `||`, which also replaces `0` and `""`). Both are common in templates while data is still loading.

OPTIONAL.JS

main.js

```
const user = { name: 'Meena', address: null, score: 0 }

console.log(user.address?.city)          // undefined, no crash
console.log(user.getAge?.())             // optional call
console.log(user.score || 'no score')    // || treats 0 as missing
console.log(user.score ?? 'no score')    // ?? keeps 0
console.log(user.nickname ?? 'Guest')
```

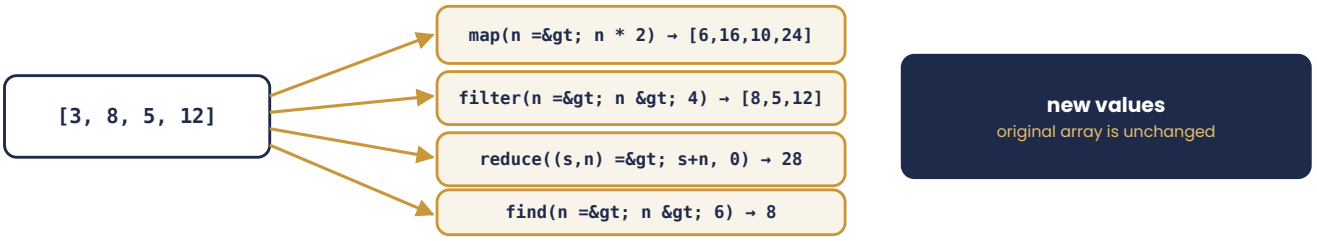
OUTPUT

```
undefined
undefined
no score
0
Guest
```

2.2 Arrays and objects

2.2.1 map, filter, reduce, find, some

Vue lists are arrays, and almost every derived value in a `computed()` is built with these methods. They do not change the original array (except `sort` and `reverse`, see Module 6.2.7).



EACH METHOD RETURNS A NEW VALUE FROM THE SAME ARRAY

ARRAYS.JS

main.js

```
const products = [
  { name: 'Notebook', price: 60, stock: 12 },
  { name: 'Pen', price: 20, stock: 0 },
  { name: 'Backpack', price: 899, stock: 3 },
]

const names = products.map((p) => p.name)
const inStock = products.filter((p) => p.stock > 0)
const total = products.reduce((sum, p) => sum + p.price, 0)
const pen = products.find((p) => p.name === 'Pen')
const anyOut = products.some((p) => p.stock === 0)
const allCheap = products.every((p) => p.price < 1000)
const byPrice = [...products].sort((a, b) => b.price - a.price)   // copy, then sort

console.log(names)
console.log(inStock.map((p) => p.name), total)
console.log(pen, anyOut, allCheap)
console.log(byPrice.map((p) => `${p.name}:${p.price}`).join(', '))
```

OUTPUT

```
[ 'Notebook', 'Pen', 'Backpack' ]
[ 'Notebook', 'Backpack' ] 979
{ name: 'Pen', price: 20, stock: 0 } true true
Backpack:899, Notebook:60, Pen:20
```

2.2.2 Object methods: keys, values, entries

OBJECTS.JS

main.js

```
const marks = { maths: 88, science: 92, english: 79 }

console.log(Object.keys(marks))
console.log(Object.values(marks))
console.log(Object.entries(marks))

for (const [subject, m] of Object.entries(marks)) {
  console.log(`${subject.padEnd(8)} ${'#'.repeat(m / 10)} ${m}`)
}
const average = Object.values(marks).reduce((a, b) => a + b) / Object.keys(marks).length
console.log('average', average.toFixed(1))
console.log(Object.fromEntries(Object.entries(marks).filter(([, m]) => m >= 80)))
```

OUTPUT

```
[ 'maths', 'science', 'english' ]
[ 88, 92, 79 ]
[ [ 'maths', 88 ], [ 'science', 92 ], [ 'english', 79 ] ]
maths    ######## 88
science  ######### 92
english  ####### 79
average 86.3
{ maths: 88, science: 92 }
```

2.3 Modules

2.3.1 import and export

Every file in a Vue project is an **ES module**. A module shares values with `export` (named exports, any number) or `export default` (one per file); other modules bring them in with `import`. A `.vue` component is imported as a default export, and Vue's own functions are named imports.

SRC/UTILS/MONEY.JS — JavaScript

```
// src/utils/money.js
export const GST = 0.18
export function withTax(amount) {
  return Math.round(amount * (1 + GST))
}
export default function formatINR(n) {
  return '₹' + n.toLocaleString('en-IN')
}
```

SRC/MAIN.JS (EXCERPT) — JavaScript

```
// src/main.js
import formatINR, { withTax, GST } from './utils/money.js'   // default + named
import * as money from './utils/money.js'                    // namespace
import { ref, computed } from 'vue'                          // named imports from a package
import App from './App.vue'                                  // a component is a default export

console.log(formatINR(withTax(1000)), GST, money.GST)
```

MAIN.MJS — main.mjs

```
import formatINR, { withTax, GST } from './money.mjs'
import * as money from './money.mjs'
console.log(formatINR(withTax(1000)), GST, Object.keys(money))
```

OUTPUT

```
₹1,180 0.18 [ 'GST', 'default', 'withTax' ]
```

2.4 Asynchronous JavaScript

2.4.1 Promises and async/await

A **promise** represents a value that will be available later (a server response, a timer). `async` functions always return a promise, and `await` pauses the function until a promise settles, so asynchronous code reads like normal top-to-bottom code. Errors are caught with `try/catch`.

ASYNC.JS

main.js

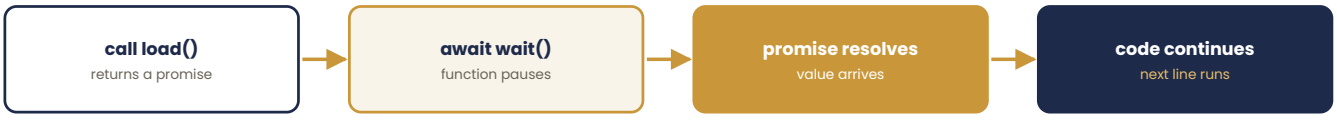
```
const wait = (ms, value) => new Promise((resolve) => setTimeout(() => resolve(value), ms))
const fail = (ms) => new Promise((_, reject) => setTimeout(() => reject(new Error('Server down')),
ms))

async function load() {
  console.log('1. start')
  const a = await wait(100, 'students')
  console.log('2. got', a)
  const [b, c] = await Promise.all([wait(50, 'courses'), wait(80, 'teachers')])  // in parallel
  console.log('3. got', b, c)
  try {
    await fail(30)
  } catch (err) {
    console.log('4. caught:', err.message)
  } finally {
    console.log('5. finally runs either way')
  }
  return 'done'
}

load().then((r) => console.log('6.', r))
console.log('0. load() returned a promise immediately')
```

OUTPUT

```
1. start
0. load() returned a promise immediately
2. got students
3. got courses teachers
4. caught: Server down
5. finally runs either way
6. done
```



AWAIT PAUSES ONLY THE ASYNC FUNCTION; THE REST OF THE PROGRAM KEEPS RUNNING

2.4.2 fetch() and JSON

`fetch(url)` returns a promise for a **Response**. `response.json()` parses the JSON body (also a promise). `fetch` only rejects on network failure, so check `response.ok` for HTTP errors such as 404. The program below starts a tiny local JSON server and calls it with `fetch`:

FETCH.MJS

fetch.mjs

```
import http from 'node:http'

// a tiny REST API for the demo
const server = http.createServer((req, res) => {
  if (req.url === '/api/students') {
    res.writeHead(200, { 'Content-Type': 'application/json' })
    res.end(JSON.stringify([{ id: 1, name: 'Asha' }, { id: 2, name: 'Ravi' }]))
  } else { res.writeHead(404); res.end('{"error":"Not found"}') }
}).listen(0)
const base = `http://localhost:${server.address().port}`

async function getJSON(url) {
  const res = await fetch(url)
  if (!res.ok) throw new Error(`HTTP ${res.status}`)
  return res.json()
}

const students = await getJSON(`${base}/api/students`)
console.log(students.map((s) => s.name))
console.log(JSON.stringify({ id: 3, name: 'Meena' }))          // object -> JSON text
console.log(JSON.parse('{"ok":true,"count":2}').count)         // JSON text -> object
try { await getJSON(`${base}/api/missing`) } catch (e) { console.log('Error:', e.message) }
server.close()
```

OUTPUT

```
[ 'Asha', 'Ravi' ]
{"id":3,"name":"Meena"}
2
Error: HTTP 404
```

2.5 JavaScript concepts behind reactivity

2.5.1 Getters and setters

A **getter** runs a function when a property is read; a **setter** runs a function when it is written. This lets an object *notice* reads and writes. Vue 2 made data reactive by converting every property into a getter/setter pair with `Object.defineProperty`, and Vue 3 still uses getters and setters for `ref().value`.

GETTERS.JS

main.js

```
function makeTracked(initial) {
  let value = initial
  return {
    get value() { console.log('  read  ->', value); return value },
    set value(v) { console.log('  write ->', v); value = v; render() },
  }
}

const count = makeTracked(0)
function render() { console.log(`  [render] Count is ${count.value}`) }

count.value++          // read, then write (which re-renders)
count.value = 10
```

OUTPUT

```
  read  -> 0
  write -> 1
  read  -> 1
  [render] Count is 1
  write -> 10
  read  -> 10
  [render] Count is 10
```

2.5.2 Proxy objects (overview)

A **Proxy** wraps an object and intercepts operations on it (get, set, delete, has…) through **trap** functions. Unlike getters and setters, a Proxy also sees properties that are **added later** and array index changes. Vue 3's `reactive()` returns a Proxy: the `get` trap records *which effect read which property* ("track"), and the `set` trap re-runs those effects ("trigger").

MINI-REACTIVITY.JS

main.js

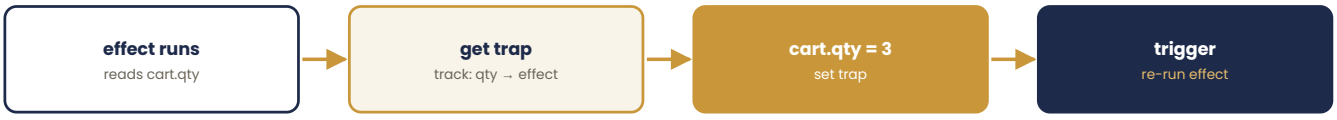
```
const deps = new Map()          // property -> effects that read it
let activeEffect = null

function reactive(obj) {
  return new Proxy(obj, {
    get(target, key) {
      if (activeEffect) {                                  // track
        if (!deps.has(key)) deps.set(key, new Set())
        deps.get(key).add(activeEffect)
      }
      return target[key]
    },
    set(target, key, value) {
      target[key] = value
      deps.get(key)?.forEach((effect) => effect())         // trigger
      return true
    },
  })
}
function effect(fn) { activeEffect = fn; fn(); activeEffect = null }

const cart = reactive({ qty: 1, price: 50 })
effect(() => console.log('total =', cart.qty * cart.price))
cart.qty = 3           // re-runs the effect automatically
cart.price = 40
cart.note = 'gift'     // no effect read "note", so nothing re-runs
```

OUTPUT

```
total = 50
total = 150
total = 120
```



TRACK ON READ, TRIGGER ON WRITE: THE IDEA BEHIND EVERY VUE UPDATE

NOTE

These 20 lines are a toy version of what Vue does. Module 5.4 and Module 17 show Vue's real behaviour: effects are batched, computed values are cached, and the component's render function is just another effect.

PRACTICE CHECKPOINT

Write functions that filter, sort and total an array of products, and fetch data from a public API with async/await.

CHECKPOINT.MJS checkpoint.mjs

```
import http from 'node:http'
const products = [
  { name: 'Notebook', price: 60, category: 'stationery' },
  { name: 'Headphones', price: 1499, category: 'electronics' },
  { name: 'Pen', price: 20, category: 'stationery' },
  { name: 'Mouse', price: 599, category: 'electronics' },
]
const byCategory = (list, cat) => list.filter((p) => p.category === cat)
const sortByPrice = (list, dir = 'asc') => [...list].sort((a, b) => (dir === 'asc' ? a.price - b.price
: b.price - a.price))
const total = (list) => list.reduce((sum, p) => sum + p.price, 0)

const elec = byCategory(products, 'electronics')
console.log(sortByPrice(elec, 'desc').map((p) => p.name), total(elec))
console.log(sortByPrice(products).map((p) => `${p.name} ${p.price}`).join(' | '))

// stand-in for a public API (e.g. https://jsonplaceholder.typicode.com/users)
const server = http.createServer((q, r) => { r.setHeader('Content-Type', 'application/json');
r.end(JSON.stringify([{ id: 1, name: 'Leanne Graham' }, { id: 2, name: 'Ervin Howell' }]))
}).listen(0)
async function loadUsers() {
  try {
    const res = await fetch(`http://localhost:${server.address().port}/users`)
    if (!res.ok) throw new Error(res.status)
    const users = await res.json()
    console.log('Users:', users.map((u) => u.name).join(', '))
  } catch (e) { console.log('Failed:', e.message) } finally { server.close() }
}
await loadUsers()
```

OUTPUT

```
[ 'Headphones', 'Mouse' ] 2098
Pen 20 | Notebook 60 | Mouse 599 | Headphones 1499
Users: Leanne Graham, Ervin Howell
```

Vue code is modern JavaScript: `const` refs, arrow functions in `computed` and `watch`, destructuring and spread for data, array methods for derived lists, modules for every file, async/await for API calls, and Proxies with getters/setters underneath the reactivity system.

MODULE 03 · BEGINNER

Setup and Single-File Components

Minimum duration **1 day** | Level **Beginner**

By the end of this module the learner can create a Vue project and understands the structure of a Single-File Component. It compares the three ways to use Vue (a CDN script, the online SFC Playground and a Vite project), walks through npm create vue@latest and its options, the dev server and the production build, explains every file in the generated project (index.html, main.js, App.vue, src/components, src/assets, public), dissects a .vue file into its template, script setup and style blocks, shows exactly how scoped styles work, how child components are imported, and finishes with the editor and browser tools.

IN THIS MODULE

3.1 Ways to use Vue
3.2 Creating a project
3.3 Project structure
3.4 Single-File Components (.vue)
3.5 Tooling

3.1 Ways to use Vue

3.1.1 CDN script for quick experiments

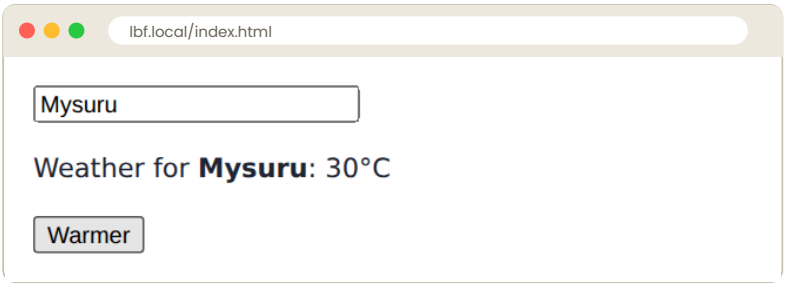
The quickest way to try Vue is one HTML file that loads Vue from a CDN (**unpkg** or **jsDelivr**). No installation or build step is needed. The global build puts everything on a global `Vue` object; templates are written directly in the page and compiled in the browser.

INDEX.HTML — HTML

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>

<div id="app">
  <input v-model="city" placeholder="Your city">
  <p>Weather for <b>{{ city || '...' }}</b>: {{ temp }}°C</p>
  <button @click="temp++">Warmer</button>
</div>

<script>
  const { createApp, ref } = Vue
  createApp({
    setup() {
      const city = ref('')
      const temp = ref(28)
      return { city, temp }
    }
  }).mount('#app')
</script>
```



AFTER TYPING "MYSURU" AND CLICKING WARMER TWICE

An ES-module version also works in every modern browser: `<script type="module">` with `import { createApp } from 'https://unpkg.com/vue@3/dist/vue.esm-browser.js'`. CDN usage is fine for learning and for adding a widget to an existing page, but it cannot use `.vue` files, TypeScript or tree-shaking, so real projects use a build tool.

3.1.2 The Vue SFC Playground (online)

The **Vue SFC Playground** at **play.vuejs.org** runs Single-File Components in the browser: the left pane holds `App.vue` (and any extra files you add), the right pane shows the live result, and tabs show the compiled JavaScript and CSS. It is ideal for trying an idea or sharing a reproduction of a bug through its URL. The same `App.vue` you write there works unchanged in a Vite project.



THE SFC PLAYGROUND: CODE ON THE LEFT, LIVE PREVIEW AND COMPILED OUTPUT ON THE RIGHT (REPRESENTATIVE)

3.1.3 Build-tool project with Vite (recommended)

For real applications the recommended setup is a **Vite** project. Vite serves your source files to the browser during development (with near-instant hot module replacement) and bundles them into optimised static files for production. It compiles `.vue` files, TypeScript and CSS, and lets you use npm packages.

	CDN script	SFC Playground	Vite project
Install needed	no	no (browser)	Node.js + npm
.vue files	no	yes	yes
TypeScript, npm packages	no	limited	yes
Production build	no	no	yes (minified, tree-shaken)
Best for	widgets, quick tests	experiments, sharing	every real project

3.2 Creating a project

3.2.1 npm create vue@latest and its options (TypeScript, Router, Pinia, Vitest, ESLint)

The official scaffolding tool is **create-vue**. It needs Node.js (an LTS version) and asks which features to include. Answering "No" gives a minimal project; each "Yes" adds the package, its configuration and an example. A typical session (representative):

Terminal (representative)

```
$ npm create vue@latest

Vue.js - The Progressive JavaScript Framework

✔ Project name: … student-portal
✔ Select features to include in your project:
   ■ TypeScript
   ■ Router (SPA development)
   ■ Pinia (state management)
   ■ Vitest (unit testing)
   □ End-to-End Testing
   ■ ESLint (error prevention)
   ■ Prettier (code formatting)

Scaffolding project in ./student-portal...
Done. Now run:

  cd student-portal
  npm install
  npm run dev
```

Option	Adds	Covered in
TypeScript	`lang="ts"` components, `tsconfig`, `vue-tsc` type checking	Module 16
Router	Vue Router, `src/router/index.js`, `src/views/`	Module 13
Pinia	Pinia, `src/stores/counter.js`	Module 14
Vitest	Vitest + Vue Test Utils, `src/components/__tests__/`	Module 19
End-to-End	Playwright, Cypress or Nightwatch	Module 19
ESLint / Prettier	linting and formatting configuration	Module 21

3.2.2 npm run dev and npm run build

`npm run dev` starts the Vite dev server (port **5173** by default). Saving a file updates the page in the browser immediately, usually without losing component state. `npm run build` produces optimised files in `dist/`, and `npm run preview` serves that build locally to check it.

Terminal (representative)

```
$ npm run dev

  VITE v7.1.9  ready in 412 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help

$ npm run build

vite v7.1.9 building for production...
✓ 34 modules transformed.
dist/index.html                  0.43 kB │ gzip:  0.29 kB
dist/assets/index-BqJf3w1k.css   1.21 kB │ gzip:  0.62 kB
dist/assets/index-D4xk9mQe.js   61.85 kB │ gzip: 24.73 kB
✓ built in 1.08s
```

Script	Command	Purpose
`npm run dev`	`vite`	development server with hot module replacement
`npm run build`	`vite build`	production bundle in `dist/`
`npm run preview`	`vite preview`	serve the production build locally
`npm run test:unit`	`vitest`	run unit tests (if Vitest was selected)
`npm run lint`	`eslint . --fix`	check and fix code style (if ESLint was selected)

3.3 Project structure

```
student-portal/
    index.html                  the page the browser loads; has <div id="app">
    package.json                dependencies and npm scripts
    vite.config.js              Vite configuration and plugins
    public/                     copied as-is to the build (no processing)
        favicon.ico
    src/                        all source code
        main.js                 entry point: createApp(App).mount("#app")
        App.vue                 root component
        assets/                 images and CSS processed by Vite
            main.css
        components/             reusable components
```

A NEW VITE + VUE PROJECT (NO OPTIONAL FEATURES SELECTED)

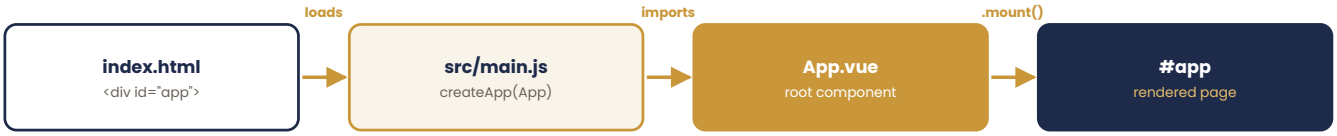
3.3.1 index.html, main.js, App.vue

Three files start every Vue application. The browser loads `index.html`; its module script loads `src/main.js`; `main.js` creates the app from the root component `App.vue` and mounts it into `<div id="app">`.

INDEX.HTML

HTML

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" href="/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Student Portal</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```



HOW A VITE + VUE APP STARTS

3.3.2 createApp() and mount()

`createApp(rootComponent)` creates an **application instance**. Before mounting you can configure it: install plugins with `app.use()`, register global components with `app.component()`, provide values with `app.provide()` and set `app.config` options such as an error handler. `app.mount(selector)` renders the root component inside that element and returns the root component instance. Mount is called once, at the end.

SRC/MAIN.JS

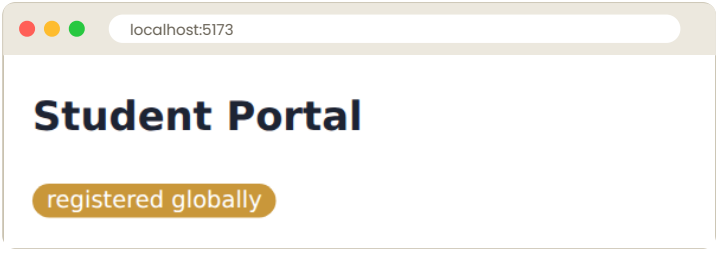
JavaScript

```javascript
import { createApp } from 'vue'
import App from './App.vue'
import BaseBadge from './components/BaseBadge.vue'

const app = createApp(App)                    // 1. create

app.component('BaseBadge', BaseBadge)         // 2. configure: global component
app.provide('appName', 'Student Portal')      //    app-level value
app.config.errorHandler = (err) => console.error('App error:', err.message)

const vm = app.mount('#app')                  // 3. mount (last)
console.log('mounted:', vm.$el.parentElement.id)
```



THE ROOT COMPONENT, A GLOBALLY REGISTERED COMPONENT AND AN APP-LEVEL VALUE

3.3.3 src/components, src/assets, public

Folder	Holds	How files are used
`src/components/`	reusable `.vue` components	imported by other components
`src/assets/`	images, fonts, CSS used by the code	imported (`import logo from './assets/logo.svg'`); Vite optimises them and adds a content hash to the file name
`public/`	files that must keep their exact name (`favicon.ico` , `robots.txt`)	copied unchanged to the build root; referenced with an absolute path `/favicon.ico`
`src/views/`	page components (added by the Router option)	used by routes (Module 13)
`src/stores/`	Pinia stores (added by the Pinia option)	Module 14

3.4 Single-File Components (.vue)

3.4.1 template, script setup, style

A **Single-File Component** (SFC) keeps one component's logic, markup and styles in one `.vue` file with three top-level blocks. Vite's Vue plugin compiles it into a normal JavaScript module.

SRC/APP.VUE

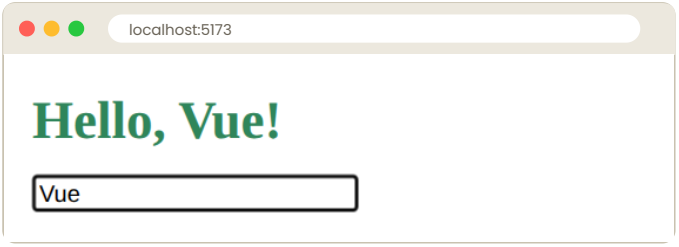
HTML

```
<script setup>
import { ref, computed } from 'vue'

const name = ref('World')
const greeting = computed(() => `Hello, ${name.value}!`)
</script>

<template>
  <div class="hello">
    <h1>{{ greeting }}</h1>
    <input v-model="name" />
  </div>
</template>

<style>
.hello { font-family: Georgia, serif; }
.hello h1 { color: #2f855a; margin: 0 0 8px; }
</style>
```



THE SFC RUNNING, AFTER TYPING "VUE" INTO THE INPUT

Block	Purpose	Notes
`<script setup>`	component logic (Composition API)	every top-level variable, function and import is available in the template; runs once per component instance
`<template>`	the HTML structure with Vue syntax	compiled to a render function at build time; may have several root elements
`<style>`	CSS for the component	`scoped` limits it to this component; `lang="scss"` for preprocessors; several style blocks allowed

Order of blocks does not matter; the official style guide recommends `<script>`, then `<template>`, then `<style>`. A component may also have a normal `<script>` block next to `<script setup>` for things that must run once per module (rare).

3.4.2 Scoped styles with style scoped

With `<style scoped>`, the compiler gives the component a unique id such as `data-v-7a7a37b1`, adds that attribute to every element the component renders, and rewrites each CSS selector to require it. The rules therefore cannot leak into other components, and other components' `.title` classes are not affected. Here is the real DOM and the real CSS the compiler produced for two components that both use `.title`:

SRC/COMPONENTS/CARDA.VUE

HTML

```
<template><h3 class="title">Card A (scoped red)</h3></template>
<style scoped>
.title { color: crimson; }
</style>
```

SRC/COMPONENTS/CARDB.VUE

HTML

```
<template><h3 class="title">Card B (no styles of its own)</h3></template>
```



ONLY CARDA'S HEADING IS RED: THE SCOPED RULE CANNOT REACH CARDB

COMPILED CSS AND RENDERED HTML

```
.title[data-v-9cd530bd] { color: crimson; }
<h3 data-v-9cd530bd="" class="title">Card A (scoped red)</h3>
<h3 class="title">Card B (no styles of its own)</h3>
```

TIP

A scoped rule also reaches the **root element** of a child component (so the parent can position it), but not elements deeper inside the child. To style inside a child use `:deep(.selector)`; Module 18.5 covers `:deep()`, `:slotted()`, `:global()`, CSS Modules and `v-bind()` in CSS.

3.4.3 Importing and using child components

In `<script setup>`, importing a component is enough to use it in the template; there is no registration step. Use PascalCase in both the import and the tag (`<StudentCard />`); the kebab-case form `<student-`

`card>` also works. Components with no content can be self-closed.

SRC/COMPONENTS/APPHEADER.VUE HTML

```
<template>
  <header>🎓 Student Portal</header>
</template>

<style scoped>
header { font-weight: 600; font-size: 18px; margin-bottom: 8px; }
</style>
```

SRC/COMPONENTS/STUDENTCARD.VUE HTML

```
<script setup>
const props = defineProps({ name: String, marks: Number })
</script>

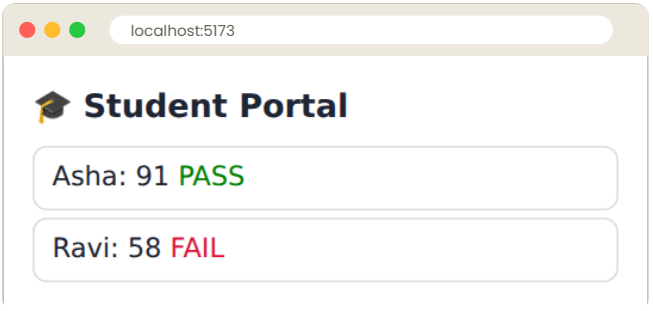
<template>
  <p class="card">
    {{ name }}: {{ marks }}
    <span :class="marks >= 60 ? 'pass' : 'fail'">{{ marks >= 60 ? 'PASS' : 'FAIL' }}</span>
  </p>
</template>

<style scoped>
.card { border: 1px solid #ddd; border-radius: 8px; padding: 6px 10px; margin: 4px 0; }
.pass { color: green; } .fail { color: crimson; }
</style>
```

SRC/APP.VUE HTML

```
<script setup>
import StudentCard from './components/StudentCard.vue'
import AppHeader from './components/AppHeader.vue'
</script>

<template>
  <AppHeader />
  <StudentCard name="Asha" :marks="91" />
  <student-card name="Ravi" :marks="58" />
</template>
```



APP.VUE COMPOSED FROM TWO CHILD COMPONENTS

3.5 Tooling

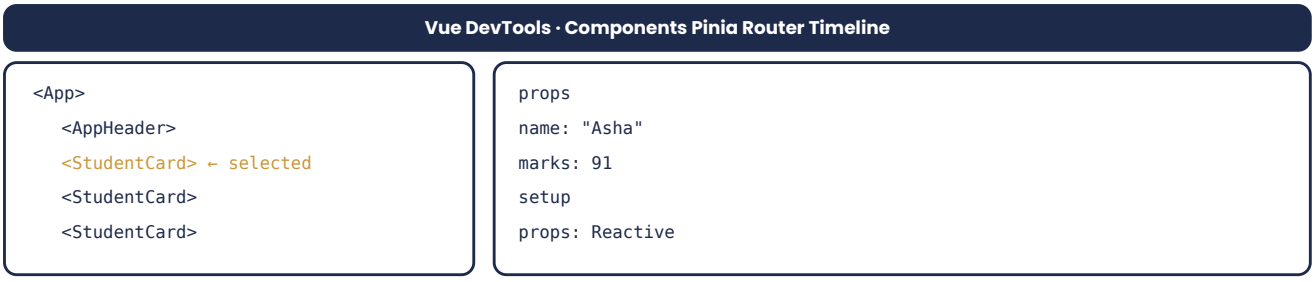
3.5.1 VS Code with the Vue (Official) extension

Install **Visual Studio Code** and the **Vue (Official)** extension (formerly "Volar"). It provides syntax highlighting for `.vue` files, autocompletion of props, events and components in templates, error checking inside templates (with TypeScript), go-to-definition, and formatting. Disable the old **Vetur** extension if it is installed; it is for Vue 2.

Feature	Example
Template autocompletion	typing `<Stu` suggests `<StudentCard>` and its props
Type checking in templates	a red underline under `{{ studnet.name }}` (typo)
Go to definition	Ctrl/Cmd-click a component tag opens its `.vue` file
Emmet in templates	`ul>li*3` + Tab expands to a list

3.5.2 Vue DevTools browser extension

Vue DevTools is available as a browser extension (Chrome, Edge, Firefox) and as a Vite plugin (`vite-plugin-vue-devtools`, offered by create-vue). It adds a Vue panel that shows the **component tree**, each component's props, state, computed values and emitted events, lets you **edit state live**, inspect Pinia stores and routes, and record a **performance timeline** (Module 17.4).



INSPECTING A COMPONENT'S PROPS AND STATE IN VUE DEVTOOLS (REPRESENTATIVE)

PRACTICE CHECKPOINT

Create a Vite Vue project, clean the starter code, and build a "Hello Vue" component with scoped styles, inspected in Vue DevTools.

After `npm create vue@latest hello-vue`, delete the example components in `src/components/` and the starter CSS import, then write these three files:

SRC/MAIN.JS JavaScript

```
import { createApp } from 'vue'
import App from './App.vue'

createApp(App).mount('#app')
```

SRC/COMPONENTS/HELLOVUE.VUE HTML

```
<script setup>
import { ref } from 'vue'

defineProps({ name: { type: String, default: 'student' } })
const waves = ref(0)
</script>

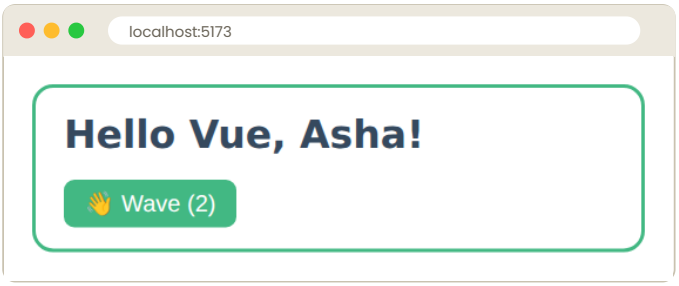
<template>
  <section class="hello">
    <h1>Hello Vue, {{ name }}!</h1>
    <button @click="waves++">👋 Wave ({{ waves }})</button>
  </section>
</template>

<style scoped>
.hello { border: 2px solid #42b883; border-radius: 12px; padding: 12px 16px; }
h1 { color: #35495e; margin: 0 0 8px; font-size: 22px; }
button { background: #42b883; color: white; border: 0; border-radius: 6px; padding: 6px 12px; }
</style>
```

SRC/APP.VUE HTML

```
<script setup>
import HelloVue from './components/HelloVue.vue'
</script>

<template>
  <HelloVue name="Asha" />
</template>
```



THE FINISHED CHECKPOINT AFTER TWO CLICKS; IN DEVTOOLS, HELLOVUE SHOWS THE PROP `NAME` AND THE STATE `WAVES: 2`

Use Vite (`npm create vue@latest`) for real projects. `index.html` loads `main.js` , which creates the app from `App.vue` and mounts it. A `.vue` file has `<script setup>` , `<template>` and `<style scoped>` ; imported components are used directly in the template.

MODULE 04 · BEGINNER

Template syntax and data binding

Minimum duration **2 days** Level **Beginner**

Vue templates are HTML enhanced with special syntax that binds data to the page. This module covers text interpolation with mustaches and the JavaScript expressions allowed inside them, raw HTML with v-html and why it is a security risk, what directives are and how arguments, modifiers and dynamic arguments work, attribute binding with v-bind and its shorthands, boolean attributes and binding many attributes from one object, class and style bindings in object and array form, and the remaining built-in directives v-text, v-once, v-pre and v-cloak. Every example was compiled and run in Chromium.

IN THIS MODULE

4.1 Text interpolation
4.2 Directives
4.3 Attribute binding
4.4 Class and style bindings
4.5 Other built-in directives

4.1 Text interpolation

4.1.1 Mustache syntax {{ }}

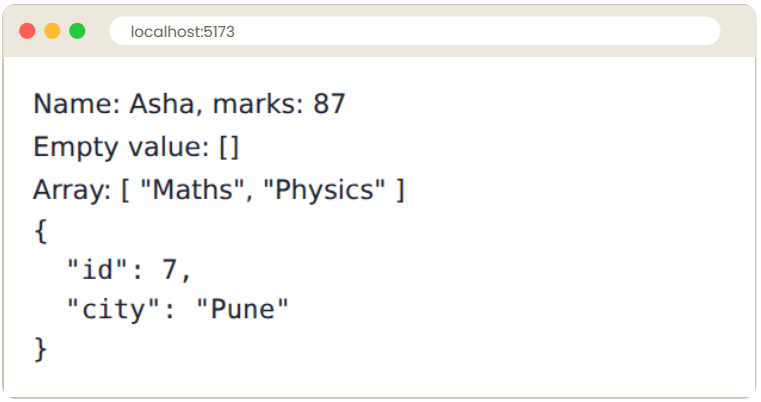
Double curly braces ("mustaches") insert the value of an expression as **text**. Whenever the data changes, the text updates. Vue converts the value to a string: `null` and `undefined` show as empty text, and arrays and plain objects are shown as formatted JSON, which is handy for debugging.

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'

const name = ref('Asha')
const marks = ref(87)
const empty = ref(null)
const subjects = ref(['Maths', 'Physics'])
const student = ref({ id: 7, city: 'Pune' })
</script>

<template>
  <p>Name: {{ name }}, marks: {{ marks }}</p>
  <p>Empty value: [{{ empty }}]</p>
  <p>Array: {{ subjects }}</p>
  <pre>{{ student }}</pre>
</template>
```



STRINGS AND NUMBERS AS TEXT, NULL AS NOTHING, OBJECTS AND ARRAYS AS JSON

NOTE

Mustaches only work in **text content**, not inside attributes: `<img src="{{ url }}">` does not work. Use `:src="url"` (4.3). Interpolated text is always **escaped**: a value containing `<b>` is shown as the characters, not as bold.

4.1.2 JavaScript expressions in templates

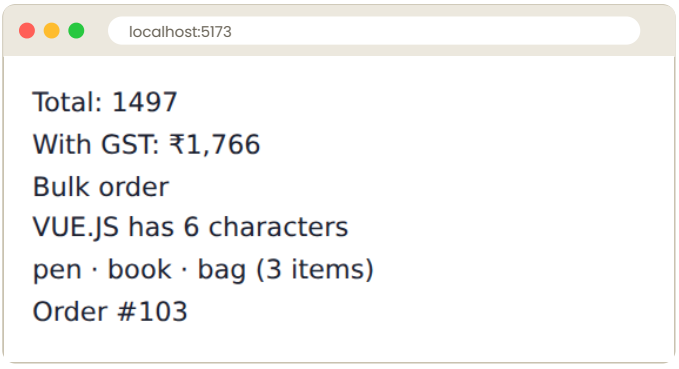
Inside `{{ }}` and directive values you can write any single JavaScript **expression**: arithmetic, ternaries, method calls, template literals, function calls. Statements (`if`, `for`, `let x = 1`) are not allowed, and only a whitelist of globals such as `Math`, `Date`, `JSON` and `Number` can be used.

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'

const price = ref(499)
const qty = ref(3)
const name = ref('vue.js')
const items = ref(['pen', 'book', 'bag'])
const formatINR = (n) => '₹' + n.toLocaleString('en-IN')
</script>

<template>
  <p>Total: {{ price * qty }}</p>
  <p>With GST: {{ formatINR(Math.round(price * qty * 1.18)) }}</p>
  <p>{{ qty > 2 ? 'Bulk order' : 'Regular order' }}</p>
  <p>{{ name.toUpperCase() }} has {{ name.length }} characters</p>
  <p>{{ items.join(' · ') }} ({{ items.length }} items)</p>
  <p>{{ `Order #${100 + qty}` }}</p>
</template>
```



EACH LINE IS ONE EXPRESSION EVALUATED AGAINST THE COMPONENT'S STATE

Allowed (expressions)	Not allowed (statements)
`{{ count + 1 }}`	`{{ let x = 1 }}`
`{{ ok ? 'Yes' : 'No' }}`	`{{ if (ok) { return 'Yes' } }}`
`{{ list.filter(n => n > 2).length }}`	`{{ for (const n of list) {} }}`
`{{ formatDate(d) }}`	`{{ window.alert(1) }}` (not a whitelisted global)

TIP

Keep templates simple. When an expression gets long or is used twice, move it into a **computed property** (Module 5.3). Functions called in templates run on **every** re-render.

Using an identifier that does not exist in the component is not an error at compile time; Vue warns at run time and shows nothing:

SRC/APP.VUE HTML

```
<script setup>
const name = 'Asha'
</script>

<template>
  <p>{{ nmae }}</p>
</template>
```

BROWSER CONSOLE

```
[Vue warn]: Property "nmae" was accessed during render but is not defined on instance.
  at <App>
```

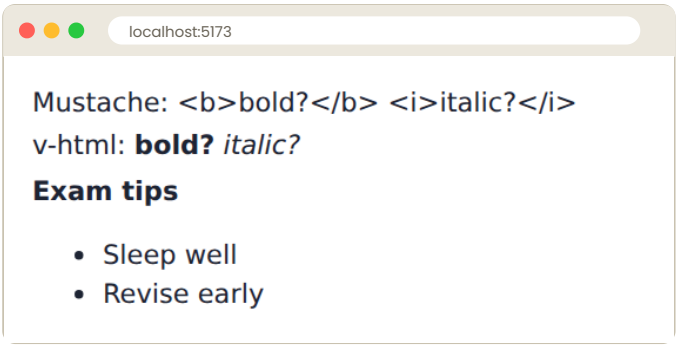
4.1.3 Raw HTML with v-html and its security risks

To render a string *as HTML*, use the `v-html` directive. It replaces the element's content with the string's HTML, without compiling it as a Vue template.

SRC/APP.VUE HTML

```
<script setup>
const plain = '<b>bold?</b> <i>italic?</i>'
const article = '<h4>Exam tips</h4><ul><li>Sleep well</li><li>Revise early</li></ul>'
</script>

<template>
  <p>Mustache: {{ plain }}</p>
  <p>v-html: <span v-html="plain"></span></p>
  <div v-html="article"></div>
</template>
```



THE SAME STRING ESCAPED BY {{ }} AND RENDERED BY V-HTML

WATCH OUT

Never use `v-html` with content that users can type (comments, profile bios, chat messages). An attacker could store `<img src=x onerror="steal(document.cookie)">`, and `v-html` would insert it so the script runs for every visitor: this is **Cross-Site Scripting (XSS)**. Only use `v-html` with trusted HTML (your own content), or sanitise it first with a library such as DOMPurify. Prefer mustaches, which are always safe.

SRC/APP.VUE HTML

```
<script setup>
// pretend this came from a database where a user typed it
const comment = '<img src="x" onerror="console.log(\'XSS! this script ran\')">Nice post'
</script>

<template>
  <p>Safe: {{ comment }}</p>
  <p>Unsafe: <span v-html="comment"></span></p>
</template>
```

BROWSER CONSOLE (THE INJECTED SCRIPT REALLY RAN)

```
XSS! this script ran
```

4.2 Directives

4.2.1 What directives are (v- prefix)

Directives are special attributes that start with `v-`. Their value is a JavaScript expression (except `v-for`, `v-on` and `v-slot`, which have their own syntax), and they apply a reactive behaviour to the element: when the expression's value changes, the directive updates the DOM.

Directive	Does	Module
`v-bind` (`:`)	binds an attribute or prop to an expression	4.3
`v-on` (`@`)	listens to an event	7
`v-if` / `v-else-if` / `v-else`	renders an element conditionally	6
`v-show`	toggles CSS `display`	6
`v-for`	renders a list	6
`v-model`	two-way binding for form inputs and components	8, 11

Directive	Does	Module
`v-slot` (`#`)	provides slot content	9
`v-html`, `v-text`, `v-once`, `v-pre`, `v-cloak`, `v-memo`	special rendering behaviours	4.1, 4.5, 17
custom (`v-focus`…)	your own DOM behaviour	18

4.2.2 Arguments, modifiers and dynamic arguments

The full syntax of a directive has up to four parts: **name**, **argument** after a colon, **modifiers** after dots, and the **value**. A **dynamic argument** in square brackets is evaluated as an expression, so the attribute or event name itself can change.



THE PARTS OF A DIRECTIVE

SRC/APP.VUE

HTML

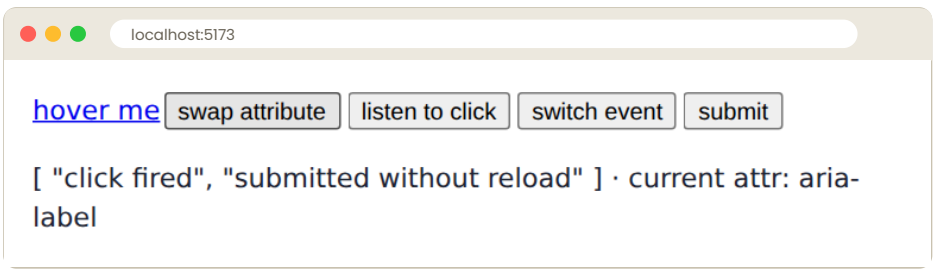
```
<script setup>
import { ref } from 'vue'

const attr = ref('title')
const eventName = ref('click')
const log = ref([])
const add = (msg) => log.value.push(msg)
</script>

<template>
  <!-- dynamic argument: binds title="..." or aria-label="..." -->
  <a href="#" :[attr]="'Hint via ' + attr">hover me</a>
  <button @click="attr = attr === 'title' ? 'aria-label' : 'title'">swap attribute</button>

  <!-- dynamic event name -->
  <button @[eventName]="add(eventName + ' fired')">listen to {{ eventName }}</button>
  <button @click="eventName = eventName === 'click' ? 'dblclick' : 'click'">switch event</button>

  <!-- modifier: .prevent calls event.preventDefault() -->
  <form @submit.prevent="add('submitted without reload')">
    <button>submit</button>
  </form>
  <p>{{ log }} · current attr: {{ attr }}</p>
</template>
```



AFTER CLICKING "LISTEN TO CLICK", "SUBMIT" AND "SWAP ATTRIBUTE"

NOTE

Dynamic argument expressions must evaluate to a string (or `null` to remove the binding) and cannot contain spaces or quotes. In in-DOM (CDN) templates, browsers lowercase attribute names, so write `:[someattr]`, not `:[someAttr]`.

4.3 Attribute binding

4.3.1 v-bind and the : shorthand

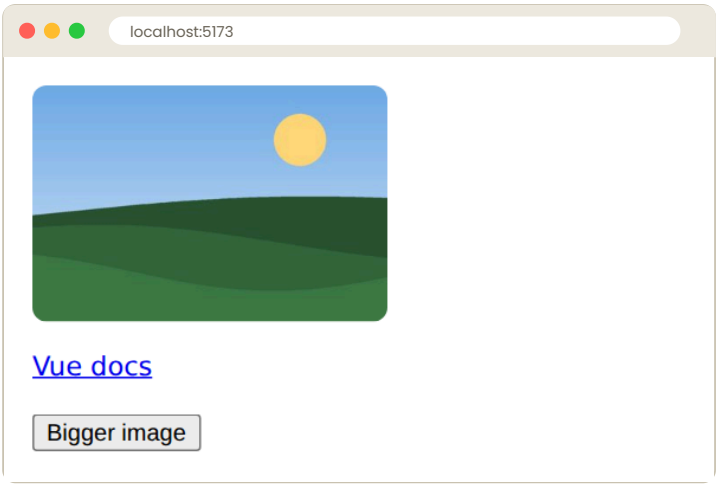
`v-bind:attr="expression"` keeps an HTML attribute (or a component prop) in sync with an expression. Because it is used everywhere, it has the shorthand `:attr`. If the value is `null` or `undefined`, the attribute is removed.

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'

const imgUrl = ref('hills-400.jpg')
const altText = ref('Hills at sunrise')
const link = ref('https://vuejs.org')
const tooltip = ref(null)
const width = ref(160)
</script>

<template>
  <img v-bind:src="imgUrl" :alt="altText" :width="width">
  <p><a :href="link" :title="tooltip">Vue docs</a></p>
  <button @click="width += 40">Bigger image</button>
</template>
```



SRC, ALT AND WIDTH BOUND FROM DATA; THE TITLE ATTRIBUTE IS ABSENT BECAUSE TOOLTIP IS NULL. AFTER ONE CLICK.

RENDERED DOM

```
<div id="app" data-v-app="">
  <img src="hills-400.jpg" alt="Hills">
  <a href="#">no title attribute</a>
</div>
```

4.3.2 Same-name shorthand :id

Since Vue 3.4, when the attribute and the variable have the same name you can omit the value: `:id` means `:id="id"`. The name is camelized, so `:aria-label` would look for a variable `ariaLabel`.

RENDERED DOM

```
<div id="app" data-v-app="">
  <img id="student-7" src="avatar.png" title="Profile picture" alt="">
</div>
```

4.3.3 Binding boolean attributes

Boolean attributes such as `disabled`, `checked`, `readonly` and `required` are present or absent. With `:disabled="expr"`, Vue adds the attribute when the value is truthy (or an empty string) and removes it when it is falsy.

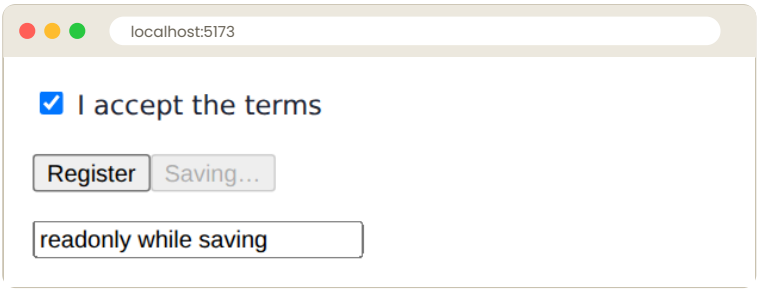
SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'

const agreed = ref(false)
const loading = ref(false)
</script>

<template>
  <label><input type="checkbox" v-model="agreed"> I accept the terms</label>
  <p>
    <button :disabled="!agreed">Register</button>
    <button :disabled="loading" @click="loading = true">{{ loading ? 'Saving…' : 'Save draft' }}
</button>
  </p>
  <input :readonly="loading" value="readonly while saving">
</template>
```



REGISTER IS ENABLED AFTER TICKING THE BOX; SAVE DRAFT DISABLES ITSELF AFTER ONE CLICK

4.3.4 Binding multiple attributes with an object

`v-bind` without an argument takes an **object** and binds each property as an attribute. This is useful for passing a group of attributes (or props) at once. Attributes written separately are merged with it; for the same name, the one written **later** wins.

RENDERED DOM

```
<div id="app" data-v-app="">
  <input id="email" type="email" placeholder="overridden (written after)" required=""
autocomplete="email">
  <input placeholder="you@example.com" id="email" type="email" required="" autocomplete="email">
</div>
```

4.4 Class and style bindings

`class` and `style` get special treatment: besides strings, `:class` and `:style` accept **objects** and **arrays**, and they **merge** with a static `class` or `style` on the same element.

4.4.1 Object syntax :class="{ active: isActive }"

In the object form each key is a class name and the class is applied when its value is truthy. Class names with dashes need quotes. The object can also come from a computed property.

SRC/APP.VUE

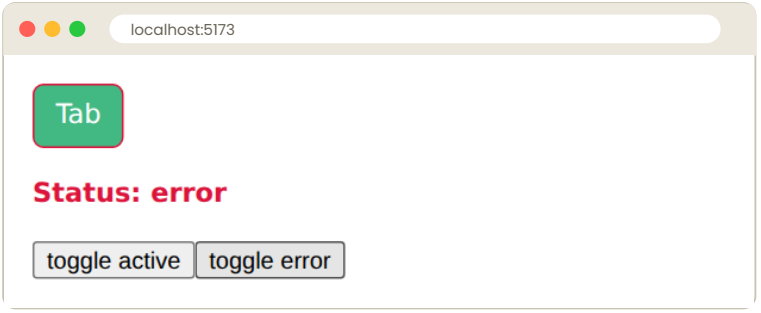
HTML

```
<script setup>
import { ref, computed } from 'vue'

const isActive = ref(true)
const hasError = ref(false)
const statusClasses = computed(() => ({
  ok: !hasError.value,
  'text-danger': hasError.value,
}))
</script>

<template>
  <div class="tab" :class="{ active: isActive, 'has-error': hasError }">Tab</div>
  <p :class="statusClasses">Status: {{ hasError ? 'error' : 'ok' }}</p>
  <button @click="isActive = !isActive">toggle active</button>
  <button @click="hasError = !hasError">toggle error</button>
</template>

<style>
.tab { padding: 6px 12px; border: 1px solid #ccc; display: inline-block; border-radius: 6px; }
.active { background: #42b883; color: white; }
.has-error { border-color: crimson; }
.ok { color: green; }
.text-danger { color: crimson; font-weight: bold; }
</style>
```



AFTER CLICKING "TOGGLE ERROR": THE STATIC CLASS TAB IS KEPT AND THE BOUND CLASSES ARE ADDED OR REMOVED

RENDERED DOM

```
<div id="app" data-v-app="">
  <div class="tab active has-error">Tab</div>
</div>
```

4.4.2 Array syntax

An array lists class names; it can mix strings, ternaries and objects.

RENDERED DOM

```
<div id="app" data-v-app="">
  <button class="btn btn-lg btn-dark active">Save</button>
</div>
```

4.4.3 Inline style bindings

`:style` takes an object of CSS properties. Use camelCase (`fontSize`) or quoted kebab-case (`'font-size'`) keys; numbers need a unit in a string. An array merges several style objects, and CSS custom properties (`--accent`) work too.

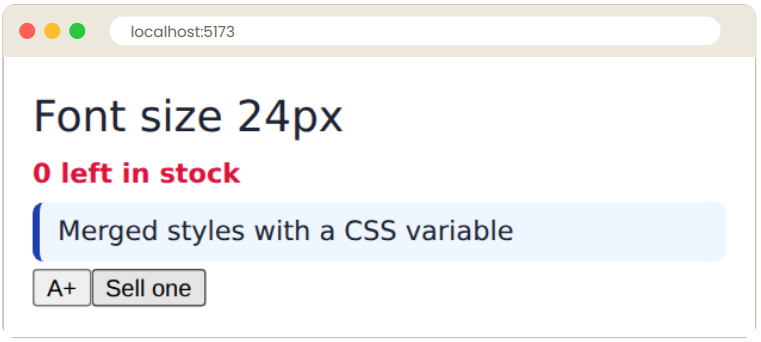
SRC/APP.VUE

HTML

```
<script setup>
import { ref, computed } from 'vue'

const size = ref(16)
const stock = ref(3)
const stockStyle = computed(() => ({
  color: stock.value === 0 ? 'crimson' : stock.value < 5 ? 'darkorange' : 'green',
  fontWeight: stock.value < 5 ? 'bold' : 'normal',
}))
const base = { padding: '6px 10px', borderRadius: '6px' }
</script>

<template>
  <p :style="{ fontSize: size + 'px' }">Font size {{ size }}px</p>
  <p :style="stockStyle">{{ stock }} left in stock</p>
  <p :style="[base, { background: '#eef6ff', '--accent': '#1e40af', borderLeft: '4px solid var(--
accent)' }]">Merged styles with a CSS variable</p>
  <button @click="size += 4">A+</button>
  <button @click="stock = Math.max(0, stock - 1)">Sell one</button>
</template>
```



AFTER CLICKING A+ TWICE AND "SELL ONE" THREE TIMES

4.5 Other built-in directives

4.5.1 v-text, v-once, v-pre, v-cloak

Directive	Effect	Equivalent / use
`v-text="msg"`	sets the element's `textContent` (replaces children)	same as `{{ msg }}` as the only content
`v-once`	renders the element and its children **once**; later updates are skipped	static content that uses data only at start (performance, Module 17)

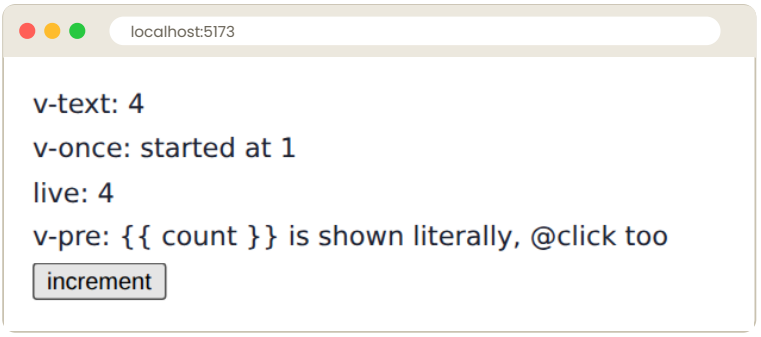
Directive	Effect	Equivalent / use
`v-pre`	skips compilation: mustaches and directives are shown as written	documentation pages that display Vue syntax
`v-cloak`	stays on the element until the app is mounted, then is removed	with CSS `[v-cloak] { display: none }`, hides raw `{{ }}` in CDN pages before Vue starts

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'

const count = ref(1)
</script>

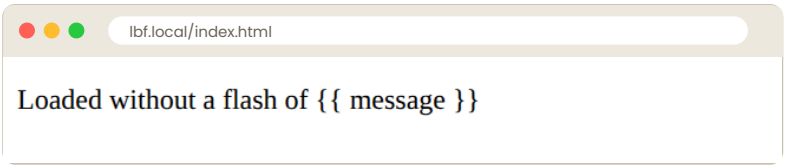
<template>
  <p v-text="'v-text: ' + count"></p>
  <p v-once>v-once: started at {{ count }}</p>
  <p>live: {{ count }}</p>
  <p v-pre>v-pre: {{ count }} is shown literally, @click too</p>
  <button @click="count++">increment</button>
</template>
```



AFTER THREE CLICKS: V-ONCE STILL SHOWS THE FIRST VALUE, V-PRE SHOWS RAW SYNTAX

INDEX.HTML HTML

```
<!DOCTYPE html>
<html>
<head>
  <style>[v-cloak] { display: none; }</style>
  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
</head>
<body>
  <div id="app" v-cloak>
    <p>{{ message }}</p>
  </div>
  <script>
    Vue.createApp({ data: () => ({ message: 'Loaded without a flash of {{ message }}' })
}).mount('#app')
  </script>
</body>
</html>
```



WITH V-CLOAK, THE RAW MUSTACHE IS NEVER VISIBLE; VUE REMOVES THE ATTRIBUTE AFTER MOUNTING

PRACTICE CHECKPOINT

A product card whose image, title, price, "sale" class and stock colour are all bound from data.

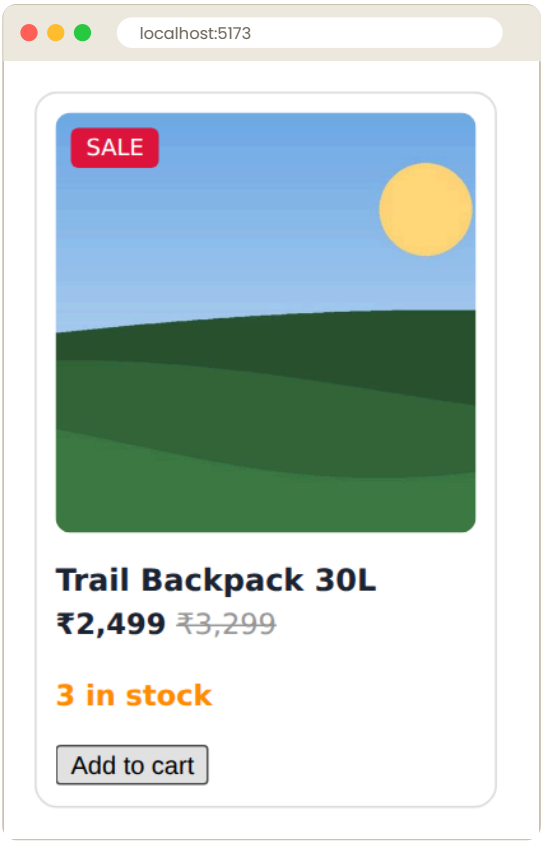
SRC/APP.VUE HTML

```
<script setup>
import { ref, computed } from 'vue'

const product = ref({
  title: 'Trail Backpack 30L',
  image: 'hills-square.jpg',
  price: 2499,
  oldPrice: 3299,
  stock: 4,
})
const onSale = computed(() => product.value.price < product.value.oldPrice)
const stockColor = computed(() => (product.value.stock === 0 ? 'crimson' : product.value.stock < 5 ?
'darkorange' : 'green'))
</script>

<template>
  <article class="card" :class="{ sale: onSale }">
    <img :src="product.image" :alt="product.title">
    <h3>{{ product.title }}</h3>
    <p class="price">
      ₹{{ product.price.toLocaleString('en-IN') }}
      <s v-if="onSale">₹{{ product.oldPrice.toLocaleString('en-IN') }}</s>
    </p>
    <p :style="{ color: stockColor, fontWeight: 600 }">
      {{ product.stock ? product.stock + ' in stock' : 'Out of stock' }}
    </p>
    <button :disabled="product.stock === 0" @click="product.stock--">Add to cart</button>
  </article>
</template>

<style scoped>
.card { width: 220px; border: 1px solid #ddd; border-radius: 12px; padding: 10px; position: relative;
}
.card img { width: 100%; border-radius: 8px; }
.sale::after { content: 'SALE'; position: absolute; top: 18px; left: 18px; background: crimson; color:
white; font-size: 12px; padding: 2px 8px; border-radius: 4px; }
h3 { margin: 6px 0 2px; font-size: 16px; }
.price { margin: 0; font-weight: bold; } s { color: #999; font-weight: normal; }
</style>
```



THE FINISHED CARD AFTER ONE REAL CLICK ON "ADD TO CART": THE STOCK DROPPED TO 3 AND STAYED ORANGE

Use `{{ }}` for text (always escaped), `v-html` only for trusted HTML, `:attr` (`v-bind`) for attributes and props, and the object or array forms of `:class` and `:style`, which merge with static classes and styles. Directives have a name, an optional argument (static or `[dynamic]`), modifiers and a value.

MODULE 05 · BEGINNER

Reactivity fundamentals

Minimum duration **2 days** | Level **Beginner**

Reactivity is Vue's core feature: when reactive data changes, the page updates automatically. This module covers ref() and the .value rule (with automatic unwrapping in templates), refs that hold objects and arrays, reactive() and its limitations (no primitives, lost reactivity when destructuring or replacing), toRefs() and toRef(), why ref() is the recommended default, computed properties with caching, writable computed and side-effect-free getters, and an overview of how reactivity works: dependency tracking with Proxies and the batched, asynchronous DOM update with nextTick().

IN THIS MODULE

5.1 ref()
5.2 reactive()
5.3 Computed properties
5.4 How reactivity works (overview)

5.1 ref()

5.1.1 Creating reactive values with ref()

`ref(initialValue)` creates a **reactive reference**: an object with one property, `.value`. Reading `.value` lets Vue **track** who uses it; writing `.value` **triggers** an update of everything that used it. A plain variable does not do this, as the example shows: both buttons change a number, but only the ref updates the page.

SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'

const reactiveCount = ref(0)   // reactive
let plainCount = 0             // NOT reactive

function incrementBoth() {
  reactiveCount.value++
  plainCount++
  console.log('plainCount is really', plainCount)
}
</script>

<template>
  <p>ref: {{ reactiveCount }}</p>
  <p>plain variable: {{ plainCount }}</p>
  <button @click="incrementBoth">increment both</button>
</template>
```



AFTER THREE CLICKS: THE REF SHOWS 3, THE PLAIN VARIABLE IS STILL DRAWN AS 0 (IT IS 3 IN MEMORY)



A REF IS A SMALL OBJECT WHOSE .VALUE GETTER TRACKS AND WHOSE SETTER TRIGGERS

5.1.2 .value in script vs automatic unwrapping in templates

In JavaScript you must write `.value`. In the template, top-level refs are **unwrapped automatically**, so you write `{{ count }}` and `@click="count++"`. Unwrapping also happens for refs nested inside a `reactive()` object, but not for refs inside a plain object or array.

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'

const count = ref(1)
const object = { id: ref(7) }     // a ref inside a PLAIN object

console.log('in script:', count.value, typeof count)   // .value needed
</script>

<template>
  <p>{{ count + 1 }}  <!-- unwrapped: 2 --></p>
  <p>{{ object.id + 1 }}  <!-- NOT unwrapped: "[object Object]1" --></p>
  <p>{{ object.id.value + 1 }}  <!-- explicit .value: 8 --></p>
  <p>{{ object.id }}  <!-- final value of an interpolation is unwrapped: 7 --></p>
</template>
```



TOP-LEVEL REFS UNWRAP; A REF NESTED IN A PLAIN OBJECT ONLY UNWRAPS WHEN IT IS THE WHOLE INTERPOLATION

Where	Write	Why
`<script setup>`	`count.value++`	JavaScript needs the property access to trigger tracking
template	`count++`, `{{ count }}`	the compiler unwraps top-level refs for you
inside `reactive()`	`state.count++`	reactive objects unwrap nested refs automatically

Where	Write	Why
forgetting .value in script	`if (count > 5)`	compares the ref object, not the number: always a bug

5.1.3 Refs holding objects and arrays

A ref can hold any value. When it holds an object or array, the value is made **deeply reactive**: changing a nested property, pushing to an array or replacing the whole value all update the page.

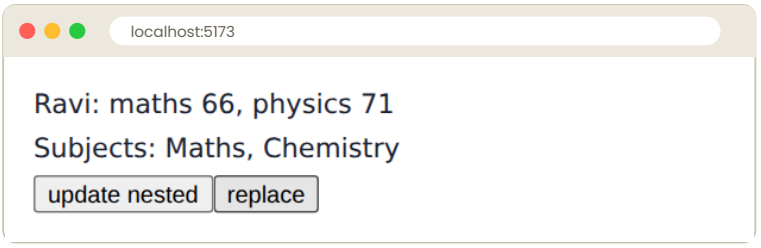
SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'

const student = ref({ name: 'Asha', marks: { maths: 80, physics: 75 } })
const subjects = ref(['Maths', 'Physics'])

function update() {
  student.value.marks.maths += 10        // nested change: reactive
  subjects.value.push('Chemistry')       // array mutation: reactive
}
function replace() {
  student.value = { name: 'Ravi', marks: { maths: 66, physics: 71 } }   // replace whole value
  subjects.value = subjects.value.filter((s) => s !== 'Physics')
}
</script>

<template>
  <p>{{ student.name }}: maths {{ student.marks.maths }}, physics {{ student.marks.physics }}</p>
  <p>Subjects: {{ subjects.join(', ') }}</p>
  <button @click="update">update nested</button>
  <button @click="replace">replace</button>
</template>
```



AFTER "UPDATE NESTED" THEN "REPLACE": BOTH KINDS OF CHANGE ARE TRACKED

TIP
For very large objects that you only ever replace (never mutate), `shallowRef()` avoids making every nested property reactive (Module 17).

5.2 reactive()

5.2.1 Reactive objects

`reactive(object)` returns a **Proxy** of the object that is deeply reactive. Unlike a ref there is no `.value`: you use it like a normal object. The Proxy is a different object from the original; only changes made **through the proxy** are tracked.

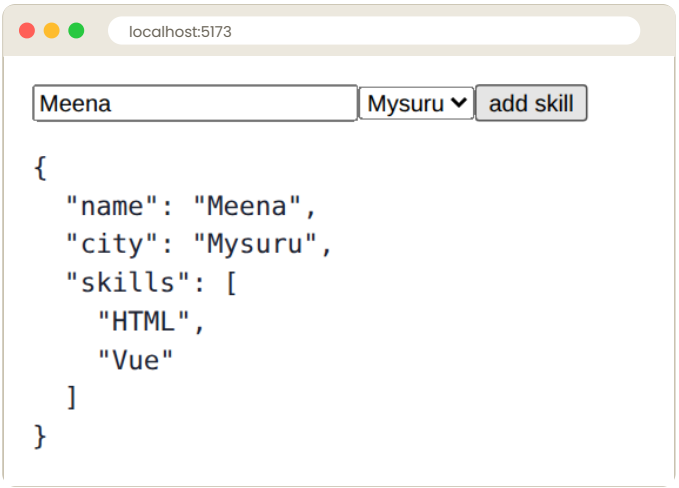
SRC/APP.VUE

HTML

```
<script setup>
import { reactive } from 'vue'

const form = reactive({
  name: '',
  city: 'Pune',
  skills: ['HTML'],
})
function addSkill() { form.skills.push('Vue') }
</script>

<template>
  <input v-model="form.name" placeholder="Name">
  <select v-model="form.city"><option>Pune</option><option>Mysuru</option></select>
  <button @click="addSkill">add skill</button>
  <pre>{{ form }}</pre>
</template>
```



AFTER TYPING A NAME, CHOOSING MYSURU AND ADDING A SKILL: REACTIVE() TRACKS EVERY PROPERTY

5.2.2 Limitations: no primitives, losing reactivity when destructuring or replacing

`reactive()` has three limitations, all because reactivity works through property access on the proxy:

Limitation	Example	What happens
Only objects	`reactive(0)`	cannot wrap a number, string or boolean; Vue warns and returns the plain value
Destructuring loses reactivity	`let { count } = state`	`count` is a plain copy; changing it does not change `state`, and changes to `state` do not reach it
Replacing the object loses it	`state = reactive({...})`	the template still uses the old proxy; you must mutate properties instead

SRC/APP.VUE HTML

```
<script setup>
import { reactive, watchEffect } from 'vue'

const bad = reactive(0)                       // 1. primitive
const state = reactive({ count: 1 })
let { count } = state                         // 2. destructured copy

watchEffect(() => console.log('state.count =', state.count))
count++                                       // changes only the local copy
console.log('local count =', count, ' state.count =', state.count)
state.count = 10                              // the effect re-runs (later, batched)
</script>
<template><p>{{ state.count }}</p></template>
```

BROWSER CONSOLE

```
[Vue warn]: value cannot be made reactive: 0
state.count = 1
local count = 2  state.count = 1
state.count = 10
```

5.2.3 toRefs() and toRef()

`toRefs(reactiveObject)` converts each property into a ref that stays **linked** to the source, so the result can be destructured safely. `toRef(object, 'key')` does the same for one property. This is how composables return reactive objects that callers can destructure (Module 12).

SRC/APP.VUE HTML

```
<script setup>
import { reactive, toRefs, toRef } from 'vue'

const state = reactive({ likes: 5, title: 'Vue tips' })
const { likes, title } = toRefs(state)    // linked refs
const likesRef = toRef(state, 'likes')    // one linked ref

function like() { likes.value++ }         // writes back to state.likes
</script>

<template>
  <p>{{ title }}: {{ likes }} likes</p>
  <p>state.likes = {{ state.likes }}, likesRef = {{ likesRef }}</p>
  <button @click="like">like</button>
  <button @click="state.likes = 0">reset via state</button>
</template>
```



AFTER TWO LIKES: THE DESTRUCTURED REF, THE ORIGINAL OBJECT AND THE TOREF STAY IN SYNC

5.2.4 Why ref() is the recommended default

	ref()	reactive()
Values	any type, including primitives	objects, arrays, Map, Set only
Replace whole value	`x.value = newValue` works	not possible (must mutate)
Destructure / pass around	pass the ref itself, stays reactive	loses reactivity unless `toRefs`
Syntax in script	needs `.value`	no `.value`
Recommended for	almost everything (official recommendation)	grouping related fields, such as a form object

NOTE

The Vue documentation recommends **`ref()` as the primary API** for declaring reactive state because it has none of reactive()'s limitations. The `.value` also makes it obvious in script code which variables are reactive.

5.3 Computed properties

5.3.1 computed() for derived values

A **computed property** derives a value from other reactive state with a getter function. It returns a read-only ref, recalculates automatically when its dependencies change, and keeps templates clean.

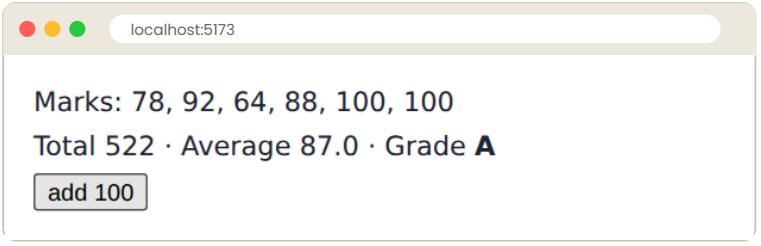
SRC/APP.VUE

HTML

```
<script setup>
import { ref, computed } from 'vue'

const marks = ref([78, 92, 64, 88])
const total = computed(() => marks.value.reduce((a, b) => a + b, 0))
const average = computed(() => (total.value / marks.value.length).toFixed(1))
const grade = computed(() => (average.value >= 85 ? 'A' : average.value >= 70 ? 'B' : 'C'))
</script>

<template>
  <p>Marks: {{ marks.join(', ') }}</p>
  <p>Total {{ total }} · Average {{ average }} · Grade <b>{{ grade }}</b></p>
  <button @click="marks.push(100)">add 100</button>
</template>
```



AFTER ADDING 100 TWICE: TOTAL, AVERAGE AND GRADE WERE RECOMPUTED IN A CHAIN

5.3.2 Computed caching vs methods

A computed value is **cached**: its getter runs once and the result is reused until a dependency changes. A **method** called in the template runs again on **every** re-render, even when its inputs did not change. In the program below, typing in the unrelated input re-renders the component 4 times; watch how often each function runs:

SRC/APP.VUE

HTML

```
<script setup>
import { ref, computed } from 'vue'

const items = ref([5, 10, 15])
const note = ref('')
let computedRuns = 0, methodRuns = 0

const totalComputed = computed(() => {
  computedRuns++
  return items.value.reduce((a, b) => a + b, 0)
})
function totalMethod() {
  methodRuns++
  return items.value.reduce((a, b) => a + b, 0)
}
function report() { console.log(`computed ran ${computedRuns}x, method ran ${methodRuns}x`) }
</script>

<template>
  <p>{{ totalComputed }} {{ totalComputed }} {{ totalMethod() }} {{ totalMethod() }}</p>
  <input v-model="note">
  <button @click="report">report</button>
</template>
```

BROWSER CONSOLE

```
computed ran 1x, method ran 10x
```

The template uses each value twice. The computed getter ran **once** in total; the method ran **twice per render** (first render + 4 re-renders = 10 calls). Use computed for derived data; use methods for actions (event handlers) or when you need a fresh value every time (such as `Date.now()`).

5.3.3 Writable computed (getter and setter)

Normally a computed is read-only. Passing an object with `get` and `set` creates a **writable computed**: assigning to it calls the setter, which should update the source state. A common use is a single input that edits several underlying values.

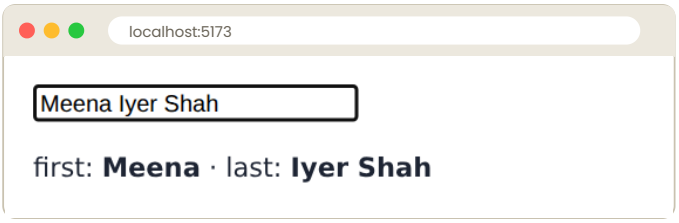
SRC/APP.VUE HTML

```
<script setup>
import { ref, computed } from 'vue'

const firstName = ref('Asha')
const lastName = ref('Rao')

const fullName = computed({
  get: () => `${firstName.value} ${lastName.value}`,
  set: (value) => {
    const [first, ...rest] = value.trim().split(' ')
    firstName.value = first
    lastName.value = rest.join(' ')
  },
})
</script>

<template>
  <input v-model="fullName">
  <p>first: <b>{{ firstName }}</b> · last: <b>{{ lastName }}</b></p>
</template>
```



TYPING "MEENA IYER SHAH" INTO THE COMPUTED'S INPUT RAN THE SETTER

SRC/APP.VUE HTML

```
<script setup>
import { ref, computed } from 'vue'
const price = ref(100)
const withTax = computed(() => price.value * 1.18)
withTax.value = 200        // read-only computed: not allowed
console.log(withTax.value)
</script>
<template><p>{{ withTax }}</p></template>
```

BROWSER CONSOLE

```
[Vue warn]: Write operation failed: computed value is readonly
118
```

5.3.4 Keeping computed getters free of side effects

A computed getter should only **calculate and return** a value. Do not change other state, make API calls, or modify the DOM inside it; use a **watcher** for side effects (Module 10). Also avoid mutating the computed's own result, for example sorting the array it read: `sort()` and `reverse()` change the source array in place.

Bad (side effect in getter)	Good
`computed(() => { count.value++; return x.value * 2 })`	`computed(() => x.value * 2)` and change `count` in an event handler

Bad (side effect in getter)	Good
`computed(() => list.value.sort())` (mutates the source)	`computed(() => [...list.value].sort())` or `list.value.toSorted()`
`computed(() => { fetch(url.value); ... })`	`watch(url, (u) => fetch(u))` (Module 10)

5.4 How reactivity works (overview)

5.4.1 Dependency tracking with Proxies

Every component's render function, every `computed` and every watcher runs inside a **reactive effect**. While an effect runs, each reactive read (a ref's `.value` getter or a proxy's `get` trap) records "this effect depends on this property": **track**. When a property is written (setter or `set` trap), Vue looks up the effects that depend on it and schedules them to run again: **trigger**. Effects that never read a property are never re-run when it changes.



EFFECTS REGISTER AS DEPENDENCIES WHILE THEY READ; WRITES LOOK UP AND RE-RUN ONLY THOSE EFFECTS

SRC/APP.VUE

HTML

```
<script setup>
import { ref, reactive, computed, watchEffect } from 'vue'

const a = ref(1), b = ref(2), unrelated = ref(0)
const sum = computed(() => { console.log('  computing sum'); return a.value + b.value })
watchEffect(() => console.log('effect sees sum =', sum.value))

setTimeout(() => { console.log('-- a = 5'); a.value = 5 }, 10)
setTimeout(() => { console.log('-- unrelated = 99 (nothing re-runs)'); unrelated.value = 99 }, 20)
setTimeout(() => { console.log('-- b = 10'); b.value = 10 }, 30)
</script>
<template><p>{{ sum }}</p></template>
```

BROWSER CONSOLE: TRACK AND TRIGGER

```
  computing sum
effect sees sum = 3
-- a = 5
  computing sum
effect sees sum = 7
-- unrelated = 99 (nothing re-runs)
-- b = 10
  computing sum
effect sees sum = 15
```

5.4.2 DOM update timing and nextTick()

When state changes, Vue does **not** update the DOM immediately. It puts the affected components in a queue and updates them once, in the next "tick" (a microtask), however many changes were made. This batching makes many changes cheap. To read the DOM **after** the update, `await nextTick()`.

SRC/APP.VUE

HTML

```
<script setup>
import { ref, nextTick, onUpdated } from 'vue'

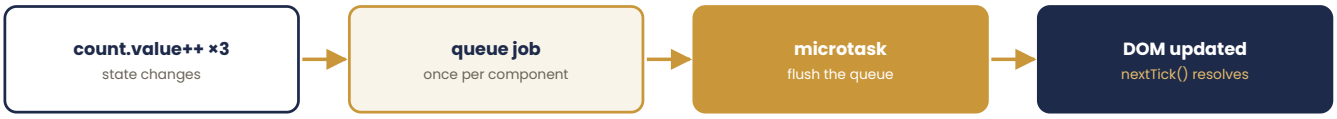
const count = ref(0)
onUpdated(() => console.log('  component re-rendered (once)'))

async function increment() {
  count.value++
  count.value++
  count.value++
  const p = document.querySelector('#out')
  console.log('right after the change, DOM shows:', p.textContent)
  await nextTick()
  console.log('after nextTick, DOM shows:', p.textContent)
}
</script>

<template>
  <p id="out">{{ count }}</p>
  <button @click="increment">+3</button>
</template>
```

BROWSER CONSOLE

```
right after the change, DOM shows: 0
  component re-rendered (once)
after nextTick, DOM shows: 3
```



SEVERAL SYNCHRONOUS CHANGES PRODUCE ONE BATCHED DOM UPDATE

PRACTICE CHECKPOINT

A shopping cart where items are a ref array and the subtotal, tax and total are computed properties that update automatically.

SRC/APP.VUE HTML

```
<script setup>
import { ref, computed } from 'vue'

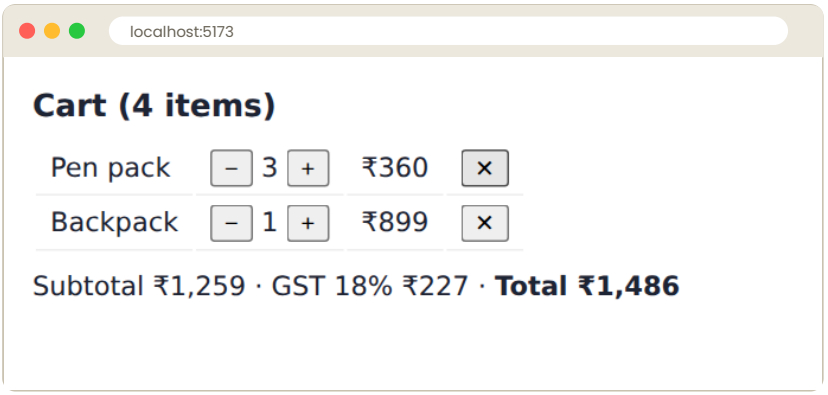
const items = ref([
  { id: 1, name: 'Notebook', price: 60, qty: 3 },
  { id: 2, name: 'Pen pack', price: 120, qty: 1 },
  { id: 3, name: 'Backpack', price: 899, qty: 1 },
])
const TAX_RATE = 0.18

const subtotal = computed(() => items.value.reduce((sum, i) => sum + i.price * i.qty, 0))
const tax = computed(() => Math.round(subtotal.value * TAX_RATE))
const total = computed(() => subtotal.value + tax.value)
const itemCount = computed(() => items.value.reduce((n, i) => n + i.qty, 0))
const inr = (n) => '₹' + n.toLocaleString('en-IN')

function remove(id) { items.value = items.value.filter((i) => i.id !== id) }
</script>

<template>
  <h3>Cart ({{ itemCount }} items)</h3>
  <table>
    <tr v-for="item in items" :key="item.id">
      <td>{{ item.name }}</td>
      <td><button @click="item.qty > 1 && item.qty--">−</button> {{ item.qty }} <button
@click="item.qty++">+</button></td>
      <td>{{ inr(item.price * item.qty) }}</td>
      <td><button @click="remove(item.id)">x</button></td>
    </tr>
  </table>
  <p>Subtotal {{ inr(subtotal) }} · GST 18% {{ inr(tax) }} · <b>Total {{ inr(total) }}</b></p>
</template>

<style scoped>
h3 { margin: 0 0 6px; } td { padding: 3px 8px; border-bottom: 1px solid #eee; } p { margin: 8px 0 0; }
</style>
```



AFTER REAL CLICKS: PEN PACK QUANTITY +1 TWICE AND THE NOTEBOOK ROW REMOVED; EVERY TOTAL RECOMPUTED

`ref()` holds any value and needs `.value` in script (not in templates); `reactive()` makes an object a deep Proxy but breaks when destructured or replaced (use `toRefs`). Computed values are cached derived state with no side effects. Vue tracks reads, triggers on writes and batches DOM updates to the next tick; use `await nextTick()` to see the updated DOM.

MODULE 06 · BEGINNER

Conditional and list rendering

Minimum duration **1 day** | Level **Beginner**

This module covers showing, hiding and repeating elements based on data: v-if, v-else-if and v-else, v-if on a template wrapper, v-show and when to choose it over v-if; v-for over arrays (with the index), objects and number ranges, the :key attribute and what goes wrong without it, v-for on a template, why v-if and v-for must not share an element, rendering filtered and sorted lists through computed properties, and which array changes Vue detects.

IN THIS MODULE

6.1 Conditional rendering
6.2 List rendering

6.1 Conditional rendering

6.1.1 v-if, v-else-if, v-else

`v-if="condition"` renders the element only when the condition is truthy. `v-else-if` and `v-else` must come **immediately after** a `v-if` or `v-else-if` element. When the condition is false the element is not hidden: it is **not in the DOM at all** (Vue leaves a `<!--v-if-->` comment as a placeholder).

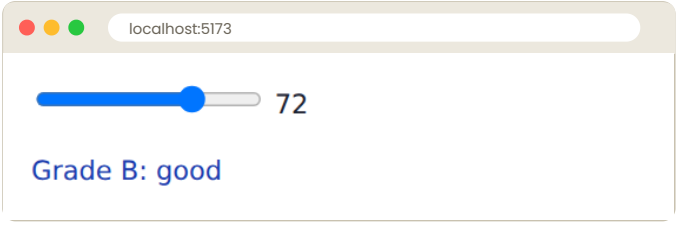
SRC/APP.VUE — HTML

```
<script setup>
import { ref } from 'vue'

const marks = ref(72)
</script>

<template>
  <input type="range" min="0" max="100" v-model.number="marks"> {{ marks }}
  <p v-if="marks >= 85" class="a">Grade A: excellent</p>
  <p v-else-if="marks >= 60" class="b">Grade B: good</p>
  <p v-else-if="marks >= 35" class="c">Grade C: pass</p>
  <p v-else class="f">Fail: try again</p>
</template>

<style>
.a { color: green; } .b { color: #1e40af; } .c { color: darkorange; } .f { color: crimson; }
</style>
```



WITH MARKS = 72, ONLY THE V-ELSE-IF BRANCH FOR GRADE B EXISTS IN THE DOM

RENDERED DOM

```
<div id="app" data-v-app="">
  <!--v-if-->
  <button>Log in</button>
</div>
```

SRC/APP.VUE

HTML

```
<script setup>
const ok = true
</script>

<template>
  <p v-if="ok">yes</p>
  <hr>
  <p v-else>no</p>
</template>
```

VITE DEV SERVER

main.js

```
$ npm run dev
```

TERMINAL: npm run dev

```
[plugin:vite:vue] [vue/compiler-sfc] v-else/v-else-if has no adjacent v-if or v-else-if.

/src/App.vue:8:2
  8 |  <p v-else>no</p>
    |    ^
```

v-else must directly follow its v-if (only whitespace and comments may sit between them). The Vite dev server reports this compile error in the terminal and as an overlay in the browser.

6.1.2 v-if on template

To toggle **several** elements together without adding a wrapper element, put `v-if` on an invisible `<template>` element. The `<template>` itself is not rendered.

RENDERED DOM

```
<div id="app" data-v-app="">
  <h3>Price: ₹999</h3>
  <p>Discount: 20%</p>
  <p>You pay: ₹799</p>
</div>
```

6.1.3 v-show

`v-show="condition"` always renders the element and only toggles its CSS `display` property. The element stays in the DOM, so its state (for example text typed into an input) is kept.

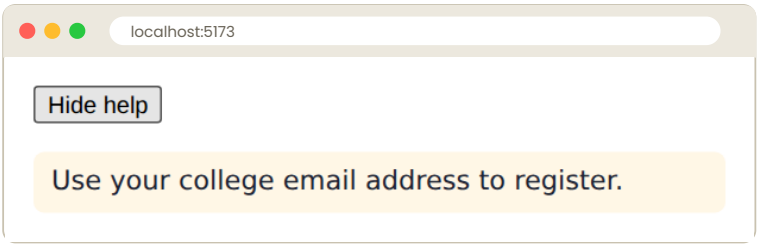
SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'
const showHelp = ref(false)
</script>

<template>
  <button @click="showHelp = !showHelp">{{ showHelp ? 'Hide' : 'Show' }} help</button>
  <p v-show="showHelp" class="help">Use your college email address to register.</p>
</template>

<style>.help { background: #fff7e6; padding: 6px 10px; border-radius: 6px; }</style>
```



AFTER ONE CLICK THE HELP TEXT IS DISPLAYED

RENDERED DOM

```
<div id="app" data-v-app="">
  <p style="display: none;">Use your college email address.</p>
  <!--v-if-->
</div>
```

6.1.4 v-if vs v-show

	v-if	v-show
When false	element removed from the DOM (child components unmounted)	element kept, `display: none`
Initial cost	nothing rendered while false (cheap)	always rendered (more initial work)
Toggle cost	higher: creates and destroys elements and components	very low: changes one CSS property
State inside	lost on every toggle	kept
Works on `<template>`, with `v-else`	yes	no
Use when	the condition rarely changes, or the content is heavy	it toggles often (tabs, dropdowns, help text)

SRC/APP.VUE

HTML

```
<script setup>
import { ref, onMounted, onUnmounted, defineComponent, h } from 'vue'

const Panel = defineComponent({
  props: ['label'],
  setup(props) {
    onMounted(() => console.log(`  ${props.label} mounted`))
    onUnmounted(() => console.log(`  ${props.label} unmounted`))
    return () => h('p', props.label)
  },
})
const on = ref(true)
setTimeout(() => { console.log('-- toggle off'); on.value = false }, 20)
setTimeout(() => { console.log('-- toggle on'); on.value = true }, 40)
</script>

<template>
  <Panel v-if="on" label="v-if panel" />
  <Panel v-show="on" label="v-show panel" />
</template>
```

BROWSER CONSOLE: V-IF RECREATES, V-SHOW KEEPS

```
  v-if panel mounted
  v-show panel mounted
-- toggle off
  v-if panel unmounted
-- toggle on
  v-if panel mounted
```

6.2 List rendering

6.2.1 v-for over arrays with index

`v-for="item in items"` renders the element once for each array item. The optional second alias is the index: `v-for="(item, index) in items"`. You can also write `of` instead of `in`, and destructure the item.

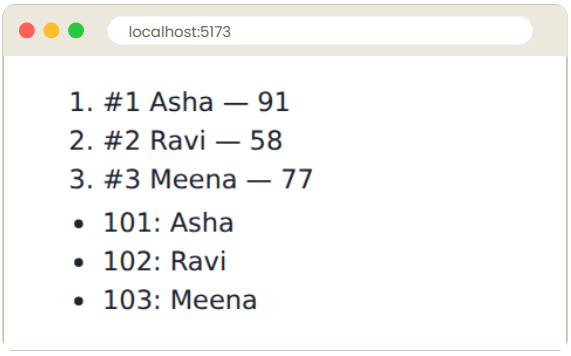
SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'

const students = ref([
  { id: 101, name: 'Asha', marks: 91 },
  { id: 102, name: 'Ravi', marks: 58 },
  { id: 103, name: 'Meena', marks: 77 },
])
</script>

<template>
  <ol>
    <li v-for="(student, index) in students" :key="student.id">
      #{{ index + 1 }} {{ student.name }} — {{ student.marks }}
    </li>
  </ol>
  <ul>
    <li v-for="{ id, name } of students" :key="id">{{ id }}: {{ name }}</li>
  </ul>
</template>
```



THE SAME ARRAY RENDERED WITH AN INDEX AND WITH DESTRUCTURING

6.2.2 v-for over objects and ranges

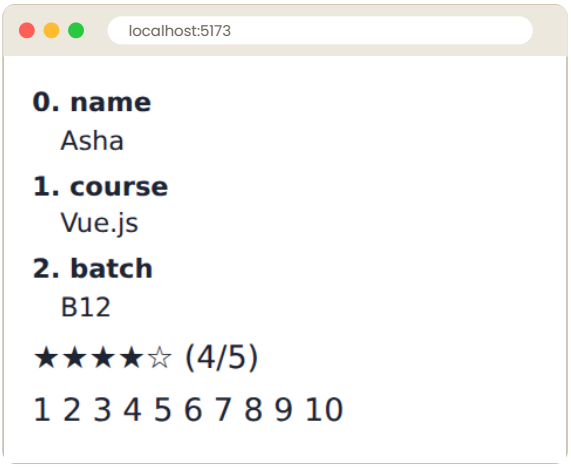
Over an object, `v-for` gives `(value, key, index)` in the order of `Object.keys()`. Over an integer `n`, it repeats `n` times with values **starting at 1**.

SRC/APP.VUE

HTML

```
<script setup>
const profile = { name: 'Asha', course: 'Vue.js', batch: 'B12' }
const rating = 4
</script>

<template>
  <dl>
    <template v-for="(value, key, index) in profile" :key="key">
      <dt>{{ index }}. {{ key }}</dt><dd>{{ value }}</dd>
    </template>
  </dl>
  <p>
    <span v-for="n in 5" :key="n">{{ n <= rating ? '★' : '☆' }}</span>
    ({{ rating }}/5)
  </p>
  <p><span v-for="n in 10" :key="n">{{ n }} </span></p>
</template>
```



AN OBJECT'S ENTRIES AND TWO NUMBER RANGES (NOTE THAT RANGES START AT 1)

6.2.3 The :key attribute and why it matters

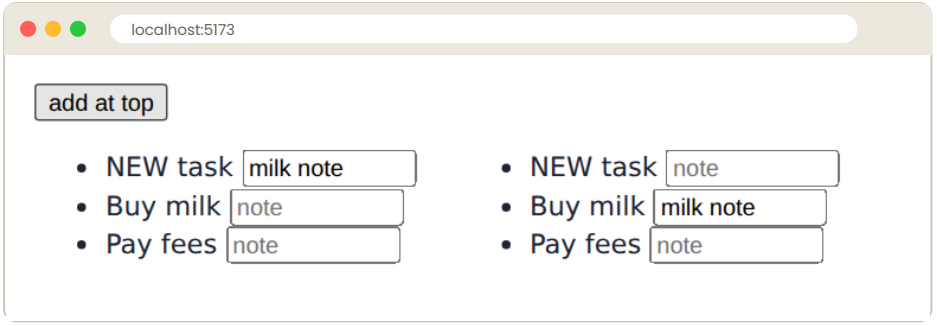
When a list changes, Vue reuses existing DOM elements as much as possible. Without a key it **patches in place by position**: if you insert an item at the top, the first element gets the new item's text, the second gets the old first's text, and so on. Anything not driven by data, such as text typed into an input or a checkbox's state, stays at its **old position** and ends up next to the wrong item. A unique, stable `:key` (usually an id) lets Vue **move** the right element instead.

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
const todos = ref([{ id: 1, text: 'Buy milk' }, { id: 2, text: 'Pay fees' }])
let next = 3
</script>

<template>
  <button @click="todos.unshift({ id: next++, text: 'NEW task' })">add at top</button>
  <div class="cols">
    <ul>
      <li v-for="t in todos">{{ t.text }} <input placeholder="note"></li>
    </ul>
    <ul>
      <li v-for="t in todos" :key="t.id">{{ t.text }} <input placeholder="note"></li>
    </ul>
  </div>
</template>

<style>.cols { display: flex; gap: 24px; } input { width: 90px; }</style>
```



TYPED "MILK NOTE" NEXT TO "BUY MILK" IN BOTH LISTS, THEN CLICKED "ADD AT TOP". LEFT (NO KEY): THE NOTE STAYED IN ROW 1, NOW BESIDE NEW TASK. RIGHT (:KEY="T.ID"): THE NOTE MOVED WITH BUY MILK.

Good key	Bad key
database id: `:key="student.id"`	the index: `:key="index"` (same problem as no key when items are inserted, removed or sorted)
a unique string: `:key="email"`	an object: `:key="student"` (keys must be strings, numbers or symbols)
a stable value that never changes for that item	`Math.random()` (new key every render: everything is recreated)

6.2.4 v-for on template

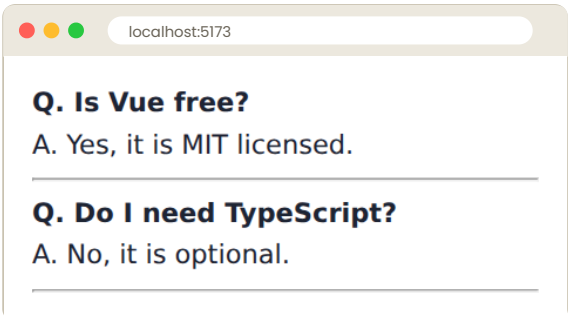
Like `v-if`, `v-for` can go on a `<template>` to repeat a **group** of elements. Put the `:key` on the `<template>` tag.

SRC/APP.VUE

HTML

```
<script setup>
const faqs = [
  { id: 1, q: 'Is Vue free?', a: 'Yes, it is MIT licensed.' },
  { id: 2, q: 'Do I need TypeScript?', a: 'No, it is optional.' },
]
</script>

<template>
  <dl>
    <template v-for="f in faqs" :key="f.id">
      <dt>Q. {{ f.q }}</dt>
      <dd>A. {{ f.a }}</dd>
      <hr>
    </template>
  </dl>
</template>
```



EACH ITEM PRODUCES THREE SIBLING ELEMENTS, WITH NO EXTRA WRAPPER

6.2.5 Why not to use v-if and v-for on the same element

When `v-if` and `v-for` are on the same element, **`v-if` has higher priority** in Vue 3: it is evaluated first, before the loop variable exists. A condition that uses the loop variable therefore fails:

SRC/APP.VUE

HTML

```
<script setup>
const todos = [{ id: 1, text: 'Learn Vue', done: true }, { id: 2, text: 'Build app', done: false }]
</script>

<template>
  <ul>
    <!-- WRONG: v-if runs first, "todo" does not exist yet -->
    <li v-for="todo in todos" v-if="!todo.done" :key="todo.id">{{ todo.text }}</li>
  </ul>
</template>
```

BROWSER CONSOLE

```
[Vue warn]: Property "todo" was accessed during render but is not defined on instance.
  at <App>
[Vue warn]: Unhandled error during execution of render function
  at <App>
Uncaught TypeError: Cannot read properties of undefined (reading 'done')
```

There are two correct patterns. To **filter** items, loop over a computed list (best, see 6.2.6). To **skip the whole list** conditionally, move `v-if` to a wrapper or a `<template>`:

CORRECT PATTERNS

HTML

```
<!-- filter: loop over a computed -->
<li v-for="todo in pendingTodos" :key="todo.id">{{ todo.text }}</li>

<!-- condition per item: put v-if on an inner template -->
<template v-for="todo in todos" :key="todo.id">
  <li v-if="!todo.done">{{ todo.text }}</li>
</template>

<!-- whole list conditional: v-if on the wrapper -->
<ul v-if="showList">
  <li v-for="todo in todos" :key="todo.id">{{ todo.text }}</li>
</ul>
```

6.2.6 Rendering filtered and sorted lists with computed properties

To show a filtered or sorted version of a list without changing the original, create a **computed** property. Copy before sorting (`[...list]` or `toSorted()`), because `sort()` mutates the array.

SRC/APP.VUE

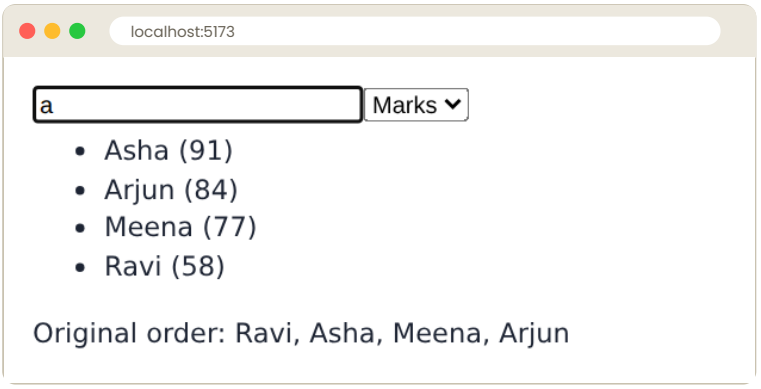
HTML

```
<script setup>
import { ref, computed } from 'vue'

const query = ref('')
const sortBy = ref('name')
const students = ref([
  { id: 1, name: 'Ravi', marks: 58 }, { id: 2, name: 'Asha', marks: 91 },
  { id: 3, name: 'Meena', marks: 77 }, { id: 4, name: 'Arjun', marks: 84 },
])

const visible = computed(() =>
  students.value
    .filter((s) => s.name.toLowerCase().includes(query.value.toLowerCase()))
    .toSorted((a, b) => (sortBy.value === 'name' ? a.name.localeCompare(b.name) : b.marks - a.marks)),
)
</script>

<template>
  <input v-model="query" placeholder="search">
  <select v-model="sortBy"><option value="name">Name</option><option value="marks">Marks</option>
</select>
  <ul><li v-for="s in visible" :key="s.id">{{ s.name }} ({{ s.marks }})</li></ul>
  <p>Original order: {{ students.map((s) => s.name).join(', ') }}</p>
</template>
```



AFTER TYPING "A" AND SORTING BY MARKS: THE COMPUTED LIST CHANGED, THE SOURCE ARRAY DID NOT

6.2.7 Array change detection

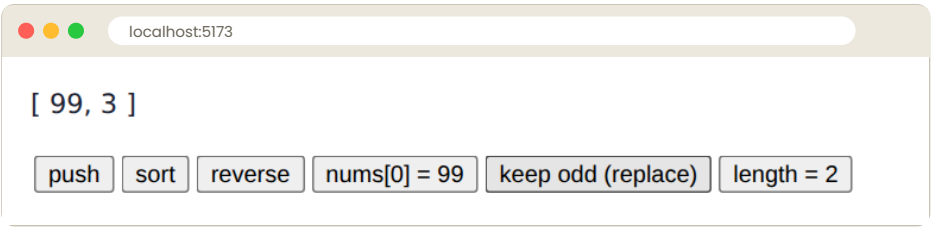
Vue wraps arrays in a Proxy, so all of these changes are detected: the **mutation methods** `push()`, `pop()`, `shift()`, `unshift()`, `splice()`, `sort()`, `reverse()`; assigning by index `arr[i] = x`; changing `length`; and **replacing** the array with the result of a non-mutating method such as `filter()`, `concat()`, `slice()` or `map()`. (Vue 2 could not detect index assignment and length changes; Vue 3 can.)

SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'
const nums = ref([3, 1, 2])
</script>

<template>
  <p>{{ nums }}</p>
  <button @click="nums.push(nums.length + 1)">push</button>
  <button @click="nums.sort((a, b) => a - b)">sort</button>
  <button @click="nums.reverse()">reverse</button>
  <button @click="nums[0] = 99">nums[0] = 99</button>
  <button @click="nums = nums.filter((n) => n % 2)">keep odd (replace)</button>
  <button @click="nums.length = 2">length = 2</button>
</template>
```



AFTER PUSH, SORT, NUMS[0] = 99 AND KEEP ODD: EVERY KIND OF CHANGE RE-RENDERED THE LIST

Mutate the array (change it in place)	Replace the array (new array)
`push pop shift unshift splice sort reverse`, `arr[i] = x`	`filter map concat slice toSorted toReversed toSpliced with`
the ref keeps the same array	assign the result: `list.value = list.value.filter(...)`

PRACTICE CHECKPOINT

A student list with search, sort by marks, a pass/fail badge and an empty-state message.

SRC/APP.VUE

HTML

```
<script setup>
import { ref, computed } from 'vue'

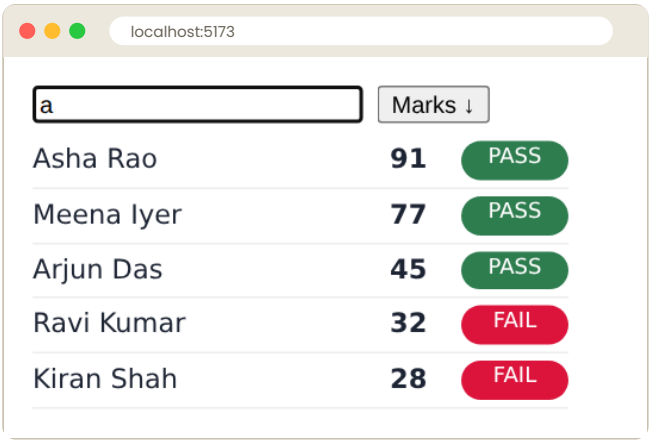
const students = ref([
  { id: 1, name: 'Asha Rao', marks: 91 }, { id: 2, name: 'Ravi Kumar', marks: 32 },
  { id: 3, name: 'Meena Iyer', marks: 77 }, { id: 4, name: 'Arjun Das', marks: 45 },
  { id: 5, name: 'Kiran Shah', marks: 28 },
])
const search = ref('')
const order = ref('desc')
const PASS = 35

const shown = computed(() => {
  const q = search.value.trim().toLowerCase()
  return students.value
    .filter((s) => s.name.toLowerCase().includes(q))
    .toSorted((a, b) => (order.value === 'desc' ? b.marks - a.marks : a.marks - b.marks))
})
</script>

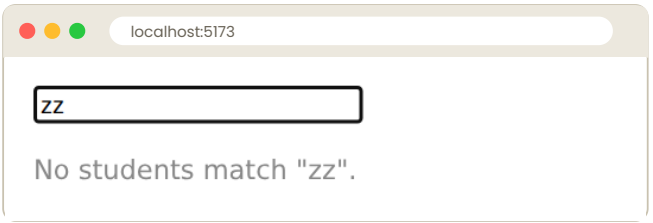
<template>
  <div class="bar">
```

```
    <input v-model="search" placeholder="Search students">
    <button @click="order = order === 'desc' ? 'asc' : 'desc'">Marks {{ order === 'desc' ? '↓' : '↑'
}}</button>
  </div>
  <ul v-if="shown.length">
    <li v-for="s in shown" :key="s.id">
      <span>{{ s.name }}</span><b>{{ s.marks }}</b>
      <em :class="s.marks >= PASS ? 'pass' : 'fail'">{{ s.marks >= PASS ? 'PASS' : 'FAIL' }}</em>
    </li>
  </ul>
  <p v-else class="empty">No students match "{{ search }}".</p>
</template>

<style scoped>
.bar { display: flex; gap: 8px; margin-bottom: 6px; }
ul { list-style: none; padding: 0; margin: 0; width: 300px; }
li { display: grid; grid-template-columns: 1fr 40px 60px; padding: 4px 0; border-bottom: 1px solid
#eee; }
em { font-style: normal; font-size: 12px; text-align: center; border-radius: 99px; color: white; }
.pass { background: #2e7d4f; } .fail { background: crimson; }
.empty { color: #888; }
</style>
```



SEARCH "A" WITH MARKS SORTED DESCENDING



THE EMPTY STATE AFTER TYPING "ZZ"

`v-if` adds or removes elements (lazy, higher toggle cost); `v-show` only toggles `display`. `v-for` loops over arrays, objects and ranges and always needs a unique, stable `:key`. Never put `v-if` and `v-for` on the same element; filter and sort through computed properties instead.

MODULE 07 · BEGINNER

Event handling and methods

Minimum duration **1 day** | Level **Beginner**

Event handling connects user actions to component logic. This module covers v-on and its @ shorthand, inline handlers versus method handlers, passing arguments and the special $event variable, the event modifiers .prevent, .stop, .once, .self, .capture and .passive, key modifiers such as .enter, .esc and .ctrl with the .exact modifier, mouse button modifiers .left, .right and .middle, and how functions declared in script setup are used by the template.

IN THIS MODULE

7.1 Listening to events
7.2 Event modifiers
7.3 Key and mouse modifiers
7.4 Functions in script setup

7.1 Listening to events

7.1.1 v-on and the @ shorthand

`v-on:event="handler"` listens to a DOM event on an element (or a custom event on a component, Module 9). Its shorthand is `@event`. Any DOM event works: `click`, `input`, `submit`, `keyup`, `mouseenter`, `scroll`…

SRC/APP.VUE

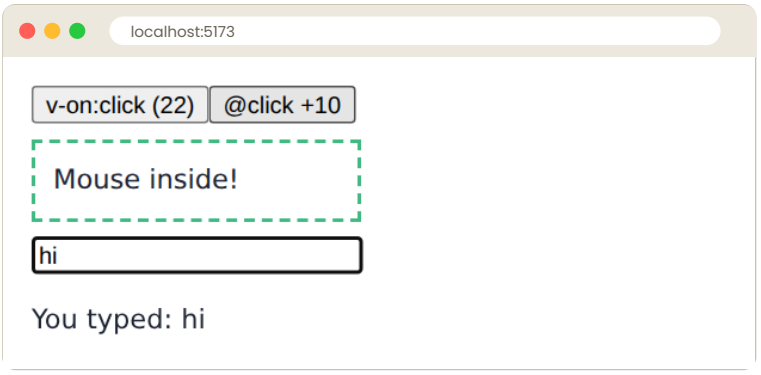
HTML

```
<script setup>
import { ref } from 'vue'

const clicks = ref(0)
const hovering = ref(false)
const typed = ref('')
</script>

<template>
  <button v-on:click="clicks++">v-on:click ({{ clicks }})</button>
  <button @click="clicks += 10">@click +10</button>
  <div class="box" @mouseenter="hovering = true" @mouseleave="hovering = false">
    {{ hovering ? 'Mouse inside!' : 'Hover me' }}
  </div>
  <input @input="typed = $event.target.value" placeholder="type here">
  <p>You typed: {{ typed }}</p>
</template>

<style>.box { margin: 8px 0; padding: 10px; border: 2px dashed #42b883; width: 160px; }</style>
```



AFTER TWO CLICKS ON EACH BUTTON, HOVERING THE BOX AND TYPING "HI"

7.1.2 Inline handlers vs method handlers

The value of `@event` can be an **inline statement** (a short piece of JavaScript run when the event fires) or a **method handler** (the name of a function). The compiler tells them apart: a plain name or property path such as `save` or `cart.add` is treated as a method and called with the event object; anything else is wrapped in a function.

Kind	Example	Use for
Inline handler	`@click="count++"`, `@click="say('hi')"`	one-line updates; calling a function with your own arguments
Method handler	`@click="increment"`	anything longer than a line; receives the native event automatically
Arrow function	`@click="() => open(id)"`	same as an inline call; rarely needed

SRC/APP.VUE HTML

```
<script setup>
function handle(event) {
  console.log('method handler got:', event.type, 'on <' + event.target.tagName.toLowerCase() + '>')
}
function say(word) {
  console.log('inline call with argument:', word)
}
</script>

<template>
  <button id="a" @click="handle">method</button>
  <button id="b" @click="say('hello')">inline</button>
</template>
```

BROWSER CONSOLE

```
method handler got: click on <button>
inline call with argument: hello
```

7.1.3 Passing arguments and $event

In an inline handler, the special variable `$event` holds the native DOM event, so you can pass both your own arguments and the event. An arrow function parameter works the same way.

SRC/APP.VUE

HTML

```
<script setup>
function vote(option, event) {
  console.log(`voted ${option}; button text = "${event.target.textContent}"`)
  event.target.disabled = true
}
</script>

<template>
  <button @click="vote('A', $event)">Option A</button>
  <button @click="(e) => vote('B', e)">Option B</button>
</template>
```

BROWSER CONSOLE

```
voted A; button text = "Option A"
voted B; button text = "Option B"
```

7.2 Event modifiers

7.2.1 .prevent, .stop, .once, .self, .capture, .passive

Instead of calling `event.preventDefault()` or `event.stopPropagation()` inside handlers, Vue offers **modifiers** written after a dot. They keep methods focused on data logic, and they can be chained.

Modifier	Effect	Typical use
`.prevent`	`event.preventDefault()`	`@submit.prevent`: no page reload; `@click.prevent` on links
`.stop`	`event.stopPropagation()`	a delete button inside a clickable card
`.once`	handler runs at most once	one-time actions
`.self`	only if `event.target` is the element itself	close a modal when clicking the backdrop, not its content
`.capture`	listen in the capture phase (outer before inner)	logging, intercepting
`.passive`	tells the browser the handler will not call `preventDefault()`	smooth `@scroll` / `@touchmove`

SRC/APP.VUE

HTML

```
<script setup>
const log = (m) => console.log(m)
</script>

<template>
  <form @submit.prevent="log('form submitted, page NOT reloaded')">
    <button id="submit">Submit</button>
  </form>

  <div @click="log('card clicked')" class="card">
    Card <button id="del" @click.stop="log('delete clicked (propagation stopped)')">Delete</button>
  </div>

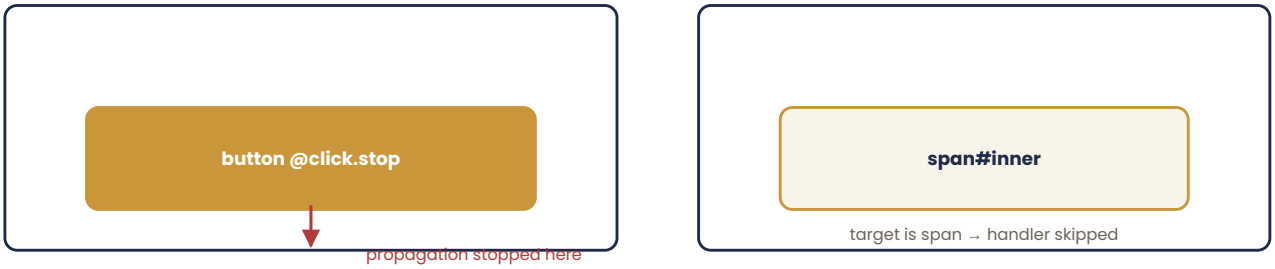
  <button id="once" @click.once="log('runs once only')">Once</button>

  <div id="outer" @click.self="log('outer itself clicked')"><span id="inner">inner text</span></div>

  <div @click.capture="log('capture: outer first')"><button id="cap" @click="log('then
inner')">Capture</button></div>
</template>
```

BROWSER CONSOLE: SUBMIT, DELETE, CARD, ONCE ×2, INNER, OUTER, CAPTURE

```
form submitted, page NOT reloaded
delete clicked (propagation stopped)
card clicked
runs once only
outer itself clicked
capture: outer first
then inner
```



.STOP STOPS THE EVENT BUBBLING TO THE CARD; .SELF IGNORES CLICKS THAT STARTED ON A CHILD

NOTE

Order matters when chaining: `@click.prevent.self` prevents the default for clicks on the element **and its children**, while `@click.self.prevent` only prevents it for clicks on the element itself.

7.3 Key and mouse modifiers

7.3.1 @keyup.enter, .esc, .ctrl, .exact

Key modifiers filter keyboard events by key: `@keyup.enter` runs only for Enter. Any key name from `event.key` can be used in kebab-case (`.page-down`, `.arrow-up`), plus the aliases `.enter`, `.tab`, `.delete` (Delete and Backspace), `.esc`, `.space`, `.up`, `.down`, `.left`, `.right`. System modifiers `.ctrl`, `.alt`, `.shift` and `.meta` require that key to be held, and `.exact` requires that **no other** system key is held.

SRC/APP.VUE

HTML

```
<script setup>
const log = (m) => console.log(m)
</script>

<template>
  <input id="t"
    @keyup.enter="log('Enter: search')"
    @keyup.esc="log('Esc: clear')"
    @keydown.ctrl.s.prevent="log('Ctrl+S: save (browser save dialog prevented)')"
    @keydown.up="log('Arrow up')">
  <button id="b" @click.ctrl.exact="log('Ctrl+click only')" @click.exact="log('plain click
only')">Click</button>
</template>
```

BROWSER CONSOLE: ENTER, ESC, CTRL+S, UP, A, CLICK, CTRL+CLICK, CTRL+SHIFT+CLICK

```
Enter: search
Esc: clear
Ctrl+S: save (browser save dialog prevented)
Arrow up
plain click only
Ctrl+click only
```

Pressing "a" logged nothing, and Ctrl+Shift+click logged nothing because `.exact` rejects the extra Shift key.

7.3.2 .left, .right, .middle

Mouse button modifiers restrict a handler to one button. `@click.right` listens to the `contextmenu` event (right-click), so combine it with `.prevent` to replace the browser's menu with your own.

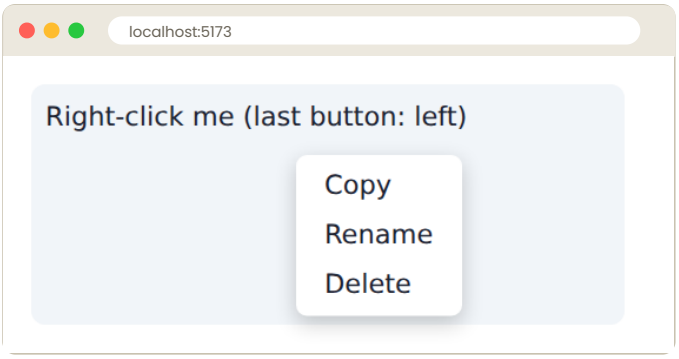
SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'
const menu = ref(null)
const last = ref('none')
function openMenu(e) { menu.value = { x: e.offsetX, y: e.offsetY } }
</script>

<template>
  <div class="area"
       @click.left="last = 'left'; menu = null"
       @click.right.prevent="openMenu"
       @click.middle="last = 'middle'">
    Right-click me (last button: {{ last }})
    <ul v-if="menu" class="menu" :style="{ left: menu.x + 'px', top: menu.y + 'px' }">
      <li>Copy</li><li>Rename</li><li>Delete</li>
    </ul>
  </div>
</template>

<style>
.area { position: relative; height: 120px; width: 320px; background: #f1f5f9; border-radius: 8px;
padding: 8px; }
.menu { position: absolute; list-style: none; margin: 0; padding: 4px 0; background: white; box-
shadow: 0 4px 12px #0003; border-radius: 6px; }
.menu li { padding: 3px 16px; }
</style>
```



A CUSTOM CONTEXT MENU, AFTER A REAL LEFT CLICK AND THEN A RIGHT CLICK AT (150, 40)

7.4 Functions in script setup

7.4.1 Declaring functions used by the template

Every function declared at the top level of `<script setup>` (with `function` or as an arrow function in a `const`) is directly usable in the template, like variables and imports. Functions can read and change refs, call other functions, be `async`, and receive arguments from the template.

SRC/APP.VUE HTML

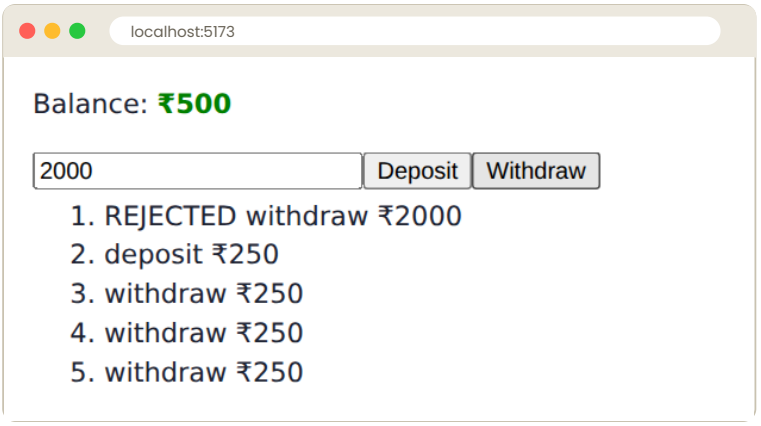
```
<script setup>
import { ref, computed } from 'vue'

const balance = ref(1000)
const history = ref([])
const amount = ref(250)

function deposit(value) {
  balance.value += value
  record('deposit', value)
}
const withdraw = (value) => {
  if (value > balance.value) return record('REJECTED withdraw', value)
  balance.value -= value
  record('withdraw', value)
}
function record(type, value) {
  history.value.unshift(`${type} ₹${value}`)
}
const status = computed(() => (balance.value < 500 ? 'low' : 'ok'))
</script>

<template>
  <p>Balance: <b :class="status">₹{{ balance }}</b></p>
  <input type="number" v-model.number="amount">
  <button @click="deposit(amount)">Deposit</button>
  <button @click="withdraw(amount)">Withdraw</button>
  <ol><li v-for="(h, i) in history" :key="i">{{ h }}</li></ol>
</template>

<style>.low { color: crimson; } .ok { color: green; }</style>
```



AFTER WITHDRAW ×3, DEPOSIT ×1 AND AN OVER-LIMIT WITHDRAW OF ₹2000

TIP

Name handlers after **what they do** (`deposit` , `removeItem`) rather than the event (`onClick`), so the same function can be reused from several events and from other functions.

PRACTICE CHECKPOINT

A counter with increment, decrement and reset, a to-do input that adds items on Enter, and a modal that closes on Escape or backdrop click.

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'

// counter
const count = ref(0)
const increment = () => count.value++
const decrement = () => { if (count.value > 0) count.value-- }
const reset = () => (count.value = 0)

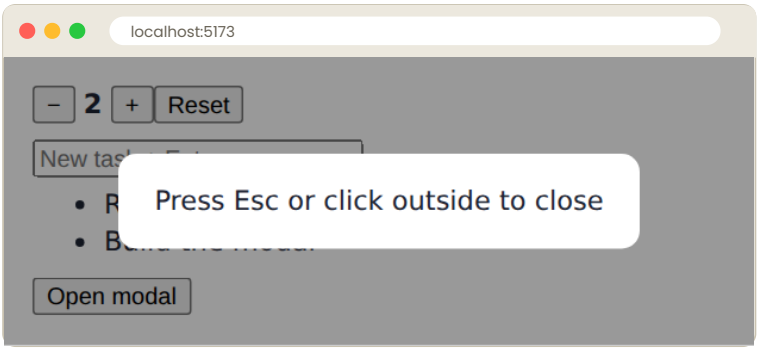
// to-do list
const newTodo = ref('')
const todos = ref([])
function addTodo() {
  const text = newTodo.value.trim()
  if (!text) return
  todos.value.push({ id: Date.now() + Math.random(), text })
  newTodo.value = ''
}

// modal
const open = ref(false)
</script>

<template>
  <section>
    <button @click="decrement">–</button> <b>{{ count }}</b> <button @click="increment">+</button>
    <button @click="reset">Reset</button>
  </section>
  <section>
    <input v-model="newTodo" @keyup.enter="addTodo" placeholder="New task + Enter">
    <ul><li v-for="t in todos" :key="t.id">{{ t.text }}</li></ul>
  </section>
  <button @click="open = true">Open modal</button>
  <div v-if="open" class="backdrop" @click.self="open = false" @keydown.esc="open = false"
```

```
tabindex="0">
    <div class="modal">Press Esc or click outside to close</div>
  </div>
</template>

<style scoped>
section { margin-bottom: 8px; } ul { margin: 4px 0; }
.backdrop { position: fixed; inset: 0; background: #0006; display: grid; place-items: center; }
.modal { background: white; padding: 16px 20px; border-radius: 10px; }
</style>
```



COUNTER +3 THEN -1, TWO TO-DOS ADDED WITH ENTER, AND THE MODAL OPEN

BROWSER CONSOLE: CLICK CONTENT, CLICK BACKDROP, REOPEN, PRESS ESC

```
modal open = false
modal open = true
modal open = false
```

`@event="..."` (`v-on`) runs an inline statement or calls a method (which receives the event); use `$event` to pass the event alongside your own arguments. Modifiers handle `preventDefault` (`.prevent`), propagation (`.stop`, `.self`, `.capture`), `.once`, `.passive`, keys (`.enter`, `.esc`, `.ctrl.exact`) and mouse buttons. Functions in `<script setup>` are available in the template directly.

MODULE 08 · BEGINNER

Forms and v-model

Minimum duration **2 days** | Level **Beginner**

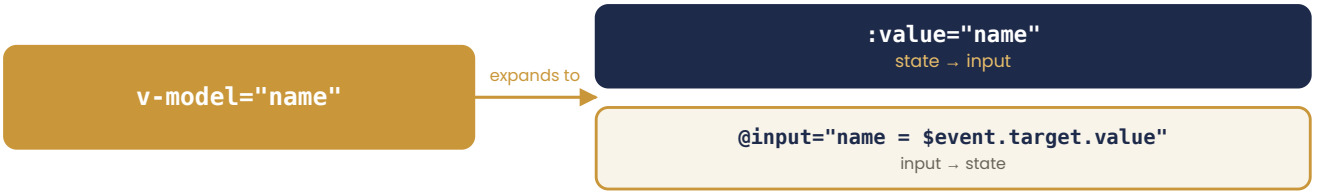
v-model creates two-way binding between form inputs and state. This module shows what v-model expands to, then uses it with text inputs and textareas, single and multiple checkboxes, radio buttons and single and multiple selects; the .lazy, .number and .trim modifiers; custom values with true-value, false-value and :value bindings (including objects); form submission with @submit.prevent and resetting the form; and manual validation with computed properties, displaying error messages and disabling submit until the form is valid.

IN THIS MODULE

8.1 v-model basics
8.2 v-model modifiers
8.3 Value bindings
8.4 Form submission
8.5 Validation

8.1 v-model basics

`v-model` keeps a form input and a piece of state in sync in **both directions**: typing updates the state, and changing the state updates the input. On a text input it is shorthand for binding the value and listening to the input event:



V-MODEL IS A VALUE BINDING PLUS AN EVENT LISTENER

`v-model` picks the right property and event for each element: `value` + `input` for text and textarea, `checked` + `change` for checkboxes and radios, `value` + `change` for selects. It ignores the HTML `value`, `checked` and `selected` attributes: the **state** is the single source of truth, so set initial values in script.

8.1.1 Text inputs and textareas

SRC/APP.VUE

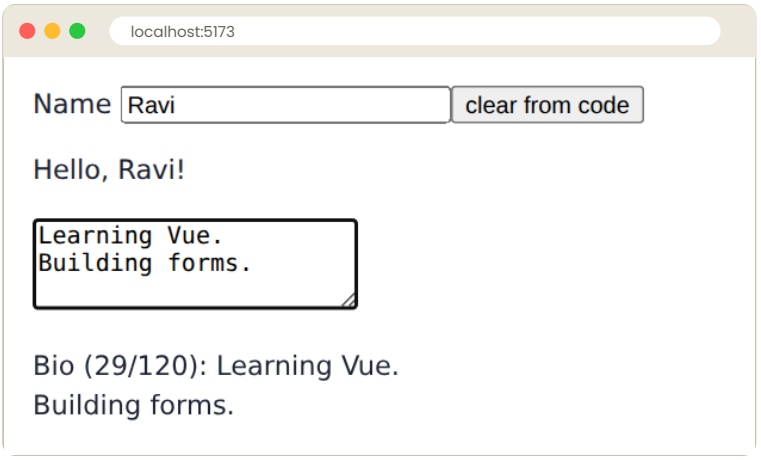
HTML

```
<script setup>
import { ref } from 'vue'

const name = ref('Asha')
const bio = ref('')
</script>

<template>
  <label>Name <input v-model="name"></label>
  <button @click="name = ''">clear from code</button>
  <p>Hello, {{ name || 'stranger' }}!</p>
  <textarea v-model="bio" rows="3" placeholder="About you"></textarea>
  <p class="pre">Bio ({{ bio.length }}/120): {{ bio }}</p>
</template>

<style>.pre { white-space: pre-line; }</style>
```



TYPED "RAVI" INTO THE NAME FIELD AND TWO LINES INTO THE TEXTAREA

WATCH OUT

Inside a `<textarea>`, interpolation `<textarea>{{ text }}</textarea>` does not work; always use `v-model`.

8.1.2 Checkboxes (single and multiple)

A single checkbox bound to a boolean toggles it. Several checkboxes bound to the **same array** add and remove their `value`s in that array.

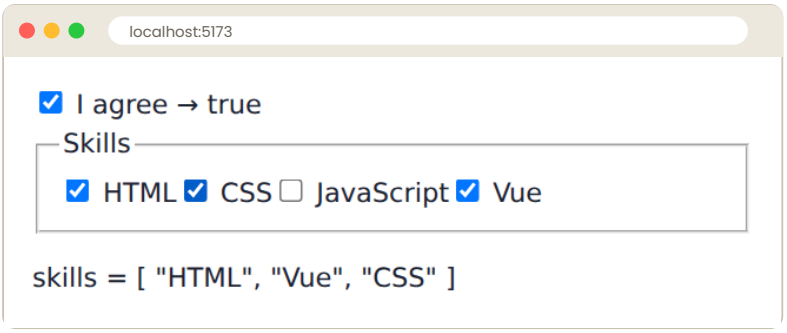
SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'

const agree = ref(false)
const skills = ref(['HTML'])
</script>

<template>
  <label><input type="checkbox" v-model="agree"> I agree</label> → {{ agree }}
  <fieldset>
    <legend>Skills</legend>
    <label><input type="checkbox" value="HTML" v-model="skills"> HTML</label>
    <label><input type="checkbox" value="CSS" v-model="skills"> CSS</label>
    <label><input type="checkbox" value="JavaScript" v-model="skills"> JavaScript</label>
    <label><input type="checkbox" value="Vue" v-model="skills"> Vue</label>
  </fieldset>
  <p>skills = {{ skills }}</p>
</template>
```



AFTER TICKING "I AGREE", VUE AND CSS: THE ARRAY KEEPS THE ORDER IN WHICH BOXES WERE TICKED

8.1.3 Radio buttons

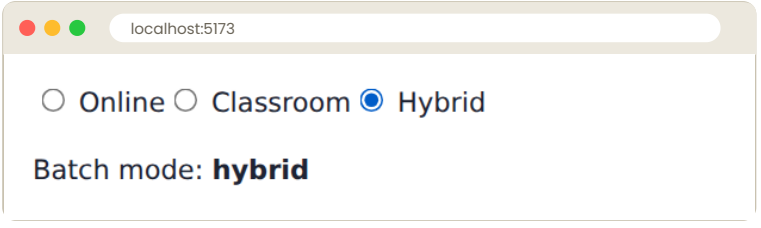
Radio buttons bound to the same variable store the `value` of the selected one.

SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'
const mode = ref('online')
</script>

<template>
  <label><input type="radio" value="online" v-model="mode"> Online</label>
  <label><input type="radio" value="classroom" v-model="mode"> Classroom</label>
  <label><input type="radio" value="hybrid" v-model="mode"> Hybrid</label>
  <p>Batch mode: <b>{{ mode }}</b></p>
</template>
```



AFTER CHOOSING HYBRID

8.1.4 Select (single and multiple)

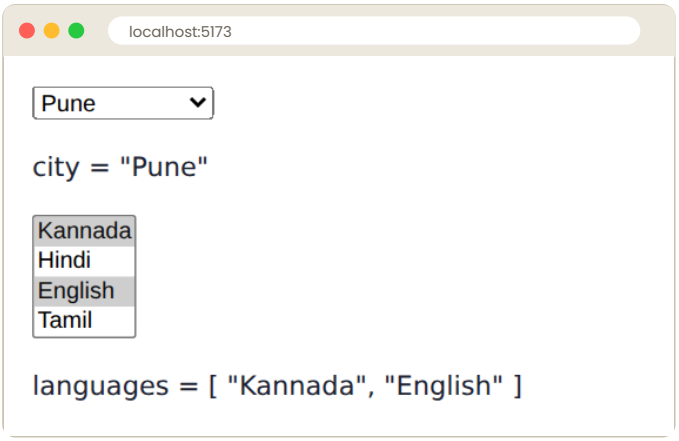
A single `<select>` stores the selected option's value. Add a disabled first option with an empty value as a placeholder. With `multiple`, the variable must be an array.

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'

const city = ref('')
const cities = ['Bengaluru', 'Mysuru', 'Pune', 'Chennai']
const languages = ref([])
</script>

<template>
  <select v-model="city">
    <option disabled value="">Choose a city</option>
    <option v-for="c in cities" :key="c">{{ c }}</option>
  </select>
  <p>city = "{{ city }}"</p>
  <select v-model="languages" multiple size="4">
    <option>Kannada</option><option>Hindi</option><option>English</option><option>Tamil</option>
  </select>
  <p>languages = {{ languages }}</p>
</template>
```



AFTER CHOOSING PUNE AND SELECTING KANNADA AND ENGLISH (CTRL+CLICK)

8.2 v-model modifiers

8.2.1 .lazy, .number, .trim

Modifier	Effect	Use
`.lazy`	syncs on the `change` event (when the field loses focus or Enter) instead of every keystroke	expensive updates, search on Enter
`.number`	converts the text with `parseFloat`; if that fails, keeps the original string	ages, quantities, prices (automatic on `type="number"`)
`.trim`	removes leading and trailing spaces	names, emails, usernames

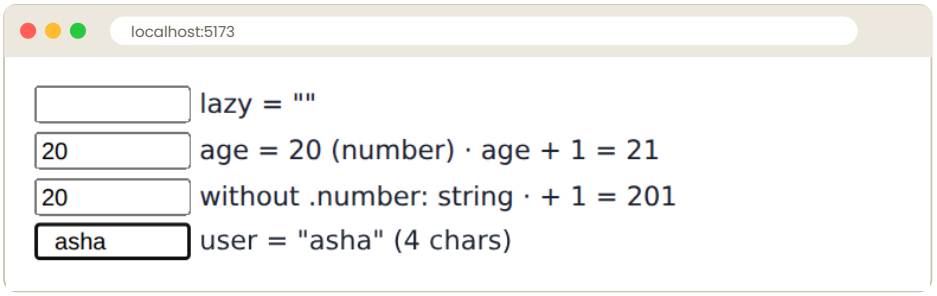
SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'

const lazy = ref('')
const age = ref(0)
const plainAge = ref(0)
const user = ref('')
</script>

<template>
  <p><input v-model.lazy="lazy"> lazy = "{{ lazy }}"</p>
  <p><input v-model.number="age"> age = {{ age }} ({{ typeof age }}) · age + 1 = {{ age + 1 }}</p>
  <p><input v-model="plainAge"> without .number: {{ typeof plainAge }} · + 1 = {{ plainAge + 1 }}</p>
  <p><input v-model.trim="user"> user = "{{ user }}" ({{ user.length }} chars)</p>
</template>
```



TYPED "VUE" (STILL FOCUSED, SO .LAZY HAS NOT SYNCED), "20" TWICE AND " ASHA "

BROWSER CONSOLE: .LAZY SYNCS ON BLUR

```
(typed "Vue": nothing yet)
lazy synced: "Vue"
```

8.3 Value bindings

8.3.1 Custom values for checkboxes, radios and options

By default a checkbox gives `true` / `false` and radios and options give strings. Bind `true-value` / `false-value` (Vue-specific attributes) on a checkbox, and `:value` on radios and options, to store any value: numbers, booleans or even **objects**.

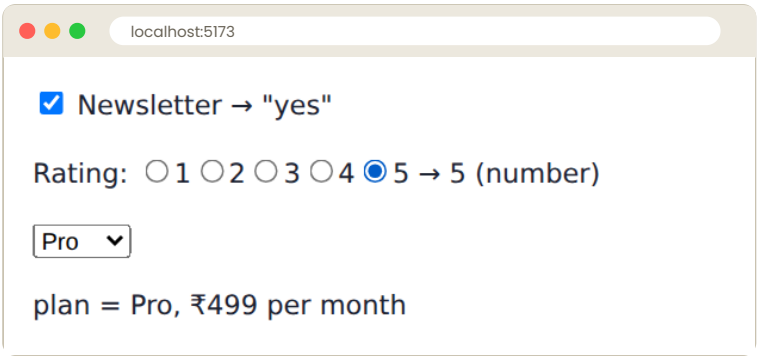
SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'

const newsletter = ref('no')
const rating = ref(3)
const plans = [
  { id: 1, name: 'Basic', price: 0 },
  { id: 2, name: 'Pro', price: 499 },
  { id: 3, name: 'Team', price: 1999 },
]
const plan = ref(plans[0])
</script>

<template>
  <label><input type="checkbox" v-model="newsletter" true-value="yes" false-value="no">
Newsletter</label>
  → "{{ newsletter }}"
  <p>
    Rating:
    <label v-for="n in 5" :key="n"><input type="radio" :value="n" v-model="rating">{{ n }}</label>
    → {{ rating }} ({{ typeof rating }})
  </p>
  <select v-model="plan">
    <option v-for="p in plans" :key="p.id" :value="p">{{ p.name }}</option>
  </select>
  <p>plan = {{ plan.name }}, ₹{{ plan.price }} per month</p>
</template>
```



AFTER TICKING NEWSLETTER, CHOOSING RATING 5 AND THE PRO PLAN (AN OBJECT)

8.4 Form submission

8.4.1 @submit.prevent

Wrap fields in a `<form>` and handle `@submit.prevent`. Submitting then works with the Enter key and the submit button, the browser's built-in validation attributes (`required`, `type="email"`, `minlength`) still apply, and `.prevent` stops the page from reloading.

SRC/APP.VUE

HTML

```
<script setup>
import { reactive } from 'vue'

const form = reactive({ email: '', password: '' })
function login() {
  console.log('submitting', JSON.stringify(form))
}
</script>

<template>
  <form @submit.prevent="login">
    <input v-model.trim="form.email" type="email" placeholder="Email" required>
    <input v-model="form.password" type="password" placeholder="Password" required minlength="6">
    <button>Log in</button>
  </form>
</template>
```

BROWSER CONSOLE: SUBMITTED WITH THE ENTER KEY

```
submitting {"email":"asha@lbf.in","password":"secret123"}
```

8.4.2 Resetting the form

Keep the initial values in one place, then reset by assigning a fresh copy. With `reactive()`, use `Object.assign(form, initial())` (you cannot replace the reactive object itself); with a `ref`, assign a new object to `.value`.

TWO RESET PATTERNS

JavaScript

```
const initial = () => ({ name: '', email: '', course: '', terms: false })

const form = reactive(initial())                  // reactive object
const reset = () => Object.assign(form, initial())

const form2 = ref(initial())                      // or a ref
const reset2 = () => (form2.value = initial())
```

8.5 Validation

8.5.1 Manual validation with computed properties

Validation rules are derived state, so they belong in **computed** properties: one computed object holds an error message per field (or an empty string), and another says whether the whole form is valid. They update as the user types.

8.5.2 Displaying error messages

Showing errors before the user has touched a field is unfriendly. Track which fields were **touched** (on blur) and whether submit was attempted, and show a field's error only then.

8.5.3 Disabling submit until valid

SRC/APP.VUE

HTML

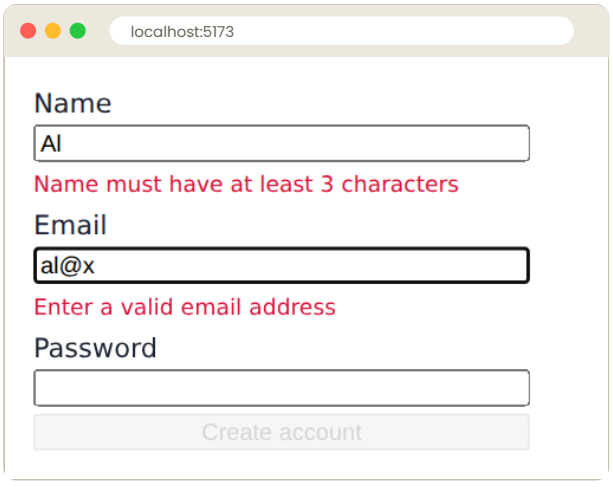
```
<script setup>
import { reactive, computed } from 'vue'

const form = reactive({ name: '', email: '', password: '' })
const touched = reactive({ name: false, email: false, password: false })

const errors = computed(() => ({
  name: form.name.trim().length < 3 ? 'Name must have at least 3 characters' : '',
  email: !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(form.email) ? 'Enter a valid email address' : '',
  password: form.password.length < 8 ? 'Password needs 8+ characters'
          : !/\d/.test(form.password) ? 'Password needs a digit' : '',
}))
const isValid = computed(() => Object.values(errors.value).every((e) => !e))
</script>

<template>
  <form @submit.prevent novalidate>
    <label>Name <input v-model.trim="form.name" @blur="touched.name = true"></label>
    <small v-if="touched.name && errors.name">{{ errors.name }}</small>
    <label>Email <input v-model.trim="form.email" @blur="touched.email = true"></label>
    <small v-if="touched.email && errors.email">{{ errors.email }}</small>
    <label>Password <input type="password" v-model="form.password" @blur="touched.password = true">
</label>
    <small v-if="touched.password && errors.password">{{ errors.password }}</small>
    <button :disabled="!isValid">Create account</button>
  </form>
</template>

<style scoped>
form { display: grid; gap: 4px; width: 280px; }
label { display: grid; } small { color: crimson; } button:disabled { opacity: .5; }
</style>
```



NAME "AL" AND EMAIL "AL@X" WERE TYPED AND LEFT (BLURRED); THE PASSWORD WAS NOT TOUCHED YET, SO ITS ERROR IS HIDDEN AND SUBMIT IS DISABLED

PRACTICE CHECKPOINT

A student registration form with all input types, validation messages, a live preview of entered data and a reset button.

SRC/APP.VUE

HTML

```
<script setup>
import { reactive, computed, ref } from 'vue'

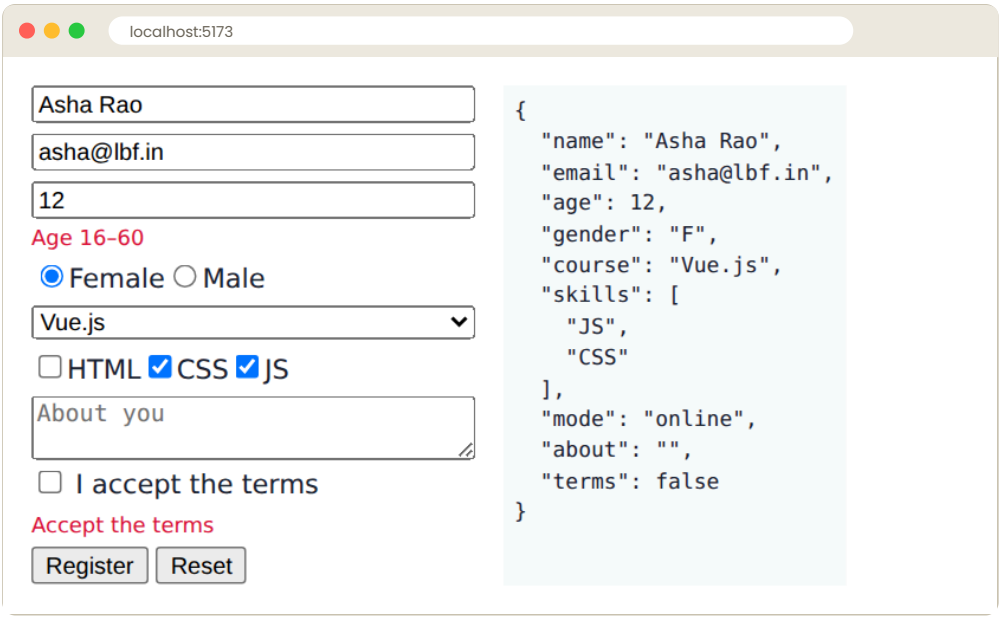
const initial = () => ({ name: '', email: '', age: null, gender: '', course: '', skills: [], mode:
'online', about: '', terms: false })
const form = reactive(initial())
const submitted = ref(false)

const errors = computed(() => ({
  name: form.name.length < 3 ? 'At least 3 characters' : '',
  email: /^\S+@\S+\.\S+$/.test(form.email) ? '' : 'Valid email required',
  age: form.age >= 16 && form.age <= 60 ? '' : 'Age 16–60',
  course: form.course ? '' : 'Pick a course',
  terms: form.terms ? '' : 'Accept the terms',
}))
const valid = computed(() => Object.values(errors.value).every((e) => !e))
const show = (f) => (submitted.value ? errors.value[f] : '')

function register() {
  submitted.value = true
  if (valid.value) console.log('Registered', { ...form })
}
function reset() { Object.assign(form, initial()); submitted.value = false }
</script>

<template>
  <div class="wrap">
    <form @submit.prevent="register">
      <input v-model.trim="form.name" placeholder="Full name"><small>{{ show('name') }}</small>
      <input v-model.trim="form.email" placeholder="Email"><small>{{ show('email') }}</small>
      <input v-model.number="form.age" type="number" placeholder="Age"><small>{{ show('age') }}
</small>
      <div><label><input type="radio" value="F" v-model="form.gender">Female</label>
           <label><input type="radio" value="M" v-model="form.gender">Male</label></div>
      <select v-model="form.course">
        <option value="" disabled>Course</option><option>Vue.js</option><option>React</option>
      </select><small>{{ show('course') }}</small>
      <div><label v-for="s in ['HTML', 'CSS', 'JS']" :key="s"><input type="checkbox" :value="s" v-
model="form.skills">{{ s }}</label></div>
      <textarea v-model="form.about" rows="2" placeholder="About you"></textarea>
      <label><input type="checkbox" v-model="form.terms"> I accept the terms</label><small>{{
show('terms') }}</small>
      <div><button>Register</button> <button type="button" @click="reset">Reset</button></div>
    </form>
    <pre>{{ form }}</pre>
  </div>
</template>

<style scoped>
.wrap { display: flex; gap: 16px; } form { display: grid; gap: 3px; width: 250px; }
small { color: crimson; font-size: 12px; min-height: 0; } pre { margin: 0; font-size: 12px;
background: #f8fafc; padding: 6px; }
</style>
```



FILLED IN PARTLY AND REGISTER CLICKED: ERRORS APPEAR FOR THE AGE AND TERMS, AND THE PREVIEW SHOWS THE LIVE DATA

`v-model` = value binding + input/change listener, adapted to each element: strings for text, booleans or arrays for checkboxes, the chosen value for radios and selects (arrays with `multiple`). `.lazy`, `.number` and `.trim` adjust the value; `:value`, `true-value` and `false-value` store custom values. Submit with `@submit.prevent`, validate with computed properties, show errors after touch/submit, and disable submit while invalid.

LEVEL 2

Intermediate level

Outcome: create reusable components with props, emits and slots, use watchers, lifecycle hooks, composables, Vue Router, Pinia and APIs.

7 MODULES · 16 DAYS MINIMUM

Module 09 · Components: props, events and slots	**3 days**
Module 10 · Watchers and lifecycle hooks	**2 days**
Module 11 · Advanced component features	**2 days**
Module 12 · Composables	**2 days**
Module 13 · Routing with Vue Router	**3 days**
Module 14 · State management with Pinia	**2 days**
Module 15 · Working with APIs and form validation	**2 days**

MODULE 09 · INTERMEDIATE

Components: props, events and slots

Minimum duration **3 days** | Level **Intermediate**

Components split the UI into reusable pieces that communicate through props, events and slots. This module covers defining and registering components (local and global), naming conventions and the component tree; props with defineProps(), types, required props, defaults, validators, reactive props destructure and the one-way data flow rule; emitting events with defineEmits(), payloads and validation; slots with fallback content, named slots, scoped slots and conditional slots with $slots; and fallthrough attributes, inheritAttrs: false and useAttrs().

IN THIS MODULE

9.1 Component basics
9.2 Props
9.3 Emitting events
9.4 Slots
9.5 Fallthrough attributes

9.1 Component basics

9.1.1 Defining and registering components

A component is defined in its own `.vue` file. Before a template can use it, the component must be **registered**, which tells Vue what a tag such as `<AlertBox>` means. In `<script setup>`, importing the component registers it automatically for that file.

SRC/COMPONENTS/ALERTBOX.VUE — HTML

```
<script setup>
import { ref } from 'vue'
const dismissed = ref(false)
</script>

<template>
  <div v-if="!dismissed" class="alert">
    Fees are due on Friday.
    <button @click="dismissed = true">x</button>
  </div>
</template>

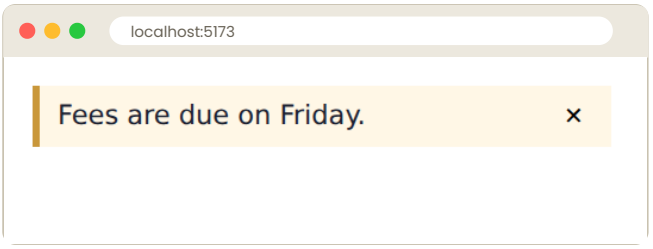
<style scoped>
.alert { background: #fff7e6; border-left: 4px solid #C9973A; padding: 6px 10px; margin: 4px 0;
display: flex; justify-content: space-between; width: 300px; }
button { border: 0; background: none; cursor: pointer; }
</style>
```

SRC/APP.VUE

HTML

```
<script setup>
import AlertBox from './components/AlertBox.vue'
</script>

<template>
  <AlertBox />
  <AlertBox />
</template>
```



TWO INSTANCES; THE SECOND WAS DISMISSED. EACH INSTANCE HAS ITS OWN STATE.

9.1.2 Local vs global registration

Local registration (importing in the file that uses it) is the default and recommended: dependencies are explicit, unused components are removed from the build (tree-shaking), and editors can follow the import. **Global** registration with `app.component()` makes a component available in every template without importing; use it only for a few base components used everywhere (buttons, icons).

SRC/MAIN.JS

JavaScript

```
// main.js — global registration
import { createApp } from 'vue'
import App from './App.vue'
import BaseButton from './components/BaseButton.vue'

createApp(App)
  .component('BaseButton', BaseButton)   // usable in every template
  .mount('#app')
```

	Local (import in script setup)	Global (app.component)
Where available	only in the importing component	every component of the app
Tree-shaking	yes: unused components are dropped	no: always in the bundle
Dependency clarity	explicit import line	hidden; hard to trace in large apps
Use for	almost everything	a handful of base components

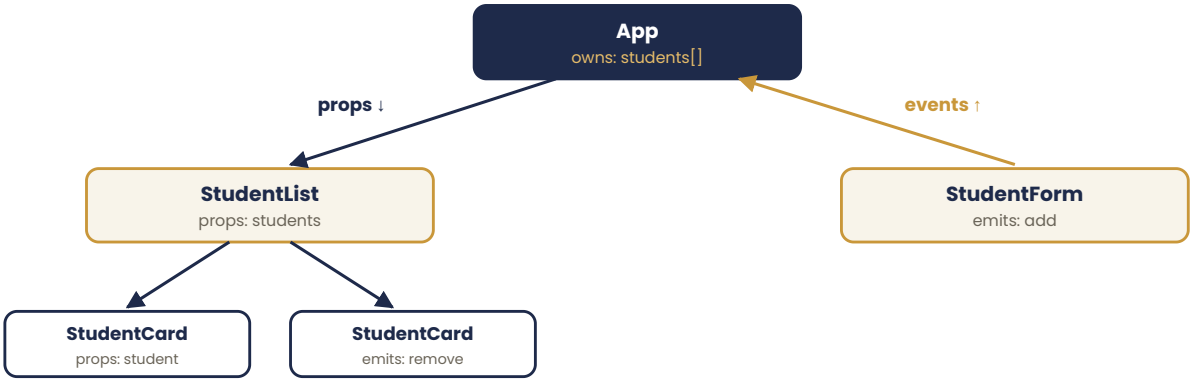
9.1.3 Naming conventions (PascalCase)

- File names and component names use **PascalCase**: `StudentCard.vue`, `TheHeader.vue`.
- Use **multi-word** names (`TodoItem`, not `Item`) so they never clash with present or future HTML elements.
- Prefix app-wide presentational components with `Base` (`BaseButton`, `BaseInput`) and single-instance layout components with `The` (`TheNavbar`).
- In templates of `.vue` files write `<StudentCard />`; `<student-card></student-card>` also works and is required in in-DOM (CDN) templates, because HTML is case-insensitive.

- Prop names are camelCase in script (`studentName`) and may be written kebab-case in templates (`student-name="..."`).

9.1.4 The component tree

Components nest inside each other to form a **tree** with `App.vue` at the root. Data flows **down** the tree through props; notifications flow **up** through events. This model keeps it clear who owns each piece of state.



PROPS FLOW DOWN, EVENTS FLOW UP

9.2 Props

9.2.1 defineProps()

Props are the inputs of a component. Declare them with the `defineProps()` macro (a compiler macro: it needs no import and only works in `<script setup>`). The parent passes them as attributes: static values as strings, anything else with `:`. In the template, props are used directly by name; in script, through the object returned by `defineProps()`.

SRC/COMPONENTS/STUDENTCARD.VUE

HTML

```
<script setup>
const props = defineProps(['name', 'marks', 'highlight'])
console.log(`StudentCard for ${props.name}: marks = ${props.marks} (${typeof props.marks})`)
</script>

<template>
  <div class="card" :class="{ star: highlight }">
    <b>{{ name }}</b> scored {{ marks }}
  </div>
</template>

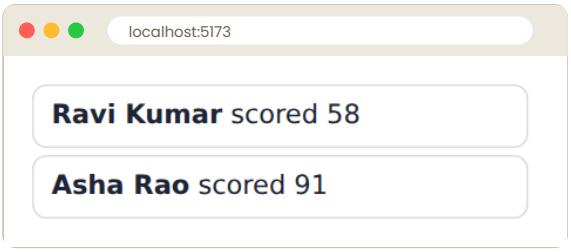
<style scoped>
.card { border: 1px solid #ddd; border-radius: 8px; padding: 6px 10px; margin: 4px 0; width: 260px; }
.star { border-color: #C9973A; background: #fff7e6; }
</style>
```

SRC/APP.VUE

HTML

```
<script setup>
import StudentCard from './components/StudentCard.vue'
const top = { name: 'Asha Rao', marks: 91 }
</script>

<template>
  <StudentCard name="Ravi Kumar" :marks="58" />
  <StudentCard :name="top.name" :marks="top.marks" highlight />
</template>
```



TWO CARDS; THE SECOND RECEIVES A BOUND OBJECT PROPERTY AND A BOOLEAN ATTRIBUTE

NOTE

`marks="58"` (without the colon) would pass the **string** "58". Use `:marks="58"` to pass a number, and `:` for every non-string value: numbers, booleans, arrays, objects, variables.

9.2.2 Prop types, required props and defaults

The object syntax declares a **type** (a native constructor such as `String`, `Number`, `Boolean`, `Array`, `Object`, `Function`, `Date`, or an array of them), whether the prop is **required**, and a **default**. Defaults for objects and arrays must be **factory functions**, so each instance gets its own copy. A `Boolean` prop that is absent becomes `false`, and present with no value becomes `true`.

SRC/COMPONENTS/COURSEBADGE.VUE

HTML

```
<script setup>
const props = defineProps({
  title: { type: String, required: true },
  hours: { type: Number, default: 30 },
  featured: Boolean,                              // absent -> false
  tags: { type: Array, default: () => ['general'] }, // factory function
  level: { type: [String, Number], default: 'Beginner' },
})
</script>

<template>
  <p :class="{ featured }">
    {{ featured ? '★' : '' }} <b>{{ title }}</b> · {{ hours }} h · {{ level }} · {{ tags.join(', ') }}
  </p>
</template>

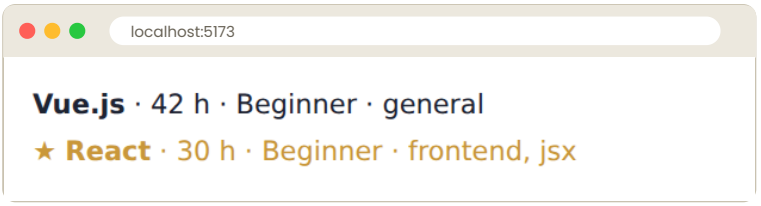
<style scoped>
p { margin: 4px 0; } .featured { color: #C9973A; }
</style>
```

SRC/APP.VUE

HTML

```
<script setup>
import CourseBadge from './components/CourseBadge.vue'
</script>

<template>
  <CourseBadge title="Vue.js" :hours="42" />
  <CourseBadge title="React" featured :tags="['frontend', 'jsx']" />
</template>
```



MISSING PROPS TOOK THEIR DEFAULTS; THE BOOLEAN PROP IS TRUE JUST BY BEING PRESENT

9.2.3 Prop validation

With types, `required` and a custom `validator` function, Vue checks every prop at run time in development and prints a warning that names the component. Validation is skipped in production builds, so it is a developer aid, not a security check.

SRC/COMPONENTS/RATINGSTARS.VUE

HTML

```
<script setup>
defineProps({
  value: {
    type: Number,
    required: true,
    validator: (v) => v >= 0 && v <= 5,
  },
})
</script>

<template><span>{{ '★'.repeat(value || 0) }}</span></template>
```

SRC/APP.VUE

HTML

```
<script setup>
import RatingStars from './components/RatingStars.vue'
</script>

<template>
  <RatingStars :value="4" />
  <RatingStars value="5" />
  <RatingStars :value="9" />
  <RatingStars />
</template>
```

BROWSER CONSOLE

```
[Vue warn]: Invalid prop: type check failed for prop "value". Expected Number with value 5, got String
with value "5".
  at <RatingStars value="5" >
  at <App>
[Vue warn]: Invalid prop: custom validator check failed for prop "value".
  at <RatingStars value=9 >
  at <App>
[Vue warn]: Missing required prop: "value"
  at <RatingStars>
  at <App>
```

9.2.4 Reactive props destructure

Since Vue 3.5, props can be **destructured** directly from `defineProps()` with JavaScript default values. The compiler rewrites every use of a destructured prop to `props.x`, so it stays reactive. When passing a destructured prop to `watch()` or a composable, wrap it in a getter: `() => count`.

SRC/COMPONENTS/PRICETAG.VUE — HTML

```
<script setup>
import { computed, watch } from 'vue'

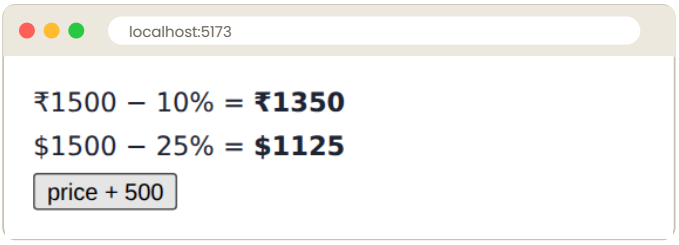
const { amount, discount = 10, currency = '₹' } = defineProps({
  amount: Number,
  discount: Number,
  currency: String,
})
const final = computed(() => amount - (amount * discount) / 100)   // reactive
watch(() => amount, (n, o) => console.log(`amount ${o} -> ${n}`))   // getter
</script>

<template>
  <p>{{ currency }}{{ amount }} − {{ discount }}% = <b>{{ currency }}{{ final }}</b></p>
</template>
```

SRC/APP.VUE — HTML

```
<script setup>
import { ref } from 'vue'
import PriceTag from './components/PriceTag.vue'
const price = ref(1000)
</script>

<template>
  <PriceTag :amount="price" />
  <PriceTag :amount="price" :discount="25" currency="$" />
  <button @click="price += 500">price + 500</button>
</template>
```



AFTER ONE CLICK: BOTH DESTRUCTURED-PROP COMPUTEDS UPDATED

SRC/APP.VUE HTML

```
<script setup>
import { watch } from 'vue'
const { count } = defineProps({ count: Number })
watch(count, (v) => console.log(v))     // passes the current number, not a source
</script>

<template><p>{{ count }}</p></template>
```

VITE DEV SERVER main.js

```
$ npm run dev
```

TERMINAL: npm run dev

```
[plugin:vite:vue] [vue/compiler-sfc] `count` is a destructured prop and should not be passed directly
to watch(). Pass a getter `() => count` instead.

/src/App.vue:4:6
  4 |  watch(count, (v) => console.log(v))     // passes the current number, not a source
    |        ^
```

The compiler catches the mistake: watch(count, …) would receive a plain number. Use watch(() => count, …).

9.2.5 One-way data flow: never mutate props

Props flow **down**: when the parent's value changes, the child receives the new value. The child must **never change** a prop; Vue makes props read-only and warns. If a child needs to change the value, it either (1) keeps a **local copy** initialised from the prop, (2) derives a **computed** from it, or (3) **emits an event** asking the parent to change it (9.3, and `defineModel()` in Module 11).

SRC/COMPONENTS/COUNTER.VUE HTML

```
<script setup>
import { ref } from 'vue'
const props = defineProps({ start: Number })

function badIncrement() {
  props.start++                    // WRONG: props are read-only
}
const count = ref(props.start)     // RIGHT: local state initialised from the prop
</script>

<template>
  <button id="bad" @click="badIncrement">bad: {{ start }}</button>
  <button id="good" @click="count++">good: {{ count }}</button>
</template>
```

SRC/APP.VUE HTML

```
<script setup>
import Counter from './components/Counter.vue'
</script>
<template><Counter :start="5" /></template>
```

BROWSER CONSOLE

```
[Vue warn]: Set operation on key "start" failed: target is readonly.
```

WATCH OUT

Objects and arrays are passed **by reference**, so a child *can* technically do `props.student.marks = 0` without a warning, and it would silently change the parent's data. Treat nested prop data as read-only too and emit an event instead.

9.3 Emitting events

9.3.1 defineEmits()

A child tells its parent that something happened by **emitting a custom event**. Declare the events with the `defineEmits()` macro, which returns an `emit` function; in the template, the shortcut `$emit` also works. The parent listens with `@event-name`, exactly like a DOM event. Component events do not bubble: only the direct parent can listen.

SRC/COMPONENTS/LIKEBUTTON.VUE — HTML

```
<script setup>
const emit = defineEmits(['like', 'reset'])
</script>

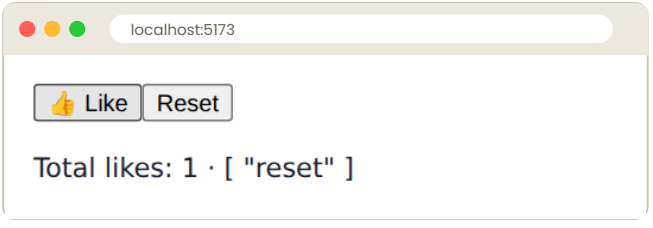
<template>
  <button @click="emit('like')">👍 Like</button>
  <button @click="$emit('reset')">Reset</button>
</template>
```

SRC/APP.VUE — HTML

```
<script setup>
import { ref } from 'vue'
import LikeButton from './components/LikeButton.vue'

const likes = ref(0)
const log = ref([])
</script>

<template>
  <LikeButton @like="likes++" @reset="likes = 0; log.push('reset')" />
  <p>Total likes: {{ likes }} · {{ log }}</p>
</template>
```



AFTER THREE LIKES, A RESET AND ONE MORE LIKE

9.3.2 Emitting with payloads

Extra arguments to `emit()` become the arguments of the parent's handler. In an inline parent handler they are available as `$event` (the first argument only).

SRC/COMPONENTS/TODOITEM.VUE HTML

```
<script setup>
const props = defineProps({ todo: { type: Object, required: true } })
const emit = defineEmits(['toggle', 'remove'])
</script>

<template>
  <div class="item">
    <input type="checkbox" :checked="todo.done" @change="emit('toggle', todo.id,
$event.target.checked)">
    <span :class="{ done: todo.done }">{{ todo.text }}</span>
    <button @click="emit('remove', todo.id)">delete</button>
  </div>
</template>

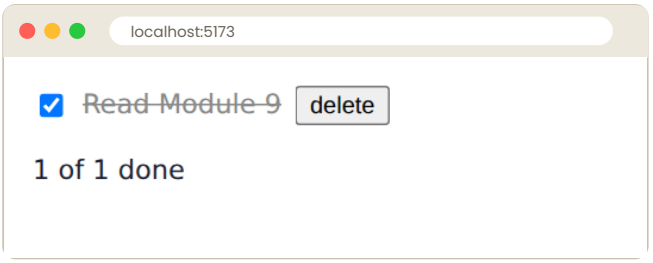
<style scoped>.done { text-decoration: line-through; color: #888; } .item { display: flex; gap: 8px; }
</style>
```

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
import TodoItem from './components/TodoItem.vue'

const todos = ref([{ id: 1, text: 'Read Module 9', done: false }, { id: 2, text: 'Build TodoItem',
done: true }])
function toggle(id, done) { todos.value.find((t) => t.id === id).done = done }
function remove(id) { todos.value = todos.value.filter((t) => t.id !== id) }
</script>

<template>
  <TodoItem v-for="t in todos" :key="t.id" :todo="t" @toggle="toggle" @remove="remove($event)" />
  <p>{{ todos.filter((t) => t.done).length }} of {{ todos.length }} done</p>
</template>
```



TICKED THE FIRST ITEM (PAYLOAD: ID AND CHECKED) AND DELETED THE SECOND (PAYLOAD: ID)

9.3.3 Validating emitted events

With the object syntax, each event can have a **validator** that receives the emitted arguments and returns `true` if they are valid; otherwise Vue warns. Declaring events is also how Vue knows an `@event` listener is a component event and not a native one (see 9.5). Emitting an undeclared event produces a warning too:

SRC/COMPONENTS/SCOREINPUT.VUE HTML

```
<script setup>
const emit = defineEmits({
  save: (score) => Number.isInteger(score) && score >= 0 && score <= 100,
  cancel: null,                    // declared, no validation
})
</script>

<template>
  <button id="ok" @click="emit('save', 87)">save 87</button>
  <button id="bad" @click="emit('save', 120)">save 120</button>
  <button id="oops" @click="emit('submit')">undeclared</button>
</template>
```

SRC/APP.VUE HTML

```
<script setup>
import ScoreInput from './components/ScoreInput.vue'
const save = (s) => console.log('parent received score', s)
</script>
<template><ScoreInput @save="save" /></template>
```

BROWSER CONSOLE

```
parent received score 87
[Vue warn]: Invalid event arguments: event validation failed for event "save".
parent received score 120
[Vue warn]: Component emitted event "submit" but it is neither declared in the emits option nor as an
"onSubmit" prop.
```

NOTE

A failing validator only logs a warning; the event is still delivered (the parent received 120). Validation helps you catch bugs during development.

9.4 Slots

9.4.1 Default slot

Props pass **data** to a child; **slots** pass **template content**. The child marks where the content goes with a `<slot>` element (the slot **outlet**), and whatever the parent writes between the component's tags is rendered there. Slot content has access to the **parent's** data, because it is written in the parent's template.

SRC/COMPONENTS/FANCYBUTTON.VUE HTML

```
<template>
  <button class="fancy">
    <slot></slot>  <!-- slot outlet: the parent's content goes here -->
  </button>
</template>

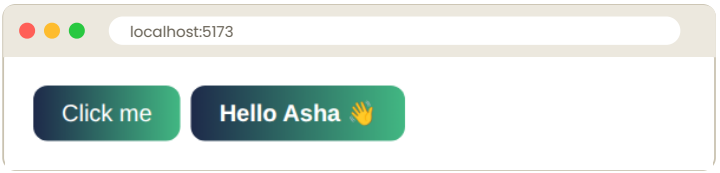
<style scoped>
.fancy { background: linear-gradient(90deg, #1E2A4A, #42b883); color: white; border: 0; padding: 8px
16px; border-radius: 8px; margin-right: 6px; }
</style>
```

SRC/APP.VUE

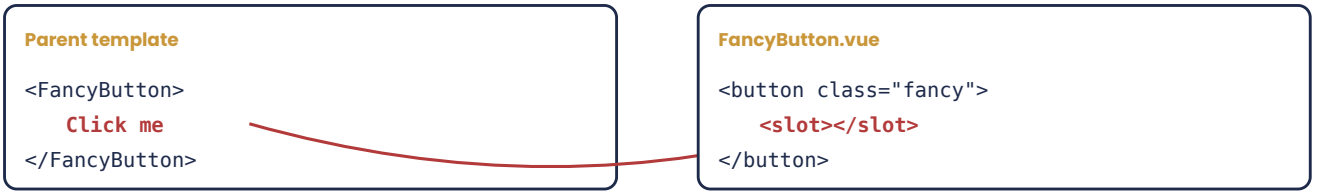
HTML

```
<script setup>
import FancyButton from './components/FancyButton.vue'
const user = 'Asha'
</script>

<template>
  <FancyButton>Click me</FancyButton>
  <FancyButton><b>Hello {{ user }}</b> <span>👋</span></FancyButton>
</template>
```



PLAIN TEXT AND A MIX OF ELEMENTS AND PARENT DATA PASSED THROUGH THE DEFAULT SLOT



SLOT CONTENT WRITTEN IN THE PARENT IS RENDERED AT THE CHILD'S SLOT OUTLET

9.4.2 Fallback content

Content between the `<slot>` tags is **fallback** (default) content, shown only when the parent provides nothing.

SRC/COMPONENTS/SUBMITBUTTON.VUE

HTML

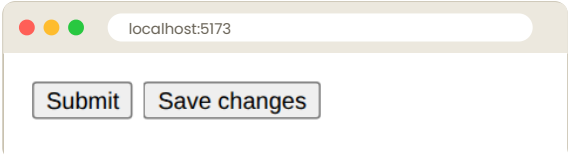
```
<template>
  <button type="submit"><slot>Submit</slot></button>
</template>
```

SRC/APP.VUE

HTML

```
<script setup>
import SubmitButton from './components/SubmitButton.vue'
</script>

<template>
  <SubmitButton />
  <SubmitButton>Save changes</SubmitButton>
</template>
```



FALLBACK "SUBMIT" IN THE FIRST BUTTON, THE PARENT'S TEXT IN THE SECOND

9.4.3 Named slots and #name shorthand

A component can have several outlets distinguished by a `name` attribute; an outlet without a name is the `default` slot. The parent targets a named slot with a `<template v-slot:name>`, shortened to `<template #name>`. Content not inside a named template goes to the default slot.

SRC/COMPONENTS/BASECARD.VUE HTML

```
<template>
  <article class="card">
    <header><slot name="header">Untitled</slot></header>
    <main><slot></slot></main>
    <footer><slot name="footer"></slot></footer>
  </article>
</template>

<style scoped>
.card { width: 260px; border: 1px solid #ddd; border-radius: 10px; overflow: hidden; }
header { background: #1E2A4A; color: white; padding: 6px 12px; } header :slotted(h3) { margin: 0;
font-size: 16px; }
main { padding: 6px 12px; } footer { padding: 6px 12px; background: #f8fafc; }
</style>
```

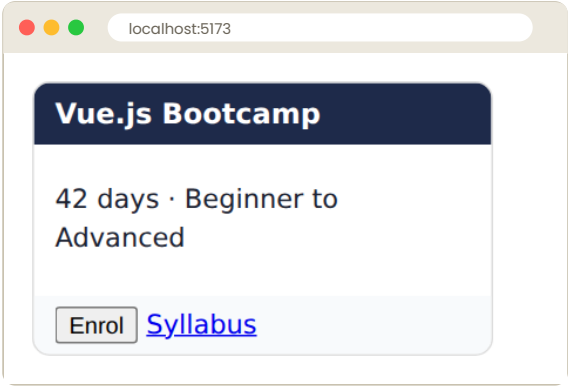
SRC/APP.VUE HTML

```
<script setup>
import BaseCard from './components/BaseCard.vue'
</script>

<template>
  <BaseCard>
    <template #header>
      <h3>Vue.js Bootcamp</h3>
    </template>

    <p>42 days · Beginner to Advanced</p>     <!-- default slot -->

    <template #footer>
      <button>Enrol</button> <a href="#">Syllabus</a>
    </template>
  </BaseCard>
</template>
```



ONE CARD COMPONENT, THREE SLOTS FILLED BY THE PARENT

9.4.4 Scoped slots

Normally slot content can only use the parent's data. A **scoped slot** lets the child pass data **to** the slot content: the child binds attributes on the `<slot>` ("slot props"), and the parent receives them with `v-`

`slot="props"` or `#name="props"` (usually destructured). This lets the child own the data and logic while the parent decides how each item looks.

SRC/COMPONENTS/FANCYLIST.VUE HTML

```
<script setup>
defineProps({ items: Array })
</script>

<template>
  <ul>
    <li v-for="(item, index) in items" :key="item.id">
      <slot name="item" :item="item" :index="index">{{ item }}</slot>
    </li>
  </ul>
</template>

<style scoped>li { padding: 3px 0; border-bottom: 1px dashed #ddd; list-style: none; } ul { padding:
0; width: 260px; }</style>
```

SRC/APP.VUE HTML

```
<script setup>
import FancyList from './components/FancyList.vue'
const courses = [
  { id: 1, name: 'Vue.js', seats: 4 },
  { id: 2, name: 'React', seats: 0 },
  { id: 3, name: 'Angular', seats: 12 },
]
</script>

<template>
  <FancyList :items="courses">
    <template #item="{ item, index }">
      {{ index + 1 }}. <b>{{ item.name }}</b>
      <span :style="{ color: item.seats ? 'green' : 'crimson' }">
        {{ item.seats ? item.seats + ' seats' : 'full' }}
      </span>
    </template>
  </FancyList>
</template>
```



THE LIST COMPONENT LOOPS; THE PARENT RENDERS EACH ITEM USING THE SLOT PROPS ITEM AND INDEX

9.4.5 Conditional slots with $slots

`$slots` (or `useSlots()` in script) contains a function for each slot the parent provided. Test it with `v-if` to render wrapper markup only when the slot has content, so an empty footer does not leave an empty bordered box.

SRC/COMPONENTS/PANELBOX.VUE HTML

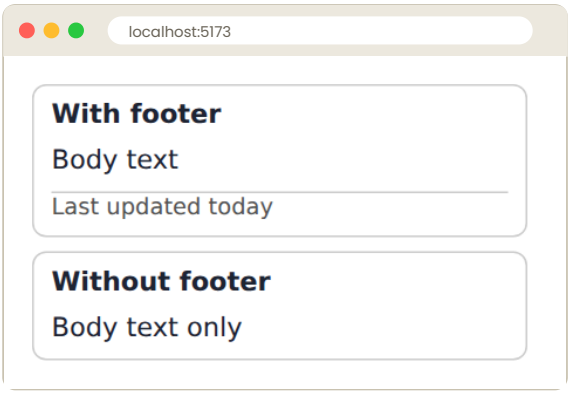
```
<template>
  <div class="panel">
    <h4 v-if="$slots.title"><slot name="title" /></h4>
    <slot />
    <div v-if="$slots.footer" class="footer"><slot name="footer" /></div>
  </div>
</template>

<style scoped>
.panel { border: 1px solid #ccc; border-radius: 8px; padding: 6px 10px; margin-bottom: 8px; width:
260px; }
h4 { margin: 0 0 4px; } .footer { border-top: 1px solid #ccc; margin-top: 6px; font-size: 13px; color:
#555; }
</style>
```

SRC/APP.VUE HTML

```
<script setup>
import PanelBox from './components/PanelBox.vue'
</script>

<template>
  <PanelBox>
    <template #title>With footer</template>
    Body text
    <template #footer>Last updated today</template>
  </PanelBox>
  <PanelBox>
    <template #title>Without footer</template>
    Body text only
  </PanelBox>
</template>
```



THE FOOTER BAR EXISTS ONLY WHEN A FOOTER SLOT WAS PROVIDED

9.5 Fallthrough attributes

9.5.1 class, style and listeners passed to the root element

A **fallthrough attribute** is an attribute or `v-on` listener passed to a component that is **not** declared as a prop or emit, such as `class`, `style`, `id`, `aria-*` or `@click`. When the component renders a **single root element**, Vue adds them to it automatically. `class` and `style` are **merged** with the root's own, and listeners are added alongside the root's own listeners.

SRC/COMPONENTS/BASEBUTTON.VUE HTML

```
<script setup>
defineProps({ variant: { type: String, default: 'primary' } })   // declared: not an attr
</script>

<template>
  <button class="btn" :class="variant">
    <slot />
  </button>
</template>
```

SRC/APP.VUE HTML

```
<script setup>
import BaseButton from './components/BaseButton.vue'
const clicked = () => console.log('clicked')
</script>

<template>
  <BaseButton variant="danger" class="large" style="margin: 4px" id="del" aria-label="Delete item"
@click="clicked">
    Delete
  </BaseButton>
</template>
```

RENDERED DOM

```
<div id="app" data-v-app="">
  <button class="btn danger large" id="del" aria-label="Delete item" style="margin: 4px;"> Delete
</button>
</div>
```

`variant` was declared as a prop, so it did not fall through; `class="large"` merged with `btn danger`, and `style`, `id`, `aria-label` and the click listener landed on the `<button>`. If the component has **several root nodes**, Vue cannot know where to put them and warns:

SRC/COMPONENTS/TWOROOTS.VUE HTML

```
<template>
  <header>Title</header>
  <main>Content</main>
</template>
```

SRC/APP.VUE HTML

```
<script setup>
import TwoRoots from './components/TwoRoots.vue'
</script>
<template><TwoRoots class="highlight" @close="() => {}" /></template>
```

BROWSER CONSOLE

```
[Vue warn]: Extraneous non-props attributes (class) were passed to component but could not be
automatically inherited because component renders fragment or text or teleport root nodes.
  at <TwoRoots class="highlight" onClose=fn<onClose> >
  at <App>
[Vue warn]: Extraneous non-emits event listeners (close) were passed to component but could not be
automatically inherited because component renders fragment or text root nodes. If the listener is
intended to be a component custom event listener only, declare it using the "emits" option.
  at <TwoRoots class="highlight" onClose=fn<onClose> >
  at <App>
```

9.5.2 inheritAttrs: false and useAttrs()

To decide yourself where fallthrough attributes go, disable automatic inheritance with `defineOptions({ inheritAttrs: false })` and bind `$attrs` (or `useAttrs()` in script) with `v-bind` wherever you want. A common case is an input component whose root is a wrapper `<label>` while attributes such as `placeholder` and `type` must reach the inner `<input>`.

SRC/COMPONENTS/BASEINPUT.VUE — HTML

```
<script setup>
import { useAttrs } from 'vue'

defineOptions({ inheritAttrs: false })
defineProps({ label: String, modelValue: String })
defineEmits(['update:modelValue'])

const attrs = useAttrs()
console.log('attrs received:', Object.keys(attrs).join(', '))
</script>

<template>
  <label class="field" :class="attrs.class">
    {{ label }}
    <input v-bind="{ ...$attrs, class: undefined }" :value="modelValue"
           @input="$emit('update:modelValue', $event.target.value)">
  </label>
</template>

<style scoped>.field { display: grid; gap: 2px; width: 220px; } .wide { width: 320px; }</style>
```

SRC/APP.VUE — HTML

```
<script setup>
import { ref } from 'vue'
import BaseInput from './components/BaseInput.vue'
const email = ref('')
</script>

<template>
  <BaseInput label="Email" type="email" placeholder="you@lbf.in" class="wide" v-model="email" />
  <p>email = {{ email }}</p>
</template>
```



TYPE AND PLACEHOLDER REACHED THE INNER INPUT; CLASS WENT TO THE LABEL. AFTER TYPING AN ADDRESS.

NOTE

`$attrs` includes listeners (as `onXxx` properties) and `class` / `style`. It is **not reactive** in script; to react to changes, use `onUpdated()` or, better, declare a prop.

PRACTICE CHECKPOINT

A reusable BaseButton, a Card with named slots (header, body, footer), and a DataTable with a scoped slot for custom cell rendering.

SRC/COMPONENTS/BASEBUTTON.VUE HTML

```
<script setup>
defineProps({ variant: { type: String, default: 'primary', validator: (v) => ['primary',
'secondary'].includes(v) } })
</script>
<template>
  <button class="btn" :class="variant"><slot>Button</slot></button>
</template>
<style scoped>
.btn { border: 0; border-radius: 6px; padding: 6px 14px; margin-right: 6px; cursor: pointer; }
.primary { background: #1E2A4A; color: white; } .secondary { background: #e2e8f0; }
.btn:disabled { opacity: .5; }
</style>
```

SRC/COMPONENTS/CARD.VUE HTML

```
<template>
  <section class="card">
    <header v-if="$slots.header"><slot name="header" /></header>
    <div class="body"><slot name="body" /></div>
    <footer v-if="$slots.footer"><slot name="footer" /></footer>
  </section>
</template>
<style scoped>
.card { width: 360px; border: 1px solid #ddd; border-radius: 10px; overflow: hidden; }
header { background: #1E2A4A; color: white; padding: 8px 12px; font-weight: 600; }
.body { padding: 8px 12px; } footer { padding: 8px 12px; background: #f8fafc; }
</style>
```

SRC/COMPONENTS/DATATABLE.VUE HTML

```
<script setup>
defineProps({ columns: Array, rows: Array })
</script>
<template>
  <table>
    <thead><tr><th v-for="c in columns" :key="c.key">{{ c.label }}</th></tr></thead>
    <tbody>
      <tr v-for="row in rows" :key="row.id">
        <td v-for="column in columns" :key="column.key">
          <slot name="cell" :row="row" :column="column" :value="row[column.key]">{{ row[column.key] }}
</slot>
        </td>
      </tr>
    </tbody>
  </table>
</template>
<style scoped>
table { border-collapse: collapse; width: 100%; } th { text-align: left; border-bottom: 2px solid
#1E2A4A; }
td, th { padding: 4px 6px; } tr + tr td { border-top: 1px solid #eee; }
</style>
```

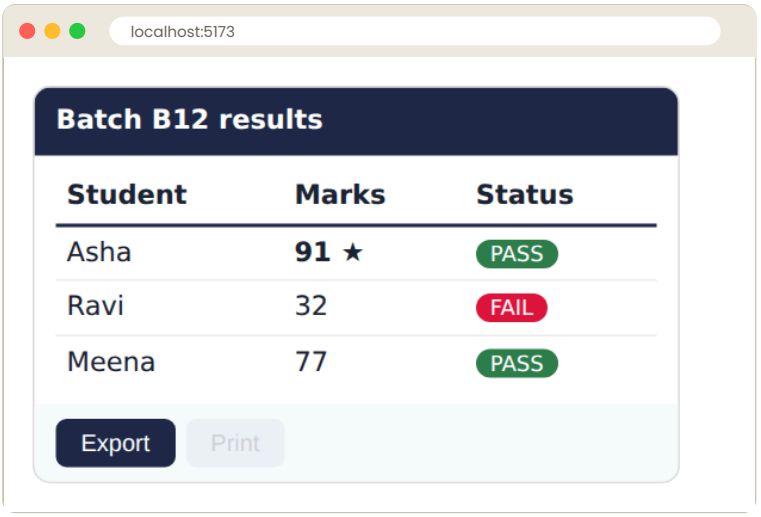
SRC/APP.VUE HTML

```
<script setup>
import BaseButton from './components/BaseButton.vue'
import Card from './components/Card.vue'
import DataTable from './components/DataTable.vue'

const columns = [{ key: 'name', label: 'Student' }, { key: 'marks', label: 'Marks' }, { key: 'status',
label: 'Status' }]
const rows = [
  { id: 1, name: 'Asha', marks: 91 }, { id: 2, name: 'Ravi', marks: 32 }, { id: 3, name: 'Meena',
marks: 77 },
].map((r) => ({ ...r, status: r.marks >= 35 ? 'pass' : 'fail' }))
const log = (m) => console.log(m)
</script>

<template>
  <Card>
    <template #header>Batch B12 results</template>
    <template #body>
      <DataTable :columns="columns" :rows="rows">
        <template #cell="{ row, column, value }">
          <span v-if="column.key === 'status'" :class="['pill', value]">{{ value.toUpperCase() }}
</span>
          <b v-else-if="column.key === 'marks' && value > 85">{{ value }} ★</b>
          <template v-else>{{ value }}</template>
        </template>
      </DataTable>
    </template>
    <template #footer>
      <BaseButton @click="log('export')">Export</BaseButton>
      <BaseButton variant="secondary" disabled>Print</BaseButton>
    </template>
  </Card>
</template>

<style>
.pill { color: white; border-radius: 99px; padding: 1px 8px; font-size: 12px; }
.pill.pass { background: #2e7d4f; } .pill.fail { background: crimson; }
</style>
```



CARD WITH NAMED SLOTS, A DATATABLE WHOSE CELLS ARE RENDERED BY THE PARENT THROUGH A SCOPED SLOT, AND TWO BASEBUTTONS

Register components by importing them in `<script setup>`. Props (`defineProps`) carry data down with types, defaults and validators, and are read-only; events (`defineEmits`) carry messages and payloads up. Slots pass template content: default, named (`#name`) and scoped (`#name="{ item }"`), and `$slots` tells which were provided. Undeclared attributes fall through to a single root unless `inheritAttrs: false`.

MODULE 10 · INTERMEDIATE

Watchers and lifecycle hooks

Minimum duration **2 days** | Level **Intermediate**

Watchers react to data changes with side effects; lifecycle hooks run code at specific points in a component's life. This module covers watch() with a ref, a getter or an array of sources, old and new values, deep watchers, the immediate and once options, watchEffect(), when to use watch, watchEffect or computed, cleanup with onWatcherCleanup(), stopping watchers; the lifecycle diagram and the hooks onBeforeMount/onMounted, onBeforeUpdate/onUpdated, onBeforeUnmount/onUnmounted, onErrorCaptured and onActivated/onDeactivated; and template refs with useTemplateRef(), DOM access after mount, and refs on child components with defineExpose().

IN THIS MODULE

10.1 Watchers
10.2 Lifecycle hooks
10.3 Template refs

10.1 Watchers

10.1.1 watch() with a ref, getter or array of sources

A **watcher** runs a callback whenever a reactive source changes. Use it for **side effects**: calling an API, saving to localStorage, logging, changing the DOM. `watch(source, callback)` accepts as the source: a **ref** (including computed), a **reactive object**, a **getter function** (for a property or an expression), or an **array** of these.

SRC/APP.VUE

HTML

```
<script setup>
import { ref, reactive, computed, watch } from 'vue'

const query = ref('')
const filters = reactive({ city: 'Pune', minMarks: 50 })
const first = ref('Asha'), last = ref('Rao')

watch(query, (q) => console.log('1. ref changed:', q))
watch(() => filters.city, (c) => console.log('2. getter (one property):', c))
watch(() => first.value + ' ' + last.value, (full) => console.log('3. getter (expression):', full))
watch([first, () => filters.minMarks], ([f, m]) => console.log('4. array of sources:', f, m))

setTimeout(() => {
  query.value = 'vue'
  filters.city = 'Mysuru'
  last.value = 'Iyer'
  filters.minMarks = 60
}, 10)
</script>
<template><p>watchers demo</p></template>
```

BROWSER CONSOLE

```
1. ref changed: vue
2. getter (one property): Mysuru
3. getter (expression): Asha Iyer
4. array of sources: Asha 60
```

WATCH OUT

Do not watch a property of a reactive object directly: `watch(filters.city, ...)` passes the current **string**, not a source, and Vue warns "Invalid watch source". Use a getter: `watch(() => filters.city, ...)`.

10.1.2 Old and new values

The callback receives `(newValue, oldValue)`. With an array source, both are arrays. For a mutated object or array, old and new are the **same object** (it was changed in place), so copy it or watch a getter that returns a new value if you need the previous state.

SRC/APP.VUE — HTML

```
<script setup>
import { ref, watch } from 'vue'

const page = ref(1)
const cart = ref(['pen'])

watch(page, (n, o) => console.log(`page ${o} -> ${n}`))
watch(cart, (n, o) => console.log('same object?', n === o, JSON.stringify(n)), { deep: true })
watch(() => [...cart.value], (n, o) => console.log('copies:', JSON.stringify(o), '->',
JSON.stringify(n)))

setTimeout(() => { page.value = 2; cart.value.push('book') }, 10)
</script>
<template><p>old and new</p></template>
```

BROWSER CONSOLE

```
page 1 -> 2
same object? true ["pen","book"]
copies: ["pen"] -> ["pen","book"]
```

10.1.3 Deep watchers

Watching a **ref** that holds an object only fires when the ref is **replaced**; changes inside it are ignored. Add `{ deep: true }` to fire on nested changes. Watching a **reactive object** directly is deep automatically. Since Vue 3.5, `deep` can also be a number: the maximum depth to traverse. Deep watching walks every nested property, so it can be expensive on large data.

SRC/APP.VUE HTML

```
<script setup>
import { ref, reactive, watch } from 'vue'

const settings = ref({ theme: { color: 'blue' }, fontSize: 14 })
const profile = reactive({ name: 'Asha', address: { city: 'Pune' } })

watch(settings, () => console.log('A (shallow ref watch) fired'))
watch(settings, () => console.log('B (deep: true) fired'), { deep: true })
watch(settings, () => console.log('C (deep: 1) fired'), { deep: 1 })
watch(profile, () => console.log('D (reactive object, deep by default) fired'))

setTimeout(() => {
  console.log('-- change settings.theme.color (depth 2)')
  settings.value.theme.color = 'green'
  profile.address.city = 'Mysuru'
}, 10)
setTimeout(() => { console.log('-- change settings.fontSize (depth 1)'); settings.value.fontSize = 16
}, 30)
setTimeout(() => { console.log('-- replace settings'); settings.value = { theme: {}, fontSize: 12 } },
50)
</script>
<template><p>deep</p></template>
```

BROWSER CONSOLE

```
-- change settings.theme.color (depth 2)
B (deep: true) fired
D (reactive object, deep by default) fired
-- change settings.fontSize (depth 1)
C (deep: 1) fired
B (deep: true) fired
-- replace settings
A (shallow ref watch) fired
C (deep: 1) fired
B (deep: true) fired
```

10.1.4 immediate and once options

A watcher is **lazy** by default: the callback runs only after the first change. `{ immediate: true }` runs it once right away (with `oldValue` undefined), which is ideal for loading data based on an initial value. `{ once: true }` (Vue 3.4+) runs the callback only for the first change and then stops the watcher.

SRC/APP.VUE HTML

```
<script setup>
import { ref, watch } from 'vue'

const userId = ref(1)
watch(userId, (id, old) => console.log(`immediate: load user ${id} (old: ${old})`), { immediate: true
})
watch(userId, (id) => console.log(`once: first change to ${id}`), { once: true })

setTimeout(() => (userId.value = 2), 10)
setTimeout(() => (userId.value = 3), 20)
</script>
<template><p>{{ userId }}</p></template>
```

BROWSER CONSOLE

```
immediate: load user 1 (old: undefined)
immediate: load user 2 (old: 1)
once: first change to 2
immediate: load user 3 (old: 2)
```

10.1.5 watchEffect()

`watchEffect(fn)` runs `fn` **immediately** and tracks every reactive value it reads automatically; when any of them changes, it runs again. There is no separate source and no old value. It is convenient when an effect depends on several values.

SRC/APP.VUE — HTML

```html
<script setup>
import { ref, watchEffect } from 'vue'

const courseId = ref(101)
const lang = ref('en')

watchEffect(() => {
  // reads courseId and lang, so it depends on both
  console.log(`fetch /api/courses/${courseId.value}?lang=${lang.value}`)
})

setTimeout(() => (courseId.value = 102), 10)
setTimeout(() => (lang.value = 'kn'), 20)
</script>
<template><p>watchEffect</p></template>
```

BROWSER CONSOLE

```
fetch /api/courses/101?lang=en
fetch /api/courses/102?lang=en
fetch /api/courses/102?lang=kn
```

NOTE

`watchEffect` only tracks values read **synchronously**. Anything read after the first `await` inside it is not tracked. Watcher callbacks run, by default, **before** the component's DOM update (`flush: 'pre'`); use `{ flush: 'post' }` (or `watchPostEffect`) to access the updated DOM, and `flush: 'sync'` rarely.

10.1.6 watch vs watchEffect vs computed

	computed	watch	watchEffect
Purpose	derive a value	side effect on specific changes	side effect on whatever it reads
Returns	a read-only ref	a stop handle	a stop handle
Runs	lazily, cached	after the source changes (lazy unless immediate)	immediately, then on changes
Dependencies	automatic	explicit source	automatic
Old value	no	yes	no
Example	total price	save to localStorage when cart changes	fetch when any filter changes

TIP

Rule of thumb: if you are **computing a value** to display, use `computed`. If you need to **do something** when data changes, use `watch` (precise source, old value, lazy) or `watchEffect` (shorter, all dependencies).

10.1.7 Cleanup with onWatcherCleanup()

When a watcher starts something asynchronous (a request, a timer), the source may change again before it finishes. `onWatcherCleanup(fn)` (Vue 3.5+, called synchronously inside the watcher) registers a function that runs **before the next run** and when the watcher is stopped, for example to abort the outdated request. Before 3.5, use the third `onCleanup` argument of the callback.

SRC/APP.VUE

HTML

```
<script setup>
import { ref, watch, onWatcherCleanup } from 'vue'

const id = ref(1)
watch(id, (newId) => {
  const controller = new AbortController()
  console.log(`start request for ${newId}`)
  const timer = setTimeout(() => console.log(`response for ${newId} arrived`), 100)

  onWatcherCleanup(() => {           // runs if id changes again first
    console.log(`abort request for ${newId}`)
    controller.abort()
    clearTimeout(timer)
  })
}, { immediate: true })

setTimeout(() => (id.value = 2), 20)   // too fast: request 1 is aborted
setTimeout(() => (id.value = 3), 40)   // request 2 is aborted
</script>
<template><p>{{ id }}</p></template>
```

BROWSER CONSOLE

```
start request for 1
abort request for 1
start request for 2
abort request for 2
start request for 3
response for 3 arrived
```

10.1.8 Stopping watchers

Watchers created synchronously in `<script setup>` are bound to the component and **stopped automatically** when it unmounts. `watch` and `watchEffect` also return a handle: call it to stop early. Since Vue 3.5 the handle also has `pause()` and `resume()`. A watcher created asynchronously (inside a `setTimeout` or after `await`) is not bound to the component and must be stopped manually to avoid a leak.

SRC/APP.VUE

HTML

```
<script setup>
import { ref, watch } from 'vue'

const n = ref(0)
const stop = watch(n, (v) => console.log('watcher saw', v))

setTimeout(() => (n.value = 1), 10)
setTimeout(() => { stop(); console.log('-- stopped'); n.value = 2 }, 30)
</script>
<template><p>{{ n }}</p></template>
```

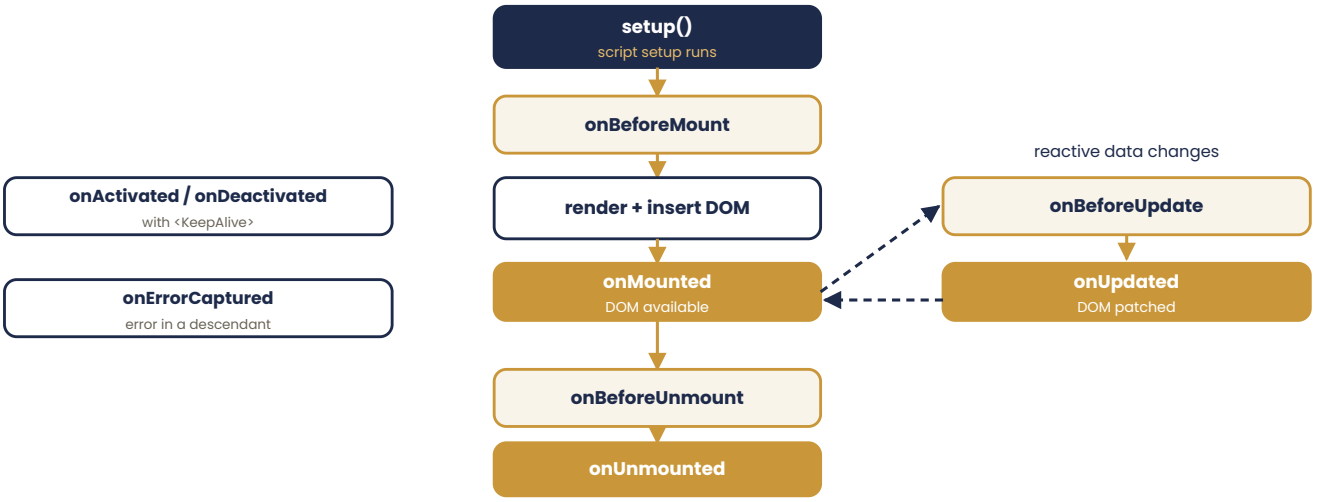
BROWSER CONSOLE

```
watcher saw 1
-- stopped
```

10.2 Lifecycle hooks

10.2.1 Lifecycle diagram

Each component instance goes through a series of steps: setup, compiling or using its render function, mounting its DOM, updating when state changes, and unmounting. **Lifecycle hooks** let you run code at these points. In the Composition API they are functions such as `onMounted(callback)`, called synchronously during `setup`.



THE COMPONENT LIFECYCLE (COMPOSITION API HOOKS)

Composition API	Options API	Runs
(script setup itself)	`beforeCreate`, `created`	when the instance is set up; state exists, no DOM
`onBeforeMount`	`beforeMount`	just before the first render is inserted
`onMounted`	`mounted`	after the component's DOM is inserted (children mounted first)
`onBeforeUpdate`	`beforeUpdate`	after a state change, before the DOM is patched
`onUpdated`	`updated`	after the DOM is patched

Composition API	Options API	Runs
`onBeforeUnmount`	`beforeUnmount`	before removal; still fully working
`onUnmounted`	`unmounted`	after removal; clean up timers, listeners, sockets
`onErrorCaptured`	`errorCaptured`	when a descendant throws
`onActivated` / `onDeactivated`	`activated` / `deactivated`	when a cached component is shown / hidden by `<KeepAlive>`

10.2.2 onBeforeMount and onMounted

`onMounted` is the most used hook: the component's elements exist, so you can measure them, focus an input, start a chart library, or begin a timer. Child components are mounted **before** their parent's `onMounted` runs.

SRC/APP.VUE

HTML

```
<script setup>
import { onBeforeMount, onMounted } from 'vue'
import ChildBox from './components/ChildBox.vue'

console.log('1. App setup: no DOM yet ->', document.querySelector('h2'))
onBeforeMount(() => console.log('2. App beforeMount'))
onMounted(() => console.log('5. App mounted, h2 text =', document.querySelector('h2').textContent))
</script>

<template>
  <h2>Parent</h2>
  <ChildBox />
</template>
```

SRC/COMPONENTS/CHILDBOX.VUE

HTML

```
<script setup>
import { onMounted } from 'vue'
console.log('3. Child setup')
onMounted(() => console.log('4. Child mounted (before its parent)'))
</script>
<template><p>child</p></template>
```

BROWSER CONSOLE

```
1. App setup: no DOM yet -> null
2. App beforeMount
3. Child setup
4. Child mounted (before its parent)
5. App mounted, h2 text = Parent
```

10.2.3 onBeforeUpdate and onUpdated

These run around every re-render caused by reactive changes. `onBeforeUpdate` can read the DOM before it changes; `onUpdated` sees the patched DOM. Do not change state inside `onUpdated`: that causes another update and can loop forever. For reacting to one specific value, prefer a watcher with `flush: 'post'`.

SRC/APP.VUE

HTML

```
<script setup>
import { ref, onBeforeUpdate, onUpdated } from 'vue'

const items = ref(['a'])
onBeforeUpdate(() => console.log('before update: li count =', document.querySelectorAll('li').length))
onUpdated(() => console.log('updated:        li count =', document.querySelectorAll('li').length))
</script>

<template>
  <ul><li v-for="i in items" :key="i">{{ i }}</li></ul>
  <button @click="items.push('b', 'c')">add two</button>
</template>
```

BROWSER CONSOLE

```
before update: li count = 1
updated:        li count = 3
```

10.2.4 onBeforeUnmount and onUnmounted

Anything the component started that the browser does not remove by itself (intervals, window event listeners, WebSocket connections, third-party widgets) must be stopped in `onUnmounted` (or `onBeforeUnmount`), or it keeps running after the component is gone: a **memory leak**.

SRC/COMPONENTS/LIVECLOCK.VUE

HTML

```
<script setup>
import { ref, onMounted, onUnmounted } from 'vue'

const now = ref(0)
let timer
const onResize = () => console.log('resized')

onMounted(() => {
  timer = setInterval(() => console.log('tick', ++now.value), 100)
  window.addEventListener('resize', onResize)
})
onUnmounted(() => {
  clearInterval(timer)                          // without this, ticks continue forever
  window.removeEventListener('resize', onResize)
  console.log('LiveClock unmounted: timer and listener removed')
})
</script>

<template><p>tick {{ now }}</p></template>
```

SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'
import LiveClock from './components/LiveClock.vue'
const show = ref(true)
setTimeout(() => (show.value = false), 250)
setTimeout(() => console.log('-- 200 ms after unmount: no more ticks'), 450)
</script>

<template><LiveClock v-if="show" /></template>
```

BROWSER CONSOLE

```
tick 1
tick 2
LiveClock unmounted: timer and listener removed
-- 200 ms after unmount: no more ticks
```

10.2.5 onErrorCaptured

`onErrorCaptured((err, instance, info) => ...)` is called when an error is thrown in a **descendant** component (in setup, render, a watcher, a lifecycle hook or an event handler). Returning `false` stops the error from propagating further (to ancestors and to `app.config.errorHandler`). It is the building block for error boundaries (Module 18.4).

SRC/COMPONENTS/BUGGYWIDGET.VUE — HTML

```
<script setup>
const data = null
</script>

<template>
  <p>Score: {{ data.score }}</p>    <!-- data is null: render throws -->
</template>
```

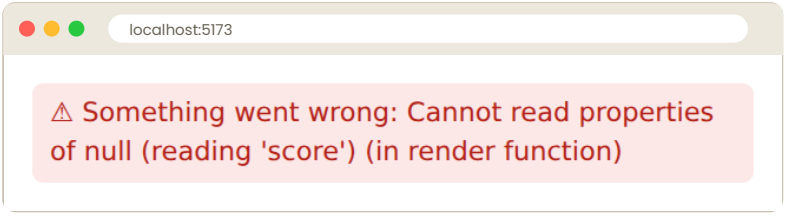
SRC/APP.VUE — HTML

```
<script setup>
import { ref, onErrorCaptured } from 'vue'
import BuggyWidget from './components/BuggyWidget.vue'

const error = ref(null)
onErrorCaptured((err, instance, info) => {
  error.value = `${err.message} (in ${info})`
  return false                    // handled: do not propagate
})
</script>

<template>
  <p v-if="error" class="err">⚠ Something went wrong: {{ error }}</p>
  <BuggyWidget v-else />
</template>

<style>.err { background: #fde8e8; color: #b42318; padding: 6px 10px; border-radius: 6px; }</style>
```



THE PARENT CAUGHT THE CHILD'S RENDER ERROR AND SHOWED A FALLBACK INSTEAD OF A BLANK PAGE

10.2.6 onActivated and onDeactivated with KeepAlive

Components inside `<KeepAlive>` (Module 11.4) are cached instead of unmounted when they are switched away. They do not run `onUnmounted` / `onMounted` again; instead `onDeactivated` runs when hidden and `onActivated` when shown again (also on first mount). Use them to pause and resume timers or refresh data.

SRC/COMPONENTS/TABA.VUE

HTML

```
<script setup>
import { onMounted, onUnmounted, onActivated, onDeactivated } from 'vue'
onMounted(() => console.log('A mounted'))
onUnmounted(() => console.log('A unmounted'))
onActivated(() => console.log('A activated'))
onDeactivated(() => console.log('A deactivated'))
</script>
<template><p>Tab A</p></template>
```

SRC/APP.VUE

HTML

```
<script setup>
import { ref, shallowRef } from 'vue'
import TabA from './components/TabA.vue'
import TabB from './components/TabB.vue'

const current = shallowRef(TabA)
setTimeout(() => { console.log('-- switch to B'); current.value = TabB }, 20)
setTimeout(() => { console.log('-- back to A'); current.value = TabA }, 40)
</script>

<template>
  <KeepAlive><component :is="current" /></KeepAlive>
</template>
```

BROWSER CONSOLE

```
A mounted
A activated
-- switch to B
A deactivated
B mounted
B activated
-- back to A
A activated
```

10.3 Template refs

10.3.1 useTemplateRef() and ref attributes

Sometimes you need the actual DOM element: to focus it, measure it, or pass it to a library. Give the element a `ref="name"` attribute and get a ref to it with `useTemplateRef('name')` (Vue 3.5+). Before 3.5 (and still valid), declare `const name = ref(null)` with the **same name** as the attribute. The ref is `null` until the component is mounted.

SRC/APP.VUE HTML

```
<script setup>
import { useTemplateRef, ref, onMounted } from 'vue'

const input = useTemplateRef('email')     // Vue 3.5+
const heading = ref(null)                 // older style: same name as ref="heading"

console.log('in setup:', input.value)     // null: not mounted yet
onMounted(() => {
  console.log('mounted:', input.value.tagName, heading.value.textContent)
})
</script>

<template>
  <h3 ref="heading">Sign in</h3>
  <input ref="email" placeholder="Email">
</template>
```

BROWSER CONSOLE

```
in setup: null
mounted: INPUT Sign in
```

10.3.2 Accessing DOM elements after mount (focus, scroll, measure)

SRC/APP.VUE HTML

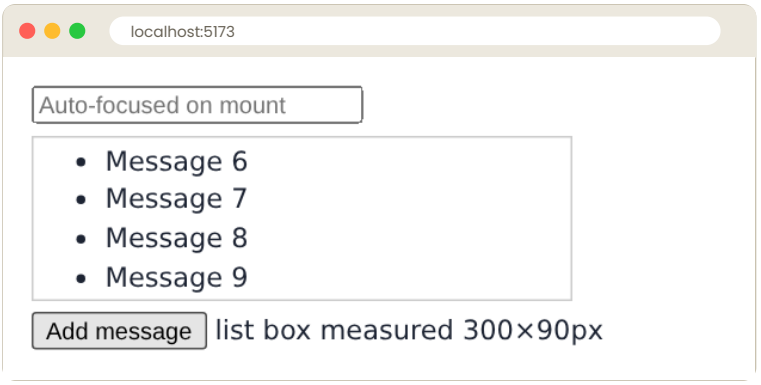
```
<script setup>
import { ref, useTemplateRef, onMounted, nextTick } from 'vue'

const search = useTemplateRef('search')
const box = useTemplateRef('box')
const messages = ref(Array.from({ length: 8 }, (_, i) => `Message ${i + 1}`))
const size = ref('')

onMounted(() => {
  search.value.focus()                                  // focus
  size.value = `${box.value.clientWidth}×${box.value.clientHeight}px` // measure
})
async function addMessage() {
  messages.value.push(`Message ${messages.value.length + 1}`)
  await nextTick()                                      // wait for the new <li>
  box.value.scrollTop = box.value.scrollHeight          // scroll to bottom
}
</script>

<template>
  <input ref="search" placeholder="Auto-focused on mount">
  <ul ref="box" class="box"><li v-for="m in messages" :key="m">{{ m }}</li></ul>
  <button @click="addMessage">Add message</button> list box measured {{ size }}
</template>

<style>.box { height: 90px; overflow: auto; border: 1px solid #ccc; margin: 6px 0; width: 260px; }
</style>
```



THE INPUT IS FOCUSED ON MOUNT, THE LIST WAS MEASURED, AND AFTER ONE CLICK IT SCROLLED TO THE NEW LAST MESSAGE

In a `v-for`, a template ref holds an **array** of elements (in no guaranteed order). A **function ref** `:ref="(el) => ..."` is called with the element on every update and with `null` on unmount.

10.3.3 Refs on child components and defineExpose()

A template ref on a **child component** gives its component instance. Components using `<script setup>` are **closed by default**: the parent cannot see anything inside them. The child must list what it makes public with the `defineExpose()` macro.

SRC/COMPONENTS/STOPWATCH.VUE HTML

```
<script setup>
import { ref } from 'vue'

const seconds = ref(42)
const secret = 'internal'
function reset() { seconds.value = 0 }

defineExpose({ seconds, reset })     // public API for the parent
</script>

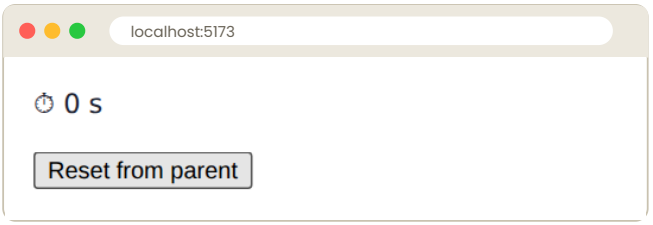
<template><p>⏱ {{ seconds }} s</p></template>
```

SRC/APP.VUE HTML

```
<script setup>
import { useTemplateRef } from 'vue'
import StopWatch from './components/StopWatch.vue'

const watchRef = useTemplateRef('sw')
function resetFromParent() {
  console.log('exposed keys:', Object.keys(watchRef.value).filter((k) => !k.startsWith('$')))
  console.log('secret is', watchRef.value.secret)    // undefined: not exposed
  watchRef.value.reset()                             // call the exposed method
}
</script>

<template>
  <StopWatch ref="sw" />
  <button @click="resetFromParent">Reset from parent</button>
</template>
```



AFTER CLICKING THE PARENT'S BUTTON: THE CHILD'S EXPOSED RESET() RAN

TIP

Reach for child refs sparingly. Props and events keep data flow visible; exposed methods are best for imperative actions that do not fit them, such as `focus()`, `open()` or `validate()` on a form component.

PRACTICE CHECKPOINT

A search box that fetches results when the query changes (with debounce and cleanup), auto-focuses on mount, and saves recent searches to localStorage with a watcher.

SRC/APP.VUE HTML

```
<script setup>
import { ref, watch, onMounted, useTemplateRef, onWatcherCleanup } from 'vue'

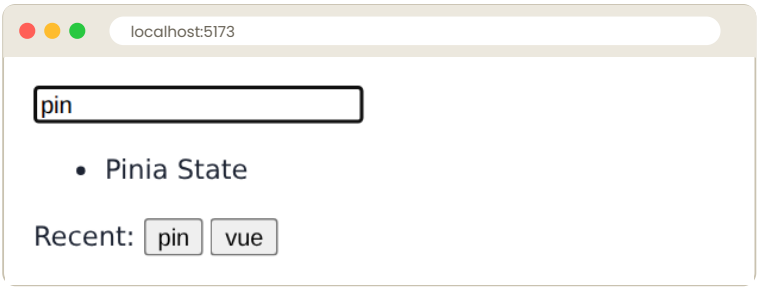
const query = ref('')
const results = ref([])
const loading = ref(false)
const recent = ref(JSON.parse(localStorage.getItem('recent') || '[]'))
const input = useTemplateRef('input')

onMounted(() => input.value.focus())

watch(query, (q) => {
  if (q.trim().length < 2) { results.value = []; return }
  const controller = new AbortController()
  const timer = setTimeout(async () => {          // debounce 300 ms
    loading.value = true
    try {
      const res = await fetch(`/api/courses?q=${encodeURIComponent(q)}`, { signal: controller.signal
})
      results.value = await res.json()
      recent.value = [q, ...recent.value.filter((r) => r !== q)].slice(0, 5)
    } catch (e) {
      if (e.name !== 'AbortError') console.error(e)
    } finally { loading.value = false }
  }, 300)
  onWatcherCleanup(() => { clearTimeout(timer); controller.abort() })
})

watch(recent, (r) => localStorage.setItem('recent', JSON.stringify(r)), { deep: true })
</script>

<template>
  <input ref="input" v-model="query" placeholder="Search courses">
  <span v-if="loading"> searching…</span>
  <ul><li v-for="r in results" :key="r">{{ r }}</li></ul>
  <p v-if="recent.length">Recent: <button v-for="r in recent" :key="r" @click="query = r">{{ r }}
</button></p>
</template>
```



TYPED "VUE" (RESULTS ARRIVED AFTER THE 300 MS DEBOUNCE), THEN "PIN": BOTH SEARCHES ARE SAVED AS RECENT

Use `watch` for side effects on specific sources (refs, getters, arrays; `deep`, `immediate`, `once`; old and new values) and `watchEffect` when the effect should track whatever it reads; clean up async work with `onWatcherCleanup`. Lifecycle hooks (`onMounted`, `onUpdated`, `onUnmounted`…) run at fixed points; release timers and listeners in `onUnmounted`. `useTemplateRef` gives DOM elements after mount, and `defineExpose` publishes a child's API to parent refs.

MODULE 11 · INTERMEDIATE

Advanced component features

Minimum duration **2 days** | Level **Intermediate**

These built-in features handle two-way component binding, deep data passing, dynamic components and transitions. The module covers v-model on components with defineModel(), multiple v-model bindings and custom modifiers; provide and inject to avoid prop drilling, providing reactive values, app-level provide and Symbol injection keys; dynamic components with component :is and lazy loading with defineAsyncComponent(); the built-in components KeepAlive, Teleport and Suspense; and animations with Transition, TransitionGroup and JavaScript hooks.

IN THIS MODULE

11.1 Component v-model
11.2 Provide and inject
11.3 Dynamic and async components
11.4 Built-in components
11.5 Transitions and animations

11.1 Component v-model

11.1.1 defineModel()

`v-model` also works on components. On a component, `v-model="x"` expands to the prop `modelValue` plus a listener for the event `update:modelValue`. Inside the child, the `defineModel()` macro (Vue 3.4+) declares both and returns a ref: reading it gives the parent's value, and assigning to it emits the update. The child can even bind it with its own `v-model`.

SRC/COMPONENTS/STARRATING.VUE — HTML

```
<script setup>
const model = defineModel({ type: Number, default: 0 })
</script>

<template>
  <span v-for="n in 5" :key="n" class="star" @click="model = n">
    {{ n <= model ? '★' : '☆' }}
  </span>
</template>

<style scoped>.star { font-size: 26px; color: #C9973A; cursor: pointer; }</style>
```

SRC/COMPONENTS/SEARCHINPUT.VUE HTML

```
<script setup>
const model = defineModel()          // modelValue + update:modelValue
</script>

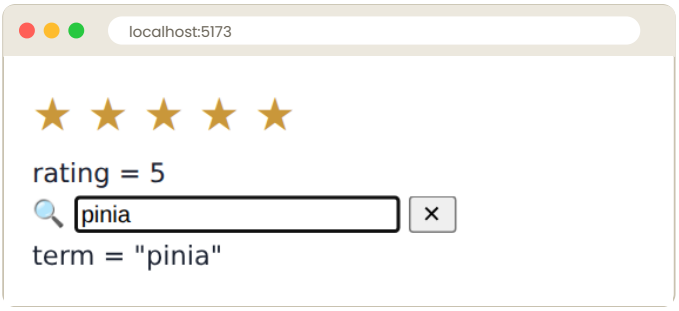
<template>
  <div class="search">🔍 <input v-model="model" placeholder="Search"> <button @click="model =
''">x</button></div>
</template>
```

SRC/APP.VUE HTML

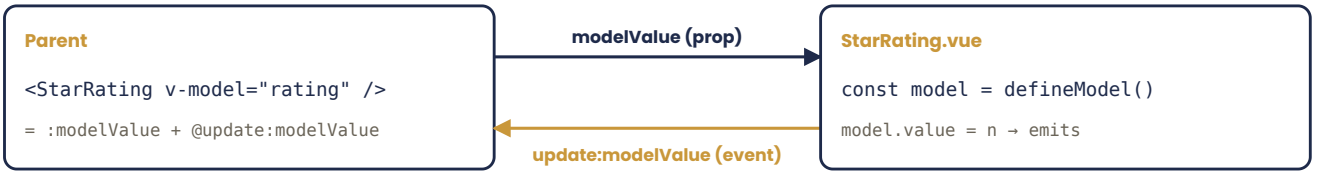
```
<script setup>
import { ref } from 'vue'
import StarRating from './components/StarRating.vue'
import SearchInput from './components/SearchInput.vue'

const rating = ref(3)
const term = ref('vue')
</script>

<template>
  <StarRating v-model="rating" />
  <p>rating = {{ rating }}</p>
  <SearchInput v-model="term" />
  <p>term = "{{ term }}"</p>
</template>
```



CLICKED THE FIFTH STAR AND TYPED "PINIA" INTO THE CHILD'S INPUT: THE PARENT'S REFS CHANGED



DEFINEMODEL() WIRES THE PROP AND THE UPDATE EVENT FOR YOU

Before Vue 3.4 the same was written by hand, which you will see in older code: `const props = defineProps(['modelValue'])`, `const emit = defineEmits(['update:modelValue'])`, and `:value="modelValue" @input="emit('update:modelValue', $event.target.value)"`.

11.1.2 Multiple v-model bindings

Give `v-model` an argument to bind a different prop: `v-model:first-name="first"` uses the prop `firstName` and the event `update:firstName`. A component can accept several such models, each declared with `defineModel('name')`.

SRC/COMPONENTS/NAMEFIELDS.VUE

HTML

```
<script setup>
const firstName = defineModel('firstName')
const lastName = defineModel('lastName')
</script>

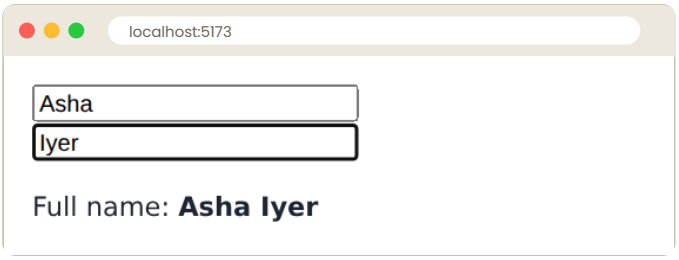
<template>
  <input v-model="firstName" placeholder="First">
  <input v-model="lastName" placeholder="Last">
</template>
```

SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'
import NameFields from './components/NameFields.vue'
const first = ref('Asha')
const last = ref('Rao')
</script>

<template>
  <NameFields v-model:first-name="first" v-model:last-name="last" />
  <p>Full name: <b>{{ first }} {{ last }}</b></p>
</template>
```



TYPED A NEW LAST NAME: ONLY LASTNAME WAS UPDATED

11.1.3 Custom v-model modifiers

Built-in modifiers such as `.trim` work on component v-models. You can also create your own: destructure `defineModel()` to get `[model, modifiers]`, and use the `set` (and `get`) transformers to change the value on its way out.

SRC/COMPONENTS/SMARTINPUT.VUE

HTML

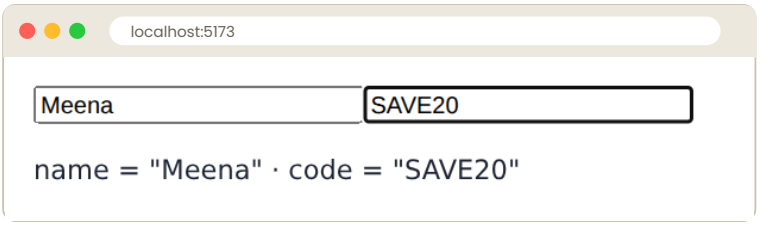
```
<script setup>
const [model, modifiers] = defineModel({
  set(value) {
    if (modifiers.capitalize) value = value.charAt(0).toUpperCase() + value.slice(1)
    if (modifiers.uppercase) value = value.toUpperCase()
    if (modifiers.nospace) value = value.replace(/\s+/g, '')
    return value
  },
})
console.log('modifiers:', JSON.stringify(modifiers))
</script>

<template><input v-model="model"></template>
```

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
import SmartInput from './components/SmartInput.vue'
const name = ref('')
const code = ref('')
</script>

<template>
  <SmartInput v-model.capitalize="name" placeholder="name" />
  <SmartInput v-model.uppercase.nospace="code" placeholder="coupon code" />
  <p>name = "{{ name }}" · code = "{{ code }}"</p>
</template>
```



TYPED "MEENA" AND "SAVE 20": EACH INPUT APPLIED ITS OWN MODIFIERS

11.2 Provide and inject

11.2.1 Avoiding prop drilling

When a deeply nested component needs data from a distant ancestor, passing it as a prop through every component in between is called **prop drilling**: the middle components carry props they do not use. With `provide(key, value)` an ancestor makes a value available to **all** its descendants, and any descendant reads it with `inject(key, default)`, no matter how deep.



PROVIDE/INJECT SKIPS THE COMPONENTS IN BETWEEN

SRC/APP.VUE HTML

```
<script setup>
import { provide } from 'vue'
import PageLayout from './components/PageLayout.vue'

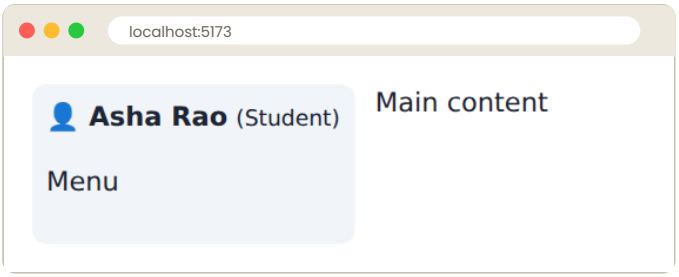
provide('user', { name: 'Asha Rao', role: 'Student' })
</script>

<template><PageLayout /></template>
```

SRC/COMPONENTS/USERAVATAR.VUE

HTML

```
<script setup>
import { inject } from 'vue'
const user = inject('user', { name: 'Guest', role: '-' })   // default if not provided
</script>
<template><b>👤 {{ user.name }}</b> <small>({{ user.role }})</small></template>
```



USERAVATAR, THREE LEVELS DOWN, READ THE USER PROVIDED BY APP; PAGELAYOUT AND SIDEBAR KNOW NOTHING ABOUT IT

11.2.2 Providing reactive values

If the provided value is a **ref** or reactive object, injectors stay in sync with it. Keep **mutations in the provider**: provide the state (read-only with `readonly()`) together with functions that change it, so it is always clear where changes come from.

SRC/APP.VUE

HTML

```
<script setup>
import { ref, provide, readonly } from 'vue'
import ThemeToggle from './components/ThemeToggle.vue'
import ThemedCard from './components/ThemedCard.vue'

const theme = ref('light')
function toggleTheme() { theme.value = theme.value === 'light' ? 'dark' : 'light' }

provide('theme', { theme: readonly(theme), toggleTheme })
</script>

<template>
  <ThemeToggle />
  <ThemedCard>Card one</ThemedCard>
  <ThemedCard>Card two</ThemedCard>
</template>
```

SRC/COMPONENTS/THEMETOGGLE.VUE

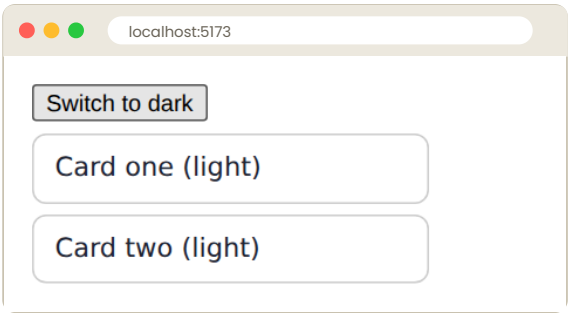
HTML

```
<script setup>
import { inject } from 'vue'
const { theme, toggleTheme } = inject('theme')
</script>
<template><button @click="toggleTheme">Switch to {{ theme === 'light' ? 'dark' : 'light' }}</button>
</template>
```

SRC/COMPONENTS/THEMEDCARD.VUE

HTML

```
<script setup>
import { inject } from 'vue'
const { theme } = inject('theme')
</script>
<template><div class="card" :class="theme"><slot /> ({{ theme }})</div></template>
<style scoped>
.card { padding: 8px 12px; margin-top: 6px; border-radius: 8px; width: 200px; border: 1px solid #ccc;
}
.dark { background: #1E2A4A; color: white; }
</style>
```



ONE CLICK ON THE TOGGLE: EVERY INJECTOR UPDATED

11.2.3 App-level provide

`app.provide(key, value)` in `main.js` makes a value available to **every** component of the app. Plugins use this to share services (the router and Pinia inject themselves this way).

SRC/MAIN.JS

JavaScript

```
import { createApp } from 'vue'
import App from './App.vue'

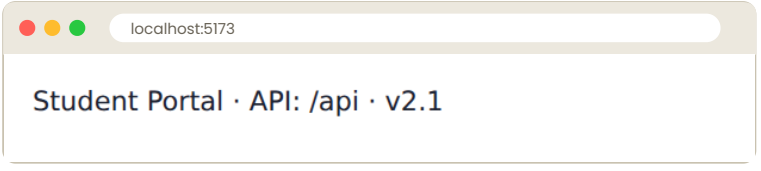
const app = createApp(App)
app.provide('config', { appName: 'Student Portal', apiBase: '/api', version: '2.1' })
app.mount('#app')
```

SRC/APP.VUE

HTML

```
<script setup>
import { inject } from 'vue'
const config = inject('config')
</script>

<template><p>{{ config.appName }} · API: {{ config.apiBase }} · v{{ config.version }}</p></template>
```



A VALUE PROVIDED BY THE APP INSTANCE, INJECTED IN A COMPONENT

11.2.4 Injection keys with Symbols

String keys can clash in large apps and in libraries. Use a **Symbol** exported from a shared file as the key: it is unique and documents where the value comes from. With TypeScript, an `InjectionKey<T>` also gives

the injected value a type (Module 16.3.3). Injecting a key that nobody provided returns `undefined` and warns:

SRC/KEYS.JS JavaScript

```
export const CartKey = Symbol('cart')
```

SRC/APP.VUE HTML

```
<script setup>
import { provide, inject } from 'vue'
import { CartKey } from './keys.js'
import CartCount from './components/CartCount.vue'

provide(CartKey, { items: 3 })
console.log('missing key:', inject('notProvided'))
</script>
<template><CartCount /></template>
```

SRC/COMPONENTS/CARTCOUNT.VUE HTML

```
<script setup>
import { inject } from 'vue'
import { CartKey } from '../keys.js'
const cart = inject(CartKey)
console.log('child injected', JSON.stringify(cart), 'via', CartKey.toString())
</script>
<template><span>{{ cart.items }}</span></template>
```

BROWSER CONSOLE

```
[Vue warn]: injection "notProvided" not found.
  at <App>
missing key: undefined
child injected {"items":3} via Symbol(cart)
```

11.3 Dynamic and async components

11.3.1 component :is=""

The built-in `<component :is="...">` renders whichever component `:is` points to and swaps it when the value changes. `:is` can be an imported component object or the name of a registered component (or an HTML tag name). Store component objects in `shallowRef` (or a plain object), not `ref`, because Vue warns if a component definition is made deeply reactive.

SRC/APP.VUE

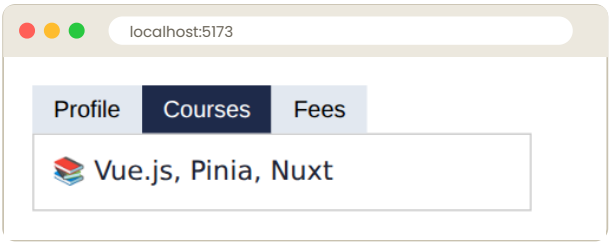
HTML

```
<script setup>
import { shallowRef } from 'vue'
import ProfileTab from './components/ProfileTab.vue'
import CoursesTab from './components/CoursesTab.vue'
import FeesTab from './components/FeesTab.vue'

const tabs = { Profile: ProfileTab, Courses: CoursesTab, Fees: FeesTab }
const current = shallowRef('Profile')
</script>

<template>
  <nav>
    <button v-for="(_, name) in tabs" :key="name" :class="{ on: current === name }" @click="current =
name">{{ name }}</button>
  </nav>
  <component :is="tabs[current]" class="panel" />
</template>

<style>
nav button { border: 0; padding: 6px 12px; background: #e2e8f0; } nav .on { background: #1E2A4A;
color: white; }
.panel { border: 1px solid #ccc; padding: 10px; width: 260px; }
</style>
```



AFTER CLICKING THE COURSES TAB

11.3.2 defineAsyncComponent() for lazy loading

`defineAsyncComponent(() => import('./Heavy.vue'))` creates a component that loads its code only when it is first rendered. The bundler (Vite) puts the imported file in a separate chunk, so the initial download is smaller. The options form adds a **loading** component (shown after `delay`), an **error** component, and a **timeout**.

SRC/APP.VUE

HTML

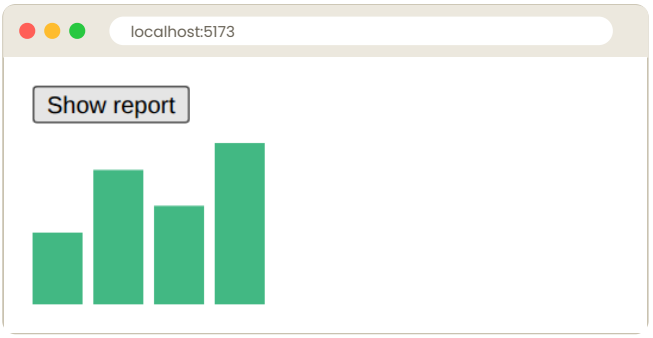
```
<script setup>
import { ref, defineAsyncComponent } from 'vue'
import LoadingSpinner from './components/LoadingSpinner.vue'

const ReportChart = defineAsyncComponent({
  loader: () => new Promise((resolve) =>      // simulate a slow network: import() after 1.5 s
    setTimeout(() => resolve(import('./components/ReportChart.vue')), 1500)),
  loadingComponent: LoadingSpinner,
  delay: 200,          // show the spinner only if loading takes longer than 200 ms
  timeout: 10000,
})
const show = ref(false)
</script>

<template>
  <button @click="show = true">Show report</button>
  <ReportChart v-if="show" />
</template>
```



HALF A SECOND AFTER CLICKING: THE CHUNK IS STILL LOADING, SO THE LOADING COMPONENT IS SHOWN



ONCE LOADED, THE REAL COMPONENT REPLACES THE PLACEHOLDER

11.4 Built-in components

11.4.1 KeepAlive for caching component state

When `<component :is>` switches, the old component is normally **unmounted** and its state (typed text, scroll position, counters) is lost. Wrapping it in `<KeepAlive>` **caches** inactive instances instead. `include` / `exclude` (component names, comma lists, regex or arrays) choose what is cached and `max` limits the cache size (least recently used is dropped).

SRC/APP.VUE

HTML

```
<script setup>
import { shallowRef } from 'vue'
import TabEditor from './components/TabEditor.vue'
import TabPreview from './components/TabPreview.vue'
const current = shallowRef(TabEditor)
</script>

<template>
  <button @click="current = TabEditor">Editor</button>
  <button @click="current = TabPreview">Preview</button>
  <div class="cols">
    <div><b>without KeepAlive</b><component :is="current" /></div>
    <div><b>with KeepAlive</b><KeepAlive><component :is="current" /></KeepAlive></div>
  </div>
</template>

<style>.cols { display: flex; gap: 20px; margin-top: 6px; } .cols > div { display: grid; }</style>
```



TYPED A DRAFT IN BOTH EDITORS, SWITCHED TO PREVIEW AND BACK: ONLY THE CACHED EDITOR KEPT ITS TEXT

CACHING ONLY SOME COMPONENTS, AT MOST FIVE

HTML

```
<KeepAlive include="TabEditor,TabSettings" :max="5">
  <component :is="current" />
</KeepAlive>
```

11.4.2 Teleport for modals and toasts

`<Teleport to="body">` renders its content in another part of the DOM (any CSS selector or element) while it stays **logically** inside the component: it still uses the component's state, props and events. Modals, toasts and tooltips need this to escape parents with `overflow: hidden`, `transform` or low `z-index`.

SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'
const open = ref(true)
</script>

<template>
  <div class="card" style="overflow: hidden">
    <p>Inside a card with overflow: hidden</p>
    <Teleport to="body">
      <div v-if="open" class="modal">I am rendered directly in body <button @click="open =
false">close</button></div>
    </Teleport>
  </div>
</template>
```

RENDERED DOM

```
<body>
  <div id="app" data-v-app="">
    <div class="card" style="overflow: hidden;">
      <p>Inside a card with overflow: hidden</p>
      <!--teleport start-->
      <!--teleport end-->
    </div>
  </div>
  <script src="main.js">
  </script>
  <div class="modal">I am rendered directly in body <button>close</button>
</div>
</body>
```

The modal's element was moved to `<body>`, after the app's root; in its original place Vue left two comments as anchors. `:disabled="true"` keeps the content in place (useful on mobile layouts), and `defer` (Vue 3.5) allows teleporting into a target rendered later in the same component tree.

11.4.3 Suspense (experimental, overview)

`<Suspense>` waits for **async dependencies** in its tree, components with an async `setup()` (in `<script setup>`, any top-level `await`) and async components, and shows the `#fallback` slot until all are resolved. It is still marked experimental, but Nuxt uses it for every page.

SRC/COMPONENTS/STUDENTPROFILE.VUE

HTML

```
<script setup>
// top-level await makes this component an async dependency
const res = await fetch('/api/students/7')
const student = await res.json()
</script>

<template><p>✅ {{ student.name }} · {{ student.course }}</p></template>
```

SRC/COMPONENTS/STUDENTSTATS.VUE

HTML

```
<script setup>
const stats = await (await fetch('/api/students/7/stats')).json()
</script>

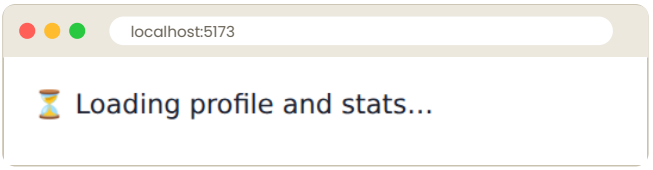
<template><p>📊 {{ stats.attendance }}% attendance</p></template>
```

SRC/APP.VUE

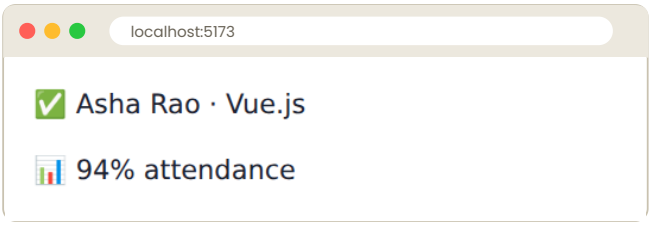
HTML

```
<script setup>
import StudentProfile from './components/StudentProfile.vue'
import StudentStats from './components/StudentStats.vue'
</script>

<template>
  <Suspense>
    <div>
      <StudentProfile />
      <StudentStats />
    </div>
    <template #fallback>
      <p>⏳ Loading profile and stats…</p>
    </template>
  </Suspense>
</template>
```



IMMEDIATELY AFTER LOADING: THE FALLBACK IS SHOWN WHILE BOTH REQUESTS ARE PENDING

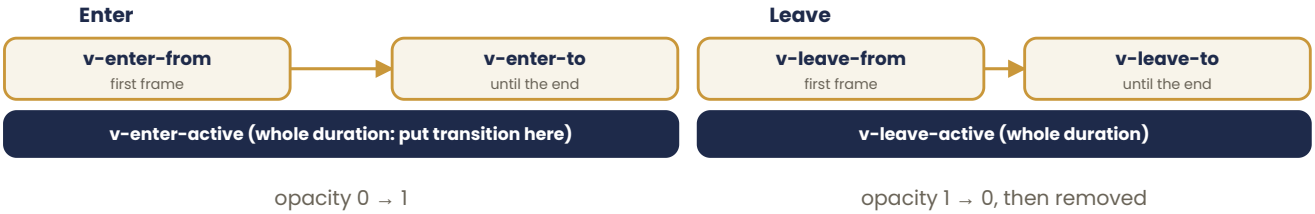


AFTER 1.2 S: BOTH ASYNC COMPONENTS RESOLVED, AND SUSPENSE SWAPPED IN THE WHOLE TREE AT ONCE

11.5 Transitions and animations

11.5.1 Transition with CSS classes

`<Transition>` animates a single element or component when it **enters** (inserted by `v-if`, shown by `v-show`, or a new key / `:is`) or **leaves**. Vue adds six CSS classes at the right moments; you write the CSS. With `name="fade"` the classes are `fade-enter-from`, `fade-enter-active`, `fade-enter-to` and the same with `leave`.



opacity 0 → 1

opacity 1 → 0, then removed

Replace "v" with the Transition's name: <Transition name="fade"> → fade-enter-from …

TRANSITION CLASSES OVER TIME

SRC/APP.VUE

HTML

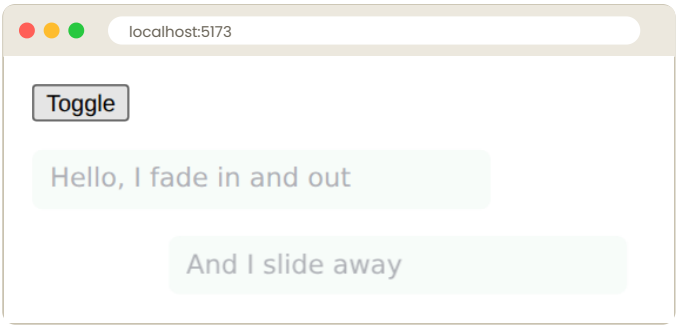
```
<script setup>
import { ref } from 'vue'
const show = ref(true)
</script>

<template>
  <button @click="show = !show">Toggle</button>
  <Transition name="fade">
    <p v-if="show" class="msg">Hello, I fade in and out</p>
  </Transition>
  <Transition name="slide">
    <p v-if="show" class="msg">And I slide away</p>
  </Transition>
</template>

<style>
.fade-enter-active, .fade-leave-active { transition: opacity 2s ease; }
.fade-enter-from, .fade-leave-to { opacity: 0; }

.slide-enter-active, .slide-leave-active { transition: all 2s ease; }
.slide-enter-from, .slide-leave-to { opacity: 0; transform: translateX(120px); }

.msg { background: #e8f5ef; padding: 6px 10px; border-radius: 6px; width: 240px; }
</style>
```



CAPTURED ABOUT 0.9 S INTO THE 2 S LEAVE TRANSITION: BOTH ELEMENTS ARE HALF FADED (AND THE SECOND HALF SLID)

`mode="out-in"` waits for the leaving element to finish before the new one enters, which avoids two elements overlapping when switching (for example between components or between `v-if` / `v-else` branches). `appear` also animates the initial render, and CSS `@keyframes` animations work the same way through `*-enter-active`.

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
const show = ref(true)
setTimeout(() => (show.value = false), 800)
</script>

<template>
  <Transition name="fade" @before-leave="(el) => console.log('before-leave:', el.className)"
              @after-leave="() => console.log('after-leave: element removed')">
    <p v-if="show" class="box">box</p>
  </Transition>
</template>

<style>
.fade-leave-active { transition: opacity .3s; }
.fade-leave-to { opacity: 0; }
</style>
```

BROWSER CONSOLE: CLASSES DURING THE LEAVE TRANSITION

```
before-leave: box
during leave: box fade-leave-active fade-leave-to
after-leave: element removed
afterwards, .box exists? false
```

11.5.2 TransitionGroup for lists

`<TransitionGroup>` animates items of a `v-for` list as they are added and removed (every child needs a key). It also adds a `name-move` class to items that change position, so with a `transform` transition, sorting and removal animate smoothly (the FLIP technique). `tag="ul"` renders a wrapper element.

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
const items = ref([1, 2, 3, 4, 5])
let next = 6
const add = () => items.value.splice(Math.floor(items.value.length / 2), 0, next++)
const remove = () => items.value.splice(1, 1)
const shuffle = () => (items.value = items.value.toSorted(() => Math.random() - 0.5))
</script>

<template>
  <button @click="add">Add</button> <button @click="remove">Remove 2nd</button> <button
@click="shuffle">Shuffle</button>
  <TransitionGroup name="list" tag="ul">
    <li v-for="item in items" :key="item">{{ item }}</li>
  </TransitionGroup>
</template>

<style>
ul { display: flex; gap: 6px; padding: 0; list-style: none; position: relative; }
li { width: 34px; height: 34px; display: grid; place-items: center; background: #1E2A4A; color: white;
border-radius: 6px; }
.list-move, .list-enter-active, .list-leave-active { transition: all 1.5s ease; }
.list-enter-from, .list-leave-to { opacity: 0; transform: translateY(30px); }
.list-leave-active { position: absolute; }
</style>
```



MID-ANIMATION AFTER CLICKING ADD: ITEM 6 IS RISING INTO PLACE WHILE ITEMS TO ITS RIGHT SLIDE OVER (LIST-MOVE)

11.5.3 JavaScript hooks and animation libraries (overview)

`<Transition>` also emits JavaScript hooks: `@before-enter`, `@enter(el, done)`, `@after-enter`, `@enter-cancelled` and the same for leave (and appear). When you animate entirely in JavaScript, call `done()` when finished and add `:css="false"` so Vue does not look for CSS transitions. This is how animation libraries such as **GSAP**, **Motion One** or the Web Animations API are integrated:

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
const show = ref(false)
setTimeout(() => (show.value = true), 20)

function onEnter(el, done) {
  console.log('enter: animating with the Web Animations API')
  const anim = el.animate([{ transform: 'scale(0)' }, { transform: 'scale(1.1)' }, { transform:
'scale(1)' }], { duration: 300 })
  anim.onfinish = () => { console.log('animation finished, calling done()'); done() }
}
</script>

<template>
  <Transition :css="false" @enter="onEnter" @after-enter="console.log('after-enter')">
    <div v-if="show" class="badge">NEW</div>
  </Transition>
</template>
```

BROWSER CONSOLE

```
enter: animating with the Web Animations API
animation finished, calling done()
after-enter
```

PRACTICE CHECKPOINT

Tabbed settings with dynamic components and KeepAlive, a teleported modal with fade transition, a theme provided app-wide with provide/inject, and a custom input using defineModel().

SRC/COMPONENTS/LABELEDINPUT.VUE HTML

```
<script setup>
const model = defineModel({ type: String, default: '' })
defineProps({ label: String })
</script>
<template>
  <label style="display:grid;margin:6px 0">{{ label }} <input v-model.trim="model"></label>
</template>
```

SRC/COMPONENTS/GENERALTAB.VUE HTML

```
<script setup>
import { ref, inject } from 'vue'
import LabeledInput from './LabeledInput.vue'
const name = ref('Asha')
const theme = inject('theme')
</script>
<template>
  <LabeledInput v-model="name" label="Display name" />
  <p>Hello {{ name }} (theme: {{ theme }})</p>
</template>
```

SRC/APP.VUE HTML

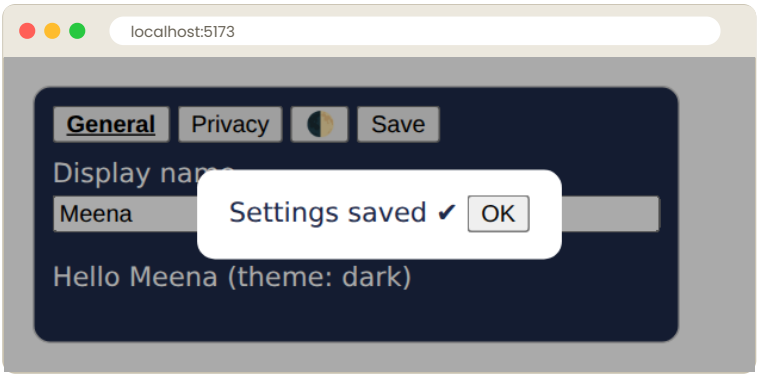
```
<script setup>
import { ref, shallowRef, provide, computed } from 'vue'
import GeneralTab from './components/GeneralTab.vue'
import PrivacyTab from './components/PrivacyTab.vue'

const theme = ref('light')
provide('theme', theme)

const tabs = { General: GeneralTab, Privacy: PrivacyTab }
const current = shallowRef('General')
const showModal = ref(false)
</script>

<template>
  <div class="app" :class="theme">
    <header>
      <button v-for="(_, t) in tabs" :key="t" :class="{ on: current === t }" @click="current = t">{{ t
}}</button>
      <button @click="theme = theme === 'light' ? 'dark' : 'light'">◐</button>
      <button @click="showModal = true">Save</button>
    </header>
    <KeepAlive><component :is="tabs[current]" /></KeepAlive>
  </div>
  <Teleport to="body">
    <Transition name="fade">
      <div v-if="showModal" class="backdrop" @click.self="showModal = false">
        <div class="modal">Settings saved ✔ <button @click="showModal = false">OK</button></div>
      </div>
    </Transition>
  </Teleport>
</template>

<style>
.app { width: 340px; padding: 10px; border-radius: 10px; border: 1px solid #ccc; }
.app.dark { background: #1E2A4A; color: white; }
header button { margin-right: 4px; } header .on { font-weight: bold; text-decoration: underline; }
.backdrop { position: fixed; inset: 0; background: #0005; display: grid; place-items: center; }
.modal { background: white; color: #1E2A4A; padding: 14px 18px; border-radius: 10px; }
.fade-enter-active, .fade-leave-active { transition: opacity .3s; } .fade-enter-from, .fade-leave-to {
opacity: 0; }
</style>
```



TYPED A NAME (KEPT BY KEEPALIVE ACROSS A TAB SWITCH), SWITCHED TO DARK THEME, AND OPENED THE TELEPORTED MODAL

Component `v-model` = `modelValue` prop + `update:modelValue` event, declared with `defineModel()` (named models and custom modifiers too). `provide` / `inject` share values down the tree without prop drilling (use Symbols and keep mutations in the provider). `<component :is>` switches components, `defineAsyncComponent` lazy-loads them, `<KeepAlive>` caches them, `<Teleport>` renders elsewhere in the DOM, `<Suspense>` waits for async setup, and `<Transition>` / `<TransitionGroup>` animate entering, leaving and moving elements.

MODULE 12 · INTERMEDIATE

Composables

Minimum duration **2 days** | Level **Intermediate**

Composables are functions that package reusable stateful logic using the Composition API. This module explains what composables are, how logic is extracted from components into them, the use prefix naming convention and why composables replaced mixins; how to write them (returning refs, accepting refs or getters through toValue(), side effects and cleanup); five common composables written from scratch (useFetch, useLocalStorage, useMouse and useWindowSize, useDebounce, useToggle); and the VueUse library.

IN THIS MODULE

12.1 What composables are
12.2 Writing composables
12.3 Common composables
12.4 VueUse library

12.1 What composables are

12.1.1 Extracting logic from components

A **composable** is a plain function that uses Vue's Composition API (`ref`, `computed`, `watch`, lifecycle hooks…) to hold and manage **state together with the logic that changes it**, and returns what the caller needs. Each call creates its own, independent state. Any logic you would otherwise copy between components (fetching, tracking the mouse, a counter with limits, form handling) can be extracted into one.

Here is a counter with limits written inside a component, and then the same logic extracted into `useCounter()` and used twice:

BEFORE: LOGIC LOCKED INSIDE ONE COMPONENT

HTML

```
<script setup>
import { ref, computed } from 'vue'
const count = ref(0)
const min = 0, max = 5
const atMax = computed(() => count.value >= max)
const inc = () => { if (count.value < max) count.value++ }
const dec = () => { if (count.value > min) count.value-- }
</script>
```

SRC/COMPOSABLES/USECOUNTER.JS

JavaScript

```
import { ref, computed } from 'vue'

export function useCounter(initial = 0, { min = 0, max = Infinity } = {}) {
  const count = ref(initial)                       // state (new for every call)
  const atMax = computed(() => count.value >= max) // derived state
  const atMin = computed(() => count.value <= min)

  function inc() { if (!atMax.value) count.value++ }  // logic
  function dec() { if (!atMin.value) count.value-- }
  function reset() { count.value = initial }

  return { count, atMax, atMin, inc, dec, reset }    // public API
}
```

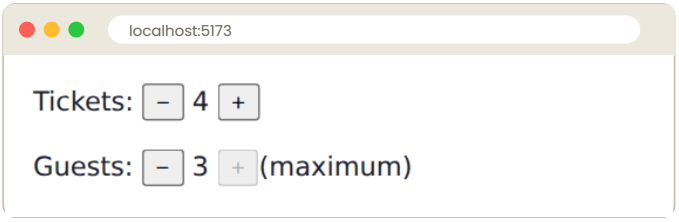
SRC/APP.VUE

HTML

```
<script setup>
import { useCounter } from './composables/useCounter.js'

const tickets = useCounter(1, { min: 1, max: 4 })
const { count: guests, inc, dec, atMax } = useCounter(0, { max: 3 })   // destructure: refs stay
reactive
</script>

<template>
  <p>Tickets: <button @click="tickets.dec()">-</button> {{ tickets.count }} <button
@click="tickets.inc()">+</button></p>
  <p>Guests: <button @click="dec">-</button> {{ guests }} <button @click="inc" :disabled="atMax">+
</button>
     <span v-if="atMax">(maximum)</span></p>
</template>
```



TWO INDEPENDENT COUNTERS FROM ONE COMPOSABLE: TICKETS CLICKED + 4 TIMES (CAPPED AT 4), GUESTS + 3 TIMES

NOTE

The template above writes `tickets.count`: a ref inside a plain returned object is not unwrapped by the template unless it is the final value of the interpolation (here it is, so it shows the number). Destructuring the returned object, as for `guests`, is the most common style and keeps each ref reactive.

12.1.2 Naming convention (use prefix)

- Name composables **useSomething** in camelCase: `useMouse`, `useFetch`, `useCart`. The prefix tells readers that the function holds reactive state and may register hooks.
- Put them in `src/composables/`, one file per composable, named after it (`useFetch.js`).
- Call them **synchronously** at the top of `<script setup>` (or inside another composable), not inside event handlers, conditions or after `await`, so that their lifecycle hooks and watchers attach to the component.
- Return a **plain object of refs** (not a `reactive()` object) so callers can destructure safely.

12.1.3 Composables vs mixins (legacy)

In Vue 2 (and the Options API), shared logic was packaged as **mixins**: objects of options merged into the component. Mixins have three problems that composables solve:

Problem with mixins	Composables
Unclear source: with several mixins it is hard to tell which one provides `this.loading`	every value comes from an explicit call: `const { loading } = useFetch()`
Name collisions: two mixins defining `data.loading` silently override each other	rename while destructuring: `const { loading: usersLoading } = useFetch(...)`
Implicit communication: mixins depend on properties that other mixins or the component must define	pass inputs as arguments and return outputs explicitly
Cannot use the same mixin twice with different settings	call the composable as many times as needed

SRC/APP.VUE

HTML

```
<script>
const loadingMixin = { data: () => ({ loading: 'from loadingMixin' }) }
const otherMixin = { data: () => ({ loading: 'from otherMixin' }) }

export default {
  mixins: [loadingMixin, otherMixin],
  mounted() { console.log('this.loading =', this.loading, '(which mixin won?)') },
}
</script>
<template><p>{{ loading }}</p></template>
```

BROWSER CONSOLE: MIXIN NAME COLLISION

```
this.loading = from otherMixin (which mixin won?)
```

12.2 Writing composables

12.2.1 Returning refs

A composable returns refs (and functions). The caller decides whether to keep the object or destructure it. If a caller prefers object-style access, it can wrap the result: `const mouse = reactive(useMouse())` unwraps the refs, so `mouse.x` needs no `.value`.

12.2.2 Accepting refs or getters as arguments with toValue()

A good composable accepts its inputs as plain values, **refs** or **getters**. `toValue(x)` (Vue 3.3+) normalises all three: it returns the value, the ref's `.value`, or the getter's result. Calling `toValue()` inside a `watchEffect` or `computed` makes the composable **react** when the input is a ref or getter.

SRC/COMPOSABLES/USETITLECASE.JS

JavaScript

```
import { computed, toValue } from 'vue'

// accepts a value, a ref or a getter
export function useTitleCase(input) {
  return computed(() =>
    toValue(input).replace(/\b\w/g, (ch) => ch.toUpperCase()),
  )
}
```

SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'
import { useTitleCase } from './composables/useTitleCase.js'

const name = ref('asha rao')
const a = useTitleCase('plain value')           // value
const b = useTitleCase(name)                    // ref
const c = useTitleCase(() => name.value + ' (B12)')  // getter

console.log(a.value, '|', b.value, '|', c.value)
setTimeout(() => { name.value = 'meena iyer'; console.log(a.value, '|', b.value, '|', c.value) }, 10)
</script>
<template><p></p></template>
```

BROWSER CONSOLE

```
Plain Value | Asha Rao | Asha Rao (B12)
Plain Value | Meena Iyer | Meena Iyer (B12)
```

12.2.3 Side effects and cleanup inside composables

Composables may start side effects: event listeners, timers, subscriptions. Register them in `onMounted` (when they need the DOM) and remove them in `onUnmounted`, so every component that uses the composable cleans up after itself automatically. For composables that may also be used outside components, `onScopeDispose()` is the general-purpose cleanup hook (it runs when the component or effect scope that called the composable is destroyed).

SRC/COMPOSABLES/USEINTERVAL.JS

JavaScript

```
import { onMounted, onUnmounted } from 'vue'

export function useInterval(callback, ms) {
  let id
  onMounted(() => { id = setInterval(callback, ms); console.log('interval started') })
  onUnmounted(() => { clearInterval(id); console.log('interval cleared') })
}
```

SRC/COMPONENTS/CLOCK.VUE

HTML

```
<script setup>
import { ref } from 'vue'
import { useInterval } from '../composables/useInterval.js'
const ticks = ref(0)
useInterval(() => console.log('tick', ++ticks.value), 100)
</script>
<template><p>{{ ticks }}</p></template>
```

BROWSER CONSOLE

```
interval started
tick 1
tick 2
tick 3
interval cleared
```

12.3 Common composables

The following five composables are written from scratch to show the techniques; section 12.4 shows their ready-made VueUse equivalents.

12.3.1 useFetch

`useFetch(url)` returns `data`, `error` and `loading` refs, and refetches whenever the URL (a ref or getter) changes, aborting the outdated request.

SRC/COMPOSABLES/USEFETCH.JS

JavaScript

```
import { ref, watchEffect, toValue, onWatcherCleanup } from 'vue'

export function useFetch(url) {
  const data = ref(null)
  const error = ref(null)
  const loading = ref(false)

  watchEffect(async () => {
    const controller = new AbortController()
    onWatcherCleanup(() => controller.abort())     // cancel the previous request
    const target = toValue(url)                    // tracked: refetch on change
    loading.value = true
    error.value = null
    try {
      const res = await fetch(target, { signal: controller.signal })
      if (!res.ok) throw new Error(`HTTP ${res.status}`)
      data.value = await res.json()
    } catch (e) {
      if (e.name !== 'AbortError') error.value = e.message
    } finally {
      loading.value = false
    }
  })

  return { data, error, loading }
}
```

SRC/APP.VUE

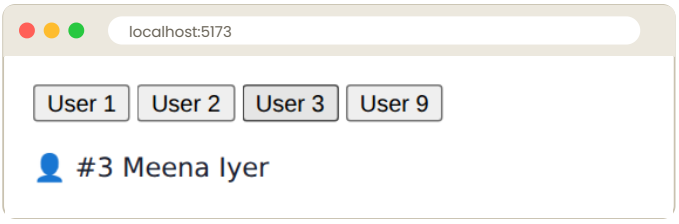
HTML

```
<script setup>
import { ref } from 'vue'
import { useFetch } from './composables/useFetch.js'

const id = ref(1)
const { data: user, error, loading } = useFetch(() => `/api/users/${id.value}`)
</script>

<template>
  <button v-for="n in [1, 2, 3, 9]" :key="n" @click="id = n">User {{ n }}</button>
  <p v-if="loading">Loading…</p>
  <p v-else-if="error" class="err">Error: {{ error }}</p>
  <p v-else-if="user">👤 #{{ user.id }} {{ user.name }}</p>
</template>

<style>.err { color: crimson; } button { margin-right: 4px; }</style>
```



AFTER CLICKING "USER 3": THE COMPOSABLE REFETCHED BECAUSE THE GETTER'S VALUE CHANGED



USER 9 DOES NOT EXIST: THE SERVER ANSWERED 404 AND THE COMPOSABLE EXPOSED THE ERROR

12.3.2 useLocalStorage

`useLocalStorage(key, initial)` returns a ref that starts from the stored value and saves every change, deeply, back to `localStorage` with a watcher.

SRC/COMPOSABLES/USELOCALSTORAGE.JS

JavaScript

```
import { ref, watch } from 'vue'

export function useLocalStorage(key, initialValue) {
  const stored = localStorage.getItem(key)
  const value = ref(stored !== null ? JSON.parse(stored) : initialValue)

  watch(value, (v) => localStorage.setItem(key, JSON.stringify(v)), { deep: true })

  return value
}
```

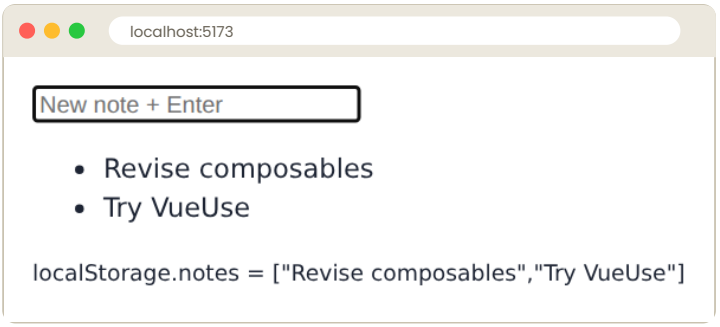
SRC/APP.VUE

HTML

```
<script setup>
import { useLocalStorage } from './composables/useLocalStorage.js'

const notes = useLocalStorage('notes', [])
const draft = useLocalStorage('draft', '')
function add() { if (draft.value.trim()) { notes.value.push(draft.value.trim()); draft.value = '' } }
</script>

<template>
  <input v-model="draft" @keyup.enter="add" placeholder="New note + Enter">
  <ul><li v-for="(n, i) in notes" :key="i">{{ n }}</li></ul>
  <small>localStorage.notes = {{ JSON.stringify(notes) }}</small>
</template>
```



TWO NOTES ADDED; AFTER A PAGE RELOAD THEY WOULD STILL BE THERE

12.3.3 useMouse and useWindowSize

SRC/COMPOSABLES/USEMOUSE.JS

JavaScript

```
import { ref, onMounted, onUnmounted } from 'vue'

export function useMouse() {
  const x = ref(0)
  const y = ref(0)
  const update = (e) => { x.value = e.pageX; y.value = e.pageY }
  onMounted(() => window.addEventListener('mousemove', update))
  onUnmounted(() => window.removeEventListener('mousemove', update))
  return { x, y }
}
```

SRC/COMPOSABLES/USEWINDOWSIZE.JS

JavaScript

```
import { ref, onMounted, onUnmounted } from 'vue'

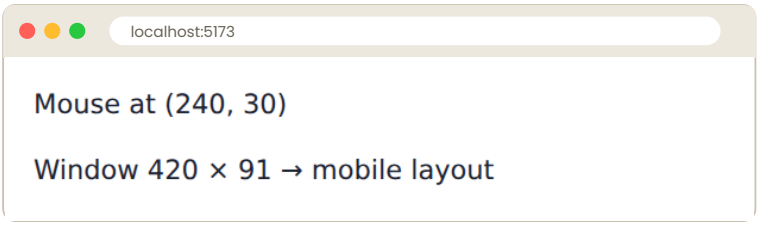
export function useWindowSize() {
  const width = ref(window.innerWidth)
  const height = ref(window.innerHeight)
  const update = () => { width.value = window.innerWidth; height.value = window.innerHeight }
  onMounted(() => window.addEventListener('resize', update))
  onUnmounted(() => window.removeEventListener('resize', update))
  return { width, height }
}
```

SRC/APP.VUE HTML

```
<script setup>
import { useMouse } from './composables/useMouse.js'
import { useWindowSize } from './composables/useWindowSize.js'

const { x, y } = useMouse()
const { width, height } = useWindowSize()
</script>

<template>
  <p>Mouse at ({{ x }}, {{ y }})</p>
  <p>Window {{ width }} × {{ height }} → {{ width < 500 ? 'mobile' : 'desktop' }} layout</p>
</template>
```



AFTER MOVING THE REAL MOUSE TO (240, 30) IN A 420 PX WIDE WINDOW

12.3.4 useDebounce

Debouncing waits until a value has stopped changing for a given time before using it, for example to search only after the user pauses typing instead of on every keystroke. `useDebounce(source, ms)` returns a ref that follows the source with a delay.

SRC/COMPOSABLES/USEDEBOUNCE.JS JavaScript

```
import { ref, watch, onUnmounted } from 'vue'

export function useDebounce(source, delay = 300) {
  const debounced = ref(source.value)
  let timer
  watch(source, (value) => {
    clearTimeout(timer)
    timer = setTimeout(() => (debounced.value = value), delay)
  })
  onUnmounted(() => clearTimeout(timer))
  return debounced
}
```

SRC/APP.VUE HTML

```
<script setup>
import { ref, watch } from 'vue'
import { useDebounce } from './composables/useDebounce.js'

const query = ref('')
const debounced = useDebounce(query, 300)

watch(query, (q) => console.log('typed:', q))
watch(debounced, (q) => console.log('>> search for:', q))

const typing = ['v', 'vu', 'vue', 'vue r', 'vue ro', 'vue router']
typing.forEach((t, i) => setTimeout(() => (query.value = t), i * 80))   // fast typing
</script>
<template><p>{{ debounced }}</p></template>
```

BROWSER CONSOLE

```
typed: v
typed: vu
typed: vue
typed: vue r
typed: vue ro
typed: vue router
>> search for: vue router
```

12.3.5 useToggle

SRC/COMPOSABLES/USETOGGLE.JS — JavaScript

```javascript
import { ref } from 'vue'

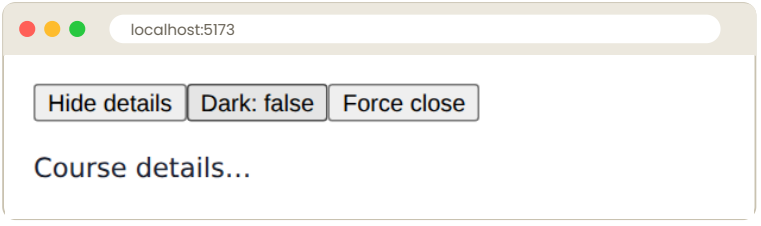
export function useToggle(initial = false) {
  const state = ref(initial)
  const toggle = (value) => {
    state.value = typeof value === 'boolean' ? value : !state.value
  }
  return [state, toggle]        // array: callers choose the names
}
```

SRC/APP.VUE — HTML

```html
<script setup>
import { useToggle } from './composables/useToggle.js'

const [isOpen, toggleOpen] = useToggle()
const [isDark, toggleDark] = useToggle(true)
</script>

<template>
  <button @click="toggleOpen()">{{ isOpen ? 'Hide' : 'Show' }} details</button>
  <button @click="toggleDark()">Dark: {{ isDark }}</button>
  <button @click="toggleOpen(false)">Force close</button>
  <p v-if="isOpen">Course details…</p>
</template>
```



AFTER "SHOW DETAILS" AND ONE CLICK ON DARK

TIP

Returning an **array** (like React's `useState`) lets each caller name the values freely; returning an **object** is better when there are more than two values, because callers can pick only the ones they need.

12.4 VueUse library

12.4.1 Popular VueUse composables

VueUse (`npm i @vueuse/core`) is a community collection of more than 200 tested composables for browser APIs, sensors, state, timing, animation and more. It works with any Vue 3 app and with Nuxt, is tree-shakable (only what you import is bundled), and handles cleanup and SSR edge cases for you. Before writing a composable, check whether VueUse already has it.

Composable	Returns / does
`useMouse()`	`{ x, y }` pointer position
`useWindowSize()`	`{ width, height }`
`useLocalStorage(key, init)` / `useStorage`	a ref synced with storage (also across tabs)
`refDebounced(ref, ms)`, `useDebounceFn(fn, ms)`, `useThrottleFn`	debounced ref or function
`useToggle(init)`	`[value, toggle]`
`useFetch(url).json()`	`{ data, error, isFetching, execute }`
`onClickOutside(elRef, handler)`	close dropdowns and modals
`useDark()`, `useTitle()`, `useOnline()`, `useClipboard()`	theme, document title, network status, copy text
`useIntervalFn`, `useNow`, `useDateFormat`	timers and time display
`useVirtualList`, `useElementSize`, `useIntersectionObserver`	large lists and layout (Module 17)

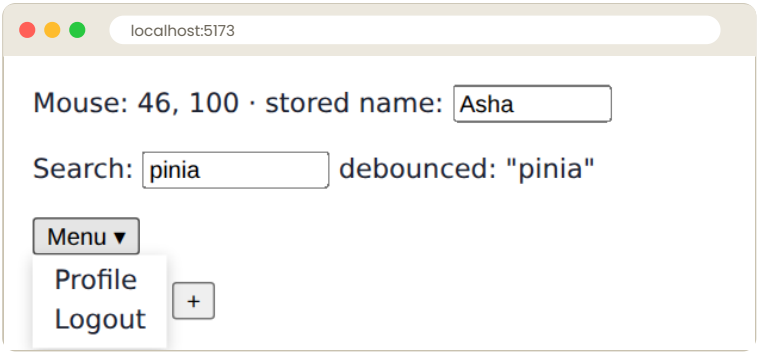
SRC/APP.VUE HTML

```
<script setup>
import { ref, useTemplateRef } from 'vue'
import { useMouse, useLocalStorage, refDebounced, useToggle, onClickOutside, useCounter } from
'@vueuse/core'

const { x, y } = useMouse()
const name = useLocalStorage('vueuse-name', 'Asha')
const search = ref('')
const debounced = refDebounced(search, 400)
const [menuOpen, toggleMenu] = useToggle(false)
const menu = useTemplateRef('menu')
onClickOutside(menu, () => (menuOpen.value = false))
const { count, inc, dec } = useCounter(1, { min: 1, max: 5 })
</script>

<template>
  <p>Mouse: {{ x }}, {{ y }} · stored name: <input v-model="name" size="8"></p>
  <p>Search: <input v-model="search" size="10"> debounced: "{{ debounced }}"</p>
  <div ref="menu" class="menu">
    <button @click="toggleMenu()">Menu ▾</button>
    <ul v-if="menuOpen"><li>Profile</li><li>Logout</li></ul>
  </div>
  <p>Qty: <button @click="dec()">−</button> {{ count }} <button @click="inc()">+</button></p>
</template>

<style>.menu { position: relative; display: inline-block; } .menu ul { position: absolute; background:
white; box-shadow: 0 2px 8px #0003; margin: 0; padding: 4px 12px; list-style: none; }</style>
```



SEVERAL VUEUSE COMPOSABLES TOGETHER: THE MENU IS OPEN, "PINIA" WAS TYPED (AND DEBOUNCED), QUANTITY +2

PRACTICE CHECKPOINT

Write useFetch, useLocalStorage and useDebounce composables and use them across three components.

The three composables from 12.3 live in `src/composables/`. Three components share them: a search box (useDebounce + useFetch), a favourites list (useLocalStorage) and a header that reads the same favourites key:

SRC/COMPONENTS/COURSESEARCH.VUE HTML

```
<script setup>
import { ref } from 'vue'
import { useDebounce } from '../composables/useDebounce.js'
import { useFetch } from '../composables/useFetch.js'
import { useLocalStorage } from '../composables/useLocalStorage.js'

const query = ref('')
const q = useDebounce(query, 300)
const { data: results, loading } = useFetch(() => `/api/courses?q=${encodeURIComponent(q.value)}`)
const favourites = useLocalStorage('favourites', [])
</script>

<template>
  <section>
    <h4>Search</h4>
    <input v-model="query" placeholder="course name">
    <p v-if="loading">…</p>
    <ul v-else><li v-for="c in results" :key="c">{{ c }} <button
@click="favourites.push(c)">★</button></li></ul>
  </section>
</template>
```

SRC/COMPONENTS/FAVOURITELIST.VUE HTML

```
<script setup>
import { useLocalStorage } from '../composables/useLocalStorage.js'
const favourites = useLocalStorage('favourites', [])
</script>

<template>
  <section>
    <h4>Favourites</h4>
    <ol><li v-for="(f, i) in favourites" :key="f">{{ f }} <button @click="favourites.splice(i,
1)">x</button></li></ol>
  </section>
</template>
```

SRC/COMPONENTS/APPHEADER.VUE HTML

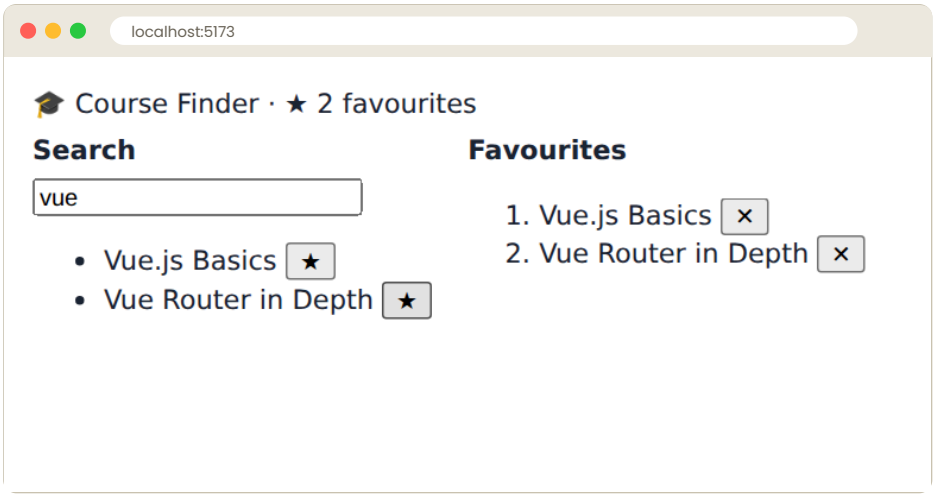
```
<script setup>
import { useLocalStorage } from '../composables/useLocalStorage.js'
const favourites = useLocalStorage('favourites', [])
</script>
<template><header>🎓 Course Finder · ★ {{ favourites.length }} favourites</header></template>
```

SRC/APP.VUE HTML

```
<script setup>
import AppHeader from './components/AppHeader.vue'
import CourseSearch from './components/CourseSearch.vue'
import FavouriteList from './components/FavouriteList.vue'
</script>

<template>
  <AppHeader />
  <div class="cols"><CourseSearch /><FavouriteList /></div>
</template>

<style>.cols { display: flex; gap: 20px; } h4 { margin: 4px 0; }</style>
```



SEARCHED "VUE" AND STARRED TWO RESULTS: THE HEADER AND THE LIST (TWO OTHER COMPONENTS) UPDATED FROM THE SHARED STORAGE REF

A composable is a `useXxx()` function that creates reactive state with the Composition API, wraps the logic around it, registers its own lifecycle cleanup, and returns refs and functions. Accept inputs as values, refs or getters and read them with `toValue()` inside reactive effects. Composables replace mixins (explicit sources, no name clashes), and VueUse provides hundreds of ready-made ones.

MODULE 13 · INTERMEDIATE

Routing with Vue Router

Minimum duration **3 days** | Level **Intermediate**

Vue Router maps URLs to components to build multi-page single-page applications. This module covers installing and creating the router with createRouter and createWebHistory, RouterView and RouterLink; active link classes, programmatic navigation with useRouter(), named routes, redirects and aliases; dynamic routes with params, reading params and query with useRoute(), passing params as props, reacting to param changes and catch-all 404 routes; nested routes and named views; global, per-route and in-component navigation guards and route meta for authentication and roles; lazy-loaded routes; and scroll behaviour, route transitions and page titles from meta.

IN THIS MODULE

13.1 Setup
13.2 Navigation
13.3 Dynamic routes
13.4 Nested routes and layouts
13.5 Navigation guards
13.6 Lazy loading routes
13.7 Other features

13.1 Setup

13.1.1 Installing and creating the router

A **single-page application** (SPA) loads one HTML page and then swaps components as the URL changes, without full page reloads. **Vue Router** is the official router. Install it with `npm install vue-router` (or choose "Router" in `npm create vue@latest`), define the routes in `src/router/index.js`, and install the router into the app with `app.use(router)`.

Terminal (representative)

```
$ npm install vue-router

added 2 packages in 1s
```

13.1.2 createRouter and createWebHistory

`createRouter()` takes a **history mode** and an array of **routes**, each mapping a `path` to a `component`. `createWebHistory()` uses normal URLs (`/about`) through the browser History API; it needs the server to send `index.html` for every path (Module 21.1.3). `createWebHashHistory()` uses `/#/about` and works on any static host. `createMemoryHistory()` keeps the URL in memory, for tests and server rendering.

SRC/ROUTER/INDEX.JS

JavaScript

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import AboutView from '../views/AboutView.vue'
import CoursesView from '../views/CoursesView.vue'

const routes = [
  { path: '/', component: HomeView },
  { path: '/about', component: AboutView },
  { path: '/courses', component: CoursesView },
]

export const router = createRouter({
  history: createWebHistory(),
  routes,
})
```

SRC/MAIN.JS

JavaScript

```
import { createApp } from 'vue'
import App from './App.vue'
import { router } from './router'

createApp(App)
  .use(router)          // installs RouterLink, RouterView, $router, $route
  .mount('#app')
```

13.1.3 RouterView and RouterLink

`<RouterView />` is the placeholder where the component of the current route is rendered. `<RouterLink to="/about">` renders an `<a>` that navigates **without reloading** the page (a plain `<a href>` would reload the whole app). Both are registered globally by `app.use(router)`.

SRC/APP.VUE

HTML

```
<template>
  <nav>
    <RouterLink to="/">Home</RouterLink>
    <RouterLink to="/about">About</RouterLink>
    <RouterLink to="/courses">Courses</RouterLink>
  </nav>
  <RouterView />
</template>
```



AFTER CLICKING "ABOUT": THE URL CHANGED TO /ABOUT AND ROUTERVIEW SWAPPED IN ABOUTVIEW, WITH NO PAGE RELOAD

RENDERED DOM

```
<div id="app" data-v-app="">
  <nav>
    <a href="/" class="">Home</a>
    <a href="/about" class="">About</a>
    <a href="/courses" class="router-link-active router-link-exact-active" aria-
current="page">Courses</a>
  </nav>
  <h2>📚 Courses</h2>
  <ul>
    <li>Vue.js</li>
    <li>React</li>
    <li>Angular</li>
  </ul>
</div>
```

13.2 Navigation

13.2.1 Active link classes

RouterLink adds `router-link-active` when the current URL **starts with** the link's route (so `/courses` stays active on `/courses/2`), and `router-link-exact-active` only on an **exact** match. It also sets `aria-current="page"` on the exact match. Rename the classes with the `activeClass` / `exactActiveClass` props or the router options `linkActiveClass` / `linkExactActiveClass`. The DOM above shows the classes on the "Courses" link.

13.2.2 Programmatic navigation with useRouter()

To navigate from code (after a form is saved, after login), get the router with `useRouter()` and call `router.push(location)`; `router.replace()` navigates without adding a history entry, and `router.back()` / `router.go(n)` move through history. A location can be a string or an object with `path` or `name`, `params`, `query` and `hash`. `push` returns a promise that resolves after navigation (with a failure object if it was blocked or duplicated).

SRC/APP.VUE HTML

```
<script setup>
import { useRouter } from 'vue-router'
const router = useRouter()

async function go() {
  await router.push('/about')
  console.log('1. now at', router.currentRoute.value.fullPath)
  await router.push({ path: '/courses', query: { level: 'beginner' }, hash: '#list' })
  console.log('2. now at', router.currentRoute.value.fullPath)
  const failure = await router.push('/courses?level=beginner#list')
  console.log('3. same location again:', failure?.message)
  router.back()
  setTimeout(() => console.log('4. after back():', router.currentRoute.value.fullPath), 50)
}
</script>

<template>
  <button @click="go">Run navigation</button>
  <RouterView />
</template>
```

BROWSER CONSOLE

```
1. now at /about
2. now at /courses?level=beginner#list
3. same location again: Avoided redundant navigation to current location: "/courses?
level=beginner#list".
4. after back(): /about
```

13.2.3 Named routes

Giving a route a `name` lets you link to it with `{ name, params }` instead of building URL strings. If the path changes later, named links keep working, and params are encoded automatically.

SRC/APP.VUE — HTML

```html
<template>
  <RouterLink :to="{ name: 'course', params: { id: 42 } }">Course 42</RouterLink>
  <RouterLink :to="{ name: 'home', query: { ref: 'mail' } }">Home</RouterLink>
</template>
```

RENDERED DOM

```
<div id="app" data-v-app="">
  <a href="/courses/42" class="">Course 42</a>
  <a href="/?ref=mail" class="router-link-active router-link-exact-active" aria-
current="page">Home</a>
</div>
```

13.2.4 Redirects and aliases

A **redirect** sends a URL to another one: the address bar changes to the target. It can be a path, a named location or a function of the requested route. An **alias** serves the same component under a second URL, and the address bar **keeps** the alias.

SRC/ROUTER/INDEX.JS — JavaScript

```javascript
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import CoursesView from '../views/CoursesView.vue'

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', component: HomeView },
    { path: '/home', redirect: '/' },                                  // path redirect
    { path: '/courses', component: CoursesView, alias: ['/classes'] },  // alias keeps the URL
    { path: '/search/:q', redirect: (to) => ({ path: '/courses', query: { q: to.params.q } }) },
  ],
})
```

BROWSER CONSOLE

```
/home              -> /                 component: HomeView
/classes           -> /classes          component: CoursesView
/search/vue        -> /courses?q=vue    component: CoursesView  (redirected from /search/vue)
/courses           -> /courses          component: CoursesView
```

13.3 Dynamic routes

13.3.1 Route params /product/:id

A path segment starting with a colon is a **param**: `/products/:id` matches `/products/7` and `/products/abc` with `params.id` set to the matched text. A route can have several params, a custom regular expression in parentheses (`:id(\\d+)` only digits), and modifiers: `?` optional, `+` one or more, `*` zero or more (repeatable params are arrays).

Pattern	Matches	params
`/products/:id`	`/products/7`	`{ id: '7' }`
`/users/:uid/posts/:pid`	`/users/3/posts/12`	`{ uid: '3', pid: '12' }`
`/orders/:id(\\d+)`	`/orders/55`, not `/orders/new`	`{ id: '55' }`
`/blog/:slug?`	`/blog` and `/blog/vue-tips`	`{ slug: '' }` or `{ slug: 'vue-tips' }`
`/docs/:path+`	`/docs/a/b/c`	`{ path: ['a', 'b', 'c'] }`

13.3.2 Reading params and query with useRoute()

`useRoute()` returns the **current route** as a reactive object: `params`, `query` (the `?key=value` part), `hash`, `path`, `fullPath`, `name`, `meta` and `matched`. In templates, `$route` is the same object. Params and query values are always **strings**.

SRC/ROUTER/INDEX.JS — JavaScript

```
import { createRouter, createWebHistory } from 'vue-router'
import ProductView from '../views/ProductView.vue'

export const router = createRouter({
  history: createWebHistory(),
  routes: [{ path: '/products/:id(\\d+)', name: 'product', component: ProductView }],
})
```

SRC/VIEWS/PRODUCTVIEW.VUE — HTML

```
<script setup>
import { useRoute } from 'vue-router'
const route = useRoute()
</script>

<template>
  <h2>Product {{ route.params.id }}</h2>
  <p>colour = {{ route.query.colour ?? 'any' }}, size = {{ route.query.size ?? 'any' }}</p>
  <pre>{{ { path: route.path, fullPath: route.fullPath, name: route.name, params: route.params, query:
route.query, hash: route.hash } }}</pre>
</template>
```



THE PAGE OPENED AT /PRODUCTS/7?COLOUR=BLUE&SIZE=M#REVIEWS

13.3.3 Passing params as props

Reading `useRoute()` ties a component to the router. With `props: true` on the route, the params are passed as **props** instead, so the component is reusable and easy to test. `props` can also be an object (static props) or a function that builds props from the route (for example to convert types or include query values).

SRC/ROUTER/INDEX.JS JavaScript

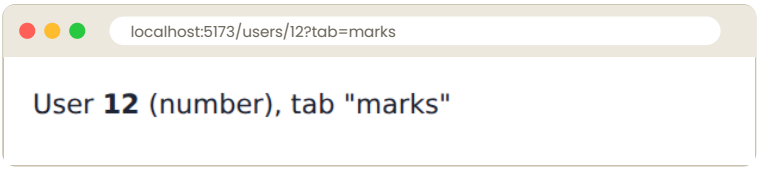
```
import { createRouter, createWebHistory } from 'vue-router'
import UserCard from '../views/UserCard.vue'

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/users/:id',
      component: UserCard,
      props: (route) => ({ id: Number(route.params.id), tab: route.query.tab || 'profile' }),
    },
  ],
})
```

SRC/VIEWS/USERCARD.VUE HTML

```
<script setup>
defineProps({ id: Number, tab: String })      // no router code at all
</script>

<template><p>User <b>{{ id }}</b> ({{ typeof id }}), tab "{{ tab }}"</p></template>
```



OPENED /USERS/12?TAB=MARKS: THE ID ARRIVED AS A NUMBER PROP

13.3.4 Reacting to param changes

When you navigate from `/courses/1` to `/courses/2`, both URLs use the **same** component, so Vue Router **reuses** the instance: `setup` and `onMounted` do not run again. To load new data, watch the param (or use the in-component guard `onBeforeRouteUpdate`, 13.5.3).

SRC/VIEWS/COURSEDETAIL.VUE HTML

```
<script setup>
import { onMounted, watch } from 'vue'
import { useRoute } from 'vue-router'

const route = useRoute()
onMounted(() => console.log('mounted (runs once)'))

watch(() => route.params.id, (id) => console.log(`load course ${id}`), { immediate: true })
</script>
<template><h2>Course {{ route.params.id }}</h2></template>
```

BROWSER CONSOLE

```
load course 1
mounted (runs once)
load course 2
load course 1
```

13.3.5 Catch-all 404 routes

A route with a param that matches everything, `/:pathMatch(.*)*`, catches every URL that no other route matched. Put it **anywhere** in the list: Vue Router ranks routes by specificity, not order, so static and simple-param routes always win over the catch-all. The unmatched segments are available as `params.pathMatch`.

SRC/ROUTER/INDEX.JS JavaScript

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import NotFound from '../views/NotFound.vue'

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/:pathMatch(.*)*', name: 'not-found', component: NotFound },   // order does not matter
    { path: '/', component: HomeView },
  ],
})
```

SRC/VIEWS/NOTFOUND.VUE

HTML

```
<script setup>
import { useRoute } from 'vue-router'
const route = useRoute()
</script>

<template>
  <h2>404 · Page not found</h2>
  <p>No page at <code>{{ route.path }}</code> (segments: {{ route.params.pathMatch }})</p>
  <RouterLink to="/">← Back home</RouterLink>
</template>
```

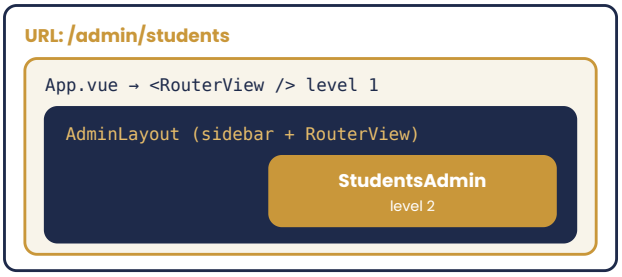


VISITING /OLD/PAGE/123

13.4 Nested routes and layouts

13.4.1 Child routes

Real apps have **layouts**: an admin area with its own sidebar, an account section with tabs. A route can have `children`; the parent component contains its own `<RouterView />`, where the matched child is rendered. Child paths without a leading slash are relative to the parent; a child with an empty path is the default shown at the parent's URL.




```
{ path: '/admin', component: AdminLayout,
children: [
{ path: '', component: Dashboard },
{ path: 'students', component: StudentsAdmin },
] }
```

EACH ROUTERVIEW RENDERS THE ROUTE RECORD AT ITS OWN NESTING LEVEL

SRC/ROUTER/INDEX.JS JavaScript

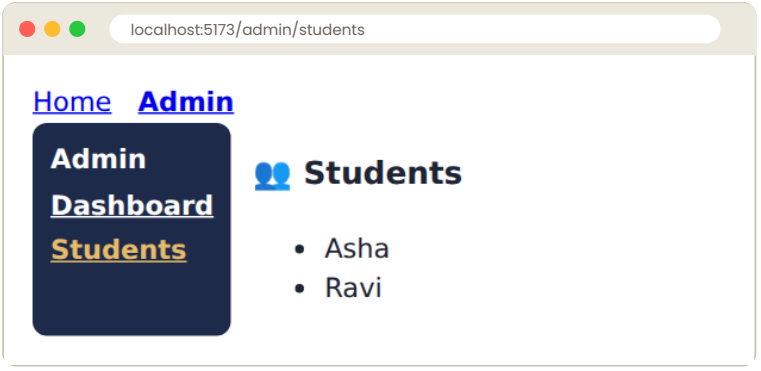
```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import AdminLayout from '../views/admin/AdminLayout.vue'
import AdminDashboard from '../views/admin/AdminDashboard.vue'
import AdminStudents from '../views/admin/AdminStudents.vue'

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', component: HomeView },
    {
      path: '/admin',
      component: AdminLayout,
      children: [
        { path: '', name: 'admin', component: AdminDashboard },         // /admin
        { path: 'students', name: 'admin-students', component: AdminStudents }, // /admin/students
      ],
    },
  ],
})
```

SRC/VIEWS/ADMIN/ADMINLAYOUT.VUE HTML

```
<template>
  <div class="admin">
    <aside>
      <b>Admin</b>
      <RouterLink :to="{ name: 'admin' }">Dashboard</RouterLink>
      <RouterLink :to="{ name: 'admin-students' }">Students</RouterLink>
    </aside>
    <main><RouterView /></main>    <!-- child routes render here -->
  </div>
</template>

<style scoped>
.admin { display: flex; gap: 12px; } aside { display: grid; gap: 4px; background: #1E2A4A; padding:
10px; border-radius: 8px; align-content: start; }
aside, aside a { color: white; } .router-link-exact-active { color: #E6BC6A; }
</style>
```



AT /ADMIN/STUDENTS AFTER CLICKING "STUDENTS" IN THE ADMIN SIDEBAR: TWO LEVELS OF ROUTERVIEW

13.4.2 Named views

To show **several components side by side** for one route (for example a main area and a sidebar that changes per page), give RouterViews a `name` and use `components` (plural) in the route, mapping each view name to a component. The unnamed RouterView is called `default`.

SRC/ROUTER/INDEX.JS JavaScript

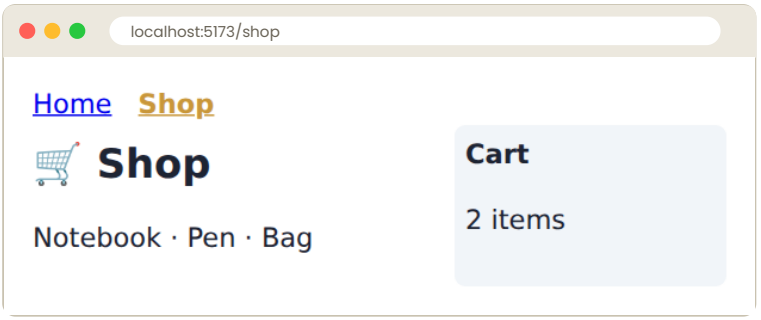
```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import ShopView from '../views/ShopView.vue'
import CartSidebar from '../views/CartSidebar.vue'
import NewsSidebar from '../views/NewsSidebar.vue'

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', components: { default: HomeView, sidebar: NewsSidebar } },
    { path: '/shop', components: { default: ShopView, sidebar: CartSidebar } },
  ],
})
```

SRC/APP.VUE HTML

```
<template>
  <nav><RouterLink to="/">Home</RouterLink> <RouterLink to="/shop">Shop</RouterLink></nav>
  <div class="layout">
    <RouterView class="main" />
    <RouterView name="sidebar" class="side" />
  </div>
</template>

<style>.layout { display: flex; gap: 10px; } .main { flex: 1; } .side { width: 140px; background:
#f1f5f9; padding: 6px; border-radius: 6px; }</style>
```



AT /SHOP: THE DEFAULT VIEW AND THE "SIDEBAR" NAMED VIEW EACH SHOW THEIR OWN COMPONENT

13.5 Navigation guards

Navigation guards run during navigation and can **allow** it, **cancel** it (return `false`) or **redirect** it (return a location). They are used for authentication, permissions, unsaved-changes warnings and analytics. Guards can be async: the navigation waits for the returned promise.



THE NAVIGATION RESOLUTION FLOW (SIMPLIFIED)

13.5.1 Global guards: beforeEach, afterEach

`router.beforeEach((to, from) => ...)` runs before **every** navigation. Return nothing or `true` to continue, `false` to cancel, or a route location to redirect. `router.afterEach((to, from, failure) => ...)` runs after navigation and cannot change it, which suits analytics and page titles.

13.5.2 Per-route beforeEnter

A `beforeEnter` guard on a route record runs only when **entering** that route (not when only its params, query or hash change). It can be a function or an array of functions, which makes reusable guards easy to attach.

13.5.3 In-component guards: onBeforeRouteLeave, onBeforeRouteUpdate

Inside a route component, `onBeforeRouteLeave` runs before navigating **away** from it (typical use: warn about unsaved changes), and `onBeforeRouteUpdate` runs when the same component is **reused** for new params (load the new data before the view updates). They have the same return values as other guards.

SRC/VIEWS/EDITVIEW.VUE

HTML

```
<script setup>
import { ref } from 'vue'
import { onBeforeRouteLeave, onBeforeRouteUpdate } from 'vue-router'

const text = ref('')
const dirty = ref(false)

onBeforeRouteUpdate((to, from) => {
  console.log(`update: ${from.params.id} -> ${to.params.id}, load new data`)
})
onBeforeRouteLeave(() => {
  if (dirty.value) {
    console.log('leave blocked: unsaved changes')   // a real app would ask with confirm()
    return false
  }
  console.log('leave allowed')
})
</script>

<template>
  <input v-model="text" @input="dirty = true">
  <button @click="dirty = false">Save</button>
</template>
```

BROWSER CONSOLE: EDIT 2, TYPE, HOME (BLOCKED), SAVE, HOME

```
update: 1 -> 2, load new data
leave blocked: unsaved changes
leave allowed
```

13.5.4 Route meta fields for authentication and roles

`meta` attaches any data to a route record, such as `{ requiresAuth: true, roles: ['admin'] }`. In a guard, `to.meta` merges the meta of the route and all its parents, so marking a parent protects all its children. The example below combines a global guard reading meta, a per-route guard and `afterEach`:

SRC/ROUTER/INDEX.JS JavaScript

```
import { createRouter, createWebHistory } from 'vue-router'
import { auth } from '../auth.js'
import HomeView from '../views/HomeView.vue'
import LoginView from '../views/LoginView.vue'
import AdminView from '../views/AdminView.vue'
import ReportsView from '../views/ReportsView.vue'

function onlyWeekdays(to) {                              // reusable per-route guard
  if (auth.day === 'Sunday') { console.log('  beforeEnter: reports closed on Sunday'); return false }
}

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', component: HomeView },
    { path: '/login', name: 'login', component: LoginView },
    {
      path: '/admin',
      meta: { requiresAuth: true, roles: ['admin'] },     // inherited by children
      children: [
        { path: '', component: AdminView },
        { path: 'reports', component: ReportsView, beforeEnter: [onlyWeekdays] },
      ],
    },
  ],
})

router.beforeEach((to, from) => {
  console.log(`beforeEach: ${from.fullPath} -> ${to.fullPath}`)
  if (to.meta.requiresAuth && !auth.user) {
    return { name: 'login', query: { redirect: to.fullPath } }      // redirect
  }
  if (to.meta.roles && !to.meta.roles.includes(auth.user?.role)) {
    console.log('  denied: role', auth.user?.role)
    return false                                                    // cancel
  }
})
router.afterEach((to, from, failure) => {
  console.log(failure ? `afterEach: FAILED (${failure.message})` : `afterEach: arrived at
${to.fullPath}`)
})
```

BROWSER CONSOLE

```
beforeEach: / -> /
afterEach: arrived at /
--- 1. not logged in
beforeEach: / -> /admin
beforeEach: / -> /login?redirect=/admin
afterEach: arrived at /login?redirect=/admin
--- 2. logged in as student
beforeEach: /login?redirect=/admin -> /admin
  denied: role student
afterEach: FAILED (Navigation aborted from "/login?redirect=/admin" to "/admin" via a navigation
guard.)
--- 3. logged in as admin
beforeEach: /login?redirect=/admin -> /admin
afterEach: arrived at /admin
--- 4. reports on Sunday
beforeEach: /admin -> /admin/reports
  beforeEnter: reports closed on Sunday
afterEach: FAILED (Navigation aborted from "/admin" to "/admin/reports" via a navigation guard.)
```

13.6 Lazy loading routes

13.6.1 Dynamic imports for route components

Importing every page at the top of `router/index.js` puts them all in the main bundle. With `component: () => import('../views/AdminView.vue')`, Vite splits each page into its own chunk, and the router loads it only when the route is first visited (and caches it). This makes the first load faster; use it for every route except the most common landing page.

SRC/ROUTER/INDEX.JS (LAZY ROUTES) — JavaScript

```javascript
const routes = [
  { path: '/', component: HomeView },                                  // eager: in the main bundle
  { path: '/about', component: () => import('../views/AboutView.vue') }, // lazy: own chunk
  {
    path: '/admin',
    component: () => import('../views/admin/AdminLayout.vue'),
    children: [
      { path: 'reports', component: () => import('../views/admin/ReportsView.vue') },
    ],
  },
]
```

BROWSER CONSOLE

```
records before: function
first visit resolved the import()
records after: object -> AboutView
page shows: ℹ️ About
```

Terminal (representative): each lazy route became a separate chunk

```
$ npm run build
dist/index.html                   0.45 kB
dist/assets/AboutView-Cd3x9K1q.js     0.31 kB | gzip:  0.25 kB
dist/assets/AdminLayout-B7mQ2w8a.js   1.02 kB | gzip:  0.61 kB
dist/assets/index-DkfC9a4l.js        98.40 kB | gzip: 38.12 kB
```

13.7 Other features

13.7.1 Scroll behaviour

By default the page keeps its scroll position when the route changes. The `scrollBehavior(to, from, savedPosition)` router option returns where to scroll: `savedPosition` on back/forward (restores the old position), `{ el: to.hash }` for anchors, or `{ top: 0 }` for new pages.

SRC/ROUTER/INDEX.JS — JavaScript

```
export const router = createRouter({
  history: createWebHistory(),
  routes,
  scrollBehavior(to, from, savedPosition) {
    if (savedPosition) return savedPosition                    // back / forward
    if (to.hash) return { el: to.hash, top: 70, behavior: 'smooth' } // #section, below a fixed header
    return { top: 0 }                                           // new page: start at the top
  },
})
```

13.7.2 Route transitions

RouterView exposes the current route component through its **slot** (`v-slot="{ Component, route }"`), so you can wrap it in `<Transition>` (and `<KeepAlive>`). Using `route.meta.transition` lets each route choose its own animation.

SRC/APP.VUE — HTML

```
<template>
  <nav><RouterLink to="/">Home</RouterLink> <RouterLink to="/about">About</RouterLink></nav>
  <RouterView v-slot="{ Component, route }">
    <Transition :name="route.meta.transition || 'fade'" mode="out-in">
      <component :is="Component" :key="route.path" />
    </Transition>
  </RouterView>
</template>

<style>
.fade-enter-active, .fade-leave-active { transition: opacity .5s; }
.fade-enter-from, .fade-leave-to { opacity: 0; }
.slide-enter-active, .slide-leave-active { transition: all .5s; }
.slide-enter-from { transform: translateX(60px); opacity: 0; }
.slide-leave-to { transform: translateX(-60px); opacity: 0; }
</style>
```



CAPTURED DURING THE SLIDE TRANSITION INTO ABOUT (OUT-IN: HOME LEFT FIRST, ABOUT IS SLIDING IN)

13.7.3 Page titles from route meta

Store each page's title in `meta.title` and set `document.title` in a global `afterEach` hook, so the browser tab, history and bookmarks show a meaningful name.

SRC/ROUTER/INDEX.JS JavaScript

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import CoursesView from '../views/CoursesView.vue'
import CourseDetail from '../views/CourseDetail.vue'

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', component: HomeView, meta: { title: 'Home' } },
    { path: '/courses', component: CoursesView, meta: { title: 'All courses' } },
    { path: '/courses/:id', component: CourseDetail, meta: { title: (r) => `Course #${r.params.id}` }
},
  ],
})

router.afterEach((to) => {
  const t = to.meta.title
  document.title = `${typeof t === 'function' ? t(to) : t ?? 'Page'} | Student Portal`
})
```

BROWSER CONSOLE

```
/            → title: Home | Student Portal
/courses     → title: All courses | Student Portal
/courses/3   → title: Course #3 | Student Portal
```

PRACTICE CHECKPOINT

A store with Home, Products, Product Detail, Cart, Login and a protected Admin area with nested routes, lazy loading and a 404 page.

SRC/ROUTER/INDEX.JS JavaScript

```
import { createRouter, createWebHistory } from 'vue-router'
import { session } from '../session.js'
import HomeView from '../views/HomeView.vue'

const routes = [
  { path: '/', name: 'home', component: HomeView, meta: { title: 'Home' } },
  { path: '/products', name: 'products', component: () => import('../views/ProductsView.vue'), meta: {
title: 'Products' } },
  { path: '/products/:id(\\d+)', name: 'product', component: () =>
import('../views/ProductDetail.vue'), props: (r) => ({ id: Number(r.params.id) }) },
  { path: '/cart', name: 'cart', component: () => import('../views/CartView.vue'), meta: { title:
'Cart' } },
  { path: '/login', name: 'login', component: () => import('../views/LoginView.vue'), meta: { title:
'Login' } },
  {
    path: '/admin',
    component: () => import('../views/admin/AdminLayout.vue'),
    meta: { requiresAdmin: true, title: 'Admin' },
    children: [
      { path: '', name: 'admin', component: () => import('../views/admin/AdminOrders.vue') },
      { path: 'stock', name: 'admin-stock', component: () => import('../views/admin/AdminStock.vue')
},
    ],
  },
  { path: '/:pathMatch(.*)*', name: 'not-found', component: () => import('../views/NotFound.vue'),
meta: { title: 'Not found' } },
]

export const router = createRouter({ history: createWebHistory(), routes, scrollBehavior: () => ({
top: 0 }) })

router.beforeEach((to) => {
  if (to.meta.requiresAdmin && !session.isAdmin) return { name: 'login', query: { redirect:
to.fullPath } }
})
router.afterEach((to) => { document.title = `${to.meta.title ?? 'Shop'} | LBF Store` })
```

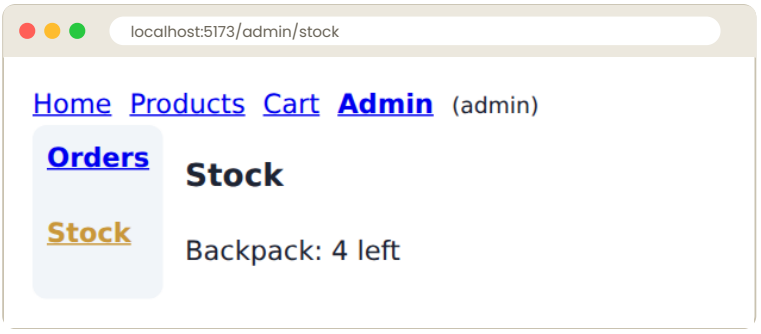
SRC/APP.VUE HTML

```
<script setup>
import { session } from './session.js'
</script>

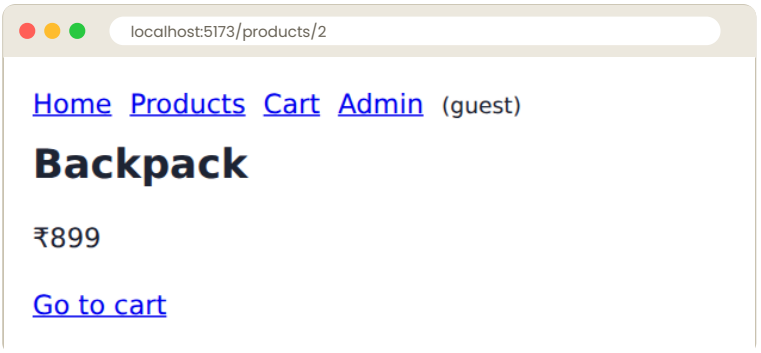
<template>
  <nav>
    <RouterLink to="/">Home</RouterLink>
    <RouterLink to="/products">Products</RouterLink>
    <RouterLink to="/cart">Cart</RouterLink>
    <RouterLink to="/admin">Admin</RouterLink>
    <small>{{ session.isAdmin ? '(admin)' : '(guest)' }}</small>
  </nav>
  <RouterView />
</template>
```



A GUEST CLICKED "ADMIN": THE GUARD REDIRECTED TO /LOGIN?REDIRECT=/ADMIN



AFTER "LOG IN AS ADMIN" AND THE STOCK LINK: THE PROTECTED NESTED ROUTE RENDERED INSIDE THE LAZILY LOADED ADMINLAYOUT



PRODUCT DETAIL (/PRODUCTS/2) REACHED FROM THE PRODUCTS PAGE; THE ID PARAM ARRIVED AS A NUMERIC PROP

`createRouter({ history: createWebHistory(), routes })` + `app.use(router)`; `<RouterLink>` navigates and `<RouterView>` renders the match. Navigate in code with `useRouter().push()`; read the current route with `useRoute()` (params and query are strings, or pass them as props). Use named routes, redirects, aliases, params with regex, a catch-all 404, nested children with their own RouterView, and named views. Guards (`beforeEach`, `beforeEnter`, `onBeforeRouteLeave` / `Update`, `afterEach`) with `meta` protect routes; lazy `() => import()` components split the bundle.

MODULE 14 · INTERMEDIATE

State management with Pinia

Minimum duration **2 days** | Level **Intermediate**

Pinia is Vue's official state management library for sharing state across the whole app. This module explains when local state is not enough and how Pinia relates to Vuex; installing and registering Pinia; defining stores with defineStore() in the setup (Composition API) style and the option style with state, getters and actions; using stores in components, keeping reactivity with storeToRefs(), getters, sync and async actions, and the store methods $reset, $patch and $subscribe; organising one store per domain and using one store inside another; a plugin that persists state to localStorage; and inspecting stores in Vue DevTools.

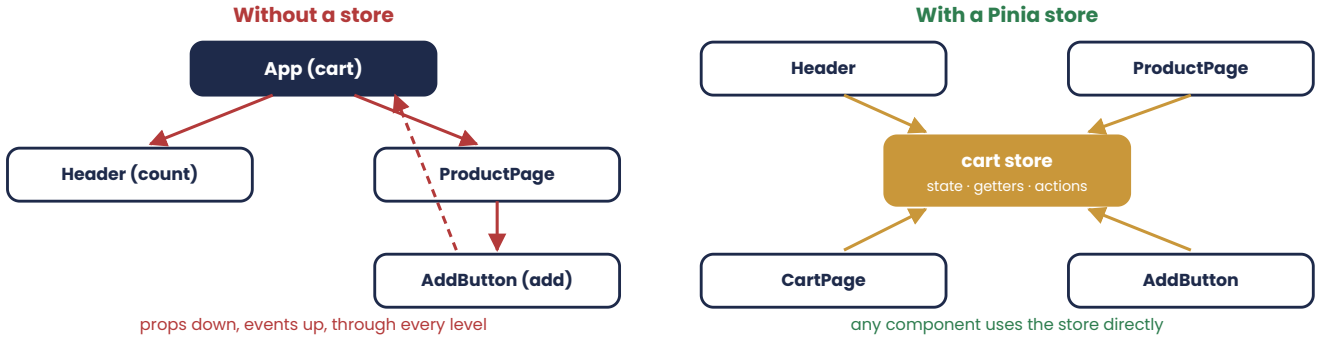
IN THIS MODULE

14.1 Why a store
14.2 Pinia basics
14.3 Using stores
14.4 Organising stores
14.5 Plugins
14.6 Pinia in Vue DevTools

14.1 Why a store

14.1.1 Local state vs shared state

State that belongs to one component (an input's text, whether a dropdown is open) stays **local** in that component. Some state is needed by **many components in different parts of the tree**: the logged-in user, the shopping cart, a list of products, a theme. Passing it through props and events across distant components becomes tangled, and provide/inject works only down one subtree. A **store** is a single, global, reactive place for such shared state and the logic that changes it.



A STORE REMOVES LONG CHAINS OF PROPS AND EVENTS

14.1.2 Pinia vs Vuex (legacy)

Vuex was the official store for Vue 2. **Pinia** (started as an experiment for "Vuex 5") is now the official recommendation, and Vuex is in maintenance mode.

	Vuex 4	Pinia
Structure	one store with nested modules	many small, independent stores

	Vuex 4	Pinia
Changing state	`commit` mutations (sync) + `dispatch` actions	actions only (sync or async), or direct assignment
Boilerplate	mutation types, namespaced strings	plain functions
TypeScript	weak inference	full inference, no extra types
Composition API	awkward	setup stores are composables
Size	about 10 KB	about 1.5 KB

14.2 Pinia basics

14.2.1 Installing and registering Pinia

Install with `npm install pinia` (or choose "Pinia" in create-vue), create the Pinia instance with `createPinia()` and register it with `app.use()` before mounting.

SRC/MAIN.JS

JavaScript

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'

createApp(App)
  .use(createPinia())
  .mount('#app')
```

14.2.2 defineStore()

`defineStore(id, definition)` defines a store and returns a **function**, by convention named `useXxxStore`. The `id` must be unique in the app (DevTools shows it). The store itself is created the first time the function is called and then **shared**: every component that calls `useCartStore()` gets the same instance.

14.2.3 Setup stores (Composition API style)

A **setup store** passes a function, just like `<script setup>`: `ref()`s become **state**, `computed()`s become **getters**, and functions become **actions**. Everything the store should expose must be returned. Setup stores can use watchers, other composables and `inject()`.

SRC/STORES/CART.JS (SETUP STORE) JavaScript

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useCartStore = defineStore('cart', () => {
  // state
  const items = ref([])
  // getters
  const count = computed(() => items.value.reduce((n, i) => n + i.qty, 0))
  const total = computed(() => items.value.reduce((s, i) => s + i.price * i.qty, 0))
  // actions
  function add(product) {
    const line = items.value.find((i) => i.id === product.id)
    if (line) line.qty++
    else items.value.push({ ...product, qty: 1 })
  }
  function remove(id) {
    items.value = items.value.filter((i) => i.id !== id)
  }
  return { items, count, total, add, remove }
})
```

14.2.4 Option stores: state, getters, actions

An **option store** passes an object, like the Options API: `state` is a function returning the initial state, `getters` receive `state` (and can use `this` for other getters), and `actions` use `this` to reach state, getters and other actions.

SRC/STORES/COUNTER.JS (OPTION STORE) JavaScript

```
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', {
  state: () => ({ count: 0, step: 1, history: [] }),
  getters: {
    double: (state) => state.count * 2,
    isBig() { return this.double > 10 },           // uses another getter
  },
  actions: {
    increment() {
      this.count += this.step
      this.history.push(this.count)
    },
    async incrementLater(ms = 500) {
      await new Promise((r) => setTimeout(r, ms))
      this.increment()
    },
  },
})
```

	Setup store	Option store
Syntax	function returning refs, computeds, functions	object with state, getters, actions
State	`ref()` / `reactive()`	`state: () => ({...})`
Getters	`computed()`	`getters: { x: (state) => ... }`
`$reset()`	not built in (write your own action)	built in

	Setup store	Option store
Extra power	watchers, composables, inject	-

14.3 Using stores

14.3.1 Accessing state in components

Call the store function in `<script setup>`. The store is a `reactive` object: state, getters and actions are accessed as properties without `.value`, and templates update when they change. Here three separate components use the same cart store:

SRC/COMPONENTS/PRODUCTLIST.VUE

HTML

```
<script setup>
import { useCartStore } from '../stores/cart.js'
const cart = useCartStore()
const products = [{ id: 1, name: 'Notebook', price: 60 }, { id: 2, name: 'Pen set', price: 120 }, {
id: 3, name: 'Backpack', price: 899 }]
</script>
<template>
  <ul><li v-for="p in products" :key="p.id">{{ p.name }} ₹{{ p.price }} <button
@click="cart.add(p)">Add</button></li></ul>
</template>
```

SRC/COMPONENTS/CARTBADGE.VUE

HTML

```
<script setup>
import { useCartStore } from '../stores/cart.js'
const cart = useCartStore()
</script>
<template><span class="badge">🛒 {{ cart.count }} · ₹{{ cart.total }}</span></template>
<style scoped>.badge { background: #1E2A4A; color: white; padding: 2px 10px; border-radius: 99px; }
</style>
```

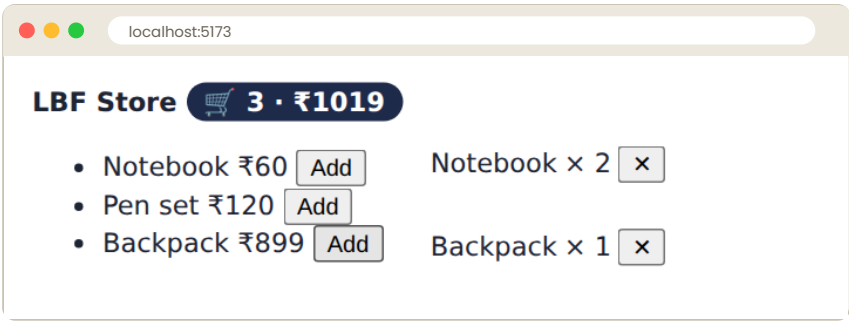
SRC/APP.VUE

HTML

```
<script setup>
import CartBadge from './components/CartBadge.vue'
import ProductList from './components/ProductList.vue'
import CartTable from './components/CartTable.vue'
</script>

<template>
  <header>LBF Store <CartBadge /></header>
  <div class="cols"><ProductList /><CartTable /></div>
</template>

<style>.cols { display: flex; gap: 24px; } header { font-weight: 600; }</style>
```



ADDED NOTEBOOK TWICE AND THE BACKPACK ONCE: THE BADGE AND THE CART TABLE (TWO OTHER COMPONENTS) UPDATED

14.3.2 storeToRefs() to keep reactivity

A store is a `reactive` object, so **destructuring** it copies plain values and **breaks reactivity** (Module 5.2.2). `storeToRefs(store)` converts the state and getters into refs that stay linked. Actions can be destructured directly from the store, because they are plain functions bound to it.

SRC/MAIN.JS JavaScript

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'

createApp(App)
  .use(createPinia())
  .mount('#app')
```

SRC/STORES/CART.JS JavaScript

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useCartStore = defineStore('cart', () => {
  // state
  const items = ref([])
  // getters
  const count = computed(() => items.value.reduce((n, i) => n + i.qty, 0))
  const total = computed(() => items.value.reduce((s, i) => s + i.price * i.qty, 0))
  // actions
  function add(product) {
    const line = items.value.find((i) => i.id === product.id)
    if (line) line.qty++
    else items.value.push({ ...product, qty: 1 })
  }
  function remove(id) {
    items.value = items.value.filter((i) => i.id !== id)
  }
  return { items, count, total, add, remove }
})
```

SRC/COMPONENTS/CARTBADGE.VUE HTML

```
<script setup>
import { useCartStore } from '../stores/cart.js'
const cart = useCartStore()
</script>
<template><span class="badge">🛒 {{ cart.count }} · ₹{{ cart.total }}</span></template>
<style scoped>.badge { background: #1E2A4A; color: white; padding: 2px 10px; border-radius: 99px; }
</style>
```

SRC/COMPONENTS/PRODUCTLIST.VUE HTML

```
<script setup>
import { useCartStore } from '../stores/cart.js'
const cart = useCartStore()
const products = [{ id: 1, name: 'Notebook', price: 60 }, { id: 2, name: 'Pen set', price: 120 }, {
id: 3, name: 'Backpack', price: 899 }]
</script>
<template>
  <ul><li v-for="p in products" :key="p.id">{{ p.name }} ₹{{ p.price }} <button
@click="cart.add(p)">Add</button></li></ul>
</template>
```

SRC/COMPONENTS/CARTTABLE.VUE HTML

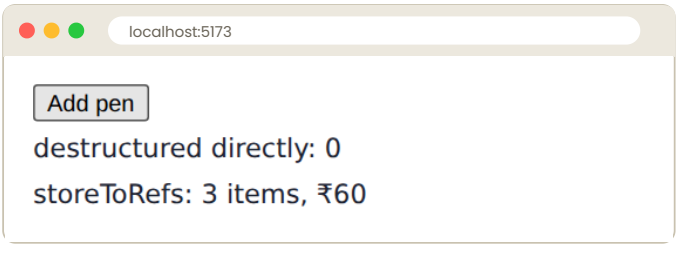
```
<script setup>
import { useCartStore } from '../stores/cart.js'
const cart = useCartStore()
</script>
<template>
  <table v-if="cart.items.length">
    <tr v-for="i in cart.items" :key="i.id"><td>{{ i.name }}</td><td>x {{ i.qty }}</td><td><button
@click="cart.remove(i.id)">x</button></td></tr>
  </table>
  <p v-else>Cart is empty</p>
</template>
```

SRC/APP.VUE HTML

```
<script setup>
import { storeToRefs } from 'pinia'
import { useCartStore } from './stores/cart.js'

const cart = useCartStore()
const { count: brokenCount } = cart                // plain copy: NOT reactive
const { count, total } = storeToRefs(cart)          // refs: reactive
const { add } = cart                                // actions: fine to destructure
</script>

<template>
  <button @click="add({ id: 1, name: 'Pen', price: 20 })">Add pen</button>
  <p>destructured directly: {{ brokenCount }}</p>
  <p>storeToRefs: {{ count }} items, ₹{{ total }}</p>
</template>
```



AFTER THREE CLICKS: ONLY THE STORETOREFS VALUES UPDATED

14.3.3 Getters for derived state

Getters are the store's computed values: cached, recalculated only when their dependencies change, and readable from any component. A getter can **return a function** to accept arguments (then it is no longer cached per argument).

SRC/STORES/PRODUCTS.JS

JavaScript

```
import { defineStore } from 'pinia'

export const useProductStore = defineStore('products', {
  state: () => ({
    products: [
      { id: 1, name: 'Notebook', category: 'stationery', price: 60, stock: 10 },
      { id: 2, name: 'Pen set', category: 'stationery', price: 120, stock: 0 },
      { id: 3, name: 'Headphones', category: 'electronics', price: 1499, stock: 4 },
      { id: 4, name: 'Mouse', category: 'electronics', price: 599, stock: 7 },
    ],
  }),
  getters: {
    inStock: (state) => state.products.filter((p) => p.stock > 0),
    countByCategory: (state) =>
      state.products.reduce((acc, p) => ({ ...acc, [p.category]: (acc[p.category] || 0) + 1 }), {}),
    cheaperThan: (state) => (max) => state.products.filter((p) => p.price < max),  // getter with
argument
  },
})
```

SRC/APP.VUE

HTML

```
<script setup>
import { useProductStore } from './stores/products.js'
const store = useProductStore()

console.log('inStock:', store.inStock.map((p) => p.name).join(', '))
console.log('by category:', JSON.stringify(store.countByCategory))
console.log('cheaper than 500:', store.cheaperThan(500).map((p) => p.name).join(', '))
store.products[2].stock = 0
console.log('after Headphones sold out:', store.inStock.map((p) => p.name).join(', '))
</script>
<template><p></p></template>
```

BROWSER CONSOLE

```
inStock: Notebook, Headphones, Mouse
by category: {"stationery":2,"electronics":2}
cheaper than 500: Notebook, Pen set
after Headphones sold out: Notebook, Mouse
```

14.3.4 Actions for logic and async calls

Actions are the store's methods. They can be synchronous or `async`, call APIs, call other actions, and change state directly (no mutations needed). Keeping data-loading logic in actions means components stay simple: they call `store.fetchStudents()` and display `store.loading`, `store.error` and `store.students`.

SRC/STORES/STUDENTS.JS JavaScript

```
import { defineStore } from 'pinia'
import { ref } from 'vue'

export const useStudentStore = defineStore('students', () => {
  const students = ref([])
  const loading = ref(false)
  const error = ref(null)

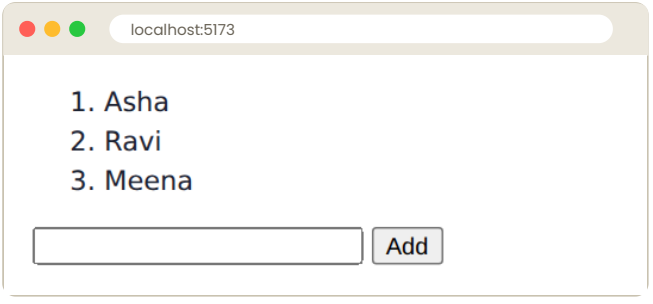
  async function fetchStudents() {
    loading.value = true
    error.value = null
    try {
      const res = await fetch('/api/students')
      if (!res.ok) throw new Error(`HTTP ${res.status}`)
      students.value = await res.json()
    } catch (e) {
      error.value = e.message
    } finally {
      loading.value = false
    }
  }
  async function addStudent(name) {
    const res = await fetch('/api/students', { method: 'POST', body: JSON.stringify({ name }),
headers: { 'Content-Type': 'application/json' } })
    students.value.push(await res.json())
  }
  return { students, loading, error, fetchStudents, addStudent }
})
```

SRC/APP.VUE HTML

```
<script setup>
import { onMounted, ref } from 'vue'
import { storeToRefs } from 'pinia'
import { useStudentStore } from './stores/students.js'

const store = useStudentStore()
const { students, loading, error } = storeToRefs(store)
const name = ref('')
onMounted(() => store.fetchStudents())
</script>

<template>
  <p v-if="loading">Loading students…</p>
  <p v-else-if="error">⚠ {{ error }}</p>
  <ol v-else><li v-for="s in students" :key="s.id">{{ s.name }}</li></ol>
  <input v-model="name"> <button @click="store.addStudent(name); name = ''">Add</button>
</template>
```



AFTER LOADING, "MEENA" WAS ADDED THROUGH THE ASYNC ADDSTUDENT ACTION (POST)

14.3.5 $reset, $patch and $subscribe

Every store has built-in methods. `$patch` changes several properties in **one** update (with an object, or a function for complex changes such as array pushes); `$reset()` restores the initial state (option stores only); `$subscribe(callback)` runs after every state change with a description of the mutation; `$onAction` observes action calls; and `$state` reads or replaces the whole state.

SRC/APP.VUE

HTML

```
<script setup>
import { useCounterStore } from './stores/counter.js'
const store = useCounterStore()

store.$subscribe((mutation, state) => {
  console.log(`  $subscribe: type="${mutation.type}" count=${state.count} step=${state.step}`)
})
store.$onAction(({ name, args, after }) => {
  console.log(`  $onAction: ${name}(${args.join(', ')}) called`)
  after(() => console.log(`  $onAction: ${name} finished, count=${store.count}`))
})

setTimeout(() => { console.log('1. direct change'); store.count = 5 }, 10)
setTimeout(() => { console.log('2. $patch object'); store.$patch({ count: 10, step: 5 }) }, 20)
setTimeout(() => { console.log('3. $patch function'); store.$patch((s) => { s.history.push(99);
s.count++ }) }, 30)
setTimeout(() => { console.log('4. action'); store.increment() }, 40)
setTimeout(() => { console.log('5. $reset'); store.$reset(); console.log('   state now',
JSON.stringify(store.$state)) }, 50)
</script>
<template><p>{{ store.count }}</p></template>
```

BROWSER CONSOLE

```
1. direct change
  $subscribe: type="direct" count=5 step=1
2. $patch object
  $subscribe: type="patch object" count=10 step=5
3. $patch function
  $subscribe: type="patch function" count=11 step=5
4. action
  $onAction: increment() called
  $onAction: increment finished, count=16
  $subscribe: type="direct" count=16 step=5
5. $reset
  $subscribe: type="patch function" count=0 step=1
   state now {"count":0,"step":1,"history":[]}
```

14.4 Organising stores

14.4.1 One store per domain (cart, auth, products)

Split state by **domain** (a business area), not by component: `useAuthStore` for the user and token, `useProductStore` for the catalogue, `useCartStore` for the cart. Each lives in its own file in `src/stores/`, and only the stores a page uses are loaded. Keep purely local UI state in components.



ONE STORE PER DOMAIN

14.4.2 Using one store inside another

A store can use another store by calling its `useXxxStore()` **inside** its setup function, getter or action (not at the top level of the module). For example, checkout in the cart store needs the logged-in user from the auth store:

SRC/STORES/CART.JS

JavaScript

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import { useAuthStore } from './auth.js'

export const useCartStore = defineStore('cart', () => {
  // state
  const items = ref([])
  // getters
  const count = computed(() => items.value.reduce((n, i) => n + i.qty, 0))
  const total = computed(() => items.value.reduce((s, i) => s + i.price * i.qty, 0))
  // actions
  function add(product) {
    const line = items.value.find((i) => i.id === product.id)
    if (line) line.qty++
    else items.value.push({ ...product, qty: 1 })
  }
  function remove(id) {
    items.value = items.value.filter((i) => i.id !== id)
  }
  async function checkout() {
    const auth = useAuthStore()                // another store, used inside an action
    if (!auth.isLoggedIn) return 'Please log in before checkout'
    const summary = `Order for ${auth.user.email}: ${count.value} item(s), ₹${total.value}`
    items.value = []
    return summary
  }
  return { items, count, total, add, remove, checkout }
})
```

BROWSER CONSOLE

```
Please log in before checkout
Order for asha@lbf.in: 1 item(s), ₹60
cart after checkout: 0 items
```

WATCH OUT

Two stores must not read each other's state **while being created** (a circular dependency at setup time). Using each other inside getters and actions is fine.

14.5 Plugins

14.5.1 Persisting state to localStorage

A **Pinia plugin** is a function that runs for every store when it is created: `pinia.use(({ store, options }) => { ... })`. It can add properties, wrap actions or subscribe to changes. A persistence plugin restores state from `localStorage` on creation and saves it on every change. (The community package **pinia-plugin-persistedstate** does this with more options; this version shows how it works.)

SRC/PLUGINS/PERSIST.JS — JavaScript

```
// a store opts in with { persist: true } as the third argument of defineStore()
export function persistPlugin({ store, options }) {
  if (!options.persist) return
  const key = `pinia:${store.$id}`
  const saved = localStorage.getItem(key)
  if (saved) store.$patch(JSON.parse(saved))                    // restore
  store.$subscribe((_mutation, state) => {                      // save on change
    localStorage.setItem(key, JSON.stringify(state))
  }, { detached: true })
}
```

SRC/MAIN.JS — JavaScript

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import { persistPlugin } from './plugins/persist.js'
import App from './App.vue'

const pinia = createPinia()
pinia.use(persistPlugin)
createApp(App).use(pinia).mount('#app')
```

BROWSER CONSOLE (AFTER A PAGE RELOAD WITH A SAVED CART)

```
restored from localStorage: 0 items
saved: {"items":[{"id":1,"name":"Pen","price":20,"qty":1}]}
restored from localStorage: 2 items
saved: {"items":[{"id":9,"name":"Saved bag","price":899,"qty":2},
{"id":1,"name":"Pen","price":20,"qty":1}]}
```

14.6 Pinia in Vue DevTools

Vue DevTools has a **Pinia** panel: it lists every store created so far, shows its state and getters (editable live), and a **timeline** records each mutation and action call with its arguments and the state before and after, which makes it easy to see who changed what. With setup stores, only returned state appears; keep all state you want to inspect in the returned object.



INSPECTING STORES, GETTERS AND THE ACTION TIMELINE (REPRESENTATIVE)

PRACTICE CHECKPOINT

Move the store's cart, auth and product data into Pinia stores, persist the cart, and show the cart count in the header.

SRC/STORES/AUTH.JS JavaScript

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
export const useAuthStore = defineStore('auth', () => {
  const user = ref(null)
  const isLoggedIn = computed(() => !!user.value)
  const login = (email) => (user.value = { email })
  const logout = () => (user.value = null)
  return { user, isLoggedIn, login, logout }
})
```

SRC/STORES/PRODUCTS.JS JavaScript

```
import { defineStore } from 'pinia'
export const useProductStore = defineStore('products', {
  state: () => ({ products: [], loading: false }),
  actions: {
    async fetchProducts() {
      this.loading = true
      this.products = await (await fetch('/api/products')).json()
      this.loading = false
    },
  },
})
```

SRC/APP.VUE HTML

```
<script setup>
import { onMounted } from 'vue'
import { storeToRefs } from 'pinia'
import { useAuthStore } from './stores/auth.js'
import { useProductStore } from './stores/products.js'
import { useCartStore } from './stores/cart.js'

const auth = useAuthStore()
const productsStore = useProductStore()
const cart = useCartStore()
const { count, total } = storeToRefs(cart)
onMounted(() => productsStore.fetchProducts())
</script>

<template>
  <header>
    <b>LBF Store</b>
    <span>🛒 {{ count }} (₹{{ total }})</span>
    <button v-if="!auth.isLoggedIn" @click="auth.login('asha@lbf.in')">Log in</button>
    <span v-else>👤 {{ auth.user.email }}</span>
  </header>
  <p v-if="productsStore.loading">Loading…</p>
  <ul v-else>
    <li v-for="p in productsStore.products" :key="p.id">{{ p.name }} ₹{{ p.price }} <button
@click="cart.add(p)">Add</button></li>
  </ul>
</template>

<style>header { display: flex; gap: 12px; align-items: center; border-bottom: 1px solid #ddd; padding-
bottom: 6px; }</style>
```



LOGGED IN, ADDED BACKPACK AND NOTEBOOK; THE HEADER COUNT COMES FROM THE PERSISTED CART STORE

Use Pinia for state shared across the app. `defineStore(id, setupFn | options)` returns `useXxxStore()`; every call returns the same store. State = refs/ `state`, getters = computeds/ `getters`, actions = functions (sync or async). Destructure state and getters only through `storeToRefs()`. Use `$patch`, `$reset`, `$subscribe` and `$onAction`, one store per domain, other stores inside actions, and plugins for cross-cutting features such as persistence.

MODULE 15 · INTERMEDIATE

Working with APIs and form validation

Minimum duration **2 days** | Level **Intermediate**

This module connects Vue apps to back-end APIs and adds robust form handling. It covers REST basics and the five HTTP methods, fetch versus Axios, Axios instances with base URLs and interceptors, environment variables with import.meta.env, an API service layer, loading, error and empty states, sending authentication tokens, CORS and the Vite dev proxy; server-state caching with TanStack Query for Vue (queries, refetching and mutations); form validation with VeeValidate and Zod or Yup schemas, field-level and form-level validation; and file uploads with FormData and image previews.

IN THIS MODULE

15.1 API communication
15.2 Server-state libraries
15.3 Form validation libraries
15.4 File uploads

15.1 API communication

15.1.1 REST basics: GET, POST, PUT, PATCH, DELETE

Most front ends talk to a **REST API**: the server exposes **resources** at URLs (`/api/students`, `/api/students/7`) and the HTTP **method** says what to do with them. Data travels as JSON, and the **status code** reports the result.

Method	URL	Meaning	Typical success
`GET`	/api/students	read the list (filters in the query string)	200 OK + array
`GET`	/api/students/7	read one	200 OK, or 404 Not Found
`POST`	/api/students	create (body = new data)	201 Created + new item
`PUT`	/api/students/7	replace the whole item	200 OK
`PATCH`	/api/students/7	update some fields	200 OK
`DELETE`	/api/students/7	remove	204 No Content

Status	Meaning
2xx	success (200 OK, 201 Created, 204 No Content)
400 / 422	invalid data sent by the client
401 / 403	not logged in / not allowed
404	resource not found
5xx	server error

15.1.2 fetch vs Axios

`fetch()` is built into browsers; **Axios** (`npm i axios`) is a popular library that wraps HTTP requests with conveniences. The same requests in both:

SRC/APP.VUE HTML

```
<script setup>
import axios from 'axios'

setTimeout(async () => {
  // fetch: manual JSON, manual error check, manual body encoding
  const res = await fetch('/api/students/1')
  if (!res.ok) throw new Error(`HTTP ${res.status}`)
  console.log('fetch:', (await res.json()).name)
  const missing = await fetch('/api/students/99')
  console.log('fetch does NOT reject on 404, ok =', missing.ok, 'status =', missing.status)

  // axios: JSON parsed for you, rejects on non-2xx, body encoded for you
  const { data } = await axios.get('/api/students/1')
  console.log('axios:', data.name)
  try {
    await axios.get('/api/students/99')
  } catch (err) {
    console.log('axios rejects on 404:', err.message, '|', err.response.data.message)
  }
}, 10)
</script>
<template><p></p></template>
```

BROWSER CONSOLE

```
fetch: Asha Rao
fetch does NOT reject on 404, ok = false status = 404
axios: Asha Rao
axios rejects on 404: Request failed with status code 404 | Student not found
```

	fetch	Axios
Install	built in	`npm i axios` (about 13 KB gzipped)
JSON	`await res.json()`	automatic: `response.data`
HTTP errors (4xx/5xx)	resolve; check `res.ok` yourself	reject with an `AxiosError` (`err.response.status`)
Request body	`JSON.stringify` + Content-Type header	pass an object
Base URL, defaults, interceptors	write your own wrapper	built in
Timeout, cancel	`AbortController`	`timeout` option, `signal`

15.1.3 Axios instances, base URLs and interceptors

`axios.create()` makes an **instance** with its own defaults: `baseURL`, `timeout`, headers. **Interceptors** run for every request or response of that instance: a request interceptor can attach the auth token, and a response interceptor can unwrap data or handle 401 errors in one place.

SRC/API/HTTP.JS

JavaScript

```
import axios from 'axios'

export const http = axios.create({
  baseURL: import.meta.env.VITE_API_URL,      // e.g. "/api" (see 15.1.4)
  timeout: 5000,
  headers: { Accept: 'application/json' },
})

// request interceptor: runs before every request
http.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  console.log(`→ ${config.method.toUpperCase()} ${config.url}`)
  return config
})

// response interceptor: runs after every response
http.interceptors.response.use(
  (response) => {
    console.log(`← ${response.status} ${response.config.url}`)
    return response
  },
  (error) => {
    const status = error.response?.status
    console.log(`← ${status ?? 'network'} error: ${error.response?.data?.message ?? error.message}`)
    if (status === 401) console.log('   (would redirect to /login here)')
    return Promise.reject(error)
  },
)
```

BROWSER CONSOLE

```
→ GET /students
← 200 /students
→ POST /students
← 401 error: Login required
   (would redirect to /login here)
→ POST /students
← 201 /students
created id 3
```

15.1.4 Environment variables with import.meta.env

Values that differ between environments (API URLs, public keys) belong in `.env` files, not in code. Vite loads `.env`, `.env.local`, `.env.[mode]` and `.env.[mode].local`; only variables prefixed with **VITE_** are exposed to the browser code, as `import.meta.env.VITE_NAME`. Built-in values: `MODE`, `DEV`, `PROD`, `BASE_URL`, `SSR`.

.ENV FILES text

```
# .env                    (all modes, committed)
VITE_APP_TITLE=Student Portal

# .env.development        (npm run dev)
VITE_API_URL=http://localhost:3000/api

# .env.production         (npm run build)
VITE_API_URL=https://api.lbf.in/v1

# .env.local              (your machine only, in .gitignore)
DB_PASSWORD=secret        # no VITE_ prefix: NOT exposed to the browser
```

SRC/APP.VUE HTML

```
<script setup>
const env = import.meta.env
</script>

<template>
  <h3>{{ env.VITE_APP_TITLE }}</h3>
  <p>mode: <b>{{ env.MODE }}</b> · dev: {{ env.DEV }} · prod: {{ env.PROD }}</p>
  <p>API: <code>{{ env.VITE_API_URL }}</code></p>
  <p>DB_PASSWORD visible? {{ env.DB_PASSWORD ?? 'no (not prefixed with VITE_)' }}</p>
</template>
```



THE VALUES VITE INJECTED IN DEVELOPMENT MODE

WATCH OUT
Everything in `import.meta.env.VITE_*` is **compiled into the JavaScript bundle** and visible to anyone. Never put secrets (database passwords, private API keys) in `VITE_` variables; keep them on the server.

15.1.5 API service layer

Instead of calling Axios from every component, put all requests for one resource in a **service module**. Components (or Pinia actions) call meaningful functions such as `studentApi.list()`; URLs, headers and data shapes live in one place, and tests can mock the service.

SRC/API/STUDENTAPI.JS

JavaScript

```
import { http } from './http.js'

export const studentApi = {
  list: (params) => http.get('/students', { params }).then((r) => r.data),
  get: (id) => http.get(`/students/${id}`).then((r) => r.data),
  create: (student) => http.post('/students', student).then((r) => r.data),
  update: (id, student) => http.put(`/students/${id}`, student).then((r) => r.data),
  patch: (id, fields) => http.patch(`/students/${id}`, fields).then((r) => r.data),
  remove: (id) => http.delete(`/students/${id}`),
}
```

15.1.6 Loading, error and empty states

Every remote request has **four** visible states: **loading** (show a spinner or skeleton), **error** (explain and offer a retry), **empty** (a friendly message, not a blank page) and **success**. Keep them in refs and render exactly one at a time.

SRC/APP.VUE

HTML

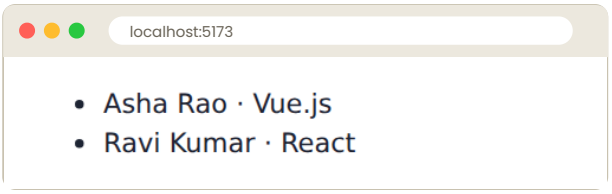
```
<script setup>
import { ref, onMounted } from 'vue'
import { studentApi } from './api/studentApi.js'

const students = ref([])
const loading = ref(false)
const error = ref(null)

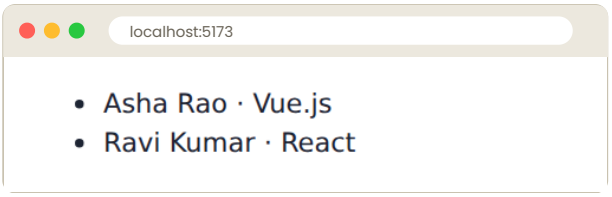
async function load() {
  loading.value = true
  error.value = null
  try {
    students.value = await studentApi.list()
  } catch (e) {
    error.value = e.response?.data?.message ?? e.message
  } finally {
    loading.value = false
  }
}
onMounted(load)
</script>

<template>
  <div v-if="loading" class="skeleton">Loading students…</div>
  <div v-else-if="error" class="error">⚠ {{ error }} <button @click="load">Retry</button></div>
  <p v-else-if="!students.length" class="empty">No students yet. Add the first one!</p>
  <ul v-else><li v-for="s in students" :key="s.id">{{ s.name }} · {{ s.course }}</li></ul>
</template>

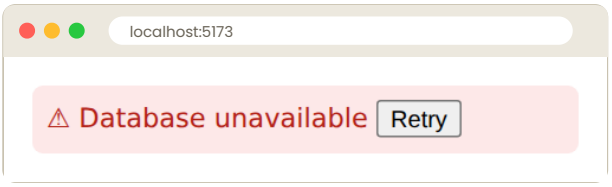
<style>
.skeleton { background: #eef2f7; padding: 10px; border-radius: 6px; color: #64748b; }
.error { background: #fde8e8; color: #b42318; padding: 8px; border-radius: 6px; } .empty { color:
#64748b; }
</style>
```



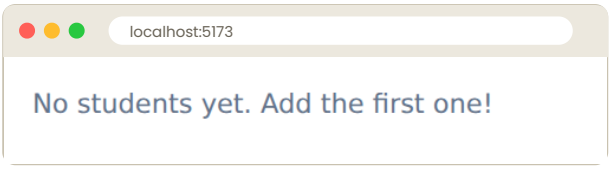
LOADING STATE (CAPTURED 100 MS AFTER START)



SUCCESS STATE



ERROR STATE (THE SERVER RETURNED 500)



EMPTY STATE (THE SERVER RETURNED [])

15.1.7 Authentication tokens

After login, the server returns a **token** (often a JWT). The client stores it and sends it in the `Authorization: Bearer <token>` header of every request, best done once in a request interceptor (15.1.3). A response interceptor handles **401** by logging out and redirecting to the login page. Where to store the token is a trade-off:

Storage	Pros	Cons
Memory (Pinia state)	not readable by other scripts after reload; simple	lost on page reload
`localStorage`	survives reloads; simple	readable by any script on the page: an XSS bug can steal it
`httpOnly` cookie (set by the server)	not readable by JavaScript at all	needs CSRF protection and same-site/CORS setup on the server

15.1.8 CORS and the Vite dev proxy

Browsers block JavaScript from reading responses from a **different origin** (protocol + host + port) unless that server allows it with **CORS** headers (`Access-Control-Allow-Origin`). In development the app runs on `http://localhost:5173` and the API on, say, `http://localhost:3000`: different origins. Instead of enabling CORS on the API, Vite's **dev proxy** forwards requests: the browser calls `/api/...` on its own origin, and the Vite server passes them to the API.

VITE.CONFIG.JS

JavaScript

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:3000',   // the real API server
        changeOrigin: true,
        // rewrite: (path) => path.replace(/^\/api/, ''),   // if the API has no /api prefix
      },
    },
  },
})
```



SAME-ORIGIN REQUEST FROM THE BROWSER; THE VITE SERVER FORWARDS IT (DEVELOPMENT ONLY)

NOTE

The proxy exists only in `npm run dev`. In production, serve the app and API from the same domain (a reverse proxy such as Nginx), or configure CORS on the API.

15.2 Server-state libraries

15.2.1 TanStack Query for Vue (caching, refetching, mutations)

Data that lives on the server ("server state") brings recurring problems: caching, de-duplicating identical requests, refetching stale data, updating lists after a change, retries. **TanStack Query** (`@tanstack/vue-query`) solves them: `useQuery({ queryKey, queryFn })` caches results by key, shares them between components, and refetches when the key changes or data is stale; `useMutation()` runs changes and lets you invalidate related queries.

SRC/MAIN.JS

JavaScript

```
import { createApp } from 'vue'
import { VueQueryPlugin } from '@tanstack/vue-query'
import App from './App.vue'

createApp(App).use(VueQueryPlugin).mount('#app')
```

SRC/COMPONENTS/STUDENTCOUNT.VUE HTML

```
<script setup>
import { useQuery } from '@tanstack/vue-query'
import { studentApi } from '../api/studentApi.js'
// same key: shares the cached data, no second request
const { data } = useQuery({ queryKey: ['students'], queryFn: () => studentApi.list() })
</script>
<template><b>{{ data?.length ?? '…' }} students</b></template>
```

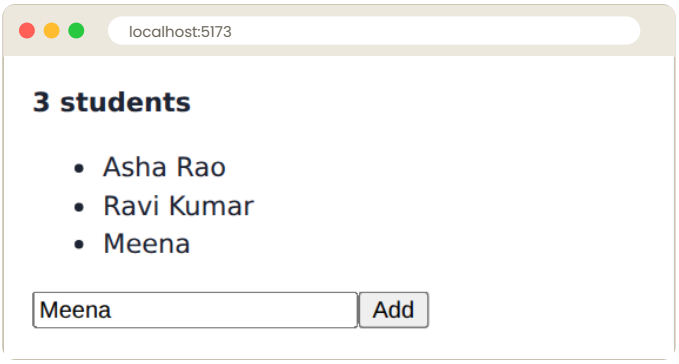
SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
import { useQuery, useMutation, useQueryClient } from '@tanstack/vue-query'
import { studentApi } from './api/studentApi.js'
import StudentCount from './components/StudentCount.vue'

const queryClient = useQueryClient()
const { data: students, isPending, isFetching } = useQuery({
  queryKey: ['students'],
  queryFn: () => studentApi.list(),
  staleTime: 30_000,                      // cached data counts as fresh for 30 s
})

const name = ref('')
const addMutation = useMutation({
  mutationFn: (student) => studentApi.create(student),
  onSuccess: () => queryClient.invalidateQueries({ queryKey: ['students'] }),   // refetch the list
})
</script>

<template>
  <StudentCount /> <small v-if="isFetching">↻ refreshing</small>
  <p v-if="isPending">Loading…</p>
  <ul v-else><li v-for="s in students" :key="s.id">{{ s.name }}</li></ul>
  <input v-model="name" placeholder="name">
  <button :disabled="addMutation.isPending.value" @click="addMutation.mutate({ name, course: 'Vue.js',
marks: 0 })">Add</button>
</template>
```



ADDED "MEENA": THE MUTATION INVALIDATED ["STUDENTS"], SO THE LIST AND THE COUNT (ANOTHER COMPONENT) REFETCHED

BROWSER CONSOLE: FOUR COMPONENTS, ONE REQUEST

```
[api] GET /api/students 200
```

Without a server-state library	With TanStack Query
each component fetches its own copy	one cache entry per query key, shared
manual loading / error refs everywhere	`isPending`, `isFetching`, `error` provided
after a change, remember to reload lists	`invalidateQueries({ queryKey })` refetches dependents
no stale data handling	`staleTime`, refetch on window focus or reconnect, retries

15.3 Form validation libraries

15.3.1 VeeValidate with Zod or Yup schemas

Hand-written validation (Module 8.5) gets repetitive in large forms. **VeeValidate** (`npm i vee-validate`) manages form values, touched/dirty state, errors and submission. Rules are described as a **schema** with **Zod** (`npm i zod @vee-validate/zod`) or **Yup**; with Zod the same schema also gives TypeScript types. `useForm({ validationSchema })` returns `defineField()` to bind inputs, `errors`, `meta` and `handleSubmit()`.

SRC/APP.VUE

HTML

```
<script setup>
import { useForm } from 'vee-validate'
import { toTypedSchema } from '@vee-validate/zod'
import { z } from 'zod'

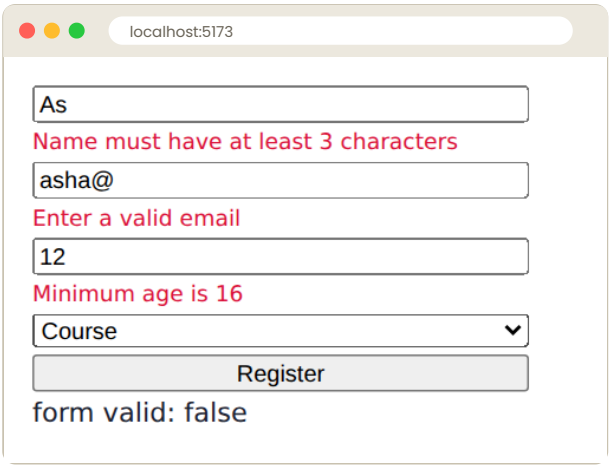
const schema = toTypedSchema(
  z.object({
    name: z.string().min(3, 'Name must have at least 3 characters'),
    email: z.string().email('Enter a valid email'),
    age: z.coerce.number().min(16, 'Minimum age is 16').max(60, 'Maximum age is 60'),
    course: z.enum(['Vue.js', 'React', 'Angular'], { message: 'Choose a course' }),
  }),
)
const { defineField, errors, handleSubmit, meta, resetForm } = useForm({
  validationSchema: schema,
  initialValues: { name: '', email: '', age: '', course: '' },
})
const [name, nameAttrs] = defineField('name')
const [email, emailAttrs] = defineField('email')
const [age, ageAttrs] = defineField('age')
const [course, courseAttrs] = defineField('course')

const onSubmit = handleSubmit((values) => {
  console.log('valid, submitting', JSON.stringify(values))   // age is a number here
  resetForm()
})
</script>

<template>
  <form @submit="onSubmit" novalidate>
    <input v-model="name" v-bind="nameAttrs" placeholder="Name"><small>{{ errors.name }}</small>
    <input v-model="email" v-bind="emailAttrs" placeholder="Email"><small>{{ errors.email }}</small>
    <input v-model="age" v-bind="ageAttrs" placeholder="Age"><small>{{ errors.age }}</small>
    <select v-model="course" v-bind="courseAttrs">
      <option value="" disabled>Course</option><option>Vue.js</option><option>React</option>
<option>Angular</option>
    </select><small>{{ errors.course }}</small>
    <button>Register</button> <span>form valid: {{ meta.valid }}</span>
  </form>
```

```
</template>

<style>form { display: grid; gap: 2px; width: 280px; } small { color: crimson; min-height: 0; }
</style>
```



TYPED "AS", A BAD EMAIL AND AGE 12, LEFT EACH FIELD, THEN CLICKED REGISTER: VEEVALIDATE SHOWED EVERY SCHEMA ERROR

BROWSER CONSOLE: SUBMITTING A VALID FORM

```
valid, submitting {"name":"Asha Rao","email":"asha@lbf.in","age":21,"course":"Vue.js"}
```

The same rules with **Yup** use `@vee-validate/yup`'s `toTypedSchema` or a plain Yup object schema:

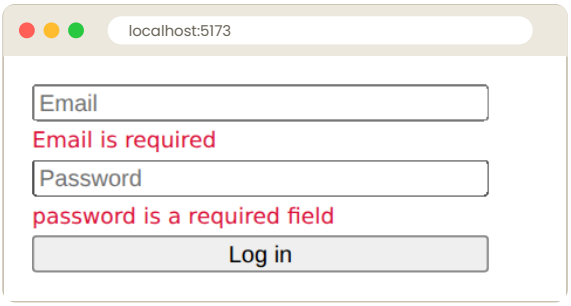
SRC/APP.VUE HTML

```
<script setup>
import { useForm } from 'vee-validate'
import * as yup from 'yup'

const { defineField, errors, handleSubmit } = useForm({
  validationSchema: yup.object({
    email: yup.string().required('Email is required').email('Enter a valid email'),
    password: yup.string().required().min(8, 'At least 8 characters'),
  }),
})
const [email, emailAttrs] = defineField('email')
const [password, passwordAttrs] = defineField('password')
const login = handleSubmit((v) => console.log('login', v.email))
</script>

<template>
  <form @submit="login" novalidate>
    <input v-model="email" v-bind="emailAttrs" placeholder="Email"><small>{{ errors.email }}</small>
    <input v-model="password" v-bind="passwordAttrs" type="password" placeholder="Password"><small>{{
errors.password }}</small>
    <button>Log in</button>
  </form>
</template>

<style>form { display: grid; gap: 2px; width: 260px; } small { color: crimson; }</style>
```



YUP SCHEMA: AN EMPTY SUBMIT SHOWS BOTH MESSAGES

15.3.2 Field-level and form-level validation

Form-level validation puts all rules in one schema passed to `useForm` (as above): good for whole forms and cross-field rules. **Field-level** validation attaches rules to one field with `useField(name, rules)`, where a rule is a function returning `true` or an error message. It suits reusable input components that carry their own rules.

SRC/COMPONENTS/PHONEINPUT.VUE HTML

```
<script setup>
import { useField } from 'vee-validate'
const props = defineProps({ name: String })

// field-level rules: return true or an error message
const isRequired = (v) => (v && v.trim() ? true : 'Phone number is required')
const isIndianMobile = (v) => /^[6-9]\d{9}$/.test(v) || 'Enter a 10-digit mobile number starting with
6-9'

const { value, errorMessage, handleBlur } = useField(() => props.name, [isRequired, isIndianMobile])
</script>

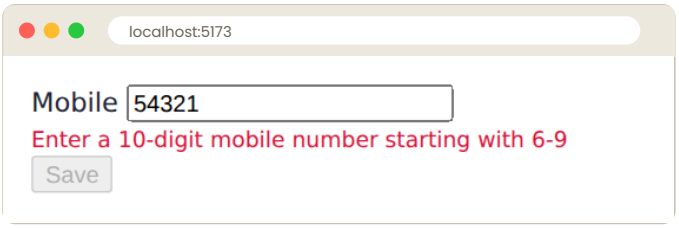
<template>
  <label>Mobile <input v-model="value" @blur="handleBlur" inputmode="numeric"></label>
  <small style="color: crimson; display: block">{{ errorMessage }}</small>
</template>
```

SRC/APP.VUE HTML

```
<script setup>
import { useForm } from 'vee-validate'
import PhoneInput from './components/PhoneInput.vue'

const { handleSubmit, meta } = useForm({ initialValues: { phone: '' } })
const save = handleSubmit((values) => console.log('saved', values.phone))
</script>

<template>
  <form @submit="save">
    <PhoneInput name="phone" />
    <button :disabled="!meta.valid">Save</button>
  </form>
</template>
```



TYPED "54321" AND LEFT THE FIELD: THE FIELD'S OWN RULE FAILED AND SAVE IS DISABLED

	Form-level (schema)	Field-level (useField rules)
Where rules live	one schema object for the form	inside each field / input component
Cross-field rules	easy (refine on the whole object)	awkward
Types from rules	yes with Zod	no
Best for	pages with one form	reusable inputs with built-in rules

15.4 File uploads

15.4.1 Uploading with FormData and image previews

Files are chosen with `<input type="file">` (v-model does not work on it; listen to `@change` and read `event.target.files`). Show a preview with `URL.createObjectURL(file)` (and release it with `URL.revokeObjectURL`), check type and size before uploading, and send the file in a `FormData` body, which the browser encodes as `multipart/form-data`. Axios reports progress with `onUploadProgress`.

SRC/APP.VUE HTML

```
<script setup>
import { ref, onUnmounted } from 'vue'

const file = ref(null)
const preview = ref('')
const error = ref('')
const MAX = 2 * 1024 * 1024        // 2 MB

function onPick(e) {
  const f = e.target.files[0]
  error.value = ''
  if (!f) return
  if (!f.type.startsWith('image/')) return (error.value = 'Please choose an image')
  if (f.size > MAX) return (error.value = 'Image must be under 2 MB')
  if (preview.value) URL.revokeObjectURL(preview.value)
  file.value = f
  preview.value = URL.createObjectURL(f)
}
async function upload() {
  const form = new FormData()
  form.append('avatar', file.value)          // the file
  form.append('studentId', '7')              // other fields
  const res = await fetch('/api/upload', { method: 'POST', body: form })   // no Content-Type: the
browser sets it
  console.log('uploaded:', (await res.json()).message)
}
onUnmounted(() => preview.value && URL.revokeObjectURL(preview.value))
</script>

<template>
  <input type="file" accept="image/*" @change="onPick">
  <p v-if="error" style="color: crimson">{{ error }}</p>
  <div v-if="file" class="preview">
    <img :src="preview" alt="preview">
    <p>{{ file.name }} · {{ (file.size / 1024).toFixed(1) }} KB · {{ file.type }}</p>
    <button @click="upload">Upload</button>
  </div>
</template>

<style>.preview img { width: 120px; height: 120px; object-fit: cover; border-radius: 50%; } .preview p
{ margin: 4px 0; font-size: 13px; }</style>
```



A REAL IMAGE FILE CHOSEN IN THE FILE INPUT: THE PREVIEW COMES FROM A LOCAL OBJECT URL, BEFORE ANY UPLOAD

PRACTICE CHECKPOINT

A full CRUD Student Management app against a REST API with Axios, a Pinia store, VeeValidate + Zod forms and proper loading and error handling.

SRC/STORES/STUDENTS.JS JavaScript

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import { studentApi } from '../api/studentApi.js'

export const useStudentStore = defineStore('students', () => {
  const students = ref([])
  const loading = ref(false)
  const error = ref(null)

  async function run(task) {
    loading.value = true; error.value = null
    try { return await task() } catch (e) { error.value = e.response?.data?.message ?? e.message }
finally { loading.value = false }
  }
  const load = () => run(async () => (students.value = await studentApi.list()))
  const add = (s) => run(async () => students.value.push(await studentApi.create(s)))
  const update = (id, s) => run(async () => { const u = await studentApi.update(id, s); students.value
= students.value.map((x) => (x.id === id ? u : x)) })
  const remove = (id) => run(async () => { await studentApi.remove(id); students.value =
students.value.filter((x) => x.id !== id) })

  return { students, loading, error, load, add, update, remove }
})
```

SRC/COMPONENTS/STUDENTFORM.VUE HTML

```
<script setup>
import { watch } from 'vue'
import { useForm } from 'vee-validate'
import { toTypedSchema } from '@vee-validate/zod'
import { z } from 'zod'

const props = defineProps({ student: Object })      // null = create, object = edit
const emit = defineEmits(['save', 'cancel'])

const { defineField, errors, handleSubmit, resetForm } = useForm({
  validationSchema: toTypedSchema(z.object({
    name: z.string().trim().min(3, 'Name: 3+ characters'),
    course: z.enum(['Vue.js', 'React', 'Angular'], { message: 'Pick a course' }),
    marks: z.coerce.number().int().min(0, 'Marks 0-100').max(100, 'Marks 0-100'),
  })),
  initialValues: { name: '', course: '', marks: '' },
})
const [name, nameAttrs] = defineField('name')
const [course, courseAttrs] = defineField('course')
const [marks, marksAttrs] = defineField('marks')

watch(() => props.student, (s) => resetForm({ values: s ? { name: s.name, course: s.course, marks:
s.marks } : { name: '', course: '', marks: '' } }), { immediate: true })
const submit = handleSubmit((values) => { emit('save', values); resetForm() })
</script>

<template>
  <form @submit="submit" novalidate class="form">
    <b>{{ student ? 'Edit #' + student.id : 'New student' }}</b>
    <input v-model="name" v-bind="nameAttrs" placeholder="Name"><small>{{ errors.name }}</small>
    <select v-model="course" v-bind="courseAttrs"><option value="" disabled>Course</option>
<option>Vue.js</option><option>React</option><option>Angular</option></select><small>{{ errors.course
}}</small>
    <input v-model="marks" v-bind="marksAttrs" placeholder="Marks"><small>{{ errors.marks }}</small>
    <div><button>{{ student ? 'Update' : 'Add' }}</button> <button v-if="student" type="button"
@click="emit('cancel')">Cancel</button></div>
  </form>
</template>

<style scoped>.form { display: grid; gap: 2px; width: 200px; } small { color: crimson; font-size:
12px; }</style>
```

SRC/APP.VUE HTML

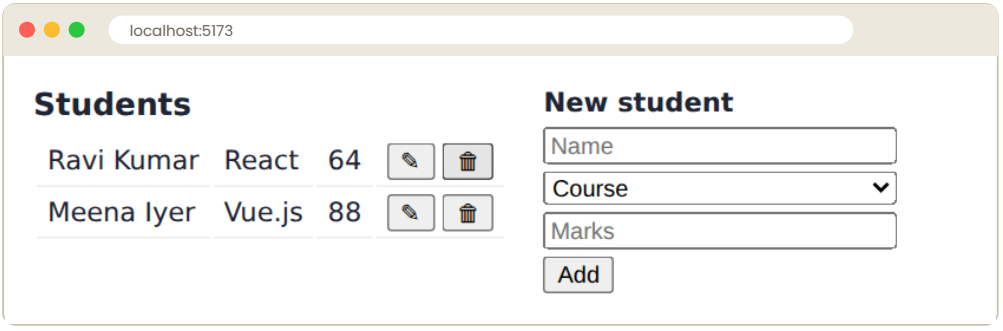
```
<script setup>
import { ref, onMounted } from 'vue'
import { useStudentStore } from './stores/students.js'
import StudentForm from './components/StudentForm.vue'

const store = useStudentStore()
const editing = ref(null)
onMounted(store.load)

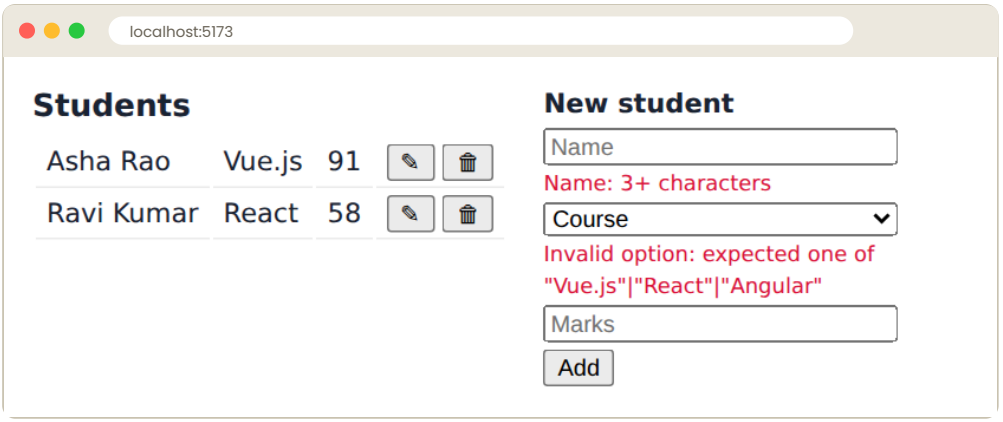
async function save(values) {
  if (editing.value) await store.update(editing.value.id, values)
  else await store.add(values)
  editing.value = null
}
</script>

<template>
  <div class="app">
    <section>
      <h3>Students <small v-if="store.loading">⏳</small></h3>
      <p v-if="store.error" class="err">⚠ {{ store.error }}</p>
      <p v-else-if="!store.loading && !store.students.length">No students yet.</p>
      <table v-else>
        <tr v-for="s in store.students" :key="s.id">
          <td>{{ s.name }}</td><td>{{ s.course }}</td><td>{{ s.marks }}</td>
          <td><button @click="editing = s">✎</button> <button @click="store.remove(s.id)">🗑</button>
</td>
        </tr>
      </table>
    </section>
    <StudentForm :student="editing" @save="save" @cancel="editing = null" />
  </div>
</template>

<style>.app { display: flex; gap: 20px; } h3 { margin: 0 0 4px; } td { padding: 2px 6px; border-
bottom: 1px solid #eee; } .err { color: crimson; }</style>
```



ADDED "MEENA IYER" (POST), EDITED RAVI'S MARKS TO 64 (PUT) AND DELETED ASHA (DELETE), EACH THROUGH THE STORE AND THE API



SUBMITTING THE FORM EMPTY: EVERY ZOD RULE REPORTED ITS MESSAGE

Talk to REST APIs with fetch or Axios (instances with `baseURL` and interceptors for tokens and errors), configure URLs with `VITE_` environment variables, and wrap endpoints in a service layer. Always render loading, error, empty and success states, and use the Vite proxy for CORS in development. TanStack Query caches and shares server state; VeeValidate with Zod/Yup schemas (form-level) or `useField` rules (field-level) validates forms; upload files with `FormData` and preview them with object URLs.

LEVEL 3

Advanced level

Use TypeScript, master reactivity and performance, extend Vue with directives and plugins, test thoroughly and build full-stack apps with Nuxt

6 MODULES · 15 DAYS MINIMUM

Module 16 · TypeScript with Vue	**2 days**
Module 17 · Reactivity in depth and performance	**2 days**
Module 18 · Advanced patterns, plugins and UI libraries	**2 days**
Module 19 · Testing Vue applications	**2 days**
Module 20 · Full-stack Vue with Nuxt	**4 days**
Module 21 · Deployment, projects and interview preparation	**3 days**

MODULE 16 · ADVANCED

TypeScript with Vue

Minimum duration **2 days** Level **Advanced**

TypeScript gives Vue projects type-safe props, events, stores and API data, so many mistakes are reported in the editor instead of in the browser. This module creates a Vue + TypeScript project, uses <script setup lang="ts"> and type-checks with vue-tsc; types props (with defaults), emits, v-model and slots, and builds a generic component; types refs, reactive objects, computed values, template refs and provide/inject with InjectionKey; and finally types Pinia stores, route meta and API responses. Every type error shown in this module was produced by running the TypeScript compiler on the code shown.

IN THIS MODULE

16.1 Setup
16.2 Typing components
16.3 Typing reactivity
16.4 Typing the ecosystem

16.1 Setup

16.1.1 Creating a Vue + TypeScript project

Answer **Yes** to "Add TypeScript?" in `npm create vue@latest` (Module 3.2.1). The generated project uses `main.ts`, adds TypeScript configuration files and a `type-check` script, and the editor's Vue (Official) extension gives type errors and autocompletion inside `.vue` files.

Terminal

```
$ npm create vue@latest student-portal
✔ Add TypeScript? … Yes
✔ Select features to include in your project: › Router (SPA development), Pinia (state management),
Vitest (unit testing), ESLint (error prevention)
Scaffolding project in ./student-portal...
Done. Now run:
  cd student-portal
  npm install
  npm run dev
```

File	Purpose
`src/main.ts`	the entry file, now TypeScript
`tsconfig.json`	references the two configs below
`tsconfig.app.json`	settings for the app code in `src/` (strict mode, the `@/` path alias, DOM types)
`tsconfig.node.json`	settings for config files that run in Node (`vite.config.ts`)
`env.d.ts`	`/// <reference types="vite/client" />` : types for `import.meta.env` and asset imports

File	Purpose
`package.json`	adds `vue-tsc` and the scripts `"type-check": "vue-tsc --build"` and `"build": "run-p type-check build-only"`

TIP

Vite itself does **not** type-check: it only strips the types (very fast). Type errors are reported by the editor and by `vue-tsc` (16.1.3), which the build script runs before `vite build`.

16.1.2 <script setup lang="ts">

Add `lang="ts"` to the script block and write TypeScript: interfaces, type annotations, union types and generics. The template stays the same, but the editor now knows the type of every expression in it.

SRC/APP.VUE

HTML

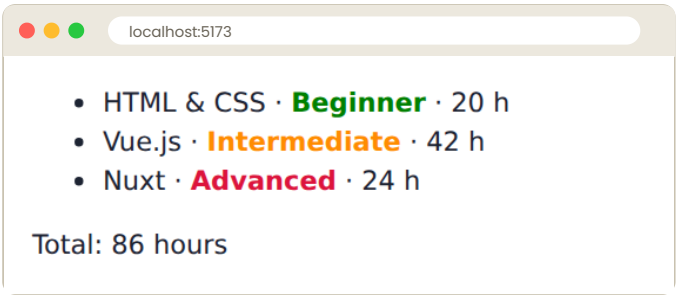
```
<script setup lang="ts">
import { ref, computed } from 'vue'

interface Course {
  id: number
  title: string
  level: 'Beginner' | 'Intermediate' | 'Advanced'
  hours: number
}

const courses = ref<Course[]>([
  { id: 1, title: 'HTML & CSS', level: 'Beginner', hours: 20 },
  { id: 2, title: 'Vue.js', level: 'Intermediate', hours: 42 },
  { id: 3, title: 'Nuxt', level: 'Advanced', hours: 24 },
])
const totalHours = computed<number>(() => courses.value.reduce((sum, c) => sum + c.hours, 0))

function levelColor(level: Course['level']): string {
  return { Beginner: 'green', Intermediate: 'darkorange', Advanced: 'crimson' }[level]
}
</script>

<template>
  <ul>
    <li v-for="c in courses" :key="c.id">
      {{ c.title }} · <b :style="{ color: levelColor(c.level) }">{{ c.level }}</b> · {{ c.hours }} h
    </li>
  </ul>
  <p>Total: {{ totalHours }} hours</p>
</template>
```



A TYPESCRIPT COMPONENT RENDERS EXACTLY LIKE A JAVASCRIPT ONE: TYPES ARE REMOVED AT BUILD TIME

16.1.3 Type checking with vue-tsc

`vue-tsc` is the TypeScript compiler (`tsc`) extended to understand `.vue` files: it checks the script **and the template**, including the props passed to child components. Run it with `npm run type-check` (or `npx vue-tsc --noEmit`) and in CI. Here a child component and its parent contain four mistakes:

SRC/COMPONENTS/STUDENTCARD.VUE HTML

```
<script setup lang="ts">
interface Props {
  name: string
  marks: number
  course?: string
}
const props = withDefaults(defineProps<Props>(), { course: 'Vue.js' })
const emit = defineEmits<{ select: [id: number] }>()
const passed = props.marks >= 40
</script>

<template>
  <div class="card" @click="emit('select', 7)">
    <h3>{{ name }} · {{ course.toUpperCase() }}</h3>
    <p>{{ marks.toFixed(1) }} · {{ passed ? 'Pass' : 'Fail' }}</p>
  </div>
</template>
```

SRC/APP.VUE HTML

```
<script setup lang="ts">
import { ref } from 'vue'
import StudentCard from './components/StudentCard.vue'

interface Student { id: number; name: string; marks: number }
const students = ref<Student[]>([{ id: 1, name: 'Asha', marks: 91 }])
const heading = ref('Students')
heading.value = 42
</script>

<template>
  <h1>{{ heading.toUpperCase() }}</h1>
  <StudentCard v-for="s in students" :key="s.id" :name="s.name" :marks="s.mark" />
  <StudentCard name="Ravi" marks="58" />
  <StudentCard :name="heading" />
</template>
```

TYPE CHECK WITH VUE-TSC main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
src/App.vue(8,1): error TS2322: Type 'number' is not assignable to type 'string'.
src/App.vue(13,75): error TS2551: Property 'mark' does not exist on type 'Student'. Did you mean
'marks'?
src/App.vue(14,28): error TS2322: Type 'string' is not assignable to type 'number'.
src/App.vue(15,4): error TS2345: Argument of type '{ name: string; }' is not assignable to parameter
of type 'Props'.
  Property 'marks' is missing in type '{ name: string; }' but required in type 'Props'.
```

Each line is file(line,column): error code and message. The template mistakes (a misspelt property, a string where a number is expected, a missing required prop) are all caught without opening the browser.

Fixing the four lines (`heading.value = 'Results'`, `s.marks`, `:marks="58"` and adding `:marks` to the last card) gives a clean run:

TYPE CHECK WITH VUE-TSC main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
(no output: no type errors)
```

16.2 Typing components

16.2.1 Type-based defineProps<Props>() and defaults

Pass an interface (or type literal) as the type argument of `defineProps`. The compiler generates the runtime prop options from the type (`String`, `Number`, `required` for non-optional members), so there is one source of truth. Give optional props default values either with **reactive props destructure** (Vue 3.5) or with `withDefaults()`:

DEFAULTS WITH REACTIVE PROPS DESTRUCTURE HTML

```
<script setup lang="ts">
// Option 1 (Vue 3.5+): destructure with JavaScript default values
const { title, level = 'Beginner', tags = [] } = defineProps<{
  title: string
  level?: 'Beginner' | 'Intermediate' | 'Advanced'
  tags?: string[]
}>()
</script>
```

DEFAULTS WITH WITHDEFAULTS() HTML

```
<script setup lang="ts">
// Option 2: withDefaults (object and array defaults are written as factory functions)
interface Props {
  title: string
  level?: 'Beginner' | 'Intermediate' | 'Advanced'
  tags?: string[]
}
const props = withDefaults(defineProps<Props>(), {
  level: 'Beginner',
  tags: () => [],
})
</script>
```

SRC/COMPONENTS/COURSEBADGE.VUE HTML

```
<script setup lang="ts">
const { title, level = 'Beginner', tags = [] } = defineProps<{
  title: string
  level?: 'Beginner' | 'Intermediate' | 'Advanced'
  tags?: string[]
}>()
</script>

<template>
  <div class="badge">
    <b>{{ title }}</b> <small>{{ level }}</small>
    <span v-for="t in tags" :key="t" class="tag">{{ t }}</span>
  </div>
</template>

<style scoped>
.badge { border: 1px solid #ccd; border-radius: 8px; padding: 6px 10px; margin: 6px 0; }
.tag { background: #eef; margin-left: 6px; padding: 0 6px; border-radius: 4px; font-size: 12px; }
</style>
```

SRC/APP.VUE HTML

```
<script setup lang="ts">
import CourseBadge from './components/CourseBadge.vue'
</script>

<template>
  <CourseBadge title="Vue.js" level="Intermediate" :tags="['SPA', 'SFC']" />
  <CourseBadge title="HTML" />
</template>
```



THE SECOND BADGE USES THE DEFAULT LEVEL AND AN EMPTY TAG LIST

NOTE

The type argument must be a type the compiler can analyse: an interface or type literal in the same file, or one imported from another file (supported since Vue 3.3). Complex conditional types cannot be turned into runtime options.

16.2.2 Typed defineEmits

Declare every event with its payload as a **named tuple**: `defineEmits<{ save: [student: Student]; cancel: [] }>()`. Emitting an unknown event or a wrong payload becomes a type error, and parents get the payload type in their handlers.

SRC/COMPONENTS/STUDENTFORM.VUE

HTML

```
<script setup lang="ts">
import { ref } from 'vue'

export interface Student { name: string; marks: number }

const emit = defineEmits<{
  save: [student: Student]
  cancel: []
}>()

const name = ref('')
const marks = ref(0)

function submit() {
  emit('save', { name: name.value, marks: marks.value })
}
function bad() {
  emit('save', name.value)        // wrong payload
  emit('delete')                  // event that was never declared
}
</script>

<template>
  <input v-model="name"> <input v-model.number="marks" type="number">
  <button @click="submit">Save</button> <button @click="emit('cancel')">Cancel</button>
</template>
```

TYPE CHECK WITH VUE-TSC

main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
src/components/StudentForm.vue(18,16): error TS2345: Argument of type 'string' is not assignable to
parameter of type 'Student'.
src/components/StudentForm.vue(19,8): error TS2345: Argument of type '"delete"' is not assignable to
parameter of type '"cancel"'.
```

The older call-signature syntax `defineEmits<{ (e: 'save', s: Student): void }>()` is equivalent; the tuple syntax is shorter.

16.2.3 Typed defineModel

`defineModel<T>()` declares the `v-model` prop and its update event with one type. Without a default its type is `Ref<T | undefined>`; with `{ required: true }` or a `default` it is `Ref<T>`.

SRC/COMPONENTS/STARRATING.VUE HTML

```
<script setup lang="ts">
const rating = defineModel<number>({ required: true })
const max = 5
</script>

<template>
  <span class="stars">
    <button v-for="n in max" :key="n" :class="{ on: n <= rating }" @click="rating = n">★</button>
  </span>
</template>

<style scoped>
button { border: 0; background: none; font-size: 24px; color: #ccc; cursor: pointer; padding: 0 2px; }
.on { color: #e0a800; }
</style>
```

SRC/APP.VUE HTML

```
<script setup lang="ts">
import { ref } from 'vue'
import StarRating from './components/StarRating.vue'

const score = ref(3)
</script>

<template>
  <StarRating v-model="score" />
  <p>Your rating: {{ score }} / 5</p>
</template>
```



CLICKED THE FOURTH STAR: THE PARENT'S NUMBER REF UPDATED THROUGH V-MODEL

TYPE CHECK WITH VUE-TSC main.js

```
$ npx vue-tsc --noEmit
```

```
TERMINAL
src/App.vue(9,15): error TS2322: Type 'string' is not assignable to type 'number'.
src/App.vue(10,4): error TS2345: Argument of type '{}' is not assignable to parameter of type '{
modelValue: number; }'.
  Property 'modelValue' is missing in type '{}' but required in type '{ modelValue: number; }'.
```

Binding a string ref to a number model, and leaving out a required model, are both reported.

16.2.4 Typing slots with defineSlots

`defineSlots` describes which slots a component offers and the props each slot receives. Each slot is written as a function that takes the slot props. Parents then get autocompletion and type checks for the slot props they destructure.

SRC/COMPONENTS/DATALIST.VUE HTML

```
<script setup lang="ts">
interface Item { id: number; name: string; marks: number }

defineProps<{ items: Item[] }>()
defineSlots<{
  item(props: { item: Item; index: number }): any
  empty(): any
}>()
</script>

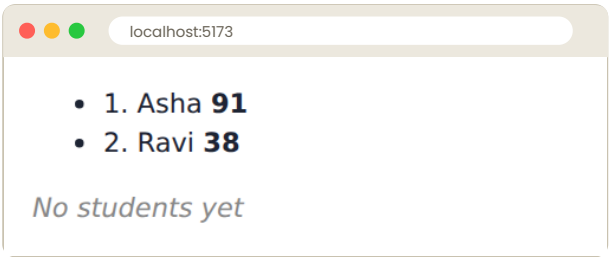
<template>
  <ul v-if="items.length">
    <li v-for="(item, index) in items" :key="item.id"><slot name="item" :item="item" :index="index" />
</li>
  </ul>
  <p v-else class="empty"><slot name="empty" /></p>
</template>

<style scoped>.empty { color: #888; font-style: italic; }</style>
```

SRC/APP.VUE HTML

```
<script setup lang="ts">
import DataList from './components/DataList.vue'
const students = [{ id: 1, name: 'Asha', marks: 91 }, { id: 2, name: 'Ravi', marks: 38 }]
</script>

<template>
  <DataList :items="students">
    <template #item="{ item, index }">
      {{ index + 1 }}. {{ item.name }} <b>{{ item.marks }}</b>
    </template>
    <template #empty>No students yet</template>
  </DataList>
  <DataList :items="[]">
    <template #empty>No students yet</template>
  </DataList>
</template>
```



THE ITEM SLOT RECEIVES TYPED { ITEM, INDEX } PROPS; THE SECOND LIST SHOWS THE EMPTY SLOT

16.2.5 Generic components with generic="T"

A **generic component** works with any item type while keeping the types connected: `<script setup lang="ts" generic="T extends { id: number }">` declares a type parameter `T` that props, emits and models can use. Vue infers `T` from what the parent passes, so a list of students makes the selected model a `Student`.

SRC/COMPONENTS/SELECTLIST.VUE HTML

```
<script setup lang="ts" generic="T extends { id: number }">
const props = defineProps<{
  items: T[]
  label: (item: T) => string
}>()
const selected = defineModel<T | null>({ default: null })
</script>

<template>
  <ul class="pick">
    <li v-for="item in items" :key="item.id" :class="{ on: selected?.id === item.id }"
@click="selected = item">
      {{ label(item) }}
    </li>
  </ul>
</template>

<style scoped>
.pick { list-style: none; padding: 0; display: flex; gap: 6px; }
.pick li { border: 1px solid #ccd; border-radius: 6px; padding: 4px 10px; cursor: pointer; }
.pick .on { background: #1E2A4A; color: white; }
</style>
```

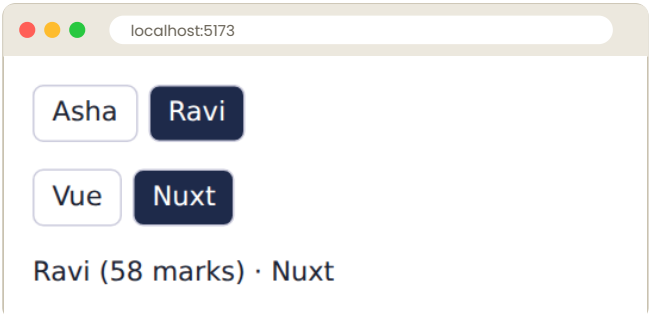
SRC/APP.VUE HTML

```
<script setup lang="ts">
import { ref } from 'vue'
import SelectList from './components/SelectList.vue'

interface Student { id: number; name: string; marks: number }
interface Course { id: number; title: string }

const students: Student[] = [{ id: 1, name: 'Asha', marks: 91 }, { id: 2, name: 'Ravi', marks: 58 }]
const courses: Course[] = [{ id: 10, title: 'Vue' }, { id: 11, title: 'Nuxt' }]
const student = ref<Student | null>(null)
const course = ref<Course | null>(null)
</script>

<template>
  <SelectList :items="students" :label="(s) => s.name" v-model="student" />
  <SelectList :items="courses" :label="(c) => c.title" v-model="course" />
  <p>{{ student?.name ?? '-' }} ({{ student?.marks ?? 0 }} marks) · {{ course?.title ?? '-' }}</p>
</template>
```



ONE COMPONENT, TWO ITEM TYPES: CLICKED RAVI AND NUXT

Inside each usage the label function's parameter is inferred (`s` is a `Student`, `c` a `Course`), and mixing the types is an error:

TYPE CHECK WITH VUE-TSC

main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
src/App.vue(16,49): error TS2339: Property 'name' does not exist on type 'Course'.
src/App.vue(16,55): error TS2322: Type 'Student | null' is not assignable to type 'Course | null |
undefined'.
  Property 'title' is missing in type 'Student' but required in type 'Course'.
```

With courses as items, T is Course: c.name does not exist, and a Student ref cannot hold a Course.

16.3 Typing reactivity

16.3.1 ref<T>(), reactive, computed

Types are usually **inferred** from the initial value: `ref(0)` is `Ref<number>`. Pass a type argument when the initial value does not show the full type (a union, `null` now and an object later, an empty array). Annotate `reactive()` objects with an interface, and `computed()` infers its type from the getter (or takes one explicitly).

SRC/COMPOSABLES/USEUSERFORM.TS

TypeScript

```ts
import { ref, reactive, computed } from 'vue'

export interface User { id: number; name: string }
type Status = 'idle' | 'loading' | 'success' | 'error'

export function useUserForm() {
  const count = ref(0)                          // Ref<number> (inferred)
  const status = ref<Status>('idle')            // union: only these four strings
  const user = ref<User | null>(null)           // null now, a User later
  const tags = ref<string[]>([])                // empty array needs the element type
  const form = reactive<{ email: string; age: number | null }>({ email: '', age: null })
  const greeting = computed(() => (user.value ? `Hi ${user.value.name}` : 'Guest'))   //
ComputedRef<string>

  status.value = 'done'                         // ✗ not one of the four
  count.value = '5'                             // ✗ string into a number ref
  tags.value.push(42)                           // ✗ number into string[]
  form.age = 'twenty'                           // ✗
  console.log(user.value.name)                  // ✗ user may still be null
  greeting.value = 'Hello'                      // ✗ computed without a setter is read-only

  return { count, status, user, tags, form, greeting }
}
```

TYPE CHECK WITH VUE-TSC

main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
src/composables/useUserForm.ts(14,3): error TS2322: Type '"done"' is not assignable to type 'Status'.
src/composables/useUserForm.ts(15,3): error TS2322: Type 'string' is not assignable to type 'number'.
src/composables/useUserForm.ts(16,19): error TS2345: Argument of type 'number' is not assignable to
parameter of type 'string'.
src/composables/useUserForm.ts(17,3): error TS2322: Type 'string' is not assignable to type 'number'.
src/composables/useUserForm.ts(18,15): error TS18047: 'user.value' is possibly 'null'.
src/composables/useUserForm.ts(19,12): error TS2540: Cannot assign to 'value' because it is a read-
only property.
```

16.3.2 Typing template refs

`useTemplateRef<HTMLInputElement>('search')` gives a `ShallowRef<HTMLInputElement | null>`: the element exists only after mounting, so TypeScript makes you handle `null` (use `?.` or a check). For a child component use `useTemplateRef<InstanceType<typeof Child>>('child')` to see the methods it exposes with `defineExpose`.

SRC/COMPONENTS/SEARCHBOX.VUE

HTML

```
<script setup lang="ts">
import { onMounted, useTemplateRef } from 'vue'

const search = useTemplateRef<HTMLInputElement>('search')

onMounted(() => {
  search.value.focus()          // ✗ may be null
  search.value?.focus()         // ✓
  search.value?.select(42)      // ✗ select() takes no arguments
})
</script>

<template>
  <input ref="search" placeholder="Search students">
</template>
```

TYPE CHECK WITH VUE-TSC

main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
src/components/SearchBox.vue(7,3): error TS18047: 'search.value' is possibly 'null'.
src/components/SearchBox.vue(9,24): error TS2554: Expected 0 arguments, but got 1.
```

16.3.3 Typing provide/inject with InjectionKey

A string key loses the type: `inject('theme')` returns `unknown`. An `InjectionKey<T>` is a `Symbol` that carries the type: `provide` checks the value against `T` and `inject` returns `T | undefined` (or `T` when a default is given). Export keys from one file so both sides share them.

SRC/KEYS.TS TypeScript

```ts
import type { InjectionKey, Ref } from 'vue'

export interface ThemeContext {
  theme: Ref<'light' | 'dark'>
  toggle: () => void
}
export const ThemeKey: InjectionKey<ThemeContext> = Symbol('theme')
```

SRC/COMPONENTS/THEMEBUTTON.VUE HTML

```html
<script setup lang="ts">
import { inject } from 'vue'
import { ThemeKey } from '../keys'

const ctx = inject(ThemeKey)                 // ThemeContext | undefined
if (!ctx) throw new Error('ThemeButton must be used inside a theme provider')
const { theme, toggle } = ctx                // now ThemeContext
</script>

<template>
  <button @click="toggle">{{ theme === 'light' ? '🌙 Dark' : '☀ Light' }}</button>
</template>
```

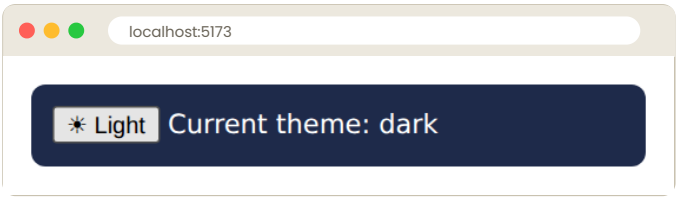
SRC/APP.VUE HTML

```html
<script setup lang="ts">
import { provide, ref } from 'vue'
import { ThemeKey } from './keys'
import ThemeButton from './components/ThemeButton.vue'

const theme = ref<'light' | 'dark'>('light')
provide(ThemeKey, { theme, toggle: () => (theme.value = theme.value === 'light' ? 'dark' : 'light') })
</script>

<template>
  <div :class="theme" class="page"><ThemeButton /> Current theme: {{ theme }}</div>
</template>

<style>.page { padding: 12px; border-radius: 8px; } .dark { background: #1E2A4A; color: white; }
</style>
```



THE INJECTED TOGGLE() SWITCHED THE PROVIDER'S THEME

TYPE CHECK WITH VUE-TSC main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
src/App.vue(7,21): error TS2322: Type 'string' is not assignable to type 'Ref<"light" | "dark",
"light" | "dark">'.
```

Providing a value that does not match ThemeContext is caught at the provide() call.

16.4 Typing the ecosystem

16.4.1 Typed Pinia stores

Setup stores need almost no extra types: Pinia infers state, getters and actions from what the function returns. Type the data with interfaces and give refs a type argument when they start empty. Components then get full autocompletion, and wrong calls are errors:

SRC/STORES/CART.TS

TypeScript

```ts
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export interface Product { id: number; name: string; price: number }
export interface CartLine extends Product { qty: number }

export const useCartStore = defineStore('cart', () => {
  const items = ref<CartLine[]>([])
  const count = computed(() => items.value.reduce((n, i) => n + i.qty, 0))
  const total = computed(() => items.value.reduce((s, i) => s + i.price * i.qty, 0))

  function add(product: Product) {
    const line = items.value.find((i) => i.id === product.id)
    if (line) line.qty++
    else items.value.push({ ...product, qty: 1 })
  }
  function remove(id: number) {
    items.value = items.value.filter((i) => i.id !== id)
  }
  return { items, count, total, add, remove }
})
```

SRC/COMPONENTS/CARTBADGE.VUE

HTML

```html
<script setup lang="ts">
import { storeToRefs } from 'pinia'
import { useCartStore } from '../stores/cart'

const cart = useCartStore()
const { count, total } = storeToRefs(cart)       // Ref<number> each

cart.add({ id: 1, name: 'Notebook', price: 60 })  // ✓
cart.add({ id: 2, name: 'Pen' })                  // ✗ price missing
cart.remove('2')                                  // ✗ id is a number
const label: string = total.value                 // ✗ total is a number
</script>

<template>
  <span>🛒 {{ count }} · ₹{{ total.toFixed(2) }} · {{ cart.items[0]?.nmae }}</span>
</template>
```

TYPE CHECK WITH VUE-TSC

main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
src/components/CartBadge.vue(9,10): error TS2345: Argument of type '{ id: number; name: string; }' is
not assignable to parameter of type 'Product'.
  Property 'price' is missing in type '{ id: number; name: string; }' but required in type 'Product'.
src/components/CartBadge.vue(10,13): error TS2345: Argument of type 'string' is not assignable to
parameter of type 'number'.
src/components/CartBadge.vue(11,7): error TS2322: Type 'number' is not assignable to type 'string'.
src/components/CartBadge.vue(15,69): error TS2339: Property 'nmae' does not exist on type 'CartLine'.
```

16.4.2 Typed routes and route meta

Route meta is typed by **augmenting** vue-router's `RouteMeta` interface once (for example in `src/router/types.d.ts` or the router file). Every route's `meta` and every `to.meta` in guards then has those fields. Route params are always `string | string[]`; convert them (`Number(route.params.id)`) where numbers are needed. (The official **unplugin-vue-router** goes further and generates typed route names and params from file-based routes.)

SRC/ROUTER/INDEX.TS

TypeScript

```ts
import { createRouter, createWebHistory, type RouteRecordRaw } from 'vue-router'

declare module 'vue-router' {
  interface RouteMeta {
    title: string               // every route must have a title
    requiresAuth?: boolean
    roles?: ('admin' | 'teacher')[]
  }
}

const routes: RouteRecordRaw[] = [
  { path: '/', component: () => import('../views/HomeView.vue'), meta: { title: 'Home' } },
  { path: '/admin', component: () => import('../views/AdminView.vue'), meta: { title: 'Admin',
requiresAuth: true, roles: ['admin'] } },
  { path: '/reports', component: () => import('../views/ReportsView.vue'), meta: { requiresAuth: true
} },
  { path: '/staff', component: () => import('../views/StaffView.vue'), meta: { title: 'Staff', roles:
['principal'] } },
]

export const router = createRouter({ history: createWebHistory(), routes })

router.beforeEach((to) => {
  document.title = `${to.meta.title} · LBF Portal`     // string, known to exist
  if (to.meta.requiresAuth && !localStorage.getItem('token')) return '/login'
  const id: number = to.params.id                      // ✗ params are strings
})
```

TYPE CHECK WITH VUE-TSC

main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
src/router/index.ts(14,38): error TS2741: Property 'title' is missing in type '{ requiresAuth: true;
}' but required in type 'RouteMeta'.
src/router/index.ts(15,68): error TS2322: Type '"principal"' is not assignable to type '"admin" |
"teacher"'.
src/router/index.ts(23,9): error TS2322: Type 'string | string[]' is not assignable to type 'number'.
  Type 'string' is not assignable to type 'number'.
```

The route without a title, the unknown role and the number/string mix-up are all reported. (The lazy imports were replaced by empty components for this check.)

16.4.3 Typed API responses

Describe the shape of the server's JSON with interfaces and put them on the service functions, for example with Axios's generic `api.get<Student[]>()`. Components that call the service then know every field. Remember that these types are a **promise**, not a check: TypeScript cannot see what the server really sends, so validate untrusted responses at runtime (for example with a Zod schema, Module 15.3.1) when correctness matters.

SRC/TYPES/STUDENT.TS

TypeScript

```ts
export interface Student {
  id: number
  name: string
  email: string
  course: 'HTML' | 'Vue.js' | 'React'
  marks: number
}
export interface Paged<T> {
  data: T[]
  total: number
  page: number
}
export type NewStudent = Omit<Student, 'id'>        // what POST expects
```

SRC/SERVICES/STUDENTSERVICE.TS TypeScript

```ts
import axios from 'axios'
import type { Student, NewStudent, Paged } from '../types/student'

const api = axios.create({ baseURL: '/api' })

export const studentService = {
  async list(page = 1): Promise<Paged<Student>> {
    const { data } = await api.get<Paged<Student>>('/students', { params: { page } })
    return data
  },
  async create(s: NewStudent): Promise<Student> {
    const { data } = await api.post<Student>('/students', s)
    return data
  },
}

export async function demo() {
  const page = await studentService.list()
  page.data.forEach((s) => console.log(s.name.toUpperCase(), s.marks))   // ✓ fully typed
  console.log(page.data[0].phone)                                        // ✗ no such field
  await studentService.create({ name: 'Meena', email: 'm@lbf.in', course: 'Angular', marks: 80 })  //
✗
}
```

TYPE CHECK WITH VUE-TSC main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
src/services/studentService.ts(20,28): error TS2339: Property 'phone' does not exist on type
'Student'.
src/services/studentService.ts(21,67): error TS2322: Type '"Angular"' is not assignable to type
'"HTML" | "Vue.js" | "React"'.
```

PRACTICE CHECKPOINT

Convert the Student Management app to TypeScript with typed props, emits, stores and API responses, passing vue-tsc with no errors.

SRC/TYPES.TS TypeScript

```ts
export interface Student { id: number; name: string; course: string; marks: number }
export type NewStudent = Omit<Student, 'id'>
```

SRC/STORES/STUDENTS.TS TypeScript

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'
import type { Student, NewStudent } from '../types'

export const useStudentStore = defineStore('students', () => {
  const students = ref<Student[]>([])
  const loading = ref(false)
  const error = ref<string | null>(null)
  const average = computed(() =>
    students.value.length ? Math.round(students.value.reduce((s, x) => s + x.marks, 0) /
students.value.length) : 0)

  async function load(): Promise<void> {
    loading.value = true
    error.value = null
    try {
      const res = await fetch('/api/students')
      if (!res.ok) throw new Error(`HTTP ${res.status}`)
      students.value = (await res.json()) as Student[]
    } catch (e) {
      error.value = e instanceof Error ? e.message : String(e)
    } finally {
      loading.value = false
    }
  }
  async function add(input: NewStudent): Promise<void> {
    const res = await fetch('/api/students', { method: 'POST', headers: { 'Content-Type':
'application/json' }, body: JSON.stringify(input) })
    students.value.push((await res.json()) as Student)
  }
  return { students, loading, error, average, load, add }
})
```

SRC/COMPONENTS/STUDENTFORM.VUE HTML

```
<script setup lang="ts">
import { reactive } from 'vue'
import type { NewStudent } from '../types'

const { courses } = defineProps<{ courses: string[] }>()
const emit = defineEmits<{ add: [student: NewStudent] }>()

const form = reactive<NewStudent>({ name: '', course: courses[0] ?? '', marks: 0 })

function submit() {
  if (!form.name.trim()) return
  emit('add', { ...form })
  form.name = ''
  form.marks = 0
}
</script>

<template>
  <form @submit.prevent="submit">
    <input v-model="form.name" placeholder="Name">
    <select v-model="form.course"><option v-for="c in courses" :key="c">{{ c }}</option></select>
    <input v-model.number="form.marks" type="number" min="0" max="100">
    <button>Add</button>
  </form>
</template>
```

SRC/COMPONENTS/STUDENTROW.VUE HTML

```
<script setup lang="ts">
import type { Student } from '../types'
defineProps<{ student: Student }>()
</script>

<template>
  <tr>
    <td>{{ student.name }}</td><td>{{ student.course }}</td>
    <td :class="student.marks >= 40 ? 'pass' : 'fail'">{{ student.marks }}</td>
  </tr>
</template>

<style scoped>.pass { color: green; } .fail { color: crimson; }</style>
```

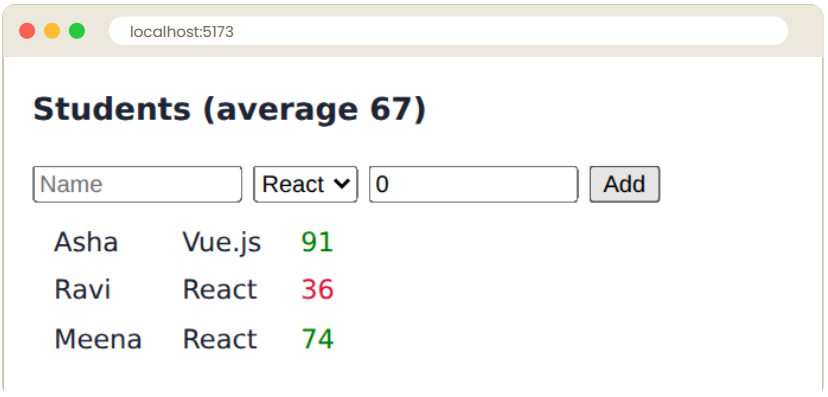
SRC/APP.VUE HTML

```
<script setup lang="ts">
import { onMounted } from 'vue'
import { storeToRefs } from 'pinia'
import { useStudentStore } from './stores/students'
import StudentForm from './components/StudentForm.vue'
import StudentRow from './components/StudentRow.vue'

const store = useStudentStore()
const { students, loading, error, average } = storeToRefs(store)
onMounted(() => store.load())
</script>

<template>
  <h3>Students (average {{ average }})</h3>
  <StudentForm :courses="['Vue.js', 'React', 'HTML']" @add="store.add" />
  <p v-if="loading">Loading…</p>
  <p v-else-if="error">⚠ {{ error }}</p>
  <table v-else>
    <StudentRow v-for="s in students" :key="s.id" :student="s" />
  </table>
</template>

<style>td { padding: 2px 10px; } form { display: flex; gap: 6px; margin-bottom: 8px; } input { width:
110px; }</style>
```



ADDED MEENA (REACT, 74) THROUGH THE TYPED FORM, EMIT AND STORE ACTION

TYPE CHECK WITH VUE-TSC

main.js

```
$ npx vue-tsc --noEmit
```

TERMINAL

```
(no output: no type errors)
```

The whole app type-checks.

Use `<script setup lang="ts">` and run `vue-tsc` (the editor and CI) because Vite does not type-check. Type props with `defineProps<Props>()` (defaults by destructuring or `withDefaults`), events with `defineEmits<{ name: [payload] }>()`, models with `defineModel<T>()`, slots with `defineSlots`, and reusable item components with `generic="T"`. Give `ref<T>()` a type when the initial value is not enough, handle `null` template refs, share `InjectionKey<T>` symbols, let setup stores infer their types, augment `RouteMeta`, and put interfaces on API services (validating at runtime where it matters).

MODULE 17 · ADVANCED

Reactivity in depth and performance

Minimum duration **2 days** Level **Advanced**

This module looks under the hood: how Vue tracks dependencies with Proxies and triggers updates, the shallow, readonly and raw variants of reactive state, effect scopes and the render debugging hooks. It then explains the rendering mechanism (the virtual DOM, patching, the compiler's patch flags and cached static content, and Vapor Mode), the practical performance techniques (stable props, v-once and v-memo, lazy loading, virtual scrolling, shallow reactivity for large data, bundle analysis) and how to measure performance with Vue DevTools, Lighthouse and the Core Web Vitals.

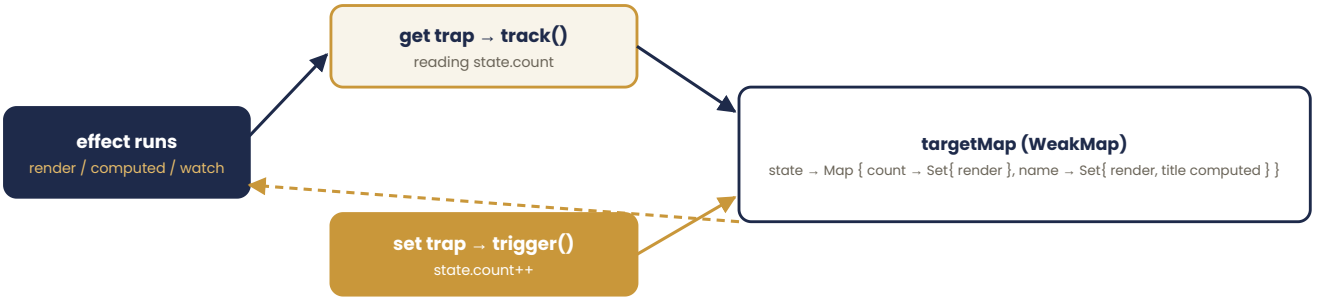
IN THIS MODULE

17.1 Reactivity internals
17.2 Rendering mechanism
17.3 Performance techniques
17.4 Measuring performance

17.1 Reactivity internals

17.1.1 Proxies, dependency tracking and triggering

`reactive()` wraps an object in a JavaScript `Proxy`. The proxy's **get** trap calls `track()`: it records that the currently running **effect** (a component's render, a `computed` or a `watch`) depends on this object's key. The **set** trap calls `trigger()`: it re-runs every effect recorded for that key. `ref()` does the same with a getter and setter on `.value`. The structure is a map of maps: `target → key → Set of effects`.



TRACK() RECORDS WHO READ A KEY; TRIGGER() RE-RUNS THEM WHEN THE KEY CHANGES

The core idea fits in a few lines of plain JavaScript (Vue's real implementation adds batching, cleanup of stale dependencies, arrays, collections and much more):

MINI-REACTIVITY.JS

main.js

```
// a tiny version of Vue's reactivity, to see the mechanism
let activeEffect = null
const targetMap = new WeakMap()

function track(target, key) {
  if (!activeEffect) return
  let deps = targetMap.get(target)
  if (!deps) targetMap.set(target, (deps = new Map()))
  let dep = deps.get(key)
  if (!dep) deps.set(key, (dep = new Set()))
  dep.add(activeEffect)
}
function trigger(target, key) {
  targetMap.get(target)?.get(key)?.forEach((effect) => effect())
}
function reactive(obj) {
  return new Proxy(obj, {
    get(t, k, r) { track(t, k); return Reflect.get(t, k, r) },
    set(t, k, v, r) { const ok = Reflect.set(t, k, v, r); trigger(t, k); return ok },
  })
}
function effect(fn) {
  const run = () => { activeEffect = run; fn(); activeEffect = null }
  run()
}

const state = reactive({ count: 1, name: 'Asha' })
effect(() => console.log(`render: count = ${state.count}`))
state.count++               // tracked key: effect re-runs
state.name = 'Ravi'         // nobody read name: nothing re-runs
state.count = 10
```

OUTPUT

```
render: count = 1
render: count = 2
render: count = 10
```

17.1.2 shallowRef and shallowReactive

`ref()` and `reactive()` are **deep**: nested objects are converted to proxies when accessed, so every nested change is tracked. `shallowRef()` tracks only `.value` itself, and `shallowReactive()` only the top-level properties. Nested changes are not seen; replace the value (or call `triggerRef()`) to update. This saves work for large or immutable data (17.3.5).

SRC/APP.VUE

HTML

```
<script setup>
import { ref, shallowRef, shallowReactive, watchEffect, triggerRef } from 'vue'

const deep = ref({ user: { name: 'Asha' } })
const shallow = shallowRef({ user: { name: 'Asha' } })
const top = shallowReactive({ count: 0, nested: { n: 0 } })

watchEffect(() => console.log('deep sees:', deep.value.user.name))
watchEffect(() => console.log('shallow sees:', shallow.value.user.name))
watchEffect(() => console.log('shallowReactive sees:', top.count, top.nested.n))

setTimeout(() => {
  console.log('--- change nested values')
  deep.value.user.name = 'Ravi'          // tracked
  shallow.value.user.name = 'Ravi'       // NOT tracked
  top.nested.n++                         // NOT tracked
}, 20)
setTimeout(() => {
  console.log('--- replace .value / top-level key, or force')
  shallow.value = { user: { name: 'Meena' } }
  top.count++
}, 40)
setTimeout(() => {
  shallow.value.user.name = 'Kiran'
  triggerRef(shallow)                    // force effects of a shallowRef to run
}, 60)
</script>
<template><p></p></template>
```

BROWSER CONSOLE

```
deep sees: Asha
shallow sees: Asha
shallowReactive sees: 0 0
--- change nested values
deep sees: Ravi
--- replace .value / top-level key, or force
shallow sees: Meena
shallowReactive sees: 1 1
shallow sees: Kiran
```

17.1.3 readonly and shallowReadonly

`readonly(obj)` returns a proxy that refuses every write (deeply) and warns in development, while still tracking reads, so it updates when the original changes. Use it to **expose state without letting consumers change it**, for example from a composable or through provide/inject. `shallowReadonly()` protects only the top level (Vue uses it for `props`).

SRC/APP.VUE HTML

```
<script setup>
import { reactive, readonly, shallowReadonly, watchEffect } from 'vue'

const state = reactive({ count: 0, settings: { theme: 'light' } })
const view = readonly(state)                // consumers get this
const top = shallowReadonly(state)

watchEffect(() => console.log('view.count =', view.count))
view.count++                                // refused, with a warning
view.settings.theme = 'dark'                // refused: readonly is deep
top.settings.theme = 'dark'                 // allowed: shallowReadonly protects only the top level
console.log('theme is now', state.settings.theme)
setTimeout(() => state.count++, 10)         // the owner can still change it, and view updates
</script>
<template><p></p></template>
```

BROWSER CONSOLE

```
view.count = 0
[Vue warn]: Set operation on key "count" failed: target is readonly.
[Vue warn]: Set operation on key "theme" failed: target is readonly.
theme is now dark
view.count = 1
```

17.1.4 markRaw and toRaw

`markRaw(obj)` flags an object so Vue **never** makes it reactive, even when it is placed inside reactive state. Use it for class instances and third-party objects that must not be proxied (a chart instance, a map, a large immutable lookup table). `toRaw(proxy)` returns the original object behind a proxy, for reading or passing to libraries without tracking.

SRC/APP.VUE HTML

```
<script setup>
import { reactive, markRaw, toRaw, isReactive } from 'vue'

class ChartInstance { constructor() { this.points = [] } draw() { return `drawing
${this.points.length} points` } }

const state = reactive({
  user: { name: 'Asha' },
  chart: markRaw(new ChartInstance()),     // stays a plain object
})

console.log('state.user reactive?', isReactive(state.user))
console.log('state.chart reactive?', isReactive(state.chart))
console.log(state.chart.draw())

const raw = toRaw(state)
console.log('toRaw gives the original object:', isReactive(raw), raw.user === toRaw(state.user))
</script>
<template><p></p></template>
```

BROWSER CONSOLE

```
state.user reactive? true
state.chart reactive? false
drawing 0 points
toRaw gives the original object: false true
```

17.1.5 effectScope (overview)

Every `computed`, `watch` and `watchEffect` created in a component's `setup` is collected in the component's **effect scope** and stopped automatically on unmount. `effectScope()` creates such a scope manually: effects created inside `scope.run()` are collected, and `scope.stop()` disposes all of them at once. Libraries (Pinia, VueUse) use it to manage effects that live outside components.

SRC/APP.VUE

HTML

```
<script setup>
import { ref, computed, watch, effectScope, onScopeDispose } from 'vue'

const count = ref(0)
const scope = effectScope()

scope.run(() => {
  const double = computed(() => count.value * 2)
  watch(count, (n) => console.log(`watcher: count=${n}, double=${double.value}`))
  onScopeDispose(() => console.log('scope disposed: cleanup ran'))
})

setTimeout(() => { count.value = 1 }, 10)
setTimeout(() => { count.value = 2 }, 20)
setTimeout(() => { scope.stop() }, 30)
setTimeout(() => { count.value = 3; console.log('count changed to 3 after stop(): no watcher output')
}, 40)
</script>
<template><p></p></template>
```

BROWSER CONSOLE

```
watcher: count=1, double=2
watcher: count=2, double=4
scope disposed: cleanup ran
count changed to 3 after stop(): no watcher output
```

17.1.6 Debugging reactivity with onRenderTracked and onRenderTriggered

Two development-only hooks show why a component renders. `onRenderTracked(e)` runs for each reactive dependency read during render; `onRenderTriggered(e)` runs when a dependency change causes a re-render, with the `key`, the operation `type` and the old and new values. Put a `debugger` statement inside to inspect in DevTools.

SRC/APP.VUE

HTML

```
<script setup>
import { ref, reactive, onRenderTracked, onRenderTriggered } from 'vue'

const count = ref(0)
const user = reactive({ name: 'Asha', city: 'Pune' })

onRenderTracked((e) => console.log(`tracked:   ${e.type} ${String(e.key)}`))
onRenderTriggered((e) => console.log(`triggered: ${e.type} ${String(e.key)}
${JSON.stringify(e.oldValue)} -> ${JSON.stringify(e.newValue)}`))

setTimeout(() => count.value++, 20)
setTimeout(() => { user.city = 'Delhi' }, 40)     // not used in the template: no re-render
setTimeout(() => { user.name = 'Ravi' }, 60)
</script>

<template>
  <p>{{ count }} · {{ user.name }}</p>
</template>
```

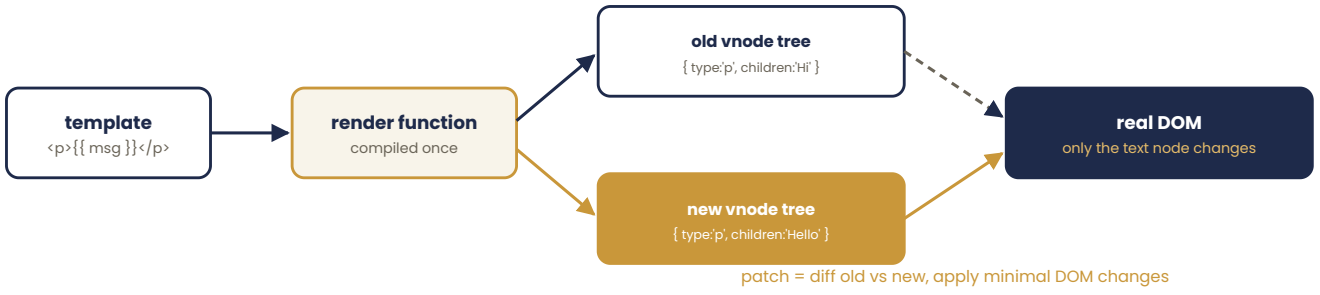
BROWSER CONSOLE

```
tracked:   get value
tracked:   get name
triggered: set value 0 -> 1
tracked:   get value
tracked:   get name
triggered: set name "Asha" -> "Ravi"
tracked:   get value
tracked:   get name
```

17.2 Rendering mechanism

17.2.1 Virtual DOM and patching

The template is compiled into a **render function** that returns a tree of **virtual nodes** (plain JavaScript objects describing elements). On the first render Vue creates real DOM from it (**mount**). When state changes, the component's render effect runs again, produces a new tree, and Vue **patches**: it compares old and new vnodes and changes only the real DOM nodes that differ. Keys (Module 6.2.3) tell the patcher which list items are the same.



RENDER, DIFF, PATCH

17.2.2 Compiler-informed optimisations (static hoisting, patch flags)

Because Vue compiles templates, the compiler knows which parts can **never** change and which **can**, and passes that to the runtime so the diff skips work:

Cached static content

Elements with no bindings (a heading, an icon) are created once and reused on every render (called "hoisting"; in Vue 3.5 the vnode is cached and marked `-1 /* CACHED */`). The patcher skips them entirely.

Patch flags

Each dynamic element gets a number saying **what** can change: `1 /* TEXT */`, `2 /* CLASS */`, `4 /* STYLE */`, `8 /* PROPS */` (with the list of prop names). Only those parts are compared.

Block tree

Each block (a template root, a `v-if` branch, a `v-for` fragment) keeps a flat array of its dynamic descendants, so the patcher visits only those instead of walking the whole tree.

WHAT THE COMPILER PRODUCES JavaScript

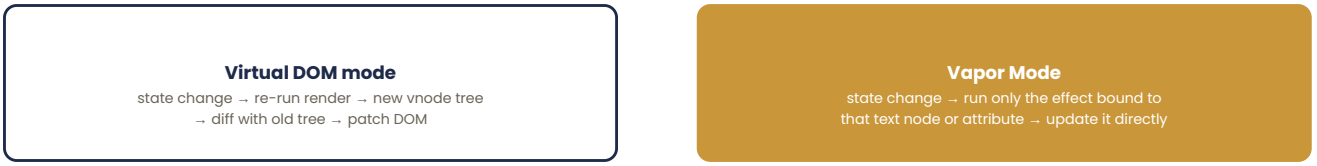
```
// Template:
//   <div>
//     <h1>Students</h1>
//     <p :class="{ active: isActive }">{{ msg }}</p>
//   </div>
// Compiled render function (abridged from the Vue SFC Playground, Vue 3.5):
export function render(_ctx, _cache) {
  return (_openBlock(), _createElementBlock("div", null, [
    _cache[0] || (_cache[0] = _createElementVNode("h1", null, "Students", -1 /* CACHED */)),
    _createElementVNode("p", {
      class: _normalizeClass({ active: _ctx.isActive })
    }, _toDisplayString(_ctx.msg), 3 /* TEXT, CLASS */)
  ]))
}
```

NOTE

Hand-written render functions (Module 18.3) do not get these hints, which is one reason templates are the recommended default. You can see the compiled output of any template in the SFC Playground's "JS" tab.

17.2.3 Vapor Mode (overview)

Vapor Mode is a new compilation strategy (introduced in Vue 3.6, opt-in per component with `<script setup vapor>`). Instead of producing virtual nodes, the compiler generates code that creates the DOM once and binds each dynamic part directly to its reactive source with fine-grained effects, like Solid.js. There is no virtual DOM and no diff, which means less memory, faster updates and a smaller runtime. Vapor components use the same `<script setup>` and Composition API and can be mixed with normal components; some features (for example the Options API) are not supported in Vapor components.



TWO WAYS TO TURN A STATE CHANGE INTO A DOM UPDATE

17.3 Performance techniques

17.3.1 Stable props and avoiding unnecessary updates

A child component re-renders when any prop it receives changes. Passing a value that changes for **every** child (for example the whole `activeId`) re-renders every row, although only two rows actually

change. Compute the per-row value in the parent (`:active="item.id === activeId"`) so props stay **stable** for the rows that are not affected. The rows below count their own renders with `onUpdated`:

SRC/COMPONENTS/ROWA.VUE HTML

```
<script setup>
import { onUpdated } from 'vue'
const props = defineProps({ item: Object, activeId: Number })
onUpdated(() => console.log('  RowA updated:', props.item.name))
</script>
<template><li :class="{ on: item.id === activeId }">{{ item.name }}</li></template>
```

SRC/COMPONENTS/ROWB.VUE HTML

```
<script setup>
import { onUpdated } from 'vue'
const props = defineProps({ item: Object, active: Boolean })
onUpdated(() => console.log('  RowB updated:', props.item.name))
</script>
<template><li :class="{ on: active }">{{ item.name }}</li></template>
```

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
import RowA from './components/RowA.vue'
import RowB from './components/RowB.vue'

const items = ref([{ id: 1, name: 'Asha' }, { id: 2, name: 'Ravi' }, { id: 3, name: 'Meena' }, { id:
4, name: 'Kiran' }])
const activeA = ref(1)
const activeB = ref(1)
setTimeout(() => { console.log('Unstable prop (activeId to every row), 1 → 2:'); activeA.value = 2 },
20)
setTimeout(() => { console.log('Stable prop (active boolean), 1 → 2:'); activeB.value = 2 }, 60)
</script>

<template>
  <ul><RowA v-for="i in items" :key="i.id" :item="i" :active-id="activeA" /></ul>
  <ul><RowB v-for="i in items" :key="i.id" :item="i" :active="i.id === activeB" /></ul>
</template>
```

BROWSER CONSOLE

```
Unstable prop (activeId to every row), 1 → 2:
  RowA updated: Asha
  RowA updated: Ravi
  RowA updated: Meena
  RowA updated: Kiran
Stable prop (active boolean), 1 → 2:
  RowB updated: Asha
  RowB updated: Ravi
```

Other ways to avoid wasted work: keep computed values stable (return the same object only when inputs change; Vue 3.4+ computeds trigger only when their value really changes), avoid creating new inline objects and functions in props of heavy children, and split large components so a change re-renders a small part.

17.3.2 v-once and v-memo

`v-once` renders an element or component **once** and treats it as static afterwards. `v-memo="[a, b]"` skips re-rendering a subtree unless one of the listed values changed; it is mainly useful on large `v-for` lists where only a few rows change. Both are micro-optimisations: use them after measuring.

SRC/APP.VUE

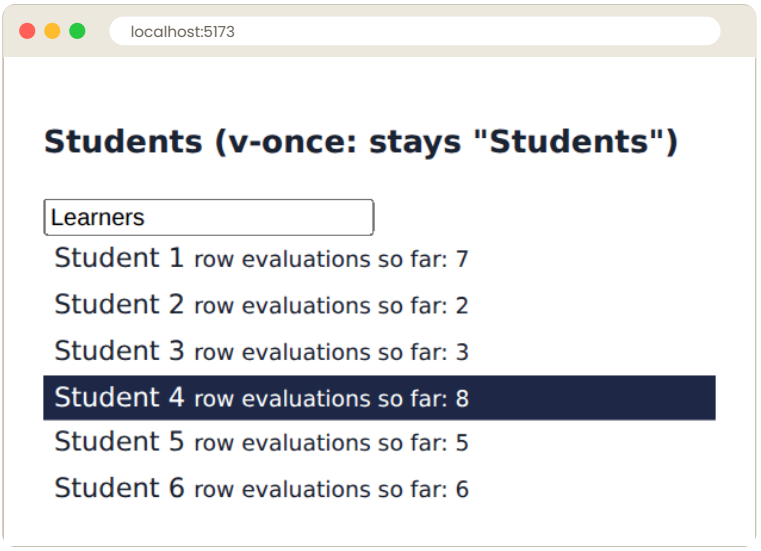
HTML

```
<script setup>
import { ref } from 'vue'

const title = ref('Students')
const selected = ref(1)
const rows = ref(Array.from({ length: 6 }, (_, i) => ({ id: i + 1, name: `Student ${i + 1}` })))
let renders = 0
const count = () => ++renders        // counts how often the row template is evaluated
</script>

<template>
  <h3 v-once>{{ title }} (v-once: stays "Students")</h3>
  <input v-model="title">
  <div v-for="r in rows" :key="r.id" v-memo="[r.id === selected]" :class="{ sel: r.id === selected }"
@click="selected = r.id">
    {{ r.name }} <small>row evaluations so far: {{ count() }}</small>
  </div>
</template>

<style>.sel { background: #1E2A4A; color: white; } div { padding: 2px 6px; cursor: pointer; }</style>
```



TYPED A NEW TITLE AND SELECTED STUDENT 4: THE V-ONCE HEADING DID NOT CHANGE, AND V-MEMO RE-RENDERED ONLY THE TWO ROWS WHOSE SELECTION CHANGED (ROWS 1 AND 4)

17.3.3 Lazy loading routes and async components

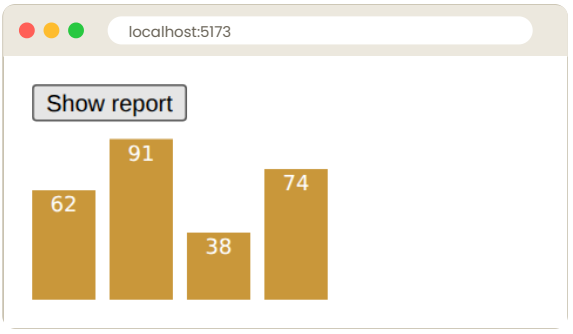
Split code so the browser downloads only what the current page needs. Routes use dynamic imports (Module 13); components that are heavy or rarely shown (a chart, a rich-text editor, a modal) use `defineAsyncComponent()`, which can show a loading component and an error component and is fetched on first render. Vite creates a separate chunk for each dynamic import.

SRC/APP.VUE HTML

```
<script setup>
import { ref, defineAsyncComponent } from 'vue'
import Spinner from './components/Spinner.vue'

const ReportChart = defineAsyncComponent({
  loader: () => import('./components/ReportChart.vue'),   // separate chunk, loaded on first use
  loadingComponent: Spinner,
  delay: 0,
})
const show = ref(false)
</script>

<template>
  <button @click="show = true">Show report</button>
  <ReportChart v-if="show" />
</template>
```



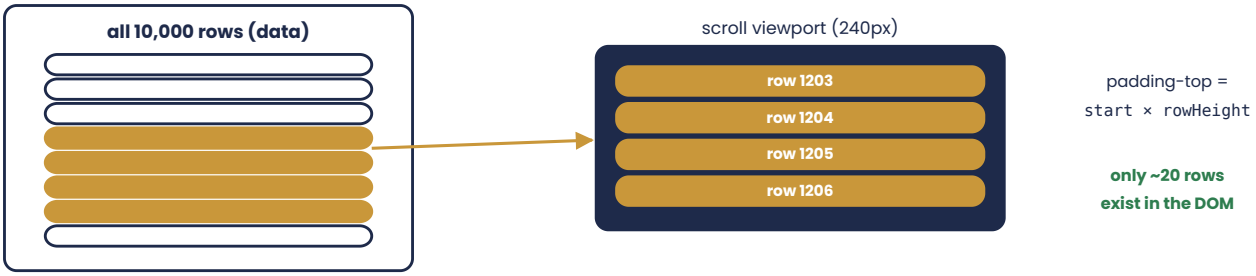
AFTER CLICKING: THE CHART COMPONENT WAS LOADED ON DEMAND AND RENDERED

LAZY ROUTE (MODULE 13) JavaScript

```
// router: each page is its own chunk
{ path: '/reports', component: () => import('@/views/ReportsView.vue') }
```

17.3.4 Virtual scrolling for large lists

Rendering 10,000 rows creates 10,000 DOM nodes and component instances, which is slow to mount and heavy on memory. **Virtual scrolling** renders only the rows visible in the scroll area (plus a small buffer) and uses spacer padding so the scrollbar still represents the full list. When the user scrolls, the window of rendered rows moves. Libraries such as **vue-virtual-scroller**, **@tanstack/vue-virtual** or VueUse's `useVirtualList` provide this; the idea fits in one component for fixed-height rows:



RENDER ONLY WHAT IS VISIBLE

SRC/COMPONENTS/VIRTUALLIST.VUE HTML

```
<script setup>
import { ref, computed } from 'vue'

const props = defineProps({
  items: { type: Array, required: true },
  rowHeight: { type: Number, default: 30 },
  height: { type: Number, default: 240 },
  buffer: { type: Number, default: 5 },
})
const scrollTop = ref(0)
const start = computed(() => Math.max(0, Math.floor(scrollTop.value / props.rowHeight) -
props.buffer))
const end = computed(() => Math.min(props.items.length, Math.ceil((scrollTop.value + props.height) /
props.rowHeight) + props.buffer))
const visible = computed(() => props.items.slice(start.value, end.value))
</script>

<template>
  <div class="viewport" :style="{ height: height + 'px' }" @scroll="scrollTop =
$event.target.scrollTop">
    <div :style="{ height: items.length * rowHeight + 'px', position: 'relative' }">
      <div :style="{ transform: `translateY(${start * rowHeight}px)` }">
        <div v-for="item in visible" :key="item.id" class="row" :style="{ height: rowHeight + 'px' }">
          <slot :item="item" />
        </div>
      </div>
    </div>
  </div>
</template>

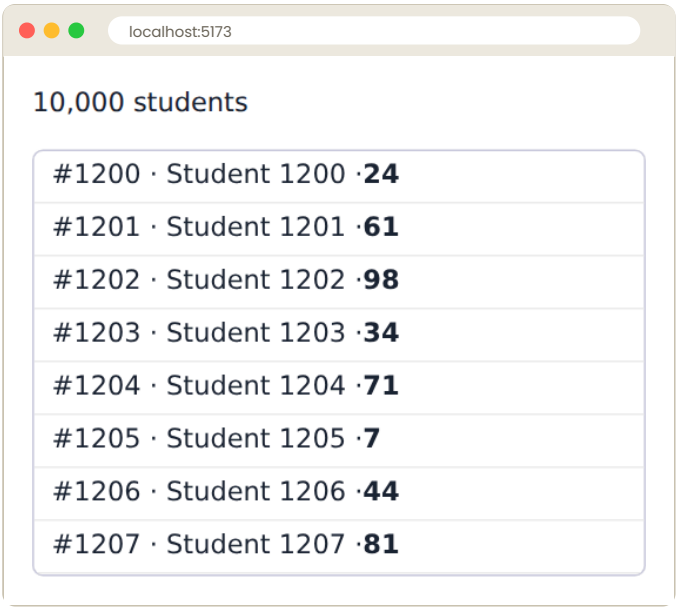
<style scoped>
.viewport { overflow-y: auto; border: 1px solid #ccd; border-radius: 6px; }
.row { display: flex; align-items: center; padding: 0 10px; border-bottom: 1px solid #eee; box-sizing:
border-box; }
</style>
```

SRC/APP.VUE HTML

```
<script setup>
import { shallowRef } from 'vue'
import VirtualList from './components/VirtualList.vue'

const students = shallowRef(Array.from({ length: 10000 }, (_, i) => ({ id: i + 1, name: `Student ${i +
1}`, marks: (i * 37) % 101 })))
</script>

<template>
  <p>{{ students.length.toLocaleString() }} students</p>
  <VirtualList :items="students" v-slot="{ item }">
    #{{ item.id }} · {{ item.name }} · <b>{{ item.marks }}</b>
  </VirtualList>
</template>
```



SCROLLED TO ROW 1,200 OF 10,000: ONLY THE VISIBLE ROWS (PLUS A BUFFER) ARE IN THE DOM

BROWSER CONSOLE: COUNTING DOM ROWS

```
rows in the DOM: 13
after scrolling, rows in the DOM: 18 | first: #1195 · Student 1195 · 41
```

17.3.5 Shallow reactivity for large data

Deep reactivity has a cost: every nested object read through a `ref` is wrapped in a proxy, and every property access is tracked. For large datasets that are replaced rather than edited in place (API results, imported files, chart data) use `shallowRef` and update by **replacing** the array. The measurement below sums a field over 100,000 rows read through a deep ref and through a shallow ref (timings from one run in Chromium):

SRC/APP.VUE HTML

```
<script setup>
import { ref, shallowRef } from 'vue'

const data = () => Array.from({ length: 100000 }, (_, i) => ({ id: i, marks: i % 100, address: { city:
'Pune' } }))
function sumMs(source) {
  const t = performance.now()
  let total = 0
  for (const row of source.value) total += row.marks + row.address.city.length
  return [(performance.now() - t).toFixed(1), total]
}
const deep = ref(data())
const shallow = shallowRef(data())

sumMs(deep); sumMs(shallow)                  // warm-up run
const [d, t1] = sumMs(deep)
const [s, t2] = sumMs(shallow)
console.log(`deep ref:    ${d} ms (sum ${t1})`)
console.log(`shallowRef:  ${s} ms (sum ${t2})`)

// updating a shallowRef: replace, do not mutate
shallow.value = shallow.value.map((r) => (r.id === 0 ? { ...r, marks: 99 } : r))
console.log('first row after replace:', shallow.value[0].marks)
</script>
<template><p></p></template>
```

BROWSER CONSOLE

```
deep ref:    72.0 ms (sum 5350000)
shallowRef:  16.5 ms (sum 5350000)
first row after replace: 99
```

The exact numbers depend on the machine; the deep version is consistently several times slower because each row and each nested object is proxied and every read is tracked.

17.3.6 Bundle size analysis and tree-shaking

Vue 3 is **tree-shakable**: features you never import (Transition, KeepAlive, v-model helpers) are dropped from the production bundle. The same applies to your own code and libraries that use ES modules: import named functions (`import { debounce } from 'lodash-es'`) rather than whole libraries. `vite build` prints the size of each chunk; a visualizer shows what is inside them.

Build output (representative)

```
$ npx vite build
vite v7.1.5 building for production...
✓ 112 modules transformed.
dist/index.html                          0.46 kB │ gzip:  0.30 kB
dist/assets/index-BqJ7d2Zk.css           3.12 kB │ gzip:  1.21 kB
dist/assets/ReportsView-C1x8aQpM.js      2.41 kB │ gzip:  1.18 kB
dist/assets/index-D9fK2mLs.js           98.73 kB │ gzip: 38.90 kB
✓ built in 1.42s
```

VITE.CONFIG.JS — JavaScript

```
// vite.config.js: npm i -D rollup-plugin-visualizer
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import { visualizer } from 'rollup-plugin-visualizer'

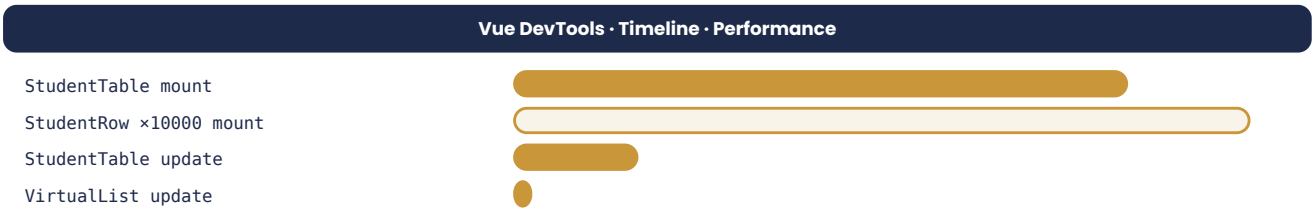
export default defineConfig({
  plugins: [vue(), visualizer({ filename: 'stats.html', gzipSize: true, open: true })],
})
```

Problem found in the report	Fix
A big library used in one page	lazy-load that route or component
A whole utility library (lodash, date libraries)	import single functions or use a smaller ES-module library
All icons of an icon set	import individual icons
Large JSON or images imported in code	move to `public/` or fetch at runtime

17.4 Measuring performance

17.4.1 Vue DevTools performance timeline

The Vue DevTools **Timeline** records component events, mount and update durations and Pinia actions. Enable "Performance" in its settings, interact with the page, and look for components that update often or take long. For deeper analysis use the browser's **Performance** panel (with `app.config.performance = true` Vue adds its render and patch timings to it).



SPOTTING SLOW MOUNTS AND UPDATES (REPRESENTATIVE)

17.4.2 Lighthouse and Core Web Vitals

Lighthouse (Chrome DevTools → Lighthouse) audits a page for performance, accessibility, best practices and SEO, and lists concrete fixes. Google's **Core Web Vitals** measure real user experience:

Metric	Measures	Good	Typical Vue fixes
LCP (Largest Contentful Paint)	loading: when the main content appears	≤ 2.5 s	code splitting, smaller bundles, SSR/SSG (Nuxt), optimised images
INP (Interaction to Next Paint)	responsiveness to clicks and typing	≤ 200 ms	less work per update, virtual lists, debounced handlers, shallow data
CLS (Cumulative Layout Shift)	visual stability	≤ 0.1	reserve space for images and async content, skeleton loaders

PRACTICE CHECKPOINT

Render a list of 10,000 rows, profile it, then optimise it with virtual scrolling and shallow reactivity and compare the results.

The naive version renders every row from a deep `ref`; the optimised version uses the `VirtualList` from 17.3.4 with a `shallowRef`. Each page measures the time from mounting to the first paint and counts the DOM rows:

SRC/APP.VUE (NAIVE) HTML

```
<script setup>
import { ref, onMounted } from 'vue'
const label = 'naive (ref + all rows)'
const t0 = performance.now()
const students = ref(Array.from({ length: 10000 }, (_, i) => ({ id: i + 1, name: `Student ${i + 1}`,
marks: (i * 37) % 101 })))
onMounted(() => requestAnimationFrame(() => console.log(`${label}: mounted + painted in
${(performance.now() - t0).toFixed(0)} ms, ${document.querySelectorAll('.row').length} rows in the
DOM`)))
</script>

<template>
  <div style="height: 240px; overflow: auto">
    <div v-for="s in students" :key="s.id" class="row">#{{ s.id }} · {{ s.name }} · {{ s.marks }}
</div>
  </div>
</template>
```

BROWSER CONSOLE: NAIVE

```
naive (ref + all rows): mounted + painted in 68 ms, 10000 rows in the DOM
```

SRC/APP.VUE (OPTIMISED) HTML

```
<script setup>
import { shallowRef, onMounted } from 'vue'
import VirtualList from './components/VirtualList.vue'
const label = 'optimised (shallowRef + virtual list)'
const t0 = performance.now()
const students = shallowRef(Array.from({ length: 10000 }, (_, i) => ({ id: i + 1, name: `Student ${i +
1}`, marks: (i * 37) % 101 })))
onMounted(() => requestAnimationFrame(() => console.log(`${label}: mounted + painted in
${(performance.now() - t0).toFixed(0)} ms, ${document.querySelectorAll('.row').length} rows in the
DOM`)))
</script>

<template>
  <VirtualList :items="students" v-slot="{ item }">#{{ item.id }} · {{ item.name }} · {{ item.marks }}
</VirtualList>
</template>
```

BROWSER CONSOLE: OPTIMISED

```
optimised (shallowRef + virtual list): mounted + painted in 17 ms, 13 rows in the DOM
```

The optimised page creates 13 DOM rows instead of 10,000 and mounts about ten times faster; scrolling stays smooth because each scroll event renders only about 20 rows. Record both versions in the DevTools Performance panel to see the difference in scripting and rendering time.

Reactivity = Proxy get → `track()`, set → `trigger()`. Use `shallowRef` / `shallowReactive` for large or replaced data, `readonly` to expose state safely, `markRaw` for objects that must never be proxied and `toRaw` to read originals; `effectScope` groups effects and `onRenderTracked/Triggered` explain renders. The compiler caches static content and adds patch flags so the virtual DOM diff is cheap; Vapor Mode removes the virtual DOM. For speed: stable props, `v-once` / `v-memo` where measured, lazy routes and async components, virtual scrolling, tree-shaking, and always measure with DevTools, Lighthouse and the Core Web Vitals.

MODULE 18 · ADVANCED

Advanced patterns, plugins and UI libraries

Minimum duration **2 days** | Level **Advanced**

This module covers extending Vue itself and building polished interfaces. It writes custom directives with their hooks (v-focus, v-click-outside, v-tooltip, v-permission), packages features as plugins with install(), global properties and global components, uses render functions with h() and introduces JSX, and builds three reusable component patterns: renderless components, compound components with provide/inject and error boundaries. It then covers the styling options of Single-File Components (scoped CSS and :deep(), CSS Modules, v-bind() in CSS, Tailwind CSS), surveys the main UI libraries (Vuetify, PrimeVue, Element Plus, Naive UI and the headless Reka UI and shadcn-vue), and finishes with accessible components, focus management and Vue I18n.

IN THIS MODULE

18.1 Custom directives
18.2 Plugins
18.3 Render functions and JSX
18.4 Component patterns
18.5 Styling in Vue
18.6 UI libraries
18.7 Accessibility and internationalisation

18.1 Custom directives

18.1.1 Directive hooks

A **custom directive** gives reusable low-level access to a DOM element, for behaviour that is about the element itself (focus, observers, outside clicks, tooltips) rather than about rendering. It is an object of **hooks** that Vue calls during the element's life. In `<script setup>`, any variable named `vXxx` is usable as `v-xxx` in the template; for app-wide use register it with `app.directive('xxx', obj)`. If only `mounted` and `updated` are needed, a function can be used instead of an object.

Hook	Called when
`created(el, binding)`	before the element's attributes and listeners are applied
`beforeMount` / `mounted`	before / after the element is inserted into the DOM
`beforeUpdate` / `updated`	before / after the containing component updates
`beforeUnmount` / `unmounted`	before / after the element is removed (clean up listeners here)

Each hook receives the element and a `binding` object: `value` (the expression's value), `oldValue`, `arg` (after the colon) and `modifiers` (after dots). Here a directive logs its hooks and binding:

SRC/APP.VUE

HTML

```
<script setup>
import { ref } from 'vue'

const vLog = {
  mounted(el, b) { console.log(`mounted: value=${b.value} arg=${b.arg}
modifiers=${JSON.stringify(b.modifiers)}`) },
  updated(el, b) { console.log(`updated: ${b.oldValue} → ${b.value}`) },
  unmounted() { console.log('unmounted: remove listeners/observers here') },
}
const count = ref(1)
const show = ref(true)
setTimeout(() => count.value++, 20)
setTimeout(() => (show.value = false), 40)
</script>

<template>
  <p v-if="show" v-log:highlight.once.fast="count">{{ count }}</p>
</template>
```

BROWSER CONSOLE

```
mounted: value=1 arg=highlight modifiers={"once":true,"fast":true}
updated: 1 → 2
unmounted: remove listeners/observers here
```

18.1.2 Examples: v-focus, v-click-outside, v-tooltip, v-permission

Four directives that real apps use, kept in one file and registered globally in `main.js`:

SRC/DIRECTIVES.JS JavaScript

```
// v-focus: focus the element when it is inserted
export const focus = {
  mounted: (el) => el.focus(),
}

// v-click-outside="handler": call handler when a click happens outside the element
export const clickOutside = {
  mounted(el, binding) {
    el._outside = (e) => { if (!el.contains(e.target)) binding.value(e) }
    document.addEventListener('click', el._outside)
  },
  unmounted(el) {
    document.removeEventListener('click', el._outside)         // always clean up
  },
}

// v-tooltip:top="'text'": show a small label on hover
export const tooltip = {
  mounted(el, { value, arg = 'top' }) {
    el.classList.add('has-tip', `tip-${arg}`)
    el.dataset.tip = value
  },
  updated(el, { value }) { el.dataset.tip = value },
}

// v-permission="'admin'": remove the element if the user lacks the role
export function createPermission(getRoles) {
  return {
    mounted(el, { value }) {
      const needed = Array.isArray(value) ? value : [value]
      if (!needed.some((r) => getRoles().includes(r))) el.remove()
    },
  }
}
```

SRC/MAIN.JS JavaScript

```
import { createApp } from 'vue'
import App from './App.vue'
import { focus, clickOutside, tooltip, createPermission } from './directives'

const currentUser = { name: 'Asha', roles: ['teacher'] }

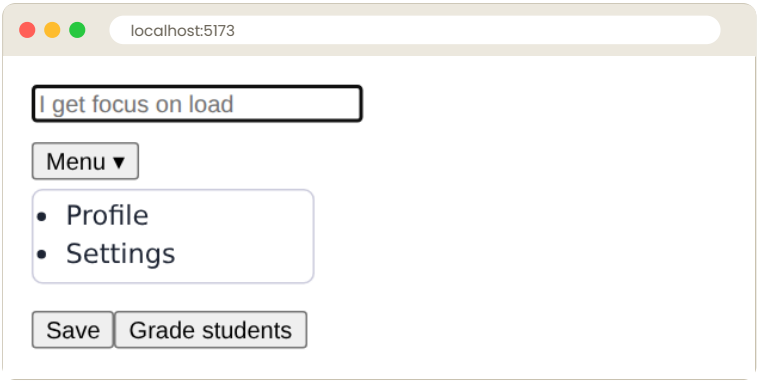
createApp(App)
  .directive('focus', focus)
  .directive('click-outside', clickOutside)
  .directive('tooltip', tooltip)
  .directive('permission', createPermission(() => currentUser.roles))
  .mount('#app')
```

SRC/APP.VUE HTML

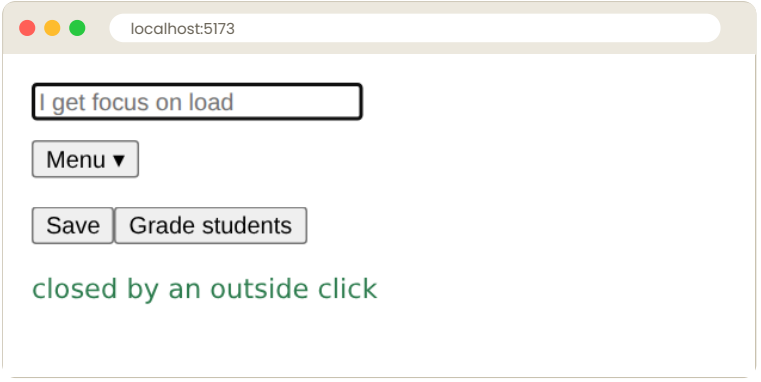
```
<script setup>
import { ref } from 'vue'
const open = ref(true)
const log = ref('')
</script>

<template>
  <input v-focus placeholder="I get focus on load">
  <div class="menu-wrap">
    <button @click.stop="open = !open">Menu ▾</button>
    <ul v-if="open" v-click-outside="() => { open = false; log = 'closed by an outside click' }"
class="menu">
      <li>Profile</li><li>Settings</li>
    </ul>
  </div>
  <p>
    <button v-tooltip:bottom="'Saves all changes'">Save</button>
    <button v-permission="'teacher'">Grade students</button>
    <button v-permission="'admin'">Delete school</button>
  </p>
  <p class="log">{{ log }}</p>
</template>

<style>
.menu-wrap { position: relative; margin: 10px 0; } .menu { margin: 4px 0; border: 1px solid #ccd;
border-radius: 6px; padding: 4px 18px; width: 120px; }
.log { color: #2E7D4F; }
</style>
```



THE INPUT WAS FOCUSED ON LOAD, THE "ADMIN" BUTTON WAS REMOVED BY V-PERMISSION, AND HOVERING SAVE SHOWS THE TOOLTIP



AFTER CLICKING ELSEWHERE ON THE PAGE: V-CLICK-OUTSIDE CLOSED THE MENU

NOTE

The tooltip appears with the CSS `:hover` rule, which a screenshot cannot trigger; hover the button in the real app. A `v-permission` directive only hides UI: the server must still check permissions.

18.2 Plugins

18.2.1 Writing a plugin with install()

A **plugin** packages app-level features so one line, `app.use(plugin, options)`, installs them. It is an object with an `install(app, options)` method (or just that function). Inside it you can register global components and directives, `app.provide()` values for composables, add global properties, and add other plugins. Vue Router, Pinia and Vue I18n are all plugins. Here is a toast notification plugin:

SRC/PLUGINS/TOAST/INDEX.JS

JavaScript

```
import { reactive, inject } from 'vue'
import ToastHost from './ToastHost.vue'

const ToastKey = Symbol('toast')

export function createToast({ duration = 2500 } = {}) {
  const state = reactive({ items: [] })
  let id = 0
  function show(message, type = 'info') {
    const item = { id: ++id, message, type }
    state.items.push(item)
    setTimeout(() => (state.items = state.items.filter((t) => t !== item)), duration)
  }
  const api = { state, show, success: (m) => show(m, 'success'), error: (m) => show(m, 'error') }

  return {
    install(app) {
      app.provide(ToastKey, api)                    // for useToast() in <script setup>
      app.config.globalProperties.$toast = api      // for templates / Options API
      app.component('ToastHost', ToastHost)         // global component
    },
  }
}

export function useToast() {
  const api = inject(ToastKey)
  if (!api) throw new Error('useToast() needs app.use(createToast())')
  return api
}
```

SRC/PLUGINS/TOAST/TOASTHOST.VUE HTML

```
<script setup>
import { useToast } from './index.js'
const { state } = useToast()
</script>

<template>
  <div class="toasts">
    <TransitionGroup name="toast">
      <div v-for="t in state.items" :key="t.id" :class="['toast', t.type]">{{ t.message }}</div>
    </TransitionGroup>
  </div>
</template>

<style scoped>
.toasts { position: fixed; top: 12px; right: 12px; display: grid; gap: 8px; }
.toast { padding: 8px 14px; border-radius: 8px; color: white; background: #1E2A4A; box-shadow: 0 4px
12px #0003; font-size: 14px; }
.success { background: #2E7D4F; } .error { background: #B33A3A; }
</style>
```

SRC/MAIN.JS JavaScript

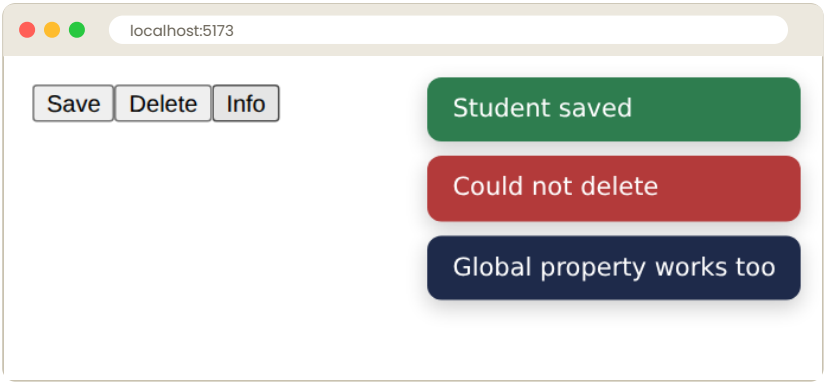
```
import { createApp } from 'vue'
import App from './App.vue'
import { createToast } from './plugins/toast'

createApp(App).use(createToast({ duration: 4000 })).mount('#app')
```

SRC/APP.VUE HTML

```
<script setup>
import { useToast } from './plugins/toast'
const toast = useToast()
</script>

<template>
  <ToastHost />
  <button @click="toast.success('Student saved')">Save</button>
  <button @click="toast.error('Could not delete')">Delete</button>
  <button @click="$toast.show('Global property works too')">Info</button>
</template>
```



CLICKED ALL THREE BUTTONS: THREE TOASTS FROM ONE PLUGIN

18.2.2 Adding global properties and components

`app.config.globalProperties.$name` makes a value available as `$name` in every template (and as `this.$name` in the Options API). In `<script setup>` prefer `provide` / `inject` with a composable (like `useToast()` above), which is explicit and typed. `app.component('Name', comp)` registers a global component usable without importing. Use both sparingly: globals hide where things come from and cannot be tree-shaken. With TypeScript, declare global properties by augmenting `ComponentCustomProperties`:

TYPING GLOBAL PROPERTIES AND COMPONENTS TypeScript

```
// src/plugins/toast/types.d.ts
import type { ToastApi } from './index'
declare module 'vue' {
  interface ComponentCustomProperties {
    $toast: ToastApi
  }
  interface GlobalComponents {
    ToastHost: typeof import('./ToastHost.vue')['default']
  }
}
export {}
```

18.3 Render functions and JSX

18.3.1 h() and when to use render functions

Templates compile to **render functions**; you can also write one yourself. `h(type, props, children)` creates a virtual node: `type` is a tag name or component, `props` holds attributes, classes, styles and `onXxx` listeners, and `children` is a string, an array of vnodes or an object of slot functions. Return a render function from `setup()`. Use render functions when the structure itself is dynamic and awkward in a template (a heading whose level is a prop, components that wrap or clone their slot content); prefer templates otherwise, because they are easier to read and get compiler optimisations (Module 17.2.2).

SRC/COMPONENTS/APPHEADING.JS JavaScript

```
import { h } from 'vue'

export default {
  props: { level: { type: Number, default: 2 }, color: String },
  setup(props, { slots }) {
    // render function: <h1>…<h6> chosen at runtime
    return () => h(`h${props.level}`, { style: { color: props.color, margin: '6px 0' } },
slots.default?.())
  },
}
```

SRC/APP.VUE

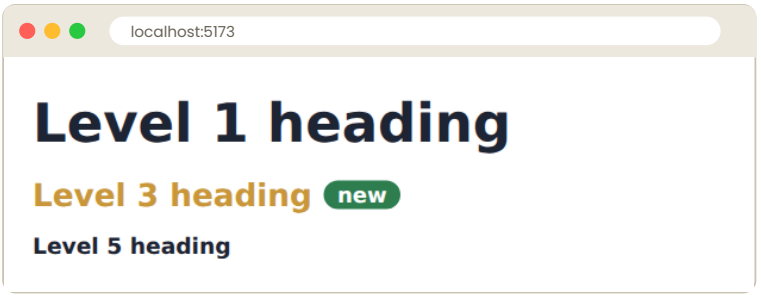
HTML

```
<script setup>
import { h } from 'vue'
import AppHeading from './components/AppHeading.js'

// a functional component: just a function of (props, { slots })
const Badge = (props, { slots }) => h('span', { class: ['badge', props.type] }, slots.default?.())
Badge.props = ['type']
</script>

<template>
  <AppHeading :level="1">Level 1 heading</AppHeading>
  <AppHeading :level="3" color="#C9973A">Level 3 heading <Badge type="new">new</Badge></AppHeading>
  <AppHeading :level="5">Level 5 heading</AppHeading>
</template>

<style>.badge { font-size: 12px; background: #2E7D4F; color: white; padding: 1px 8px; border-radius:
99px; vertical-align: middle; }</style>
```



ONE COMPONENT RENDERS H1, H3 OR H5 DEPENDING ON THE LEVEL PROP

18.3.2 JSX in Vue (overview)

JSX is an HTML-like syntax inside JavaScript that compiles to `h()` calls. Install `@vitejs/plugin-vue-jsx`, add it to `vite.config`, and write `.jsx` / `.tsx` files or `<script setup lang="tsx">`. Vue JSX differs from React's: use `class` (not `className`), `onClick` for listeners, `v-model` and `v-show` work, and slots are passed as an object. It suits teams coming from React and highly dynamic components; templates remain the default.

THE SAME IDEA IN JSX (NOT RUN: NEEDS THE JSX PLUGIN)

JavaScript

```
// AppHeading.jsx (needs @vitejs/plugin-vue-jsx)
import { defineComponent, ref } from 'vue'

export default defineComponent({
  props: { level: { type: Number, default: 2 } },
  setup(props, { slots }) {
    const clicks = ref(0)
    return () => {
      const Tag = `h${props.level}`
      return (
        <Tag class="heading" onClick={() => clicks.value++}>
          {slots.default?.()} <small>({clicks.value} clicks)</small>
        </Tag>
      )
    }
  },
})
```

18.4 Component patterns

18.4.1 Renderless components

A **renderless component** contains only logic and renders nothing of its own: it passes its state to the parent through a **scoped slot**, and the parent decides the markup. It is the template-based alternative to a composable (Module 12), handy when the logic and the slot usage belong together in the template. This one fetches a URL and exposes `data`, `loading`, `error` and `reload`:

SRC/COMPONENTS/FETCHDATA.VUE HTML

```
<script setup>
import { ref, watchEffect } from 'vue'

const props = defineProps({ url: { type: String, required: true } })
const data = ref(null)
const loading = ref(false)
const error = ref(null)

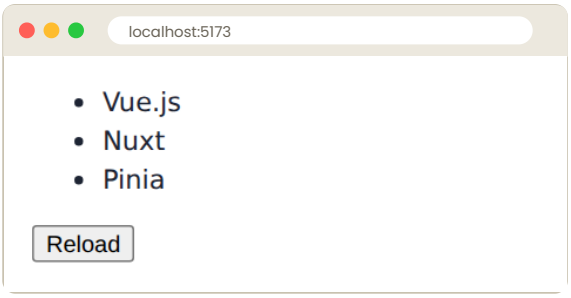
async function load() {
  loading.value = true
  error.value = null
  try {
    const res = await fetch(props.url)
    if (!res.ok) throw new Error(`HTTP ${res.status}`)
    data.value = await res.json()
  } catch (e) {
    error.value = e.message
  } finally {
    loading.value = false
  }
}
watchEffect(load)
</script>

<template>
  <slot :data="data" :loading="loading" :error="error" :reload="load" />
</template>
```

SRC/APP.VUE HTML

```
<script setup>
import FetchData from './components/FetchData.vue'
</script>

<template>
  <FetchData url="/api/courses" v-slot="{ data, loading, error, reload }">
    <p v-if="loading">Loading courses…</p>
    <p v-else-if="error">⚠ {{ error }}</p>
    <ul v-else>
      <li v-for="c in data" :key="c.id">{{ c.title }}</li>
    </ul>
    <button @click="reload">Reload</button>
  </FetchData>
</template>
```



FETCHDATA RENDERS NO MARKUP OF ITS OWN; THE PARENT DREW THE LIST

18.4.2 Compound components with provide/inject

Compound components are a set of components that work together and share hidden state, like `<select>` and `<option>`: the parent (`Tabs`) provides the state and each child (`Tab`) injects it and registers itself. The user composes them freely in the template without passing props between them.

SRC/COMPONENTS/TABS.VUE HTML

```
<script setup>
import { ref, provide } from 'vue'

const props = defineProps({ defaultTab: String })
const titles = ref([])
const active = ref(props.defaultTab)

provide('tabs', {
  active,
  register: (title) => { titles.value.push(title); if (!active.value) active.value = title },
})
</script>

<template>
  <div class="tabs">
    <nav>
      <button v-for="t in titles" :key="t" :class="{ on: t === active }" @click="active = t">{{ t }}
</button>
    </nav>
    <section><slot /></section>
  </div>
</template>

<style scoped>
nav { display: flex; gap: 4px; border-bottom: 2px solid #1E2A4A; }
button { border: 0; padding: 6px 14px; background: #eef; border-radius: 6px 6px 0 0; cursor: pointer;
}
.on { background: #1E2A4A; color: white; }
section { padding: 10px 4px; }
</style>
```

SRC/COMPONENTS/TAB.VUE

HTML

```
<script setup>
import { inject } from 'vue'

const props = defineProps({ title: { type: String, required: true } })
const tabs = inject('tabs')
tabs.register(props.title)
</script>

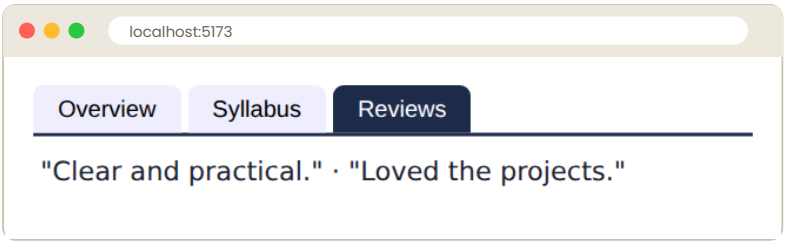
<template>
  <div v-show="tabs.active.value === title"><slot /></div>
</template>
```

SRC/APP.VUE

HTML

```
<script setup>
import Tabs from './components/Tabs.vue'
import Tab from './components/Tab.vue'
</script>

<template>
  <Tabs default-tab="Syllabus">
    <Tab title="Overview">Vue.js is a progressive framework for building user interfaces.</Tab>
    <Tab title="Syllabus">21 modules from templates to Nuxt, 42 days.</Tab>
    <Tab title="Reviews">"Clear and practical." · "Loved the projects."</Tab>
  </Tabs>
</template>
```



CLICKED "REVIEWS": TAB COMPONENTS READ THE ACTIVE TAB FROM THEIR TABS PARENT

18.4.3 Error boundaries with onErrorCaptured

`onErrorCaptured(err, instance, info)` runs in an ancestor when any descendant throws during render, in a lifecycle hook, a watcher or an event handler. Returning `false` stops the error from propagating further (to `app.config.errorHandler`). An **error boundary** component uses it to show a fallback instead of a broken page, with a way to retry:

SRC/COMPONENTS/ERRORBOUNDARY.VUE HTML

```
<script setup>
import { ref, onErrorCaptured } from 'vue'

const error = ref(null)
onErrorCaptured((err, instance, info) => {
  console.error(`caught in ${info}:`, err.message)    // report to a logging service here
  error.value = err
  return false                                        // stop propagation
})
const reset = () => (error.value = null)
</script>

<template>
  <slot v-if="error" name="fallback" :error="error" :reset="reset" />
  <slot v-else />
</template>
```

SRC/COMPONENTS/STUDENTWIDGET.VUE HTML

```
<script setup>
defineProps({ data: Object })
</script>

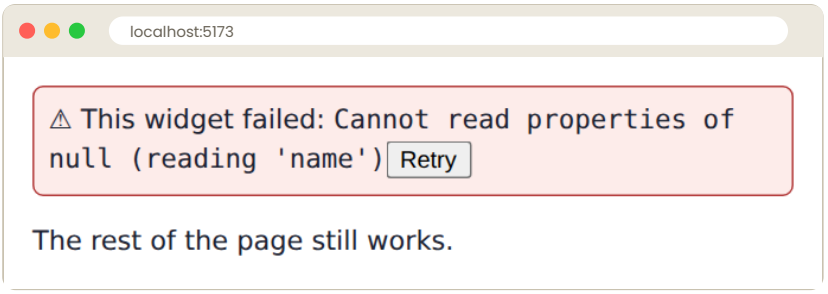
<template>
  <p>{{ data.name }} scored {{ data.marks }}</p>
</template>
```

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
import ErrorBoundary from './components/ErrorBoundary.vue'
import StudentWidget from './components/StudentWidget.vue'
const broken = ref(true)
</script>

<template>
  <ErrorBoundary>
    <StudentWidget :data="broken ? null : { name: 'Asha', marks: 91 }" />
    <template #fallback="{ error, reset }">
      <div class="fallback">
        ⚠ This widget failed: <code>{{ error.message }}</code>
        <button @click="broken = false; reset()">Retry</button>
      </div>
    </template>
  </ErrorBoundary>
  <p>The rest of the page still works.</p>
</template>

<style>.fallback { background: #fdecea; border: 1px solid #B33A3A; padding: 8px; border-radius: 6px; }
</style>
```



THE CHILD THREW DURING RENDER (DATA WAS NULL): THE BOUNDARY SHOWED ITS FALLBACK

18.5 Styling in Vue

18.5.1 Scoped CSS and deep selectors :deep()

`<style scoped>` (Module 3.4.2) adds a unique attribute such as `data-v-7ba5bd90` to the component's elements and to its selectors, so the rules cannot leak out. The **root** element of a child also gets the parent's attribute, but elements **inside** the child do not, so a parent cannot style them. `:deep(selector)` moves the attribute to the left part of the selector so it reaches into children (useful for content from libraries or `v-html`). `:slotted()` targets slot content from the child, and `:global()` writes an unscoped rule.

SRC/COMPONENTS/RICHTEXT.VUE — HTML

```
<script setup>
defineProps({ html: String })
</script>

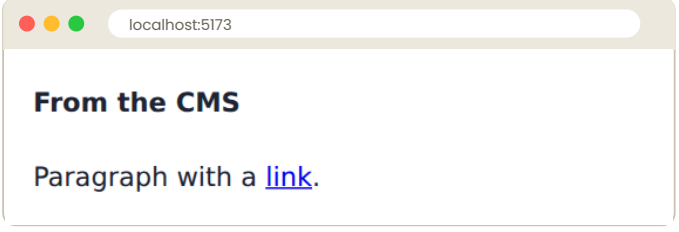
<template>
  <div class="rich" v-html="html"></div>
</template>
```

SRC/APP.VUE — HTML

```
<script setup>
import RichText from './components/RichText.vue'
</script>

<template>
  <div class="article">
    <RichText html="<h4>From the CMS</h4><p>Paragraph with a <a href='#'>link</a>.</p>" />
  </div>
</template>

<style scoped>
.article h4 { color: red; }               /* does NOT reach inside RichText */
.article :deep(h4) { color: #C9973A; }     /* compiled to .article[data-v-xxx] h4 */
.article :deep(a) { color: #2E7D4F; font-weight: bold; }
</style>
```



THE :DEEP() RULES STYLED THE H4 AND LINK RENDERED INSIDE THE CHILD

18.5.2 CSS Modules

`<style module>` compiles each class to a unique, hashed name and exposes the mapping as `$style` in the template (and through `useCssModule()` in script). Classes are bound with `:class="$style.card"`. CSS Modules are an alternative to scoped styles that also make class names available to JavaScript; `<style module="classes">` renames the object.

SRC/COMPONENTS/MODULECARD.VUE HTML

```
<script setup>
import { useCssModule } from 'vue'
const props = defineProps({ highlight: Boolean })
const style = useCssModule()          // the same map in script, e.g. style.card
</script>

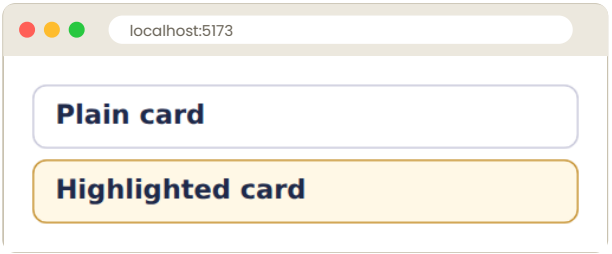
<template>
  <div :class="[$style.card, { [$style.highlight]: highlight }]">
    <h4 :class="$style.title"><slot /></h4>
  </div>
</template>

<style module>
.card { border: 1px solid #ccd; border-radius: 8px; padding: 6px 12px; margin: 6px 0; }
.highlight { border-color: #C9973A; background: #fff8e6; }
.title { margin: 0; color: #1E2A4A; }
</style>
```

SRC/APP.VUE HTML

```
<script setup>
import ModuleCard from './components/ModuleCard.vue'
</script>

<template>
  <ModuleCard>Plain card</ModuleCard>
  <ModuleCard highlight>Highlighted card</ModuleCard>
</template>
```



TWO CARDS STYLED THROUGH $STYLE

RENDERED DOM: HASHED CLASS NAMES

```
<div id="app" data-v-app="">
  <div class="_card_GfHcP">
    <h4 class="_title_gGHQt">Plain card</h4>
  </div>
  <div class="_card_GfHcP _highlight_JT9Fl">
    <h4 class="_title_gGHQt">Highlighted card</h4>
  </div>
</div>
```

18.5.3 v-bind() in CSS

`v-bind(expression)` inside `<style>` links a CSS value to component state. The compiler turns it into a CSS custom property set on the component's root element, which updates reactively, so no inline styles are needed. Wrap expressions with dots in quotes: `v-bind('theme.color')`.

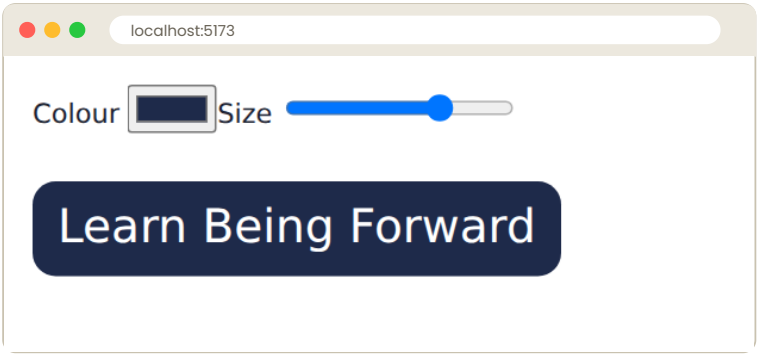
SRC/APP.VUE

HTML

```
<script setup>
import { ref, computed } from 'vue'
const color = ref('#C9973A')
const size = ref(18)
const theme = computed(() => ({ radius: size.value / 2 + 'px' }))
</script>

<template>
  <label>Colour <input v-model="color" type="color"></label>
  <label>Size <input v-model.number="size" type="range" min="12" max="32"></label>
  <p class="preview">Learn Being Forward</p>
</template>

<style scoped>
.preview {
  color: white;
  background: v-bind(color);
  font-size: v-bind(size + 'px');
  border-radius: v-bind('theme.radius');
  padding: 8px 14px;
  display: inline-block;
}
</style>
```



CHANGED THE COLOUR TO NAVY AND THE SIZE TO 26: THE CSS UPDATED THROUGH CUSTOM PROPERTIES

18.5.4 Tailwind CSS with Vue

Tailwind CSS v4 works in Vue with the Vite plugin: `npm install tailwindcss @tailwindcss/vite`, add `tailwindcss()` to `vite.config` plugins and `@import "tailwindcss";` to the main CSS file. Utility classes then go straight into templates, including dynamic `:class` bindings (write full class names so Tailwind can find them). Components keep repeated combinations in one place, which is the Vue answer to long class lists.

VITE.CONFIG.JS JavaScript

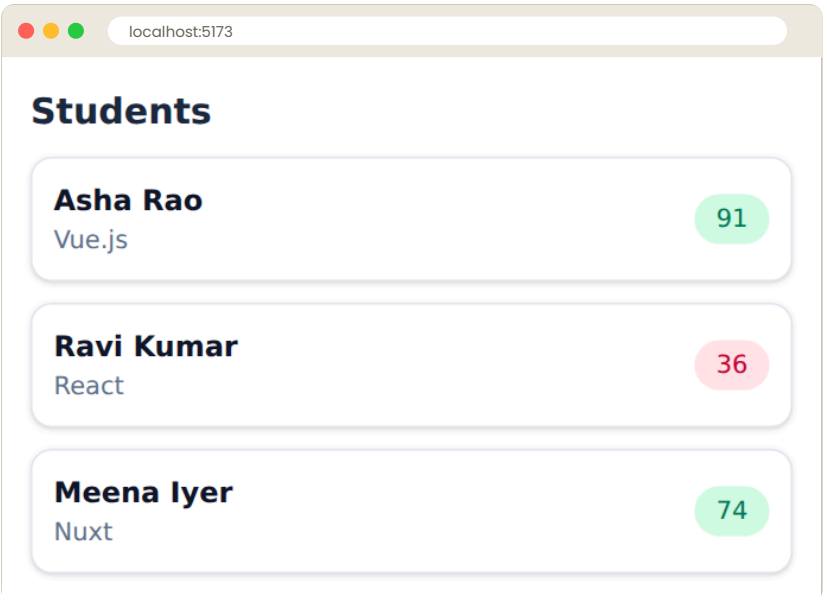
```
// vite.config.js
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({ plugins: [vue(), tailwindcss()] })
```

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
const students = ref([
  { id: 1, name: 'Asha Rao', course: 'Vue.js', marks: 91 },
  { id: 2, name: 'Ravi Kumar', course: 'React', marks: 36 },
  { id: 3, name: 'Meena Iyer', course: 'Nuxt', marks: 74 },
])
</script>

<template>
  <div class="max-w-md space-y-3">
    <h2 class="text-xl font-bold text-slate-800">Students</h2>
    <div v-for="s in students" :key="s.id"
         class="flex items-center justify-between rounded-xl border border-slate-200 bg-white p-3
shadow-sm">
      <div>
        <p class="font-semibold text-slate-900">{{ s.name }}</p>
        <p class="text-sm text-slate-500">{{ s.course }}</p>
      </div>
      <span :class="['rounded-full px-3 py-1 text-sm font-medium', s.marks >= 40 ? 'bg-emerald-100
text-emerald-700' : 'bg-rose-100 text-rose-700']">
        {{ s.marks }}
      </span>
    </div>
  </div>
</template>
```



UTILITY CLASSES IN A VUE TEMPLATE, COMPILED WITH TAILWIND CSS V4

18.6 UI libraries

A **UI component library** provides ready-made, tested, accessible components (buttons, inputs, dialogs, menus, data tables, date pickers) with a consistent design, so teams build admin panels and forms much faster. Each is installed as a plugin (or imported per component) and offers theming. The examples below show typical setup and usage; they need the libraries installed and are not run here.

18.6.1 Vuetify

Vuetify is the largest Material Design library for Vue: about 80 components, a grid system, themes (light/dark), icons and a strong data table. It suits admin dashboards and apps that want the Material look. Create a project with `npm create vuetify@latest` or add the plugin:

VUETIFY SETUP AND USAGE (REPRESENTATIVE) JavaScript

```
// main.js
import { createApp } from 'vue'
import { createVuetify } from 'vuetify'
import * as components from 'vuetify/components'
import * as directives from 'vuetify/directives'
import 'vuetify/styles'
import App from './App.vue'

createApp(App).use(createVuetify({ components, directives, theme: { defaultTheme: 'light' }
})).mount('#app')

<!-- in a component -->
<v-data-table :headers="[{ title: 'Name', key: 'name' }, { title: 'Marks', key: 'marks' }]"
:items="students" items-per-page="5" />
<v-btn color="primary" prepend-icon="mdi-content-save" @click="save">Save</v-btn>
```

18.6.2 PrimeVue

PrimeVue offers 90+ components with its own design presets (Aura, Lara, Nora) or unstyled mode for Tailwind, and an especially complete `DataTable` (sorting, filtering, pagination, selection, export). Components are imported individually, which keeps bundles small.

PRIMEVUE DATATABLE (REPRESENTATIVE) HTML

```
// main.js
import PrimeVue from 'primevue/config'
import Aura from '@primeuix/themes/aura'
app.use(PrimeVue, { theme: { preset: Aura } })

<!-- StudentsTable.vue -->
<script setup>
import DataTable from 'primevue/datatable'
import Column from 'primevue/column'
</script>
<template>
  <DataTable :value="students" paginator :rows="5" sortField="marks" :sortOrder="-1" stripedRows>
    <Column field="name" header="Name" sortable />
    <Column field="course" header="Course" sortable />
    <Column field="marks" header="Marks" sortable />
  </DataTable>
</template>
```

18.6.3 Element Plus and Naive UI (overview)

Element Plus (the Vue 3 version of Element UI) is widely used for enterprise and admin systems, with a clean desktop-oriented design and many form components (`<el-form>`, `<el-table>`, `<el-date-picker>`). **Naive UI** is written in TypeScript with a theme system based on JavaScript objects (no CSS files to override), 90+ components and good performance with large data (virtual lists built in).

18.6.4 Headless libraries: Reka UI and shadcn-vue

Headless libraries provide behaviour and accessibility (keyboard navigation, focus management, ARIA attributes) but **no styles**, so the design is completely yours, usually with Tailwind. **Reka UI** (formerly Radix Vue) provides unstyled primitives such as `DialogRoot`, `DropdownMenu` and `Combobox`. **shadcn-vue** is not a dependency at all: its CLI **copies** ready-styled components (built on Reka UI and Tailwind) into your project (`npx shadcn-vue@latest add button dialog`), so you own and can edit the code.

Library	Style	Strength	Good for
Vuetify	Material Design	largest component set, layouts, themes	admin apps, Material look
PrimeVue	presets or unstyled	DataTable, 90+ components, Tailwind mode	business apps, data-heavy screens
Element Plus	clean desktop	forms and tables, large community in Asia	enterprise back-office
Naive UI	themeable via JS	TypeScript, performance	TypeScript projects, dashboards
Reka UI	headless	accessible primitives	custom design systems
shadcn-vue	copied into project	own the code, Tailwind	modern custom UIs

18.7 Accessibility and internationalisation

18.7.1 Accessible components and focus management

Accessible components work for keyboard and screen-reader users. Start with **semantic HTML** (`<button>` not a clickable `<div>`, `<label for>` for inputs, headings in order), give icon-only buttons an `aria-label`, announce dynamic messages with `aria-live`, and keep visible focus styles. In a single-page app, **focus management** is your job: when a dialog opens move focus into it and keep it there, close it with <kbd>Escape</kbd>, and return focus to the button that opened it; after a route change, move focus to the new page's heading.

SRC/COMPONENTS/ACCESSIBLEMODAL.VUE — HTML

```
<script setup>
import { ref, watch, nextTick, useTemplateRef } from 'vue'

const open = defineModel({ type: Boolean })
const props = defineProps({ title: String })
const dialog = useTemplateRef('dialog')
let opener = null

watch(open, async (isOpen) => {
  if (isOpen) {
    opener = document.activeElement                  // remember who opened it
    await nextTick()
```

```
    dialog.value.querySelector('input, button')?.focus()
  } else {
    opener?.focus()                                  // give focus back
  }
})
function trap(e) {                                   // keep Tab inside the dialog
  const items = [...dialog.value.querySelectorAll('input, button')]
  const first = items[0], last = items[items.length - 1]
  if (e.shiftKey && document.activeElement === first) { e.preventDefault(); last.focus() }
  else if (!e.shiftKey && document.activeElement === last) { e.preventDefault(); first.focus() }
}
</script>

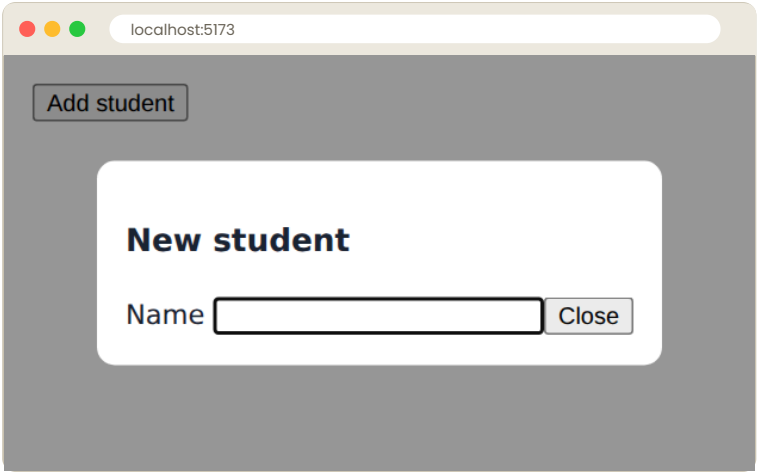
<template>
  <Teleport to="body">
    <div v-if="open" class="backdrop" @keydown.esc="open = false">
      <div ref="dialog" role="dialog" aria-modal="true" aria-labelledby="dlg-title" class="dialog"
@keydown.tab="trap">
        <h3 id="dlg-title">{{ title }}</h3>
        <slot />
        <button @click="open = false">Close</button>
      </div>
    </div>
  </Teleport>
</template>

<style scoped>
.backdrop { position: fixed; inset: 0; background: #0006; display: grid; place-items: center; }
.dialog { background: white; padding: 16px; border-radius: 10px; min-width: 260px; }
</style>
```

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
import AccessibleModal from './components/AccessibleModal.vue'
const show = ref(false)
</script>

<template>
  <button id="open" @click="show = true">Add student</button>
  <AccessibleModal v-model="show" title="New student">
    <label>Name <input id="name"></label>
  </AccessibleModal>
</template>
```



AFTER OPENING: FOCUS MOVED INTO THE DIALOG'S FIRST INPUT

BROWSER CONSOLE: WHERE IS THE FOCUS?

```
after open, focus is on: name
after Tab: Close
after another Tab (trapped, wraps around): name
after Escape, dialog open? false | focus back on: open
```

TIP

Use the browser's Lighthouse accessibility audit and the axe DevTools extension to find missing labels and contrast problems, and try every screen with the keyboard only. The headless libraries in 18.6.4 implement these patterns (focus trap, Escape, ARIA) for you.

18.7.2 Vue I18n

Vue I18n translates the interface. Create it with `createI18n({ locale, fallbackLocale, messages })` and install it as a plugin. In components, `useI18n()` returns `t()` for messages, `n()` and `d()` for locale-aware numbers and dates, and the reactive `locale`; templates can also use `$t()`. Messages support named parameters `{name}` and pluralisation with `|`. Missing keys fall back to the `fallbackLocale`. In real projects keep one JSON file per language and lazy-load them.

SRC/I18N.JS JavaScript

```
import { createI18n } from 'vue-i18n'

const messages = {
  en: {
    nav: { home: 'Home', courses: 'Courses' },
    greeting: 'Welcome, {name}!',
    students: 'no students | one student | {count} students',
    fee: 'Course fee',
    footer: 'All rights reserved',
  },
  hi: {
    nav: { home: 'होम', courses: 'पाठ्यक्रम' },
    greeting: 'स्वागत है, {name}!',
    students: 'कोई छात्र नहीं | एक छात्र | {count} छात्र',
    fee: 'पाठ्यक्रम शुल्क',
    // footer missing: falls back to English
  },
}

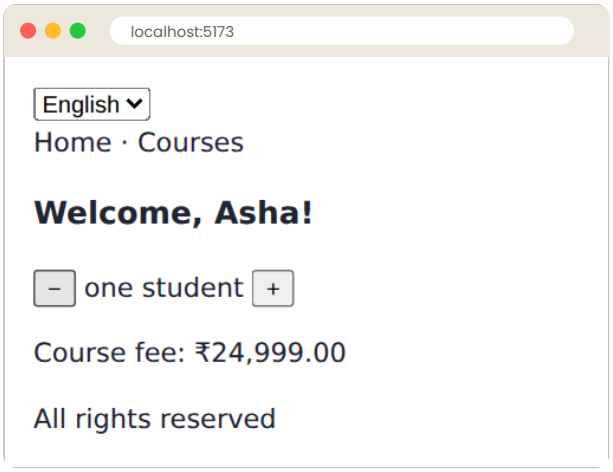
export const i18n = createI18n({
  legacy: false,
  locale: 'en',
  fallbackLocale: 'en',
  messages,
  numberFormats: {
    en: { currency: { style: 'currency', currency: 'INR' } },
    hi: { currency: { style: 'currency', currency: 'INR' } },
  },
})
```

SRC/APP.VUE HTML

```
<script setup>
import { ref } from 'vue'
import { useI18n } from 'vue-i18n'

const { t, n, locale } = useI18n()
const count = ref(3)
</script>

<template>
  <select v-model="locale">
    <option value="en">English</option>
    <option value="hi">हिन्दी</option>
  </select>
  <nav>{{ t('nav.home') }} · {{ t('nav.courses') }}</nav>
  <h3>{{ t('greeting', { name: 'Asha' }) }}</h3>
  <p>
    <button @click="count = Math.max(0, count - 1)">−</button>
    {{ t('students', count) }}
    <button @click="count++">+</button>
  </p>
  <p>{{ $t('fee') }}: {{ n(24999, 'currency') }}</p>
  <footer>{{ t('footer') }}</footer>
</template>
```



ENGLISH: NAMED PARAMETER, PLURAL FORM AND CURRENCY



SWITCHED TO HINDI: EVERY TEXT UPDATED REACTIVELY; THE MISSING FOOTER FELL BACK TO ENGLISH

PRACTICE CHECKPOINT

Create v-click-outside and v-permission directives, package a toast system as a plugin, and build an admin page with a UI library data table.

The directives (18.1.2) and the toast plugin (18.2.1) are reused. In a real project the table would be PrimeVue's `DataTable` or Vuetify's `v-data-table`; here a small sortable, searchable, paginated `DataTable` with the same column-definition API stands in, so the page runs without the library:

SRC/COMPONENTS/DATATABLE.VUE HTML

```
<script setup>
import { ref, computed } from 'vue'

const props = defineProps({
  value: { type: Array, required: true },
  columns: { type: Array, required: true },     // [{ field, header, sortable }]
  rows: { type: Number, default: 5 },
})
const query = ref('')
const sortField = ref(null)
const sortOrder = ref(1)
const page = ref(0)

const filtered = computed(() => props.value.filter((r) =>
JSON.stringify(r).toLowerCase().includes(query.value.toLowerCase())))
const sorted = computed(() => !sortField.value ? filtered.value
  : [...filtered.value].sort((a, b) => (a[sortField.value] > b[sortField.value] ? 1 : -1) *
sortOrder.value))
const pages = computed(() => Math.max(1, Math.ceil(sorted.value.length / props.rows)))
const visible = computed(() => sorted.value.slice(page.value * props.rows, (page.value + 1) *
props.rows))

function sortBy(col) {
  if (!col.sortable) return
  sortOrder.value = sortField.value === col.field ? -sortOrder.value : 1
  sortField.value = col.field
}
</script>

<template>
  <input v-model="query" placeholder="Search…" @input="page = 0">
  <table>
    <thead>
      <tr><th v-for="c in columns" :key="c.field" :class="{ sortable: c.sortable }"
@click="sortBy(c)">
        {{ c.header }} <span v-if="sortField === c.field">{{ sortOrder > 0 ? '▲' : '▼' }}</span>
      </th><th></th></tr>
    </thead>
    <tbody>
      <tr v-for="row in visible" :key="row.id">
        <td v-for="c in columns" :key="c.field">{{ row[c.field] }}</td>
        <td><slot name="actions" :row="row" /></td>
      </tr>
    </tbody>
  </table>
  <footer>
    <button :disabled="page === 0" @click="page--">‹</button> Page {{ page + 1 }} of {{ pages }}
    <button :disabled="page >= pages - 1" @click="page++">›</button> · {{ filtered.length }} records
  </footer>
</template>
```

```
<style scoped>
table { border-collapse: collapse; width: 100%; margin: 6px 0; font-size: 14px; }
th, td { text-align: left; padding: 4px 8px; border-bottom: 1px solid #e3e3ea; }
th { background: #1E2A4A; color: white; } .sortable { cursor: pointer; }
footer { font-size: 13px; }
</style>
```

SRC/MAIN.JS JavaScript

```
import { createApp } from 'vue'
import App from './App.vue'
import { createToast } from './plugins/toast'
import { clickOutside, createPermission } from './directives'

const currentUser = { roles: ['teacher'] }
createApp(App)
  .use(createToast())
  .directive('click-outside', clickOutside)
  .directive('permission', createPermission(() => currentUser.roles))
  .mount('#app')
```

SRC/APP.VUE

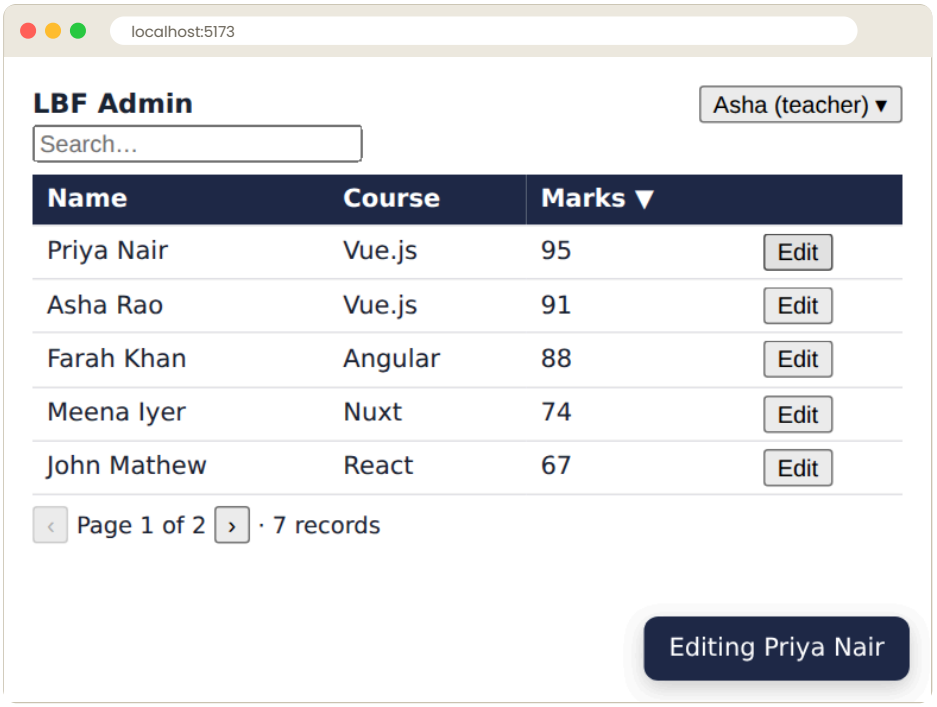
HTML

```
<script setup>
import { ref } from 'vue'
import DataTable from './components/DataTable.vue'
import { useToast } from './plugins/toast'

const toast = useToast()
const menuOpen = ref(false)
const students = ref([
  { id: 1, name: 'Asha Rao', course: 'Vue.js', marks: 91 }, { id: 2, name: 'Ravi Kumar', course:
'React', marks: 36 },
  { id: 3, name: 'Meena Iyer', course: 'Nuxt', marks: 74 }, { id: 4, name: 'Kiran Das', course:
'Vue.js', marks: 58 },
  { id: 5, name: 'Farah Khan', course: 'Angular', marks: 88 }, { id: 6, name: 'John Mathew', course:
'React', marks: 67 },
  { id: 7, name: 'Priya Nair', course: 'Vue.js', marks: 95 },
])
const columns = [{ field: 'name', header: 'Name', sortable: true }, { field: 'course', header:
'Course', sortable: true }, { field: 'marks', header: 'Marks', sortable: true }]
function remove(row) {
  students.value = students.value.filter((s) => s.id !== row.id)
  toast.success(`Removed ${row.name}`)
}
</script>

<template>
  <ToastHost />
  <header>
    <b>LBF Admin</b>
    <div class="user">
      <button @click.stop="menuOpen = !menuOpen">Asha (teacher) ▾</button>
      <ul v-if="menuOpen" v-click-outside="() => (menuOpen = false)"><li>Profile</li><li>Log out</li>
</ul>
    </div>
  </header>
  <DataTable :value="students" :columns="columns" :rows="5">
    <template #actions="{ row }">
      <button v-permission="'teacher'" @click="toast.show(`Editing ${row.name}`)">Edit</button>
      <button v-permission="'admin'" @click="remove(row)">Delete</button>
    </template>
  </DataTable>
</template>

<style>header { display: flex; justify-content: space-between; align-items: center; } .user {
position: relative; } .user ul { position: absolute; right: 0; background: white; border: 1px solid
#ccd; margin: 2px 0; padding: 4px 18px; }</style>
```



SORTED BY MARKS (DESCENDING), THEN CLICKED EDIT ON THE TOP ROW: DELETE BUTTONS ARE HIDDEN FOR A TEACHER, AND THE TOAST PLUGIN CONFIRMS THE ACTION

Directives (`vXxx` / `app.directive`) handle element-level behaviour through hooks; clean up in `unmounted`. Plugins (`install(app, options)`) bundle globals: components, directives, `provide`d APIs and `globalProperties`. Render functions with `h()` (or JSX) suit dynamic structure; templates stay the default. Patterns: renderless components expose logic through scoped slots, compound components share state with provide/inject, error boundaries use `onErrorCaptured`. Style with scoped CSS and `:deep()`, CSS Modules, `v-bind()` in CSS or Tailwind; choose a UI library (Vuetify, PrimeVue, Element Plus, Naive UI, or headless Reka UI / shadcn-vue) to move fast; manage focus and ARIA for accessibility; translate with Vue I18n.

MODULE 19 · ADVANCED

Testing Vue applications

Minimum duration **2 days** Level **Advanced**

Tests give confidence to refactor and ship. This module explains unit, component and end-to-end tests and what is worth testing in a Vue app; sets up Vitest and tests composables and utility functions, mocking modules and timers; tests components with Vue Test Utils (mount, find, trigger, setValue), including props, emits, slots and asynchronous behaviour with flushPromises, and introduces Vue Testing Library; tests Pinia stores (real and with createTestingPinia), components that use the router, and API calls mocked with MSW; runs a real end-to-end test with Playwright and compares Cypress; and finishes with coverage reports and a CI pipeline.

IN THIS MODULE

19.1 Testing fundamentals
19.2 Vitest
19.3 Component testing
19.4 Testing the ecosystem
19.5 End-to-end testing
19.6 Coverage and CI

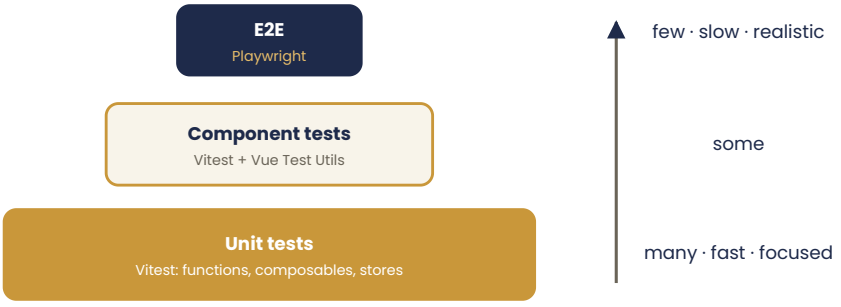
HOW THE TESTS WERE RUN

Vitest, Vue Test Utils, @pinia/testing, Vue Testing Library and MSW could not be installed offline. The spec files are written exactly as for a real project; they were run in Chromium by small stand-ins with the same APIs, which print a report in Vitest's format. The end-to-end test in Section 19.5 was run with the **real** Playwright test runner against the built app.

19.1 Testing fundamentals

19.1.1 Unit, component and end-to-end tests

Unit tests check one function or composable in isolation (fast, many). **Component tests** mount one component, interact with it like a user and check the rendered output and emitted events. **End-to-end (E2E) tests** drive the whole running app in a real browser, including the router, the store and (mocked or real) back end (slow, few). The **testing pyramid** recommends many small fast tests at the bottom and a few broad slow ones at the top.



THE TESTING PYRAMID

19.1.2 What to test in a Vue app

Test **behaviour, not implementation**: what the user sees and does, and what the component emits or calls, not internal variable names. A test that breaks when you rename a private ref is a bad test.

Test this	Not this
the text and elements rendered for given props	the exact HTML structure or CSS classes used only for styling
events emitted (and their payloads) after user actions	that an internal method was called
composables and utilities: inputs → outputs	Vue itself (that v-if works)
store actions and getters, error and loading states	third-party libraries
critical user journeys end-to-end (sign up, checkout)	every possible path end-to-end

19.2 Vitest

19.2.1 Setup and running tests

Vitest is the Vite-native test runner: it reuses `vite.config` (aliases, the Vue plugin), so `.vue` files work in tests without extra setup, and its API matches Jest (`describe`, `it` / `test`, `expect`, `vi`). Choose Vitest in `npm create vue@latest`, or install it with `npm i -D vitest @vue/test-utils jsdom` and set the DOM environment:

VITEST.CONFIG.JS — JavaScript

```
import { fileURLToPath } from 'node:url'
import { defineConfig, mergeConfig } from 'vitest/config'
import viteConfig from './vite.config'

export default mergeConfig(viteConfig, defineConfig({
  test: {
    environment: 'jsdom',              // a browser-like DOM for component tests
    globals: false,                    // import describe/it/expect explicitly
    root: fileURLToPath(new URL('./', import.meta.url)),
  },
}))
```

Command	Does
`npx vitest` (`npm run test:unit`)	watch mode: re-runs affected tests on save
`npx vitest run`	run once (CI)
`npx vitest run src/utils`	run tests whose path matches
`npx vitest --ui`	browser UI with the test tree
`npx vitest run --coverage`	coverage report (19.6)

A test file is named `*.spec.js` or `*.test.js`. A first test of a utility function, with one deliberately wrong expectation to show a failure report:

SRC/UTILS/GRADES.SPEC.JS

JavaScript

```
import { describe, it, expect } from 'vitest'
import { formatMarks, grade } from './grades'

describe('formatMarks', () => {
  it('formats marks out of 100', () => {
    expect(formatMarks(91)).toBe('91/100')
  })
  it('rejects marks outside 0-100', () => {
    expect(() => formatMarks(120)).toThrow('marks must be 0-100')
  })
})

describe('grade', () => {
  it('gives A from 90', () => expect(grade(90)).toBe('A'))
  it('gives C at the pass mark 40', () => expect(grade(40)).toBe('C'))
  it('gives F below 40', () => expect(grade(39)).toBe('B'))     // wrong on purpose
})
```

TEST RUN

main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
❯ src/utils/grades.spec.js (5 tests | 1 failed) 1ms
   ✓ formatMarks > formats marks out of 100 1ms
   ✓ formatMarks > rejects marks outside 0-100 0ms
   ✓ grade > gives A from 90 0ms
   ✓ grade > gives C at the pass mark 40 0ms
   × grade > gives F below 40 0ms

 FAIL  src/utils/grades.spec.js > grade > gives F below 40
AssertionError: expected "F" to be "B"

 Test Files  1 failed (1)
      Tests  1 failed | 4 passed (5)
   Duration  2ms
```

19.2.2 Testing composables and utility functions

A composable that only uses reactivity (`ref`, `computed`, `watch`) can be called directly in a test. Composables that use lifecycle hooks or `inject` need a component around them: mount a tiny host component (or use a `withSetup` helper) and read the result.

SRC/COMPOSABLES/COMPOSABLES.SPEC.JS JavaScript

```
import { describe, it, expect } from 'vitest'
import { mount } from '@vue/test-utils'
import { useCounter } from './useCounter'
import { useWindowWidth } from './useWindowWidth'

describe('useCounter (plain reactivity: call it directly)', () => {
  it('starts at the initial value and increments', () => {
    const { count, inc } = useCounter(3)
    inc()
    expect(count.value).toBe(4)
  })
  it('stops at max and reports atMax', () => {
    const { count, atMax, inc } = useCounter(9, { max: 10 })
    inc(); inc()
    expect(count.value).toBe(10)
    expect(atMax.value).toBe(true)
  })
})

// helper: run a composable inside a real component so lifecycle hooks work
function withSetup(composable) {
  let result
  const wrapper = mount({ setup() { result = composable(); return () => null } })
  return [result, wrapper]
}

describe('useWindowWidth (uses lifecycle hooks: needs a component)', () => {
  it('updates on resize and stops listening after unmount', () => {
    const [width, wrapper] = withSetup(() => useWindowWidth())
    window.innerWidth = 500
    window.dispatchEvent(new Event('resize'))
    expect(width.value).toBe(500)
    wrapper.unmount()
    window.innerWidth = 900
    window.dispatchEvent(new Event('resize'))
    expect(width.value).toBe(500)
  })
})
```

TEST RUN main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/composables/composables.spec.js (3 tests) 6ms
   ✓ useCounter (plain reactivity: call it directly) > starts at the initial value and increments 1ms
   ✓ useCounter (plain reactivity: call it directly) > stops at max and reports atMax 0ms
   ✓ useWindowWidth (uses lifecycle hooks: needs a component) > updates on resize and stops listening
after unmount 5ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  6ms
```

NOTE

`window.innerWidth` is assignable in jsdom (and was overridden for this run); in a real browser environment use `vi.stubGlobal` or `Object.defineProperty`.

19.2.3 Mocking modules and timers

Tests must be **fast and deterministic**, so replace slow or unpredictable dependencies. `vi.fn()` creates a mock function that records calls and can return chosen values. `vi.mock('./path', factory)` replaces a whole module for every file that imports it (the call is hoisted to the top of the file). `vi.spyOn(obj, 'method')` watches a real method. `vi.useFakeTimers()` controls `setTimeout`/`setInterval`, and `vi.advanceTimersByTime(ms)` moves time forward instantly.

SRC/COMPOSABLES/USESEARCH.SPEC.JS JavaScript

```
import { describe, it, expect, vi, beforeEach, afterEach } from 'vitest'
import { nextTick } from 'vue'
import { useSearch } from './useSearch'
import { searchStudents } from '@/services/api'

vi.mock('@/services/api', () => ({
  searchStudents: vi.fn(async (q) => [{ id: 1, name: `Result for ${q}` }]),
}))

describe('useSearch', () => {
  beforeEach(() => { vi.useFakeTimers(); vi.clearAllMocks() })
  afterEach(() => vi.useRealTimers())

  it('debounces: only the last query is sent, after 300 ms', async () => {
    const { query, results } = useSearch()
    query.value = 'a'; await nextTick()
    query.value = 'as'; await nextTick()
    query.value = 'asha'; await nextTick()
    expect(searchStudents).not.toHaveBeenCalled()

    await vi.advanceTimersByTimeAsync(300)
    expect(searchStudents).toHaveBeenCalledTimes(1)
    expect(searchStudents).toHaveBeenCalledWith('asha')
    expect(results.value[0].name).toBe('Result for asha')
  })
})
```

TEST RUN main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/composables/useSearch.spec.js (1 tests) 2ms
   ✓ useSearch > debounces: only the last query is sent, after 300 ms 2ms

 Test Files  1 passed (1)
      Tests  1 passed (1)
   Duration  2ms
```

19.3 Component testing

19.3.1 Vue Test Utils: mount, find, trigger, setValue

Vue Test Utils (VTU) is the official component-testing library. `mount(Component, { props, slots, global })` renders it and returns a **wrapper**. `find(selector)` / `findAll` locate elements (prefer `data-test` attributes, which do not change with styling); `get` is like `find` but fails loudly. `trigger('click')` fires events and `setValue('x')` types into inputs; both return a promise: **await them** so the DOM is updated.

`text()`, `html()`, `classes()`, `attributes()` and `exists()` read the result. `shallowMount` replaces child components with stubs.

SRC/COMPONENTS/__TESTS__/TODOLIST.SPEC.JS JavaScript

```
import { describe, it, expect } from 'vitest'
import { mount } from '@vue/test-utils'
import TodoList from '../TodoList.vue'

describe('TodoList', () => {
  it('adds a todo from the input', async () => {
    const wrapper = mount(TodoList)
    await wrapper.get('[data-test="new-todo"]').setValue('Revise Pinia')
    await wrapper.get('form').trigger('submit')

    const items = wrapper.findAll('[data-test="todo"]')
    expect(items).toHaveLength(1)
    expect(items[0].text()).toContain('Revise Pinia')
    expect(wrapper.get('[data-test="new-todo"]').element.value).toBe('')
  })

  it('ignores empty input', async () => {
    const wrapper = mount(TodoList)
    await wrapper.get('form').trigger('submit')
    expect(wrapper.find('[data-test="todo"]').exists()).toBe(false)
  })

  it('marks a todo as done and updates the count', async () => {
    const wrapper = mount(TodoList)
    await wrapper.get('[data-test="new-todo"]').setValue('A')
    await wrapper.get('form').trigger('submit')
    await wrapper.get('[data-test="new-todo"]').setValue('B')
    await wrapper.get('form').trigger('submit')
    await wrapper.get('[data-test="todo"] input').setValue(true)

    expect(wrapper.get('[data-test="todo"]').classes()).toContain('done')
    expect(wrapper.get('[data-test="count"]').text()).toBe('1 left')
  })
})
```

TEST RUN main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/components/__tests__/TodoList.spec.js (3 tests) 21ms
   ✓ TodoList > adds a todo from the input 16ms
   ✓ TodoList > ignores empty input 2ms
   ✓ TodoList > marks a todo as done and updates the count 3ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  21ms
```

19.3.2 Testing props, emits and slots

Pass **props** with the `props` mounting option and change them later with `await wrapper.setProps()`. `wrapper.emitted()` returns every emitted event with its payloads, e.g. `emitted('save')[0]` is the argument list of the first `save` event. Provide **slot** content with the `slots` option (strings, templates or components).

SRC/COMPONENTS/__TESTS__/STUDENTCARD.SPEC.JS

JavaScript

```
import { describe, it, expect } from 'vitest'
import { mount } from '@vue/test-utils'
import StudentCard from '../StudentCard.vue'

const asha = { id: 7, name: 'Asha', marks: 91 }

describe('StudentCard', () => {
  it('renders props and reacts to prop changes', async () => {
    const wrapper = mount(StudentCard, { props: { student: asha } })
    expect(wrapper.text()).toContain('Asha')
    expect(wrapper.classes()).toContain('pass')

    await wrapper.setProps({ passMark: 95 })
    expect(wrapper.classes()).toContain('fail')
  })

  it('emits select and remove with the student id', async () => {
    const wrapper = mount(StudentCard, { props: { student: asha } })
    await wrapper.get('h3').trigger('click')
    await wrapper.get('[data-test="remove"]').trigger('click')

    expect(wrapper.emitted('select')).toHaveLength(1)
    expect(wrapper.emitted('select')[0]).toEqual([7])
    expect(wrapper.emitted().remove[0]).toEqual([7])
  })

  it('renders custom content in the actions slot', () => {
    const wrapper = mount(StudentCard, {
      props: { student: asha },
      slots: { actions: `<template #actions="{ student }"><a href="#">Edit {{ student.name }}</a>
</template>` },
    })
    expect(wrapper.find('[data-test="remove"]').exists()).toBe(false)
    expect(wrapper.get('a').text()).toBe('Edit Asha')
  })
})
```

TEST RUN

main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/components/__tests__/StudentCard.spec.js (3 tests) 15ms
   ✓ StudentCard > renders props and reacts to prop changes 11ms
   ✓ StudentCard > emits select and remove with the student id 2ms
   ✓ StudentCard > renders custom content in the actions slot 2ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  15ms
```

19.3.3 Testing async behaviour with flushPromises

A component that fetches data on mount updates the DOM only after the request's promise resolves. `await flushPromises()` waits until all pending promises have settled; then the test can check the loaded state. Mock `fetch` (or the API module) so the test does not depend on a server.

SRC/COMPONENTS/__TESTS__/STUDENTLIST.SPEC.JS

JavaScript

```
import { describe, it, expect, vi, afterEach } from 'vitest'
import { mount, flushPromises } from '@vue/test-utils'
import StudentList from '../StudentList.vue'

const respond = (status, body) => vi.fn(async () => ({ ok: status < 400, status, json: async () =>
body }))

describe('StudentList', () => {
  afterEach(() => vi.unstubAllGlobals())

  it('shows loading, then the students', async () => {
    vi.stubGlobal('fetch', respond(200, [{ id: 1, name: 'Asha' }, { id: 2, name: 'Ravi' }]))
    const wrapper = mount(StudentList)
    expect(wrapper.text()).toBe('Loading…')

    await flushPromises()
    expect(wrapper.findAll('li').map((li) => li.text())).toEqual(['Asha', 'Ravi'])
    expect(fetch).toHaveBeenCalledWith('/api/students')
  })

  it('shows an error message when the request fails', async () => {
    vi.stubGlobal('fetch', respond(500, null))
    const wrapper = mount(StudentList)
    await flushPromises()
    expect(wrapper.get('[role="alert"]').text()).toBe('HTTP 500')
  })
})
```

TEST RUN

main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/components/__tests__/StudentList.spec.js (2 tests) 14ms
   ✓ StudentList > shows loading, then the students 13ms
   ✓ StudentList > shows an error message when the request fails 1ms

 Test Files  1 passed (1)
      Tests  2 passed (2)
   Duration  14ms
```

19.3.4 Vue Testing Library (overview)

Vue Testing Library (`@testing-library/vue`, built on VTU) pushes tests towards what users perceive: you query by **role**, **label** and **visible text** (`screen.getByRole('button', { name: 'Register' })`), never by CSS classes or component internals, and interact with `fireEvent` (or the `user-event` library). Queries come as `getBy` (must exist), `queryBy` (may be missing, returns null) and `findBy` (waits for async content). Such tests also check accessibility for free: if a button has no accessible name, the test cannot find it.

SRC/COMPONENTS/__TESTS__/REGISTERFORM.SPEC.JS

JavaScript

```
import { describe, it, expect } from 'vitest'
import { render, screen, fireEvent } from '@testing-library/vue'
import RegisterForm from '../RegisterForm.vue'

describe('RegisterForm (Vue Testing Library)', () => {
  it('shows validation messages for an empty form', async () => {
    render(RegisterForm)
    await fireEvent.click(screen.getByRole('button', { name: 'Register' }))
    expect(screen.getAllByRole('alert')).toHaveLength(3)
    expect(screen.getByText('Enter a valid email')).toBeInTheDocument()
  })

  it('emits registered with the form data when valid', async () => {
    const { emitted } = render(RegisterForm)
    await fireEvent.update(screen.getByLabelText('Name'), 'Asha Rao')
    await fireEvent.update(screen.getByLabelText('Email'), 'asha@lbf.in')
    await fireEvent.update(screen.getByLabelText('Course'), 'Vue.js')
    await fireEvent.click(screen.getByRole('button', { name: 'Register' }))

    expect(screen.queryByRole('alert')).toBeNull()
    expect(emitted().registered[0]).toEqual([{ name: 'Asha Rao', email: 'asha@lbf.in', course:
'Vue.js' }])
  })
})
```

TEST RUN

main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/components/__tests__/RegisterForm.spec.js (2 tests) 30ms
   ✓ RegisterForm (Vue Testing Library) > shows validation messages for an empty form 22ms
   ✓ RegisterForm (Vue Testing Library) > emits registered with the form data when valid 8ms

 Test Files  1 passed (1)
      Tests  2 passed (2)
   Duration  30ms
```

19.4 Testing the ecosystem

19.4.1 Testing Pinia stores

Unit-test a store by creating a fresh Pinia before each test with `setActivePinia(createPinia())`, so tests do not share state. **In component tests**, `createTestingPinia()` from `@pinia/testing` installs a testing Pinia: `initialState` sets the store state, and every action is replaced by a spy (`createSpy: vi.fn`) that does nothing by default (`stubActions: false` runs the real actions), so the component is tested on its own.

SRC/STORES/__TESTS__/CART.SPEC.JS JavaScript

```
import { describe, it, expect, beforeEach } from 'vitest'
import { setActivePinia, createPinia } from 'pinia'
import { useCartStore } from '../cart'

describe('cart store', () => {
  beforeEach(() => setActivePinia(createPinia()))       // fresh state per test

  it('adds products and merges duplicates', () => {
    const cart = useCartStore()
    cart.add({ id: 1, name: 'Notebook', price: 60 })
    cart.add({ id: 1, name: 'Notebook', price: 60 })
    expect(cart.items).toEqual([{ id: 1, name: 'Notebook', price: 60, qty: 2 }])
    expect(cart.subtotal).toBe(120)
  })

  it('applies a valid coupon and rejects others', () => {
    const cart = useCartStore()
    cart.add({ id: 2, name: 'Backpack', price: 899 })
    expect(() => cart.applyCoupon('FREE')).toThrow('Invalid coupon')
    cart.applyCoupon('LBF10')
    expect(cart.total).toBe(809)
  })

  it('starts empty in every test', () => {
    expect(useCartStore().items).toHaveLength(0)
  })
})
```

SRC/COMPONENTS/__TESTS__/CARTBADGE.SPEC.JS JavaScript

```
import { describe, it, expect, vi } from 'vitest'
import { mount } from '@vue/test-utils'
import { createTestingPinia } from '@pinia/testing'
import CartBadge from '../CartBadge.vue'
import { useCartStore } from '@/stores/cart'

describe('CartBadge with createTestingPinia', () => {
  it('renders from initial state and calls the (stubbed) action', async () => {
    const wrapper = mount(CartBadge, {
      global: {
        plugins: [createTestingPinia({
          createSpy: vi.fn,
          initialState: { cart: { items: [{ id: 1, name: 'Notebook', price: 60, qty: 2 }] } },
        })],
      },
    })
    const cart = useCartStore()
    expect(wrapper.get('[data-test="badge"]').text()).toBe('🛒 1 · ₹120')

    await wrapper.get('button').trigger('click')
    expect(cart.add).toHaveBeenCalledWith({ id: 3, name: 'Pen', price: 20 })
    expect(cart.items).toHaveLength(1)           // stubbed: the real action did not run
  })
})
```

TEST RUN

main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/stores/__tests__/cart.spec.js (3 tests) 3ms
   ✓ cart store > adds products and merges duplicates 2ms
   ✓ cart store > applies a valid coupon and rejects others 1ms
   ✓ cart store > starts empty in every test 0ms
✓ src/components/__tests__/CartBadge.spec.js (1 tests) 9ms
   ✓ CartBadge with createTestingPinia > renders from initial state and calls the (stubbed) action 9ms

 Test Files  2 passed (2)
      Tests  4 passed (4)
   Duration  12ms
```

19.4.2 Testing components that use the router

Two approaches. **Mock** the router for a unit-style test: replace `useRouter` / `useRoute` with `vi.mock('vue-router')` and assert `push` was called, or pass `global.mocks` for `$route`. Or install a **real router** with `createMemoryHistory()` (no browser URL needed), `push` the starting route, `await router.isReady()`, and check what renders after clicking links.

SRC/VIEWS/__TESTS__/STUDENTDETAIL.SPEC.JS

JavaScript

```
import { describe, it, expect } from 'vitest'
import { mount, flushPromises } from '@vue/test-utils'
import { createRouter, createMemoryHistory } from 'vue-router'
import StudentDetail from '../StudentDetail.vue'

const Home = { template: '<h2>All students</h2>' }
function makeRouter() {
  return createRouter({
    history: createMemoryHistory(),
    routes: [
      { path: '/', component: Home },
      { path: '/students/:id', name: 'student', component: StudentDetail },
    ],
  })
}
const App = { template: '<RouterView />' }

describe('StudentDetail with a real router', () => {
  it('reads the id param and navigates to the next student', async () => {
    const router = makeRouter()
    router.push('/students/7')
    await router.isReady()
    const wrapper = mount(App, { global: { plugins: [router] } })
    expect(wrapper.get('h2').text()).toBe('Student #7')

    await wrapper.get('button').trigger('click')
    await flushPromises()
    expect(router.currentRoute.value.fullPath).toBe('/students/8')
    expect(wrapper.get('h2').text()).toBe('Student #8')
  })

  it('goes back to the list through RouterLink', async () => {
    const router = makeRouter()
    router.push('/students/3')
    await router.isReady()
    const wrapper = mount(App, { global: { plugins: [router] } })
    await wrapper.get('a').trigger('click')
```

```
    await flushPromises()
    expect(wrapper.get('h2').text()).toBe('All students')
  })
})
```

TEST RUN main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/views/__tests__/StudentDetail.spec.js (2 tests) 25ms
   ✓ StudentDetail with a real router > reads the id param and navigates to the next student 22ms
   ✓ StudentDetail with a real router > goes back to the list through RouterLink 3ms

 Test Files  1 passed (1)
      Tests  2 passed (2)
   Duration  25ms
```

19.4.3 Mocking APIs with MSW

Mock Service Worker (MSW) intercepts requests at the network level, so the code under test runs unchanged (fetch, Axios, the service layer, the store) and only the server is fake. Define **handlers** with `http.get(url, resolver)` returning `HttpResponse.json(...)`, start a server in tests with `setupServer(...handlers)` (from `msw/node`), and override a handler in a single test with `server.use()` to simulate errors. The same handlers can run in the browser during development with `setupWorker`.

SRC/MOCKS/HANDLERS.JS JavaScript

```
import { http, HttpResponse } from 'msw'

export const students = [{ id: 1, name: 'Asha', course: 'Vue.js', marks: 91 }, { id: 2, name: 'Ravi',
course: 'React', marks: 36 }]

export const handlers = [
  http.get('/api/students', () => HttpResponse.json(students)),
  http.get('/api/students/:id', ({ params }) => {
    const s = students.find((x) => x.id === Number(params.id))
    return s ? HttpResponse.json(s) : HttpResponse.json({ message: 'Student not found' }, { status:
404 })
  }),
  http.post('/api/students', async ({ request }) => {
    const body = await request.json()
    return HttpResponse.json({ id: 3, ...body }, { status: 201 })
  }),
]
```

SRC/STORES/__TESTS__/STUDENTS.MSW.SPEC.JS

JavaScript

```
import { describe, it, expect, beforeAll, afterEach, afterAll, beforeEach } from 'vitest'
import { setupServer } from 'msw/node'
import { http, HttpResponse } from 'msw'
import { setActivePinia, createPinia } from 'pinia'
import { handlers } from '@/mocks/handlers'
import { useStudentStore } from '../students'

const server = setupServer(...handlers)
beforeAll(() => server.listen({ onUnhandledRequest: 'error' }))
afterEach(() => server.resetHandlers())          // undo per-test overrides
afterAll(() => server.close())

describe('student store against MSW', () => {
  beforeEach(() => setActivePinia(createPinia()))

  it('loads students through axios', async () => {
    const store = useStudentStore()
    await store.load()
    expect(store.list.map((s) => s.name)).toEqual(['Asha', 'Ravi'])
  })

  it('creates a student (POST body reaches the handler)', async () => {
    const store = useStudentStore()
    const created = await store.create({ name: 'Meena', course: 'Nuxt', marks: 80 })
    expect(created).toEqual({ id: 3, name: 'Meena', course: 'Nuxt', marks: 80 })
  })

  it('stores the server message when the API fails', async () => {
    server.use(http.get('/api/students', () => HttpResponse.json({ message: 'Database offline' }, {
status: 503 })))
    const store = useStudentStore()
    await store.load()
    expect(store.error).toBe('Database offline')
    expect(store.list).toEqual([])
  })
})
```

TEST RUN

main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/stores/__tests__/students.msw.spec.js (3 tests) 10ms
   ✓ student store against MSW > loads students through axios 3ms
   ✓ student store against MSW > creates a student (POST body reaches the handler) 6ms
   ✓ student store against MSW > stores the server message when the API fails 1ms

 Test Files  1 passed (1)
      Tests  3 passed (3)
   Duration  10ms
```

19.5 End-to-end testing

19.5.1 Playwright or Cypress

End-to-end tests start the built app and drive a real browser. **Playwright** (Microsoft) runs Chromium, Firefox and WebKit, in parallel, with auto-waiting locators (`page.getByRole`, `getByLabel`), tracing and a

code generator (`npx playwright codegen`). **Cypress** runs tests inside the browser with an interactive time-travel UI and also supports component testing. Both are offered by `npm create vue@latest`.

	Playwright	Cypress
Browsers	Chromium, Firefox, WebKit (Safari engine)	Chrome family, Firefox, (WebKit experimental)
Speed / parallel	parallel workers built in, free	parallel through the paid cloud or plugins
API style	`await`-based, auto-waiting locators	chained commands (`cy.get().click()`)
Multiple tabs, origins	yes	limited
Debugging	trace viewer, UI mode, codegen	interactive runner with time travel
Set up	`npm init playwright@latest`	`npm i -D cypress`

Here the registration flow of a small app (the RegisterForm from 19.3.4 inside an App that shows a success message) is tested with the **real** Playwright test runner against the built app:

PLAYWRIGHT.CONFIG.TS TypeScript

```ts
import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  testDir: './tests',
  reporter: 'list',
  use: { baseURL: 'http://localhost:5173', trace: 'on-first-retry' },
  projects: [{ name: 'chromium', use: { ...devices['Desktop Chrome'] } }],
  webServer: { command: 'npm run preview', url: 'http://localhost:5173', stdout: 'ignore', stderr:
'ignore' },
})
```

E2E/REGISTER.SPEC.TS TypeScript

```ts
import { test, expect } from '@playwright/test'

test.describe('registration', () => {
  test.beforeEach(async ({ page }) => {
    await page.goto('/')
  })

  test('shows errors for an empty form', async ({ page }) => {
    await page.getByRole('button', { name: 'Register' }).click()
    await expect(page.getByRole('alert')).toHaveCount(3)
    await expect(page.getByText('Enter a valid email')).toBeVisible()
  })

  test('registers a student', async ({ page }) => {
    await page.getByLabel('Name').fill('Asha Rao')
    await page.getByLabel('Email').fill('asha@lbf.in')
    await page.getByLabel('Course').selectOption('Vue.js')
    await page.getByRole('button', { name: 'Register' }).click()

    await expect(page.getByRole('status')).toHaveText('Welcome, Asha Rao! You are enrolled in
Vue.js.')
    await expect(page.getByRole('button', { name: 'Register' })).toBeHidden()
  })
})
```

PLAYWRIGHT TEST RUN (REAL PLAYWRIGHT 1.56) main.js

```
$ npx playwright test
```

```
TERMINAL
Running 2 tests using 1 worker
  ✓  1 [chromium] › tests/register.spec.ts:8:3 › registration › shows errors for an empty form (221ms)
  ✓  2 [chromium] › tests/register.spec.ts:14:3 › registration › registers a student (236ms)
  2 passed (1.5s)
```

Playwright started the server from webServer, opened Chromium and ran both tests; locators wait automatically for elements to appear.

19.6 Coverage and CI

Coverage measures which lines, branches and functions the tests executed. Install `@vitest/coverage-v8` and run `vitest run --coverage`; the report (text in the terminal, HTML in `coverage/`) shows untested code. Use it to find gaps, not as a target to game: 100% coverage with weak assertions proves little. Thresholds can fail the build when coverage drops:

COVERAGE SETTINGS JavaScript

```
// vitest.config.js (test section)
test: {
  environment: 'jsdom',
  coverage: {
    provider: 'v8',
    reporter: ['text', 'html'],
    include: ['src/**/*.{js,ts,vue}'],
    exclude: ['src/main.js', 'src/**/*.spec.js'],
    thresholds: { lines: 80, functions: 80, branches: 70 },
  },
}
```

Coverage report (representative)

```
$ npx vitest run --coverage
 ✓ src/stores/__tests__/cart.spec.js (3 tests)
 ✓ src/components/__tests__/StudentCard.spec.js (3 tests)
 ✓ src/components/__tests__/RegisterForm.spec.js (2 tests)
 % Coverage report from v8
----------------------|---------|----------|---------|---------|
File                  | % Stmts | % Branch | % Funcs | % Lines |
----------------------|---------|----------|---------|---------|
All files             |   91.42 |    84.61 |   90.00 |   91.42 |
 components           |   95.65 |    88.88 |  100.00 |   95.65 |
  RegisterForm.vue    |  100.00 |   100.00 |  100.00 |  100.00 |
  StudentCard.vue     |   90.90 |    75.00 |  100.00 |   90.90 |
 stores               |   86.36 |    80.00 |   80.00 |   86.36 |
  cart.js             |   86.36 |    80.00 |   80.00 |   86.36 |
----------------------|---------|----------|---------|---------|
```

A **CI pipeline** runs the checks on every push and pull request, so broken code cannot be merged. A GitHub Actions workflow for a Vue project:

.GITHUB/WORKFLOWS/CI.YML

text

```
name: CI
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check
      - run: npx vitest run --coverage
      - run: npx playwright install --with-deps chromium
      - run: npm run build
      - run: npx playwright test
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: reports, path: "coverage/\nplaywright-report/" }
```

PRACTICE CHECKPOINT

Write tests for a composable, the cart Pinia store, the registration form component (props, emits and validation) and one Playwright end-to-end flow.

The pieces were built in this module: the composable tests (19.2.2 and 19.2.3), the cart store (19.4.1), the registration form (19.3.4) and the Playwright flow (19.5.1). Run together, with a props test added for a reusable `CourseSelect` used by the form:

TEST RUN

main.js

```
$ npx vitest run
```

TERMINAL: npx vitest run

```
✓ src/composables/useCounter.spec.js (1 tests) 1ms
   ✓ useCounter never goes below min 1ms
✓ src/stores/__tests__/cart.spec.js (1 tests) 1ms
   ✓ cart removes a line and recalculates the total 1ms
✓ src/components/__tests__/CourseSelect.spec.js (3 tests) 8ms
   ✓ CourseSelect > renders one option per course plus the placeholder 6ms
   ✓ CourseSelect > supports v-model (emits update:modelValue) 1ms
   ✓ CourseSelect > can be disabled 1ms
✓ src/components/__tests__/RegisterForm.spec.js (1 tests) 12ms
   ✓ RegisterForm rejects a short name 12ms

 Test Files  4 passed (4)
      Tests  6 passed (6)
   Duration  21ms
```

Follow the pyramid: many fast unit tests (functions, composables, stores), some component tests, a few E2E journeys. Vitest reuses the Vite config; mock with `vi.fn`, `vi.mock`, `vi.spyOn` and fake timers. Vue Test Utils: `mount`, `get/find`, `await trigger/setValue/setProps`, `emitted()`, `slots`, `flushPromises()`; Vue Testing Library queries by role, label and text. Test stores with a fresh `createPinia()` or `createTestingPinia`, routing with `createMemoryHistory`, APIs with MSW handlers. Run E2E with Playwright (or Cypress), track coverage, and run everything in CI.

MODULE 20 · ADVANCED

Full-stack Vue with Nuxt

Minimum duration **4 days** Level **Advanced**

Nuxt is the full-stack framework for Vue: it adds server rendering, file-based routing, auto-imports and back-end API routes to everything learned so far. This module explains the rendering modes (SPA, SSR, SSG and hybrid) and why server rendering helps SEO and performance; creates a Nuxt 4 project and walks through its directory structure and auto-imports; builds pages with file-based and dynamic routes, layouts and route middleware; fetches data with useFetch, useAsyncData and $fetch, including lazy loading states; writes server API routes in server/api and connects storage; shares state with useState and Pinia, sets titles and social meta tags with useHead and useSeoMeta, surveys the main Nuxt modules and authentication; and finishes with deployment targets.

IN THIS MODULE

20.1 Why Nuxt
20.2 Nuxt basics
20.3 Routing and layouts
20.4 Data fetching
20.5 Server routes
20.6 State, SEO and modules
20.7 Deploying Nuxt

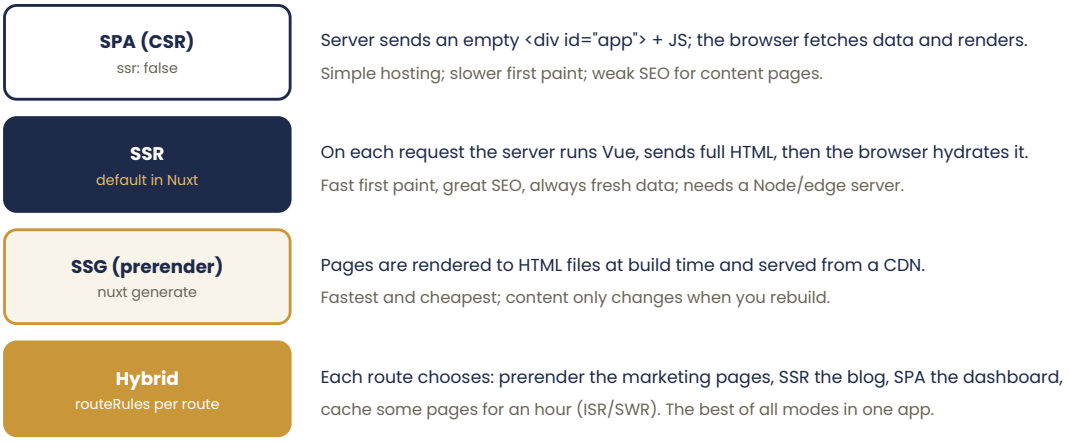
HOW THE NUXT EXAMPLES WERE RUN

Nuxt could not be installed offline. The Nuxt apps in this module are written exactly as for a real Nuxt 4 project (the app/ and server/ directories, auto-imports, file-based routes); they were run in Chromium on a compact stand-in that implements the same conventions and composables: routes, layouts and middleware are generated from the file names, server/api handlers answer the app's requests in the page, and pages render inside Suspense just as in Nuxt. Terminal sessions with the real CLI are labelled representative.

20.1 Why Nuxt

20.1.1 Rendering modes: SPA, SSR, SSG and hybrid rendering

A plain Vue app is a **single-page application** (SPA): the server sends an almost empty HTML file and JavaScript builds the page in the browser. Nuxt can also render on the server:



FOUR WAYS TO TURN ROUTES INTO HTML

Hydration is the step where Vue, running in the browser, attaches event listeners and reactivity to the HTML the server already sent, instead of creating the DOM again. Nuxt does this automatically, and the same component code runs on both sides. Rendering is configured in `nuxt.config.ts`:

NUXT.CONFIG.TS (HYBRID RENDERING) TypeScript

```ts
export default defineNuxtConfig({
  ssr: true,                                  // default; false = SPA
  routeRules: {
    '/': { prerender: true },                 // SSG at build time
    '/blog/**': { swr: 3600 },                // SSR, cached and revalidated every hour
    '/dashboard/**': { ssr: false },          // client-side only
    '/api/**': { cors: true },
  },
})
```

20.1.2 SEO and performance benefits

Benefit	Why server rendering helps
SEO	search engines and link previews (WhatsApp, LinkedIn, X) receive the full content and meta tags in the HTML, without running JavaScript
Faster first paint (LCP)	the browser can show content as soon as the HTML arrives, before the JavaScript bundle loads
Less work on slow devices	data is fetched server-side, close to the database, and sent with the page
Code splitting by default	each page is its own chunk; only the current page's code is loaded
Full-stack in one project	API routes, server middleware and the front end share one codebase, one deploy and shared types

20.2 Nuxt basics

20.2.1 Creating a Nuxt project

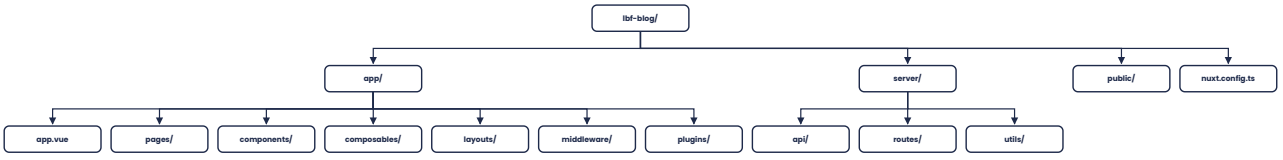
Creating and starting a Nuxt app (representative)

```
$ npm create nuxt@latest lbf-blog
✔ Which package manager would you like to use? › npm
✔ Initialize git repository? › Yes
✔ Would you like to install any of the official modules? › @nuxt/ui, @nuxt/image, @nuxt/content
✔ Installation completed.
$ cd lbf-blog
$ npm run dev
Nuxt 4.1.2 (with Nitro 2.12.4, Vite 7.1.5 and Vue 3.5.21)
  ➜ Local:    http://localhost:3000/
  ➜ Network:  use --host to expose
✔ Vite client built in 412ms
✔ Nitro server built in 731ms
```

Command	Does
`npm run dev`	development server with hot reload on port 3000
`npm run build`	production build in `.output/` (server + client)
`npm run generate`	prerender every route to static files (SSG)
`npm run preview`	run the production build locally
`npx nuxi add page about`	scaffold files (page, component, api, middleware, layout)

20.2.2 Directory structure (app/, pages/, components/, composables/, server/)

Nuxt is **convention over configuration**: where a file lives decides what it does. Nuxt 4 keeps the Vue application in `app/` and the back end in `server/`:



A NUXT 4 PROJECT

Path	Purpose
`app/app.vue`	root component (optional; default renders the layout and page)
`app/pages/`	one file per route: file-based routing (20.3.1)
`app/components/`	components, auto-imported by file name (`blog/PostCard.vue` → `<BlogPostCard>`)
`app/composables/`, `app/utils/`	functions auto-imported everywhere in the app
`app/layouts/`	page wrappers such as headers and sidebars (20.3.2)
`app/middleware/`	route guards (20.3.3)

Path	Purpose
`app/plugins/`	run once when the app starts (register libraries, provide helpers)
`server/api/`, `server/routes/`	back-end endpoints run by the Nitro server (20.5)
`public/`	static files served as-is (`favicon.ico`, `robots.txt`)
`nuxt.config.ts`	configuration: modules, head, runtime config, route rules

20.2.3 Auto-imports

Nuxt **auto-imports** Vue APIs (`ref`, `computed`, `watch`…), Nuxt composables (`useFetch`, `useState`, `useRoute`…), everything exported from `app/composables/` and `app/utils/`, and all components in `app/components/`. Only what is used ends up in the bundle, and the editor still gets full types (Nuxt generates declarations in `.nuxt/`). Explicit imports remain possible from `#imports`. In the first app below, `index.vue` uses `ref`, `useCounter` and `<AppCounter>` without a single import:

APP/COMPOSABLES/USECOUNTER.TS — TypeScript

```ts
export function useCounter(start = 0) {
  const count = ref(start)
  const inc = () => count.value++
  return { count, inc }
}
```

APP/COMPONENTS/APPCOUNTER.VUE — HTML

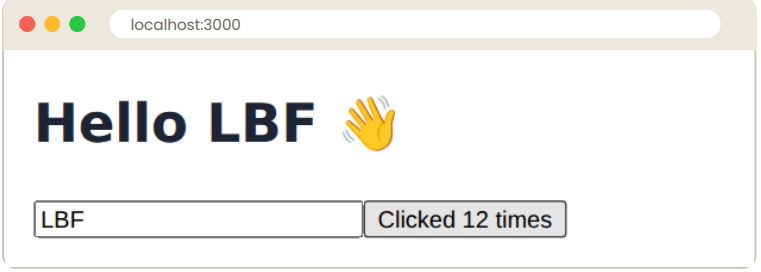
```html
<script setup lang="ts">
const { count, inc } = useCounter(10)
</script>

<template>
  <button @click="inc">Clicked {{ count }} times</button>
</template>
```

APP/PAGES/INDEX.VUE — HTML

```html
<script setup lang="ts">
const name = ref('Nuxt')
</script>

<template>
  <h1>Hello {{ name }} 👋</h1>
  <input v-model="name">
  <AppCounter />
</template>
```



NO IMPORT STATEMENTS: REF, THE COMPOSABLE AND THE COMPONENT WERE AUTO-IMPORTED

20.3 Routing and layouts

20.3.1 File-based routing and dynamic pages

Every `.vue` file in `app/pages/` becomes a route; Nuxt generates the Vue Router configuration. Square brackets make a **dynamic segment**, read with `useRoute().params`. Navigate with `<NuxtLink to="...">` (prefetches the target page's code when the link is visible) or `navigateTo()`.

File	Route	Example URL
`pages/index.vue`	`/`	/
`pages/about.vue`	`/about`	/about
`pages/blog/index.vue`	`/blog`	/blog
`pages/blog/[slug].vue`	`/blog/:slug`	/blog/vue-3-5 (params.slug = "vue-3-5")
`pages/users/[id]/edit.vue`	`/users/:id/edit`	/users/7/edit
`pages/[...slug].vue`	catch-all	/a/b/c (params.slug = ["a","b","c"])

APP/PAGES/BLOG/INDEX.VUE HTML

```
<script setup lang="ts">
const { data: posts } = await useFetch('/api/posts')
</script>

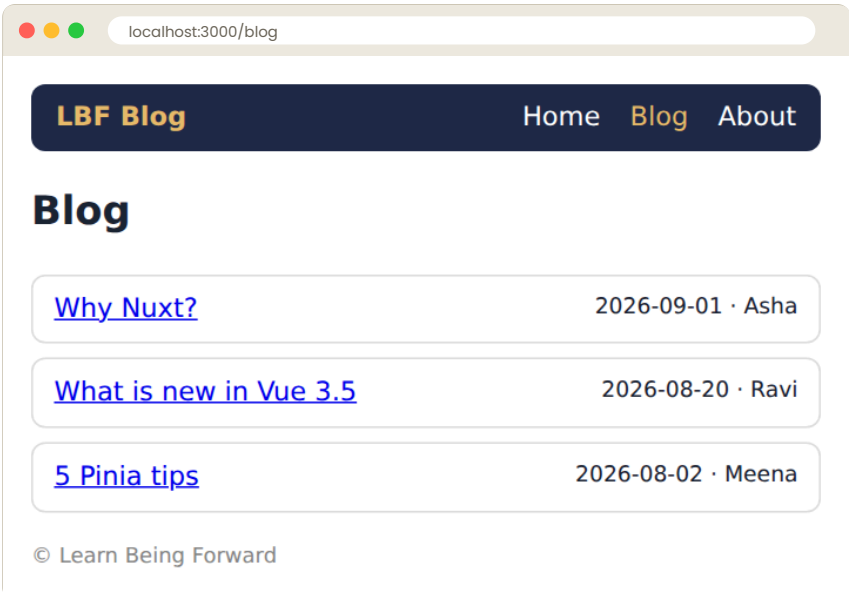
<template>
  <h2>Blog</h2>
  <article v-for="post in posts" :key="post.slug" class="card">
    <NuxtLink :to="`/blog/${post.slug}`">{{ post.title }}</NuxtLink>
    <small>{{ post.date }} · {{ post.author }}</small>
  </article>
</template>

<style>.card { border: 1px solid #ddd; border-radius: 8px; padding: 8px 12px; margin: 8px 0; display:
flex; justify-content: space-between; }</style>
```

APP/PAGES/BLOG/[SLUG].VUE HTML

```
<script setup lang="ts">
const route = useRoute()
const { data: post, error } = await useFetch(`/api/posts/${route.params.slug}`)
</script>

<template>
  <p v-if="error">⚠ {{ error.statusMessage }}</p>
  <article v-else-if="post">
    <h2>{{ post.title }}</h2>
    <small>{{ post.date }} · by {{ post.author }}</small>
    <p>{{ post.body }}</p>
    <NuxtLink to="/blog">← All posts</NuxtLink>
  </article>
</template>
```



/BLOG: THE LIST PAGE (PAGES/BLOG/INDEX.VUE)



AFTER CLICKING THE SECOND POST: /BLOG/VUE-3-5 RENDERED BY PAGES/BLOG/[SLUG].VUE

20.3.2 Layouts

A **layout** in `app/layouts/` wraps pages with shared UI. `default.vue` is used automatically (with `<NuxtLayout>` in `app.vue`, or by default when there is no `app.vue`); the page renders where the layout puts `<slot />`. A page chooses another layout with `definePageMeta({ layout: 'admin' })`, or none with `layout: false`. The blog above used this default layout:

APP/LAYOUTS/DEFAULT.VUE HTML

```
<template>
  <div class="site">
    <header>
      <NuxtLink to="/" class="logo">LBF Blog</NuxtLink>
      <nav><NuxtLink to="/">Home</NuxtLink> <NuxtLink to="/blog">Blog</NuxtLink> <NuxtLink
to="/about">About</NuxtLink></nav>
    </header>
    <main><slot /></main>
    <footer>© Learn Being Forward</footer>
  </div>
</template>

<style>
.site header { display: flex; justify-content: space-between; align-items: center; background:
#1E2A4A; padding: 8px 14px; border-radius: 8px; }
.site header a { color: white; text-decoration: none; margin-left: 12px; } .site .logo { font-weight:
700; margin: 0; color: #E6BC6A; }
.site nav a.router-link-exact-active { color: #E6BC6A; } .site footer { color: #888; font-size: 12px;
margin-top: 16px; }
</style>
```

APP/LAYOUTS/ADMIN.VUE HTML

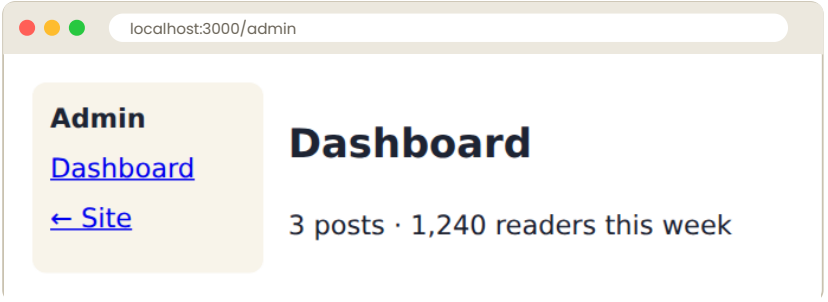
```
<template>
  <div class="admin">
    <aside><b>Admin</b><NuxtLink to="/admin">Dashboard</NuxtLink><NuxtLink to="/">← Site</NuxtLink>
</aside>
    <section><slot /></section>
  </div>
</template>

<style>
.admin { display: grid; grid-template-columns: 130px 1fr; gap: 14px; }
.admin aside { background: #F8F4EA; border-radius: 8px; padding: 10px; display: grid; gap: 6px; align-
content: start; }
</style>
```

APP/PAGES/ADMIN/INDEX.VUE HTML

```
<script setup lang="ts">
definePageMeta({ layout: 'admin' })
</script>

<template>
  <h2>Dashboard</h2>
  <p>3 posts · 1,240 readers this week</p>
</template>
```



/ADMIN USES THE ADMIN LAYOUT CHOSEN WITH DEFINEPAGEMETA

20.3.3 Route middleware

Route middleware runs before navigating to a page (on the server for the first request, then in the browser). Named middleware lives in `app/middleware/` and pages opt in with `definePageMeta({ middleware: 'auth' })`; files ending in `.global.ts` run on every route. Return `navigateTo('/login')` to redirect or `abortNavigation()` to stop; return nothing to continue.

APP/MIDDLEWARE/AUTH.TS TypeScript

```ts
export default defineNuxtRouteMiddleware((to) => {
  const { user } = useAuth()
  if (!user.value) {
    return navigateTo({ path: '/login', query: { redirect: to.fullPath } })
  }
})
```

APP/MIDDLEWARE/LOG.GLOBAL.TS TypeScript

```ts
export default defineNuxtRouteMiddleware((to, from) => {
  console.log(`[middleware] ${from.fullPath} → ${to.fullPath}`)
})
```

APP/COMPOSABLES/USEAUTH.TS TypeScript

```ts
export function useAuth() {
  const user = useState<{ name: string; role: string } | null>('user', () => null)   // shared app-wide
  const login = (name: string, role = 'admin') => { user.value = { name, role } }
  const logout = () => { user.value = null }
  return { user, login, logout }
}
```

APP/PAGES/ADMIN/INDEX.VUE HTML

```vue
<script setup lang="ts">
definePageMeta({ layout: 'admin', middleware: 'auth' })
const { user } = useAuth()
</script>

<template>
  <h2>Dashboard</h2>
  <p>Welcome, {{ user?.name }}. 3 posts · 1,240 readers this week</p>
</template>
```

APP/PAGES/LOGIN.VUE

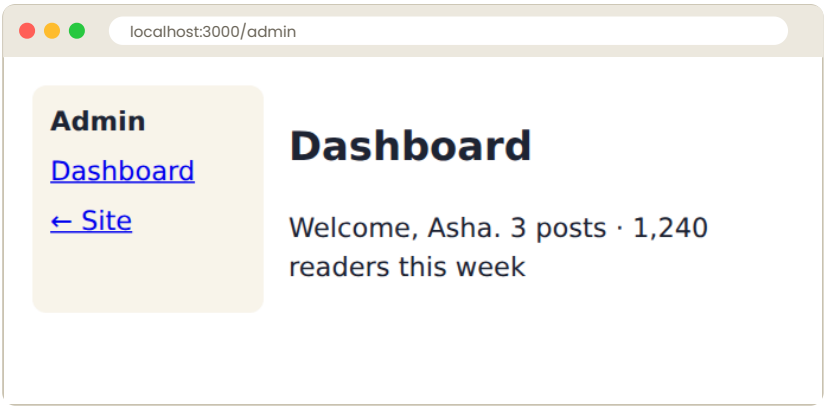
HTML

```
<script setup lang="ts">
const route = useRoute()
const { login } = useAuth()
const name = ref('')
async function submit() {
  login(name.value)
  await navigateTo((route.query.redirect as string) || '/')
}
</script>

<template>
  <h2>Log in</h2>
  <form @submit.prevent="submit"><input v-model="name" placeholder="Your name"> <button>Log
in</button></form>
</template>
```



VISITED /ADMIN WHILE LOGGED OUT: THE AUTH MIDDLEWARE REDIRECTED TO /LOGIN?REDIRECT=/ADMIN



AFTER LOGGING IN AS ASHA: SENT BACK TO THE PROTECTED DASHBOARD

20.4 Data fetching

20.4.1 useFetch and useAsyncData

`useFetch(url, options)` fetches data **once on the server** during server rendering and sends the result to the browser in the page payload, so the browser does not fetch it again during hydration. It returns refs: `data`, `status` (`idle`, `pending`, `success`, `error`), `error` and a `refresh()` function. Used with `await` in `<script setup>`, navigation waits until the data is ready. Reactive values in the URL or `query` re-fetch automatically when they change. `useAsyncData(key, () => anyAsyncWork())` is the general form for any

async function (several requests, a SDK call); `useFetch(url)` is shorthand for `useAsyncData(url, () => $fetch(url))`.

SERVER/API/POSTS/INDEX.GET.TS TypeScript

```ts
export default defineEventHandler((event) => {
  const { q = '', author = '' } = getQuery(event) as { q?: string; author?: string }
  return posts
    .filter((p) => p.title.toLowerCase().includes(q.toLowerCase()))
    .filter((p) => !author || p.author === author)
    .map(({ slug, title, author }) => ({ slug, title, author }))
})
```

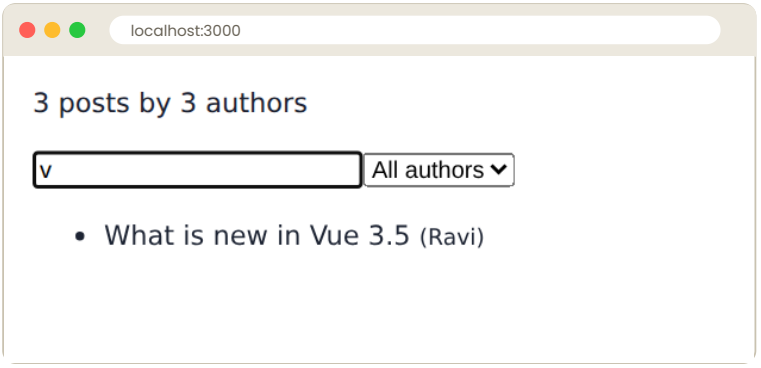
APP/PAGES/INDEX.VUE HTML

```html
<script setup lang="ts">
const q = ref('')
const author = ref('')

// re-fetches automatically when q or author change
const { data: posts, status } = await useFetch('/api/posts', { query: { q, author } })

// useAsyncData: any async function, here two requests in parallel
const { data: stats } = await useAsyncData('stats', async () => {
  const [all, authors] = await Promise.all([$fetch('/api/posts'), $fetch('/api/authors')])
  return { posts: all.length, authors: authors.length, names: authors }
})
</script>

<template>
  <p>{{ stats?.posts }} posts by {{ stats?.authors }} authors</p>
  <input v-model="q" placeholder="Search titles">
  <select v-model="author"><option value="">All authors</option><option v-for="a in stats?.names"
:key="a">{{ a }}</option></select>
  <p v-if="status === 'pending'">Searching…</p>
  <ul><li v-for="p in posts" :key="p.slug">{{ p.title }} <small>({{ p.author }})</small></li></ul>
</template>
```



TYPED "V" IN THE SEARCH: USEFETCH RE-RAN WITH THE NEW QUERY PARAMETER

20.4.2 $fetch

`$fetch(url, { method, body, query })` (from the **ofetch** library) is Nuxt's HTTP client: it parses JSON, serialises object bodies, and **throws** a `FetchError` for 4xx/5xx responses. Use it for actions triggered by the user (submitting a form, deleting an item) and inside server routes. Do not use it alone for data needed to render a page: the request would run twice (on the server and again in the browser); wrap it in `useAsyncData` or use `useFetch`.

APP/PAGES/INDEX.VUE

HTML

```
<script setup lang="ts">
const log = ref<string[]>([])
async function run() {
  const list = await $fetch('/api/posts')
  log.value.push(`GET /api/posts → ${list.length} posts, first: ${list[0].title}`)
  try {
    await $fetch('/api/posts/unknown')
  } catch (e: any) {
    log.value.push(`${e.name}: ${e.message}`)
    log.value.push(`e.data.statusMessage = ${e.data.statusMessage}`)
  }
}
</script>

<template>
  <button @click="run">Run requests</button>
  <pre>{{ log.join('\n') }}</pre>
</template>
```



$FETCH RETURNS PARSED JSON AND THROWS A FETCHERROR FOR THE 404

20.4.3 Lazy fetching and loading states

By default, awaiting `useFetch` blocks navigation until the data arrives. With `lazy: true` (or `useLazyFetch`) the page renders immediately and you show a loading state while `status` is `'pending'`; good for slow, non-critical data such as comments. `server: false` fetches only in the browser. Show progress between pages with `<NuxtLoadingIndicator />` in `app.vue`.

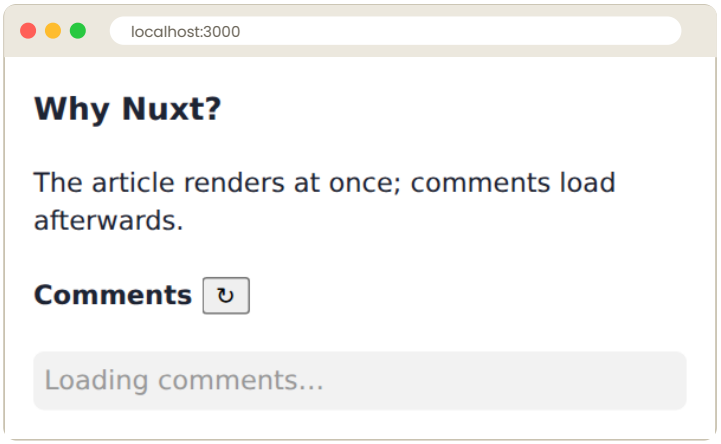
APP/PAGES/INDEX.VUE

HTML

```
<script setup lang="ts">
const { data: comments, status, refresh } = useLazyFetch('/api/comments', { default: () => [] })
</script>

<template>
  <h3>Why Nuxt?</h3>
  <p>The article renders at once; comments load afterwards.</p>
  <h4>Comments <button @click="refresh()">↻</button></h4>
  <p v-if="status === 'pending'" class="skeleton">Loading comments…</p>
  <ul v-else><li v-for="c in comments" :key="c.id"><b>{{ c.by }}:</b> {{ c.text }}</li></ul>
</template>

<style>.skeleton { color: #999; background: #f2f2f2; border-radius: 6px; padding: 6px; }</style>
```



IMMEDIATELY AFTER LOADING: THE PAGE IS VISIBLE WHILE THE COMMENTS ARE PENDING



ABOUT THREE SECONDS LATER: THE COMMENTS ARRIVED

20.5 Server routes

20.5.1 API routes in server/api

Files in `server/api/` become endpoints under `/api`, run by **Nitro**, Nuxt's server engine. A handler is `defineEventHandler((event) => ...)`; whatever it returns is sent as JSON. The file name sets the path and optionally the method: `server/api/students/[id].delete.ts` handles `DELETE /api/students/:id`. h3 utilities read the request: `getQuery(event)`, `await readBody(event)`, `getRouterParam(event, 'id')`; `createError({ statusCode, statusMessage })` sends an error status. Files in `server/utils/` are auto-imported in server code, and `server/routes/` creates routes without the `/api` prefix.

File	Endpoint
`server/api/hello.ts`	`GET|POST|… /api/hello`
`server/api/students/index.get.ts`	`GET /api/students`
`server/api/students/index.post.ts`	`POST /api/students`
`server/api/students/[id].get.ts`	`GET /api/students/7`
`server/routes/sitemap.xml.ts`	`GET /sitemap.xml`

20.5.2 Connecting a database

Server routes are where database access belongs: credentials stay on the server (read them from `useRuntimeConfig()`, filled from environment variables) and the browser only sees the API. Any Node

database client works (Prisma or Drizzle with PostgreSQL/MySQL/SQLite, the MongoDB driver, Supabase). Put the client in `server/utils/` so every route can use it. Nitro also has a built-in key-value **storage** layer, `useStorage()`, with drivers for memory, the file system, Redis, Cloudflare KV and more. A complete CRUD API for students, using a small data-access module in `server/utils/db.ts` backed by `useStorage`:

SERVER/UTILS/DB.TS TypeScript

```
// data access in one place; swap the body for Prisma/Drizzle/Mongo without touching the routes
export interface Student { id: number; name: string; course: string }

const store = () => useStorage('db')

export async function listStudents(): Promise<Student[]> {
  return (await store().getItem<Student[]>('students')) ?? [
    { id: 1, name: 'Asha Rao', course: 'Vue.js' }, { id: 2, name: 'Ravi Kumar', course: 'Nuxt' },
  ]
}
export async function saveStudents(list: Student[]) {
  await store().setItem('students', list)
}
```

SERVER/API/STUDENTS/INDEX.GET.TS TypeScript

```
export default defineEventHandler(() => listStudents())
```

SERVER/API/STUDENTS/INDEX.POST.TS TypeScript

```
export default defineEventHandler(async (event) => {
  const body = await readBody<{ name?: string; course?: string }>(event)
  if (!body?.name || body.name.trim().length < 3) {
    throw createError({ statusCode: 422, statusMessage: 'Name must have at least 3 characters' })
  }
  const list = await listStudents()
  const student = { id: Math.max(0, ...list.map((s) => s.id)) + 1, name: body.name.trim(), course:
body.course || 'Vue.js' }
  await saveStudents([...list, student])
  setResponseStatus(event, 201)
  return student
})
```

SERVER/API/STUDENTS/[ID].DELETE.TS TypeScript

```
export default defineEventHandler(async (event) => {
  const id = Number(getRouterParam(event, 'id'))
  const list = await listStudents()
  if (!list.some((s) => s.id === id)) throw createError({ statusCode: 404, statusMessage: 'Student not
found' })
  await saveStudents(list.filter((s) => s.id !== id))
  return { deleted: id }
})
```

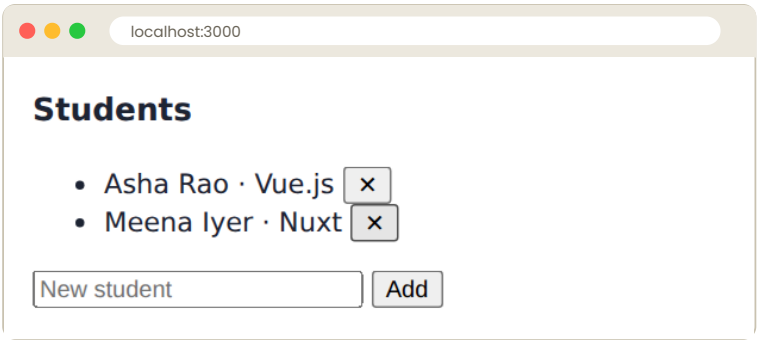
APP/PAGES/INDEX.VUE

HTML

```
<script setup lang="ts">
const { data: students, refresh } = await useFetch('/api/students')
const name = ref('')
const error = ref('')

async function add() {
  error.value = ''
  try {
    await $fetch('/api/students', { method: 'POST', body: { name: name.value, course: 'Nuxt' } })
    name.value = ''
    await refresh()
  } catch (e: any) {
    error.value = e.data?.statusMessage ?? e.message
  }
}
async function remove(id: number) {
  await $fetch(`/api/students/${id}`, { method: 'DELETE' })
  await refresh()
}
</script>

<template>
  <h3>Students</h3>
  <ul><li v-for="s in students" :key="s.id">{{ s.name }} · {{ s.course }} <button
@click="remove(s.id)">x</button></li></ul>
  <form @submit.prevent="add"><input v-model="name" placeholder="New student"> <button>Add</button>
</form>
  <p v-if="error" style="color: crimson">{{ error }}</p>
</template>
```



TRIED "AL" (REJECTED WITH 422 BY THE SERVER), ADDED MEENA IYER, THEN DELETED RAVI

20.6 State, SEO and modules

20.6.1 useState and Pinia in Nuxt

With server rendering, a module-level `const count = ref(0)` would be **shared by every user** on the server (a data leak) and would not be transferred to the browser. `useState(key, init)` creates state that is per request on the server, serialised into the payload, and shared by every component that uses the same key in the browser. For larger state, add Pinia with the `@pinia/nuxt` module: stores work as in Module 14 and are SSR-safe.

APP/COMPOSABLES/USETHEME.TS

TypeScript

```
export const useTheme = () => useState<'light' | 'dark'>('theme', () => 'light')
```

APP/COMPONENTS/THEMETOGGLE.VUE

HTML

```
<script setup lang="ts">
const theme = useTheme()
</script>

<template>
  <button @click="theme = theme === 'light' ? 'dark' : 'light'">Theme: {{ theme }}</button>
</template>
```

APP/STORES/CART.TS

TypeScript

```
export const useCartStore = defineStore('cart', () => {
  const items = ref<string[]>([])
  const add = (name: string) => items.value.push(name)
  return { items, add }
})
```

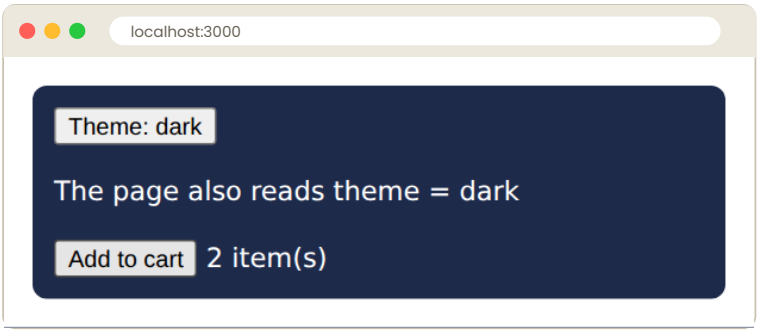
APP/PAGES/INDEX.VUE

HTML

```
<script setup lang="ts">
const theme = useTheme()                  // same state as inside ThemeToggle
const cart = useCartStore()               // Pinia via @pinia/nuxt
</script>

<template>
  <div :class="['page', theme]">
    <ThemeToggle />
    <p>The page also reads theme = {{ theme }}</p>
    <button @click="cart.add('Vue course')">Add to cart</button> {{ cart.items.length }} item(s)
  </div>
</template>

<style>.page { padding: 12px; border-radius: 8px; } .dark { background: #1E2A4A; color: white; }
</style>
```



THE TOGGLE AND THE PAGE SHARE ONE USESTATE VALUE; THE CART IS A PINIA STORE

20.6.2 useHead and useSeoMeta

`useHead()` sets the page title, meta and link tags, html attributes and scripts from any component, reactively; `useSeoMeta()` is a typed, flat shortcut for SEO and social tags (`description`, `ogTitle`, `ogImage`, `twitterCard`). With server rendering the tags are in the HTML that crawlers and link previews read. Site-wide defaults go in `nuxt.config.ts` under `app.head`, including a `titleTemplate`.

NUXT.CONFIG.TS TypeScript

```ts
export default defineNuxtConfig({
  app: {
    head: {
      titleTemplate: '%s · LBF Blog',
      htmlAttrs: { lang: 'en' },
      meta: [{ name: 'viewport', content: 'width=device-width, initial-scale=1' }],
    },
  },
})
```

APP/PAGES/BLOG/[SLUG].VUE HTML

```html
<script setup lang="ts">
const route = useRoute()
const { data: post } = await useFetch(`/api/posts/${route.params.slug}`)

useSeoMeta({
  title: () => post.value?.title,
  description: () => post.value?.body,
  ogTitle: () => post.value?.title,
  ogDescription: () => post.value?.body,
  ogImage: () => `https://lbf.example/og/${route.params.slug}.png`,
  twitterCard: 'summary_large_image',
})
useHead({ link: [{ rel: 'canonical', href: `https://lbf.example/blog/${route.params.slug}` }] })
</script>

<template>
  <article v-if="post">
    <h2>{{ post.title }}</h2>
    <p>{{ post.body }}</p>
  </article>
</template>
```

HTML SENT BY THE SERVER (VIEW SOURCE)

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Why Nuxt? · LBF Blog</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="description" content="Server rendering, file-based routing and API routes in one
framework.">
    <meta property="og:title" content="Why Nuxt?">
    <meta property="og:description" content="Server rendering, file-based routing and API routes in
one framework.">
    <meta property="og:image" content="https://lbf.example/og/why-nuxt.png">
    <meta name="twitter:card" content="summary_large_image">
    <link rel="canonical" href="https://lbf.example/blog/why-nuxt">
  </head>
  <body>
    <div id="__nuxt">
      <article>
        <h2>Why Nuxt?</h2>
        <p>Server rendering, file-based routing and API routes in one framework.</p>
      </article>
    </div>
    <script type="application/json" id="__NUXT_DATA__">{"data":{"/api/posts/why-nuxt":{"slug":"why-
nuxt","title":"Why Nuxt?","date":"2026-09-01","author":"Asha","body":"Server rendering, file-based
routing and API routes in one framework."}}}</script>
  </body>
</html>
```

The title (through the template), description, Open Graph and Twitter tags and the canonical link are in the HTML itself, so search engines and link previews see them without running JavaScript. The fetched post travels in the `__NUXT_DATA__` payload, so the browser hydrates without requesting it again.

20.6.3 Nuxt modules: Nuxt UI, Nuxt Image, Nuxt Content

Modules extend Nuxt with one line in `nuxt.config.ts` (`npx nuxi module add <name>` installs and registers them). There are 250+ at **nuxt.com/modules**. Three official ones:

Nuxt UI (@nuxt/ui)

A component library built on Tailwind CSS and Reka UI: buttons, forms with validation, tables, modals, navigation, dark mode, all accessible and themeable. Pairs with Nuxt UI Pro templates.

Nuxt Image (@nuxt/image)

`<NuxtImg>` and `<NuxtPicture>` resize, convert (WebP/AVIF) and lazy-load images, with providers such as Cloudinary or Vercel; a big win for LCP.

Nuxt Content (@nuxt/content)

A Git-based CMS: write Markdown/YAML/JSON in `content/`, query it with `queryCollection()`, and render it with `<ContentRenderer>`. Ideal for blogs and documentation.

USING OFFICIAL MODULES (REPRESENTATIVE) TypeScript

```
export default defineNuxtConfig({
  modules: ['@nuxt/ui', '@nuxt/image', '@nuxt/content', '@pinia/nuxt'],
  image: { quality: 80, formats: ['avif', 'webp'] },
})

<!-- a page using them -->
<script setup lang="ts">
const { data: post } = await useAsyncData('post', () =>
queryCollection('blog').path(useRoute().path).first())
</script>
<template>
  <UContainer>
    <NuxtImg :src="post.cover" width="800" height="420" alt="" loading="lazy" />
    <ContentRenderer :value="post" />
    <UButton to="/blog" icon="i-lucide-arrow-left">All posts</UButton>
  </UContainer>
</template>
```

20.6.4 Authentication (overview)

Authentication in Nuxt combines a **server session** with route middleware. The login route checks the credentials against the database and sets a secure, **httpOnly cookie** holding a signed session (never a token readable by JavaScript); server routes read the session to authorise requests; a middleware like the one in 20.3.3 protects pages; and a composable exposes the current user. The module **nuxt-auth-utils** provides exactly this (`setUserSession`, `requireUserSession`, `useUserSession()`) plus OAuth providers (GitHub, Google); **@sidebase/nuxt-auth** wraps Auth.js, and hosted services (Supabase Auth, Clerk, Auth0) have Nuxt modules.

SESSION AUTHENTICATION (REPRESENTATIVE) TypeScript

```
// server/api/login.post.ts  (with nuxt-auth-utils)
export default defineEventHandler(async (event) => {
  const { email, password } = await readBody(event)
  const user = await findUserByEmail(email)                    // server/utils/db.ts
  if (!user || !(await verifyPassword(user.passwordHash, password)))
    throw createError({ statusCode: 401, statusMessage: 'Invalid email or password' })
  await setUserSession(event, { user: { id: user.id, name: user.name, role: user.role } })
  return { ok: true }
})

// server/api/admin/stats.get.ts: protected API
export default defineEventHandler(async (event) => {
  const { user } = await requireUserSession(event)             // 401 if not logged in
  if (user.role !== 'admin') throw createError({ statusCode: 403 })
  return getStats()
})

// app/middleware/auth.ts: protected pages
export default defineNuxtRouteMiddleware(() => {
  const { loggedIn } = useUserSession()
  if (!loggedIn.value) return navigateTo('/login')
})
```

20.7 Deploying Nuxt

20.7.1 Static hosting vs Node/edge servers (Vercel, Netlify, Cloudflare)

`nuxt build` produces `.output/`: a server (`.output/server/index.mjs`, run with `node .output/server/index.mjs`) and the public assets. Nitro builds for many targets through **presets**, usually detected automatically on the hosting platform; `nuxt generate` prerenders a fully static site.

Target	Build	Suits	Notes
Static hosting (GitHub Pages, any CDN)	`nuxt generate`	blogs, docs, marketing sites	no server routes at runtime; data fixed at build time
Node server (VPS, Docker, Render, Railway)	`nuxt build`	full SSR apps with a database	run `node .output/server/index.mjs` behind a process manager
Vercel	zero config	SSR, hybrid, ISR	server routes become serverless functions; `routeRules` map to caching
Netlify	zero config	SSR, hybrid	Netlify Functions / Edge Functions
Cloudflare Pages / Workers	`NITRO_PRESET=cloudflare_pages`	global edge rendering	very fast cold starts; use KV/D1 for data

DOCKERFILE

text

```
# Dockerfile for a Node deployment
FROM node:22-slim AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-slim
WORKDIR /app
COPY --from=build /app/.output ./.output
ENV NUXT_DATABASE_URL="" PORT=3000
EXPOSE 3000
CMD ["node", ".output/server/index.mjs"]
```

TIP

Runtime config keys can be overridden by environment variables with the `NUXT_` prefix (`runtimeConfig.databaseUrl` ← `NUXT_DATABASE_URL`), so one build runs in staging and production with different secrets.

PRACTICE CHECKPOINT

A full-stack blog or job portal in Nuxt with server API routes, a database, SEO metadata and authentication, deployed live.

A compact job portal brings the module together: a jobs API with filtering and an authenticated "post a job" endpoint, storage through `server/utils`, a listing page with reactive filters, a detail page with SEO

tags, login with `useState` and protecting middleware, and a layout. Deploy it to Vercel or Netlify by pushing the repository, or with the Dockerfile above.

SERVER/UTILS/JOBS.TS TypeScript

```
export interface Job { id: number; title: string; company: string; city: string; type: 'Full-time' | 'Internship'; salary: string }
const seed: Job[] = [
  { id: 1, title: 'Vue.js Developer', company: 'Acme Tech', city: 'Bengaluru', type: 'Full-time', salary: '₹8-12 LPA' },
  { id: 2, title: 'Frontend Intern', company: 'PixelWorks', city: 'Pune', type: 'Internship', salary: '₹25k/month' },
  { id: 3, title: 'Nuxt Engineer', company: 'ShopKart', city: 'Bengaluru', type: 'Full-time', salary: '₹14-20 LPA' },
]
export async function allJobs(): Promise<Job[]> { return (await useStorage('db').getItem<Job[]>('jobs')) ?? seed }
export async function saveJobs(jobs: Job[]) { await useStorage('db').setItem('jobs', jobs) }
```

SERVER/API/JOBS/INDEX.GET.TS TypeScript

```
export default defineEventHandler(async (event) => {
  const { city = '', type = '' } = getQuery(event) as Record<string, string>
  return (await allJobs()).filter((j) => (!city || j.city === city) && (!type || j.type === type))
})
```

SERVER/API/JOBS/INDEX.POST.TS TypeScript

```
export default defineEventHandler(async (event) => {
  if (getHeader(event, 'authorization') !== 'Bearer demo-session') throw createError({ statusCode: 401, statusMessage: 'Log in to post jobs' })
  const body = await readBody(event)
  const jobs = await allJobs()
  const job = { id: jobs.length + 1, ...body }
  await saveJobs([job, ...jobs])
  setResponseStatus(event, 201)
  return job
})
```

APP/COMPOSABLES/USESESSION.TS TypeScript

```
export function useSession() {
  const user = useState<string | null>('user', () => null)
  const headers = computed(() => (user.value ? { authorization: 'Bearer demo-session' } : {}))
  return { user, headers, login: (name: string) => (user.value = name), logout: () => (user.value = null) }
}
```

APP/MIDDLEWARE/AUTH.TS TypeScript

```
export default defineNuxtRouteMiddleware(() => {
  if (!useSession().user.value) return navigateTo('/login')
})
```

APP/PAGES/INDEX.VUE HTML

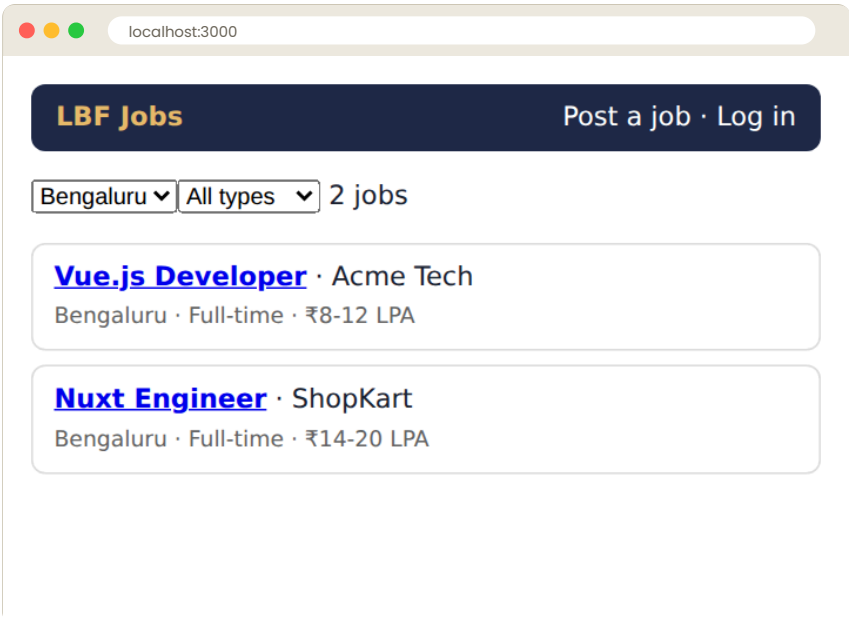
```
<script setup lang="ts">
useSeoMeta({ title: 'Frontend jobs in India', description: 'Vue and Nuxt jobs and internships' })
const city = ref('')
const type = ref('')
const { data: jobs } = await useFetch('/api/jobs', { query: { city, type } })
</script>

<template>
  <p>
    <select v-model="city"><option value="">All cities</option><option>Bengaluru</option>
<option>Pune</option></select>
    <select v-model="type"><option value="">All types</option><option>Full-time</option>
<option>Internship</option></select>
    {{ jobs?.length }} jobs
  </p>
  <div v-for="j in jobs" :key="j.id" class="job">
    <NuxtLink :to="`/jobs/${j.id}`"><b>{{ j.title }}</b></NuxtLink> · {{ j.company }}
    <br><small>{{ j.city }} · {{ j.type }} · {{ j.salary }}</small>
  </div>
</template>
```

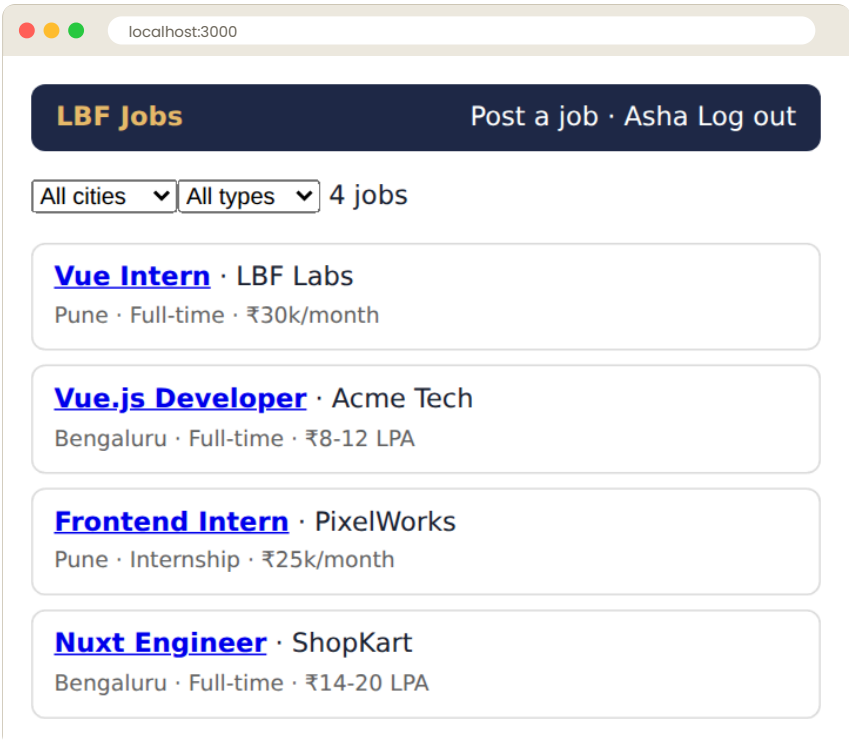
APP/PAGES/POST.VUE HTML

```
<script setup lang="ts">
definePageMeta({ middleware: 'auth' })
const { headers } = useSession()
const form = reactive({ title: '', company: '', city: 'Pune', type: 'Full-time', salary: '' })
async function publish() {
  await $fetch('/api/jobs', { method: 'POST', body: form, headers: headers.value })
  await navigateTo('/')
}
</script>

<template>
  <form @submit.prevent="publish">
    <h3>Post a job</h3>
    <input v-model="form.title" placeholder="Title"> <input v-model="form.company"
placeholder="Company">
    <input v-model="form.salary" placeholder="Salary"> <button>Publish</button>
  </form>
</template>
```



FILTERED TO BENGALURU



CLICKED "POST A JOB" (REDIRECTED TO LOGIN), LOGGED IN AS ASHA, POSTED "VUE INTERN" AT LBF LABS: IT NOW HEADS THE LIST

Nuxt = Vue + server rendering + conventions. Choose rendering per route (SPA, SSR, SSG, hybrid with `routeRules`). Files decide behaviour: `app/pages` (routes, `[param]`), `app/layouts`, `app/middleware`, `app/components` and `app/composables` (auto-imported), `server/api` (Nitro endpoints with `defineEventHandler`, `getQuery`, `readBody`, `createError`). Fetch page data with `await useFetch` / `useAsyncData` (fetched once on the server), use `$fetch` for user actions, `lazy` for non-critical data. Share state with `useState` or Pinia, set SEO tags with `useSeoMeta`, extend with modules, protect pages with middleware and sessions, and deploy static, to Node, or to Vercel/Netlify/Cloudflare.

MODULE 21 · ADVANCED

Deployment, projects and interview preparation

Minimum duration **3 days** | Level **Advanced**

The final module covers shipping Vue applications and getting hired. It builds for production with environment variables, deploys to Netlify, Vercel and GitHub Pages, configures servers for history-mode routing and automates everything with GitHub Actions; collects the best practices of the official style guide, ESLint and Prettier, a feature-based folder structure and front-end security; describes five capstone projects with their features and architecture; and prepares for interviews with a core-topics checklist answered concisely, the Vue 2 vs Vue 3 differences, solved machine-coding problems and a method for explaining your projects, before pointing to the next steps: mobile apps with Ionic or Quasar and back ends with Node.js or Laravel.

IN THIS MODULE

21.1 Deployment
21.2 Best practices
21.3 Capstone projects
21.4 Interview preparation
21.5 Next steps

21.1 Deployment

21.1.1 Production builds and environment variables

`npm run build` runs `vite build` (after `vue-tsc` in TypeScript projects) and writes an optimised, static site to `dist/`: minified JavaScript split into chunks with hashed file names (so browsers can cache them forever), extracted CSS and copied `public/` files. Vue's development warnings and DevTools hooks are removed. `npm run preview` serves `dist/` locally to check it before deploying.

Production build (representative)

```
$ npm run build
> vue-tsc --build && vite build
vite v7.1.5 building for production...
✓ 148 modules transformed.
dist/index.html                          0.52 kB │ gzip:  0.33 kB
dist/assets/index-Cq1VwZ8x.css           6.84 kB │ gzip:  2.01 kB
dist/assets/StudentsView-B4n2Kc9d.js     4.12 kB │ gzip:  1.77 kB
dist/assets/index-DkP0sT3a.js          104.20 kB │ gzip: 40.88 kB
✓ built in 1.86s
$ npm run preview
  ➜  Local:   http://localhost:4173/
```

Configuration that differs between environments goes in `.env` files (Module 15.1.4). Only variables prefixed `VITE_` reach the browser code, and they are **baked into the bundle at build time**: anyone can

read them, so never put secrets there. Use `.env.production` for production values or set the variables in the hosting dashboard, which overrides the files during the build.

File	Loaded when
`.env`	always
`.env.local`	always, git-ignored (your machine only)
`.env.development`	`vite` / `npm run dev`
`.env.production`	`vite build`
`.env.staging`	`vite build --mode staging`

21.1.2 Deploying to Netlify, Vercel or GitHub Pages

A Vue SPA is a folder of static files, so any static host works. The three most used:

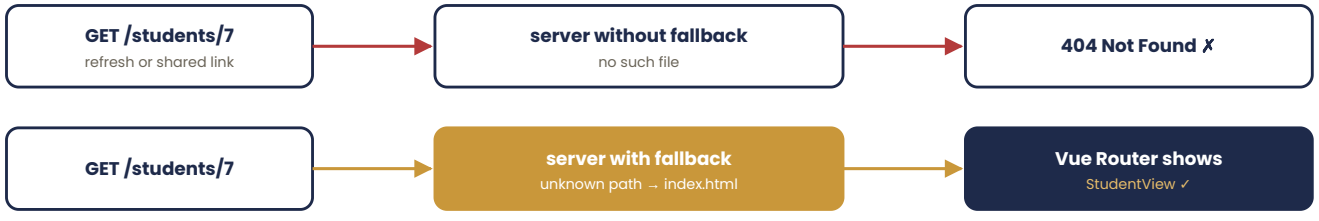
Host	How	Build settings
Netlify	connect the Git repository (or drag `dist/` onto the dashboard); every push deploys, every pull request gets a preview URL	build `npm run build`, publish `dist`
Vercel	import the repository; Vite is detected automatically; previews per branch	framework preset "Vite", output `dist`
GitHub Pages	a GitHub Actions workflow builds and publishes `dist/`; the site lives at `https://user.github.io/repo/`	set `base: '/repo/'` in `vite.config`

VITE.CONFIG.JS (GITHUB PAGES) — JavaScript

```
// vite.config.js for GitHub Pages: the app is served from /<repo>/
export default defineConfig({
  plugins: [vue()],
  base: process.env.GITHUB_ACTIONS ? '/student-portal/' : '/',
})
```

21.1.3 History-mode routing configuration

With `createWebHistory()` URLs look like `/students/7`. Navigating inside the app works, but when the user **refreshes** or opens that link directly, the browser asks the server for `/students/7`, a file that does not exist, and gets a 404. The server must answer every unknown path with `index.html` so the Vue router can take over (and the app should show its own 404 page with a catch-all route).



WHY HISTORY MODE NEEDS A SERVER REWRITE

Server / host	Fallback configuration
Netlify	`public/_redirects`: `/* /index.html 200`

Server / host	Fallback configuration
Vercel	`vercel.json`: `{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }`
Nginx	`location / { try_files $uri $uri/ /index.html; }`
Apache	`.htaccess`: `FallbackResource /index.html`
Express	`app.use(express.static('dist')); app.get('*', (req, res) => res.sendFile('dist/index.html'))`
GitHub Pages	no rewrites: copy `index.html` to `404.html` in the build, or use `createWebHashHistory()`

21.1.4 CI/CD with GitHub Actions

Continuous integration runs lint, type check, tests and the build on every push; **continuous deployment** publishes automatically when the main branch passes. This workflow deploys a Vue app to GitHub Pages:

.GITHUB/WORKFLOWS/DEPLOY.YML

text

```
name: Deploy
on:
  push: { branches: [main] }
permissions: { contents: read, pages: write, id-token: write }

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm run test:unit -- --run
      - run: npm run build
        env:
          VITE_API_URL: ${{ vars.VITE_API_URL }}
      - run: cp dist/index.html dist/404.html          # history-mode fallback on Pages
      - uses: actions/upload-pages-artifact@v3
        with: { path: dist }
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment: { name: github-pages, url: "${{ steps.deployment.outputs.page_url }}" }
    steps:
      - id: deployment
        uses: actions/deploy-pages@v4
```

21.2 Best practices

21.2.1 Vue style guide

The official **Vue style guide** (vuejs.org/style-guide) groups rules by priority. The most important ones:

Rule	Do	Avoid
Multi-word component names	`TodoItem.vue`, `AppHeader.vue`	`Item.vue` (clashes with HTML elements)
Detailed prop definitions	`defineProps({ status: { type: String, required: true } })` or TS types	`defineProps(['status'])`
Keyed v-for	`v-for="t in todos" :key="t.id"`	no key, or the index as key for editable lists
v-if with v-for	filter in a `computed`, or wrap in `<template v-for>`	both on one element
Scoped styles	`<style scoped>` or CSS Modules in components	global styles from components
File names	PascalCase `StudentCard.vue`; base components prefixed `BaseButton`, `AppIcon`	mixed casing
Simple templates	move complex expressions into `computed`	long JavaScript inside `{{ }}`
Prop naming	camelCase in script, kebab-case in templates (`:max-items`)	inconsistent casing
Block order	`<script setup>`, `<template>`, `<style>` consistently	random order per file

21.2.2 ESLint and Prettier setup

ESLint with `eslint-plugin-vue` finds bugs and style-guide violations (unused variables, missing keys, mutating props); **Prettier** formats code consistently so reviews focus on logic. create-vue sets both up when you choose them. Run them in the editor on save, in a pre-commit hook (husky + lint-staged) and in CI.

ESLINT.CONFIG.JS

JavaScript

```
// eslint.config.js (flat config)
import pluginVue from 'eslint-plugin-vue'
import js from '@eslint/js'
import skipFormatting from '@vue/eslint-config-prettier/skip-formatting'

export default [
  { files: ['**/*.{js,mjs,vue}'] },
  { ignores: ['dist/**', 'coverage/**'] },
  js.configs.recommended,
  ...pluginVue.configs['flat/recommended'],        // style guide rules (A, B and C priority)
  {
    rules: {
      'vue/multi-word-component-names': 'error',
      'vue/no-mutating-props': 'error',
      'vue/require-default-prop': 'off',
    },
  },
  skipFormatting,                                  // leave formatting to Prettier
]
```

.PRETTIERRC.JSON

JavaScript

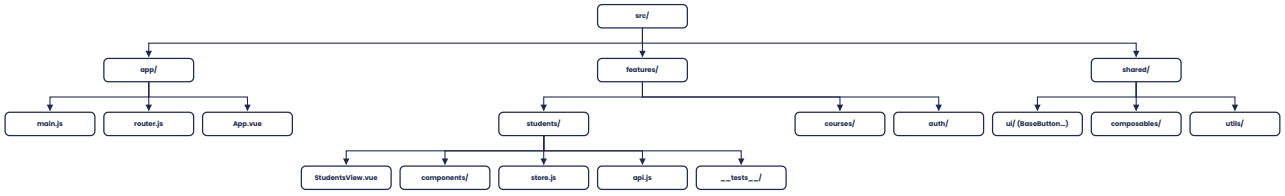
```
{
  "$schema": "https://json.schemastore.org/prettierrc",
  "semi": false,
  "singleQuote": true,
  "printWidth": 100
}
```

ESLint findings (representative)

```
$ npm run lint
/src/components/TodoList.vue
  12:7   error  Elements in iteration expect to have 'v-bind:key' directives  vue/require-v-for-key
  18:5   error  Unexpected mutation of "items" prop                           vue/no-mutating-props
/src/components/Item.vue
  1:1    error  Component name "Item" should always be multi-word             vue/multi-word-component-
names
✖ 3 problems (3 errors, 0 warnings)
```

21.2.3 Feature-based folder structure

Small apps can group files by **type** (`components/`, `views/`, `stores/`). As an app grows, group by **feature**: everything about students (pages, components, store, API service, tests) lives together, and shared, generic pieces live in `shared/`. Features can be understood, changed and even deleted as a unit.



FEATURE-BASED STRUCTURE

21.2.4 Security: avoiding v-html with user content, protecting tokens

Vue escapes interpolated text automatically, which prevents most **cross-site scripting** (XSS). The main holes are the ones you open yourself:

Risk	Safe practice
`v-html` with user content	avoid it; if HTML is needed (Markdown, CMS), sanitise with **DOMPurify** first
User-controlled URLs in `:href` / `:src`	allow only `https:` / `mailto:` (block `javascript:` URLs)
User-controlled styles	never bind raw user strings to `:style` or into `<style>`
Tokens in `localStorage`	readable by any injected script; prefer **httpOnly, Secure, SameSite** cookies set by the server
Secrets in `VITE_` variables	never: they are public in the bundle; keep secrets on the server (or Nuxt server routes)
Trusting the front end	always validate and authorise on the server; route guards and hidden buttons are UX, not security
Outdated dependencies	`npm audit`, Dependabot, update regularly

SRC/APP.VUE

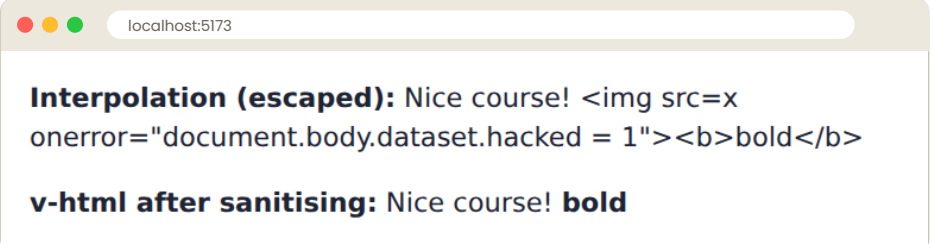
HTML

```
<script setup>
import { ref, computed } from 'vue'

// what a malicious user typed into a comment box
const comment = ref('Nice course! <img src=x onerror="document.body.dataset.hacked = 1"><b>bold</b>')

// minimal allow-list sanitiser for the demo; in a real app use DOMPurify.sanitize(comment.value)
const safeHtml = computed(() => {
  const doc = new DOMParser().parseFromString(comment.value, 'text/html')
  doc.body.querySelectorAll('*').forEach((el) => {
    if (!['B', 'I', 'EM', 'STRONG', 'P'].includes(el.tagName)) el.remove()
    else [...el.attributes].forEach((a) => el.removeAttribute(a.name))
  })
  return doc.body.innerHTML
})
</script>

<template>
  <p><b>Interpolation (escaped):</b> {{ comment }}</p>
  <p><b>v-html after sanitising:</b> <span v-html="safeHtml"></span></p>
</template>
```



INTERPOLATION SHOWS THE MARKUP AS TEXT; THE SANITISED V-HTML KEEPS ONLY THE ALLOWED BOLD TAG AND DROPS THE IMG ONERROR ATTACK

21.3 Capstone projects

Portfolio projects prove skill better than certificates. Build them like real products: a README with screenshots and a live link, clean commits, tests for the core logic, a deployed URL, and a short write-up of the decisions you made. Each capstone below lists the minimum features and the course modules it exercises.

21.3.1 E-commerce store with cart, filters, auth and admin panel

Features

product grid with search, category and price filters (query params in the URL), product detail, cart with quantities and coupon, checkout form with validation, login/register, order history, admin CRUD for products with image upload.

Architecture

Vue Router with lazy routes and auth/admin guards; Pinia stores: `auth`, `products`, `cart` (persisted); Axios service layer with interceptors; VeeValidate + Zod forms; a fake API (json-server or MSW) or Nuxt server routes.

Modules

9 to 15, 18 (UI library table in admin), 19 (cart store tests, checkout E2E), 21 (deploy)

21.3.2 Kanban task manager with drag and drop and Pinia

Features

boards with columns (To do, In progress, Done), cards with title, description, labels, assignee and due date; drag cards between columns and reorder; filter by label/assignee; undo; data saved to localStorage or an API.

Architecture

Pinia store holding boards → columns → cards with actions `moveCard(cardId, toColumn, index)`; drag and drop with the native HTML5 API or **vue-draggable-plus** / SortableJS; `TransitionGroup` for smooth moves; keyboard support for accessibility.

Modules

9, 11 (transitions, teleport for card modal), 12 (useLocalStorage composable), 14, 17 (shallow data for big boards), 19

21.3.3 Admin dashboard with charts and a UI library

Features

login, sidebar layout, KPI cards, line/bar/pie charts with date-range filters, sortable/paginated data tables with export to CSV, user management, dark mode, responsive layout.

Architecture

Vuetify or PrimeVue for layout and tables; charts with **vue-chartjs** (Chart.js) or ECharts; TanStack Query for cached API data; route meta for permissions with a `v-permission` directive.

Modules

13, 15 (TanStack Query), 16 (TypeScript), 17 (virtual scrolling), 18 (directives, UI library, i18n)

21.3.4 Weather or movie app using a public API

Features

search with debounce, results grid, detail page, favourites saved locally, loading skeletons, error and empty states, geolocation (weather) or genre filters and pagination (movies).

Architecture

composables `useFetch`, `useDebounce`, `useLocalStorage`; APIs such as Open-Meteo (no key needed) or TMDB (key through an environment variable and ideally a small proxy); Router query params for search state.

Modules

10, 12, 13, 15, 17 (lazy images), 21

21.3.5 Full-stack Nuxt application

Features

a blog, job portal or course platform: public SSR pages with SEO metadata, authentication, a dashboard to create and edit content, comments or applications, image uploads, sitemap.

Architecture

Nuxt 4 with server/api routes, a real database (PostgreSQL via Prisma/Drizzle, or Supabase), nuxt-auth-utils sessions, Nuxt UI, useSeoMeta, routeRules for hybrid rendering, deployed on Vercel/Netlify/Cloudflare.

Modules

20 (all), 16, 19

21.4 Interview preparation

21.4.1 Core topics checklist

Interviewers check that you understand **why**, not only **how**. Each answer below is the short version; be ready to back it with a small example.

Question	Answer in one breath
ref vs reactive?	`ref()` wraps any value (primitives too) in an object with `.value`; `reactive()` returns a deep Proxy of an object. reactive loses reactivity when destructured or replaced, so `ref` is the recommended default (Module 5).
computed vs watch?	`computed` derives a cached value from state, synchronously and without side effects; `watch` runs side effects (API calls, logging, saving) when a source changes, with access to old and new values (5.3, 10.1).
v-if vs v-show?	`v-if` adds/removes the element (lazy, higher toggle cost); `v-show` toggles CSS `display` (always rendered, cheap to toggle). Use v-show for frequent toggles (6.1.4).
Why keys in v-for?	Keys give each vnode an identity so the patcher reuses, moves or removes the right elements and keeps component state and input values with the right item. Use stable ids, not indexes, for lists that change order (6.2.3).
Props and emits?	One-way data flow: parents pass data down with props (read-only in the child), children notify parents with emitted events; `v-model` on components is props + `update:modelValue` (or `defineModel`) (9).
What are slots?	Placeholders for parent-provided template content: default, named and scoped slots (the child passes data back to the slot content) (9.3).
Lifecycle hooks?	`onBeforeMount`, `onMounted` (DOM ready, fetch/listeners), `onBeforeUpdate` / `onUpdated`, `onBeforeUnmount` / `onUnmounted` (cleanup); setup itself replaces created (10.2).
provide/inject?	Dependency injection down a component subtree without prop drilling; provide refs for reactivity and use InjectionKey for types (11.2).
What is a composable?	A function that uses Composition API features to encapsulate and reuse stateful logic (`useFetch`, `useMouse`), returning refs; the Vue 3 replacement for mixins (12).
Why Pinia?	Official store: simple stores per domain, state/getters/actions without mutations, full TypeScript inference, DevTools and plugins, SSR-safe (14).
Router guards?	Global `beforeEach` / `beforeResolve` / `afterEach`, per-route `beforeEnter`, in-component `onBeforeRouteLeave/Update`; return false, a route, or nothing; used for auth and unsaved-changes prompts (13.4).
Options vs Composition API?	Options API organises code by option type (data, methods, computed); Composition API organises it by feature inside `setup`, enables composables and better TypeScript. Both are supported; new projects favour `<script setup>` (1.6).
nextTick?	DOM updates are batched and applied asynchronously; `await nextTick()` waits until the DOM reflects the latest state changes (5.4.2).
How does reactivity work?	Proxies (and ref getters/setters) track which effect reads which property and trigger those effects when the property changes; updates are batched per tick (17.1).
Virtual DOM?	Render functions produce vnode trees; Vue diffs old and new trees and patches only changed DOM, helped by compiler patch flags and cached static nodes (17.2).

21.4.2 Vue 2 vs Vue 3 differences

Topic	Vue 2	Vue 3
Reactivity	`Object.defineProperty`: new properties and array index changes not detected (`Vue.set` needed)	Proxy-based: additions, deletions, Map/Set all tracked
API style	Options API (Composition API added in 2.7)	Composition API + `<script setup>`, Options API still supported
App creation	`new Vue({ render: h => h(App) }).$mount('#app')`	`createApp(App).mount('#app')`, plugins per app
Template root	exactly one root element	fragments: multiple roots allowed
v-model on components	`value` prop + `input` event; `.sync` modifier	`modelValue` + `update:modelValue`, multiple v-models, `defineModel`
New built-ins	-	`Teleport`, `Suspense`, multiple v-model, `emits` option
Removed	-	filters, `$on/$off` event bus, `Vue.set`, `$listeners` (merged into `$attrs`)
TypeScript	limited (class components)	written in TypeScript, first-class support
Performance / size	larger runtime, full re-render diffs	tree-shakable, faster diffing with patch flags, smaller
Tooling	Vue CLI (webpack), Vuex	Vite, Pinia, Vitest, Vue DevTools 7
Status	end of life 31 Dec 2023	current (3.5, Vapor Mode arriving in 3.6)

21.4.3 Machine-coding rounds: to-do, search with debounce, pagination, modal, form validation

Machine-coding rounds give 45 to 90 minutes to build a working feature. Interviewers watch your structure, state design, edge cases (empty, loading, error) and communication. A reliable routine: clarify requirements, sketch components and state, build the happy path, then handle edge cases and polish, explaining as you go. The to-do list (19.3.1), modal (18.7.1) and form validation (15.3, 19.3.4) appear earlier in the course; here are the two most frequent remaining ones.

Search with debounce: wait until the user stops typing before requesting, cancel stale requests, and handle loading, empty and error states.

SRC/COMPOSABLES/USEDEBOUNCE.JS

JavaScript

```
import { ref, watch, onScopeDispose } from 'vue'

export function useDebounce(source, delay = 300) {
  const debounced = ref(source.value)
  let timer
  watch(source, (v) => {
    clearTimeout(timer)
    timer = setTimeout(() => (debounced.value = v), delay)
  })
  onScopeDispose(() => clearTimeout(timer))
  return debounced
}
```

SRC/APP.VUE

HTML

```
<script setup>
import { ref, watch } from 'vue'
import { useDebounce } from './composables/useDebounce'

const query = ref('')
const debounced = useDebounce(query, 300)
const results = ref([])
const status = ref('idle')                    // idle | loading | done | error
let controller

watch(debounced, async (q) => {
  controller?.abort()                         // cancel the previous request
  if (!q.trim()) { results.value = []; status.value = 'idle'; return }
  controller = new AbortController()
  status.value = 'loading'
  try {
    const res = await fetch(`/api/students?q=${encodeURIComponent(q)}`, { signal: controller.signal })
    results.value = await res.json()
    status.value = 'done'
  } catch (e) {
    if (e.name !== 'AbortError') status.value = 'error'
  }
})
</script>

<template>
  <input v-model="query" placeholder="Search students" autofocus>
  <p v-if="status === 'loading'">Searching…</p>
  <p v-else-if="status === 'error'">Something went wrong</p>
  <p v-else-if="status === 'done' && !results.length">No students match "{{ debounced }}"</p>
  <ul v-else><li v-for="r in results" :key="r">{{ r }}</li></ul>
</template>
```



TYPED "ASH" QUICKLY: ONE REQUEST AFTER THE PAUSE, TWO MATCHES

BROWSER CONSOLE: THREE KEYSTROKES, ONE REQUEST

```
results for "kir": Kiran Das
results for "p": Priya Nair, Akash Patel
```

Pagination: a reusable component with previous/next, page numbers with ellipses, and the current page kept in the URL query in a real app.

SRC/COMPONENTS/APPPAGINATION.VUE HTML

```
<script setup>
import { computed } from 'vue'

const page = defineModel({ type: Number, default: 1 })
const props = defineProps({ total: { type: Number, required: true }, perPage: { type: Number, default:
10 } })
const pages = computed(() => Math.max(1, Math.ceil(props.total / props.perPage)))

// 1 … 4 5 6 … 10: first, last, current ± 1
const items = computed(() => {
  const out = []
  for (let p = 1; p <= pages.value; p++) {
    if (p === 1 || p === pages.value || Math.abs(p - page.value) <= 1) out.push(p)
    else if (out[out.length - 1] !== '…') out.push('…')
  }
  return out
})
</script>

<template>
  <nav class="pager" aria-label="Pagination">
    <button :disabled="page === 1" @click="page--">‹ Prev</button>
    <template v-for="(it, i) in items" :key="i">
      <span v-if="it === '…'">…</span>
      <button v-else :class="{ on: it === page }" :aria-current="it === page ? 'page' : null"
@click="page = it">{{ it }}</button>
    </template>
    <button :disabled="page === pages" @click="page++">Next ›</button>
  </nav>
</template>

<style scoped>
.pager { display: flex; gap: 4px; align-items: center; }
button { border: 1px solid #ccd; background: white; border-radius: 6px; padding: 3px 9px; cursor:
pointer; }
.on { background: #1E2A4A; color: white; } button:disabled { opacity: .4; }
</style>
```

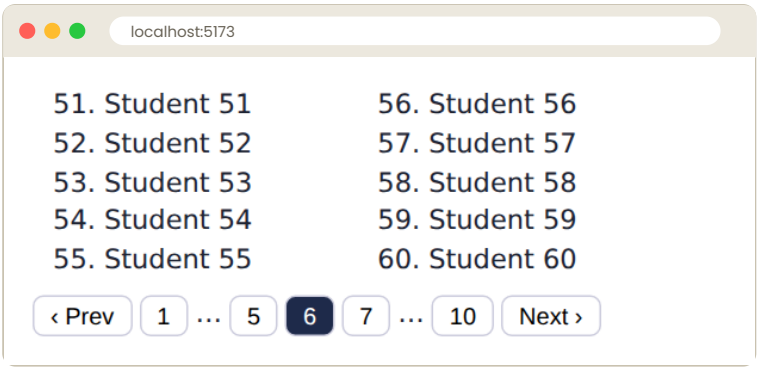
SRC/APP.VUE HTML

```
<script setup>
import { ref, computed } from 'vue'
import AppPagination from './components/AppPagination.vue'

const all = Array.from({ length: 95 }, (_, i) => `Student ${i + 1}`)
const page = ref(1)
const perPage = 10
const visible = computed(() => all.slice((page.value - 1) * perPage, page.value * perPage))
</script>

<template>
  <ol :start="(page - 1) * perPage + 1"><li v-for="s in visible" :key="s">{{ s }}</li></ol>
  <AppPagination v-model="page" :total="all.length" :per-page="perPage" />
</template>

<style>ol { columns: 2; margin: 0 0 8px; }</style>
```



WENT TO PAGE 5, THEN NEXT: PAGE 6 OF 10 WITH ELLIPSES ON BOTH SIDES

Common round	What they look for
To-do list	add/edit/delete/toggle, filters (all/active/done), keys, persistence, empty state
Search with debounce	debounce, cancelling stale requests (AbortController), loading/empty/error states
Pagination / infinite scroll	derived state with computed, URL query sync, IntersectionObserver for infinite scroll
Modal / dialog	Teleport, v-model to open/close, Escape and backdrop close, focus trap and return focus
Form validation	field-level and form-level rules, show errors after blur/submit, disable submit while sending
Star rating, accordion, tabs, autocomplete, stopwatch	reusable component API (props, emits, v-model, slots), keyboard support, cleanup of timers

21.4.4 Explaining your projects

Prepare a two-minute story for each portfolio project with the **STAR** structure and real numbers:

Situation / Task

"Our training centre tracked 600 students in spreadsheets; I built a student management portal so trainers could record marks and attendance."

Action

"Vue 3 with script setup and TypeScript, Pinia stores per domain, a typed Axios service layer, VeeValidate + Zod forms, route guards for roles, Vitest for stores and Playwright for the enrolment flow, deployed on Netlify with GitHub Actions."

Result

"Data entry time dropped from about a day to an hour a week, there were no more duplicate records, and Lighthouse performance is 98 thanks to lazy routes and virtual scrolling."

Be ready for follow-ups

Why Pinia and not provide/inject? What was the hardest bug? How would you scale it to 100,000 students? What would you do differently? What do the tests cover?

21.5 Next steps

21.5.1 Mobile apps with Ionic Vue or Quasar

Ionic Vue provides mobile UI components (tabs, lists, swipe gestures, native-looking transitions) for Vue and uses **Capacitor** to package the web app as a native iOS/Android app with access to the camera, GPS, notifications and storage. **Quasar** is a Vue framework with 70+ Material components and one codebase that builds an SPA, SSR site, PWA, mobile app (Capacitor/Cordova) and desktop app (Electron). Both reuse everything from this course: components, Pinia, Vue Router and composables.

21.5.2 Back-end with Node.js or Laravel

To build complete products, pair Vue with a back end. **Node.js** with **Express**, **Fastify** or **NestJS** keeps one language across the stack (and Nuxt server routes are already Node). **Laravel** (PHP) has a long partnership with Vue: **Inertia.js** lets Laravel controllers render Vue pages directly without building a separate API, and Laravel starter kits ship with Vue, TypeScript and Tailwind. Whichever you choose, learn REST and authentication on the server, a SQL database, and deployment with Docker.

PRACTICE CHECKPOINT

Build and deploy two capstone projects (one Vue SPA, one Nuxt full-stack app) with tests and GitHub repositories, and complete five timed machine-coding challenges.

Week	Plan
1	Pick the SPA capstone (21.3.1 to 21.3.4); write the README with features and wireframes; scaffold with create-vue (TS, Router, Pinia, Vitest, ESLint)
2	Build the core features; store and composable tests; deploy early to Netlify/Vercel with CI
3	Nuxt capstone (21.3.5): server routes, database, auth, SEO; deploy to Vercel/Netlify
4	Polish both (loading/empty/error states, accessibility, Lighthouse), add E2E tests, write the STAR stories; do one 60-minute machine-coding challenge per day: to-do, debounced search, pagination, modal, form validation

Ship with `npm run build`, `VITE_` variables (public, build-time), a history-mode fallback on the server, and CI/CD that lints, tests, builds and deploys. Follow the style guide, enforce it with ESLint and Prettier, organise by feature, and never trust user HTML, tokens in localStorage or front-end-only checks. Prove your skills with deployed capstones, answer core questions with the why, practise machine-coding rounds, and tell project stories with STAR. Next: Ionic/Quasar for mobile and Node.js or Laravel for the back end.